

# **Manual de Linux e Shell Scripting**

**Do básico ao avançado — do terminal ao kernel, com Kali Linux como referência**

Vitor Mattos

02/08/2026



# Índice

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| <b>Apresentação</b>                                                         | <b>1</b>  |
| Para quem é este manual . . . . .                                           | 1         |
| Como o manual está organizado . . . . .                                     | 1         |
| Como cada capítulo funciona . . . . .                                       | 1         |
| <br>                                                                        |           |
| <b>I. Volume 1 — Primeiros Passos: Linux e Kali</b>                         | <b>3</b>  |
| <br>                                                                        |           |
| <b>1. O que é Linux: kernel, distribuições e o modelo de software livre</b> | <b>5</b>  |
| 1.1. O que a máquina responde no primeiro minuto . . . . .                  | 5         |
| 1.2. O kernel: o Linux no sentido estrito . . . . .                         | 6         |
| 1.3. De onde veio: Unix, GNU e 1991 . . . . .                               | 7         |
| 1.4. O que é, então, uma distribuição . . . . .                             | 8         |
| 1.5. Famílias: o mapa das distribuições . . . . .                           | 9         |
| 1.6. O modelo de software livre . . . . .                                   | 10        |
| 1.7. O que muda no seu dia a dia . . . . .                                  | 10        |
| 1.8. No Kali . . . . .                                                      | 10        |
| 1.9. Laboratório . . . . .                                                  | 11        |
| 1.10. Resumo . . . . .                                                      | 13        |
| <br>                                                                        |           |
| <b>2. Por que Kali Linux: propósito, filosofia e quando não usá-lo</b>      | <b>15</b> |
| 2.1. O que o Kali é, em uma frase . . . . .                                 | 15        |
| 2.2. De onde veio: a linhagem BackTrack . . . . .                           | 15        |
| 2.3. As decisões de projeto que explicam o comportamento . . . . .          | 16        |
| 2.4. O contrato de segurança que você assina ao instalar . . . . .          | 17        |
| 2.5. Quando <b>não</b> usar Kali . . . . .                                  | 18        |
| 2.6. As alternativas que valem conhecer . . . . .                           | 19        |
| 2.7. No Kali . . . . .                                                      | 19        |
| 2.8. Laboratório . . . . .                                                  | 20        |
| 2.9. Resumo . . . . .                                                       | 21        |
| <br>                                                                        |           |
| <b>3. Instalando o Kali: máquina virtual, bare metal, WSL e live USB</b>    | <b>23</b> |
| 3.1. Antes de tudo: verificar a imagem . . . . .                            | 23        |
| 3.2. Qual imagem baixar . . . . .                                           | 24        |
| 3.3. Método 1: máquina virtual — o padrão recomendado . . . . .             | 24        |
| 3.4. Método 2: bare metal — instalação direta no disco . . . . .            | 26        |
| 3.5. Método 3: live USB com persistência . . . . .                          | 26        |
| 3.6. Método 4: WSL2 no Windows . . . . .                                    | 27        |
| 3.7. Método 5: contêiner . . . . .                                          | 28        |
| 3.8. Depois de instalar: os primeiros cinco minutos . . . . .               | 28        |
| 3.9. No Kali . . . . .                                                      | 28        |
| 3.10. Laboratório . . . . .                                                 | 29        |
| 3.11. Resumo . . . . .                                                      | 30        |

|                                                                                          |           |
|------------------------------------------------------------------------------------------|-----------|
| <b>4. Primeiro contato: sessão, ambiente Xfce e organização do sistema</b>               | <b>33</b> |
| 4.1. Do boot ao prompt: quem entrega a tela de login . . . . .                           | 33        |
| 4.2. O que é uma sessão . . . . .                                                        | 34        |
| 4.3. As peças do Xfce . . . . .                                                          | 35        |
| 4.4. O menu Kali: treze categorias e uma metodologia . . . . .                           | 35        |
| 4.5. O terminal e o prompt . . . . .                                                     | 36        |
| 4.6. Organização do sistema: onde as coisas estão . . . . .                              | 37        |
| 4.7. Os ajustes do primeiro dia . . . . .                                                | 37        |
| 4.8. No Kali . . . . .                                                                   | 38        |
| 4.9. Laboratório . . . . .                                                               | 39        |
| 4.10. Resumo . . . . .                                                                   | 40        |
| <b>5. A estrutura de diretórios do Linux (FHS)</b>                                       | <b>43</b> |
| 5.1. Uma árvore só, sem letras de unidade . . . . .                                      | 43        |
| 5.2. A árvore completa . . . . .                                                         | 43        |
| 5.3. A lógica: duas perguntas organizam tudo . . . . .                                   | 47        |
| 5.4. Onde colocar o que você instalou . . . . .                                          | 48        |
| 5.5. No Kali . . . . .                                                                   | 48        |
| 5.6. Laboratório . . . . .                                                               | 49        |
| 5.7. Resumo . . . . .                                                                    | 51        |
| <b>6. Anatomia de um comando: shell, argumentos, opções e sintaxe</b>                    | <b>53</b> |
| 6.1. O que acontece entre o Enter e a saída . . . . .                                    | 53        |
| 6.2. Anatomia da linha . . . . .                                                         | 54        |
| 6.3. Onde o shell procura o comando . . . . .                                            | 54        |
| 6.4. Opções: as convenções que existem e as que não existem . . . . .                    | 55        |
| 6.5. O código de saída . . . . .                                                         | 56        |
| 6.6. Quatro armadilhas que valem conhecer agora . . . . .                                | 57        |
| 6.7. O shell é um programa como outro qualquer . . . . .                                 | 58        |
| 6.8. No Kali . . . . .                                                                   | 58        |
| 6.9. Laboratório . . . . .                                                               | 59        |
| 6.10. Resumo . . . . .                                                                   | 61        |
| <b>7. Obtendo ajuda: man, info, tldr e a documentação do Kali</b>                        | <b>63</b> |
| 7.1. O mapa das fontes . . . . .                                                         | 63        |
| 7.2. man: a referência canônica . . . . .                                                | 63        |
| 7.3. --help: rápido e nem sempre suficiente . . . . .                                    | 65        |
| 7.4. help: os comandos que não têm arquivo . . . . .                                     | 66        |
| 7.5. info: a documentação estendida do GNU . . . . .                                     | 66        |
| 7.6. /usr/share/doc: o que a distribuição decidiu . . . . .                              | 67        |
| 7.7. tldr: exemplos em vez de referência . . . . .                                       | 67        |
| 7.8. Um último recurso: o <code>whereis</code> e a leitura do próprio programa . . . . . | 67        |
| 7.9. Sobre confiar na resposta . . . . .                                                 | 68        |
| 7.10. No Kali . . . . .                                                                  | 68        |
| 7.11. Laboratório . . . . .                                                              | 69        |
| 7.12. Resumo . . . . .                                                                   | 71        |
| <b>II. Volume 2 — O Terminal e o Shell</b>                                               | <b>73</b> |
| <b>8. Emulador de terminal e shell: bash, zsh e o padrão do Kali</b>                     | <b>75</b> |
| 8.1. A pilha: da tecla ao programa . . . . .                                             | 75        |

|                                                                   |            |
|-------------------------------------------------------------------|------------|
| 8.2. O emulador de terminal . . . . .                             | 76         |
| 8.3. O shell . . . . .                                            | 78         |
| 8.4. Arquivos de inicialização . . . . .                          | 79         |
| 8.5. No Kali . . . . .                                            | 80         |
| 8.6. Laboratório . . . . .                                        | 81         |
| 8.7. Resumo . . . . .                                             | 83         |
| <b>9. Navegação: pwd, cd, ls e caminhos absolutos e relativos</b> | <b>85</b>  |
| 9.1. O diretório de trabalho é um atributo do processo . . . . .  | 85         |
| 9.2. Caminhos absolutos e relativos . . . . .                     | 86         |
| 9.3. cd: mais do que trocar de pasta . . . . .                    | 87         |
| 9.4. ls: ler a saída, não só listar . . . . .                     | 88         |
| 9.5. Links simbólicos e o caminho “de verdade” . . . . .          | 89         |
| 9.6. Onde o script está: o caminho que mais quebra . . . . .      | 90         |
| 9.7. No Kali . . . . .                                            | 90         |
| 9.8. Laboratório . . . . .                                        | 91         |
| 9.9. Resumo . . . . .                                             | 94         |
| <b>10. Manipulando arquivos e diretórios: cp, mv, rm, mkdir</b>   | <b>95</b>  |
| 10.1. Nomes e conteúdo são coisas separadas . . . . .             | 95         |
| 10.2. mkdir: criando diretórios . . . . .                         | 96         |
| 10.3. touch: criar e carimbar . . . . .                           | 96         |
| 10.4. cp: copiar com intenção . . . . .                           | 97         |
| 10.5. mv: mover e renomear são a mesma coisa . . . . .            | 98         |
| 10.6. rm: o comando sem volta . . . . .                           | 99         |
| 10.7. Copiar árvores grandes: quando cp não basta . . . . .       | 100        |
| 10.8. No Kali . . . . .                                           | 101        |
| 10.9. Laboratório . . . . .                                       | 101        |
| 10.10. Resumo . . . . .                                           | 105        |
| <b>11. Visualizando conteúdo: cat, less, head, tail e more</b>    | <b>107</b> |
| 11.1. Escolher a ferramenta antes de abrir o arquivo . . . . .    | 107        |
| 11.2. cat: concatenar, não visualizar . . . . .                   | 108        |
| 11.3. less: o visualizador que se aprende uma vez . . . . .       | 110        |
| 11.4. head e tail: as pontas do arquivo . . . . .                 | 111        |
| 11.5. Arquivos comprimidos e outros visualizadores . . . . .      | 112        |
| 11.6. No Kali . . . . .                                           | 113        |
| 11.7. Laboratório . . . . .                                       | 113        |
| 11.8. Resumo . . . . .                                            | 116        |
| <b>12. Redirecionamento e pipes: stdin, stdout e stderr</b>       | <b>117</b> |
| 12.1. Os três descritores padrão . . . . .                        | 117        |
| 12.2. Redirecionando a saída . . . . .                            | 118        |
| 12.3. Redirecionando a entrada . . . . .                          | 120        |
| 12.4. Pipes: a ideia central do Unix . . . . .                    | 121        |
| 12.5. tee: gravar e continuar . . . . .                           | 122        |
| 12.6. Substituição de processo . . . . .                          | 122        |
| 12.7. Buffering: por que a saída parece travada . . . . .         | 123        |
| 12.8. Descritores personalizados . . . . .                        | 123        |
| 12.9. No Kali . . . . .                                           | 123        |
| 12.10. Laboratório . . . . .                                      | 124        |
| 12.11. Resumo . . . . .                                           | 127        |

|                                                                                                                     |            |
|---------------------------------------------------------------------------------------------------------------------|------------|
| <b>13. Curingas, globbing e expansão de chaves</b>                                                                  | <b>129</b> |
| 13.1. A ordem das expansões                                                                                         | 129        |
| 13.2. Globbing: os curingas do shell                                                                                | 130        |
| 13.3. Globbing recursivo                                                                                            | 131        |
| 13.4. Expansão de chaves                                                                                            | 132        |
| 13.5. Aspas desligam tudo                                                                                           | 133        |
| 13.6. Onde o globbing machuca                                                                                       | 133        |
| 13.7. No Kali                                                                                                       | 134        |
| 13.8. Laboratório                                                                                                   | 135        |
| 13.9. Resumo                                                                                                        | 138        |
| <b>14. Histórico, atalhos de teclado e produtividade no terminal</b>                                                | <b>139</b> |
| 14.1. O editor de linha                                                                                             | 139        |
| 14.2. O histórico                                                                                                   | 141        |
| 14.3. Autocompletar                                                                                                 | 144        |
| 14.4. Produtividade além dos atalhos                                                                                | 144        |
| 14.5. No Kali                                                                                                       | 145        |
| 14.6. Laboratório                                                                                                   | 146        |
| 14.7. Resumo                                                                                                        | 149        |
| <b>III. Volume 3 — Arquivos e o Sistema de Arquivos</b>                                                             | <b>151</b> |
| <b>15. Tipos de arquivo, inodes e a árvore de diretórios</b>                                                        | <b>153</b> |
| 15.1. “Tudo é um arquivo” — e o que isso quer dizer                                                                 | 153        |
| 15.2. Os sete tipos de arquivo                                                                                      | 154        |
| 15.3. O inode: onde o arquivo realmente mora                                                                        | 155        |
| 15.4. <code>stat</code> : o inode em texto legível                                                                  | 156        |
| 15.5. Diretórios são arquivos com um formato especial                                                               | 157        |
| 15.6. Quando os inodes acabam                                                                                       | 158        |
| 15.7. <code>file</code> : descobrir o que é, ignorando o nome                                                       | 158        |
| 15.8. No Kali                                                                                                       | 159        |
| 15.9. Laboratório                                                                                                   | 160        |
| 15.10. Resumo                                                                                                       | 162        |
| <b>16. Links simbólicos e hard links</b>                                                                            | <b>163</b> |
| 16.1. Hard link: outro nome para o mesmo inode                                                                      | 163        |
| 16.2. Link simbólico: um arquivo que contém um caminho                                                              | 164        |
| 16.3. <code>ln</code> : a ordem dos argumentos e outras armadilhas                                                  | 165        |
| 16.4. Como os comandos tratam os links                                                                              | 166        |
| 16.5. Resolvendo caminhos: <code>readlink</code> , <code>realpath</code> e <code>namei</code>                       | 167        |
| 16.6. Links quebrados e laços                                                                                       | 167        |
| 16.7. O caso concreto do Kali: <code>update-alternatives</code>                                                     | 167        |
| 16.8. Segurança: por que links merecem desconfiança                                                                 | 168        |
| 16.9. No Kali                                                                                                       | 169        |
| 16.10. Laboratório                                                                                                  | 169        |
| 16.11. Resumo                                                                                                       | 172        |
| <b>17. Localizando arquivos: <code>find</code>, <code>locate</code>, <code>which</code> e <code>type</code></b>     | <b>175</b> |
| 17.1. <code>find</code> : a varredura completa                                                                      | 175        |
| 17.2. <code>locate</code> : a consulta ao índice                                                                    | 178        |
| 17.3. Onde está um comando: <code>type</code> , <code>command -v</code> , <code>which</code> , <code>whereis</code> | 179        |

|                                                                            |            |
|----------------------------------------------------------------------------|------------|
| 17.4. No Kali . . . . .                                                    | 180        |
| 17.5. Laboratório . . . . .                                                | 181        |
| 17.6. Resumo . . . . .                                                     | 184        |
| <b>18. Compactação e arquivamento: tar, gzip, xz e zip</b>                 | <b>185</b> |
| 18.1. Arquivar não é comprimir . . . . .                                   | 185        |
| 18.2. tar: o comando e suas letras . . . . .                               | 186        |
| 18.3. Os compressores, e quando usar cada um . . . . .                     | 187        |
| 18.4. zip e unzip . . . . .                                                | 188        |
| 18.5. Compactação em fluxo: sem arquivo temporário . . . . .               | 189        |
| 18.6. Verificar antes de confiar . . . . .                                 | 189        |
| 18.7. No Kali . . . . .                                                    | 190        |
| 18.8. Laboratório . . . . .                                                | 191        |
| 18.9. Resumo . . . . .                                                     | 193        |
| <b>19. Montagem de dispositivos e o comando mount</b>                      | <b>195</b> |
| 19.1. Uma árvore só, sem letras de unidade . . . . .                       | 195        |
| 19.2. Ver o que está montado . . . . .                                     | 196        |
| 19.3. Montar e desmontar . . . . .                                         | 196        |
| 19.4. /etc/fstab: montagens permanentes . . . . .                          | 198        |
| 19.5. Montagem automática no ambiente gráfico . . . . .                    | 199        |
| 19.6. Arquivos-imagem: montar sem hardware . . . . .                       | 199        |
| 19.7. Bind mount e tmpfs . . . . .                                         | 199        |
| 19.8. No Kali . . . . .                                                    | 200        |
| 19.9. Laboratório . . . . .                                                | 201        |
| 19.10. Resumo . . . . .                                                    | 204        |
| <b>20. Permissões de arquivo: leitura, escrita e execução</b>              | <b>205</b> |
| 20.1. Ler a saída do <code>ls -l</code> . . . . .                          | 205        |
| 20.2. A regra que quase ninguém aprende: só uma classe se aplica . . . . . | 206        |
| 20.3. chmod: as duas notações . . . . .                                    | 206        |
| 20.4. Permissões em diretório: o que realmente significam . . . . .        | 207        |
| 20.5. chown e chgrp: mudar dono e grupo . . . . .                          | 208        |
| 20.6. umask: as permissões que os arquivos novos recebem . . . . .         | 209        |
| 20.7. Testar permissão sem tentar a operação . . . . .                     | 210        |
| 20.8. No Kali . . . . .                                                    | 210        |
| 20.9. Laboratório . . . . .                                                | 211        |
| 20.10. Resumo . . . . .                                                    | 214        |
| <b>IV. Volume 4 — Usuários, Grupos e Permissões</b>                        | <b>217</b> |
| <b>21. Usuários, grupos e o arquivo /etc/passwd</b>                        | <b>219</b> |
| <b>22. O root, sudo e a elevação de privilégios</b>                        | <b>221</b> |
| <b>23. Permissões especiais: SUID, SGID e sticky bit</b>                   | <b>223</b> |
| <b>24. ACLs e atributos estendidos de arquivo</b>                          | <b>225</b> |
| <b>25. Gerenciamento de usuários, senhas e o PAM</b>                       | <b>227</b> |
| <b>26. Trabalhando como root no Kali com segurança</b>                     | <b>229</b> |

|                                                                  |            |
|------------------------------------------------------------------|------------|
| <b>V. Volume 5 — Processos e Controle de Tarefas</b>             | <b>231</b> |
| 27.O que é um processo: PID, estados e a árvore de processos     | 233        |
| 28.Monitoramento: ps, top e htop                                 | 235        |
| 29.Sinais e controle: kill, killall, nice e prioridades          | 237        |
| 30.Jobs, primeiro e segundo plano, nohup e disown                | 239        |
| 31.Multiplexação de terminal: tmux e screen                      | 241        |
| 32.Recurso do sistema: memória, CPU, uptime e limites            | 243        |
| <b>VI. Volume 6 — Gerenciamento de Pacotes</b>                   | <b>245</b> |
| 33.APT e o modelo Debian: repositórios e o Kali rolling          | 247        |
| 34.Instalando, removendo e atualizando pacotes com apt           | 249        |
| 35.dpkg, arquivos .deb e resolução de dependências               | 251        |
| 36.Repositórios do Kali, metapacotes e kali-tweaks               | 253        |
| 37.Compilando a partir do código-fonte                           | 255        |
| 38.Alternativas: pipx, snap, flatpak e Appliance                 | 257        |
| <b>VII. Volume 7 — Boot, systemd e Serviços</b>                  | <b>259</b> |
| 39.O processo de inicialização: UEFI, GRUB e o kernel            | 261        |
| 40.systemd: units, targets e o modelo de serviços                | 263        |
| 41.Gerenciando serviços com systemctl                            | 265        |
| 42.Logs com journald e o syslog tradicional                      | 267        |
| 43.Agendamento: cron, at e timers do systemd                     | 269        |
| 44.Targets, modo de recuperação e resolução de problemas de boot | 271        |
| <b>VIII. Volume 8 — Armazenamento e Sistemas de Arquivos</b>     | <b>273</b> |
| 45.Discos, partições e a nomenclatura de dispositivos            | 275        |
| 46.Particionamento com fdisk, parted e gdisk                     | 277        |
| 47.Sistemas de arquivos: ext4, Btrfs, XFS e formatação           | 279        |
| 48.LVM: volumes lógicos flexíveis                                | 281        |
| 49.RAID por software com mdadm                                   | 283        |



|                                                               |            |
|---------------------------------------------------------------|------------|
| 50. Criptografia de disco com LUKS e a persistência no Kali   | 285        |
| <b>IX. Volume 9 — Fundamentos de Shell Scripting</b>          | <b>287</b> |
| 51. Do comando ao script: o primeiro script Bash              | 289        |
| 52. Variáveis, o ambiente e o escopo de exportação            | 291        |
| 53. Aspas, expansões e substituição de comandos               | 293        |
| 54. Entrada e saída: read, echo e printf                      | 295        |
| 55. Condicionais: test, colchetes duplos e if                 | 297        |
| 56. Códigos de saída e o encadeamento de comandos             | 299        |
| 57. Boas práticas: shebang, permissões e portabilidade        | 301        |
| <b>X. Volume 10 — Scripting Intermediário</b>                 | <b>303</b> |
| 58. Laços: for, while e until                                 | 305        |
| 59. Funções, retorno e escopo de variáveis                    | 307        |
| 60. Arrays indexados e associativos                           | 309        |
| 61. Parâmetros posicionais e getopt                           | 311        |
| 62. Expansão de parâmetros avançada                           | 313        |
| 63. Aritmética e manipulação de números no shell              | 315        |
| <b>XI. Volume 11 — Processamento de Texto</b>                 | <b>317</b> |
| 64. Expressões regulares: fundamentos e os diferentes sabores | 319        |
| 65. grep e a busca de padrões                                 | 321        |
| 66. sed: o editor de fluxo                                    | 323        |
| 67. awk: processamento por campos e registros                 | 325        |
| 68. cut, sort, uniq, tr e a caixa de ferramentas de texto     | 327        |
| 69. Dados estruturados na linha de comando: jq, JSON e CSV    | 329        |
| 70. Juntando tudo: pipelines de processamento de dados        | 331        |
| <b>XII. Volume 12 — Scripting Avançado e Robusto</b>          | <b>333</b> |
| 71. Tratamento de erros: set -euo pipefail e traps            | 335        |

|                                                                        |                |
|------------------------------------------------------------------------|----------------|
| <b>72. Depuração de scripts: set -x e ShellCheck</b>                   | <b>337</b>     |
| <b>73. Manipulando arquivos, descritores e processos em scripts</b>    | <b>339</b>     |
| <b>74. Subshells, agrupamento e here-documents</b>                     | <b>341</b>     |
| <b>75. Sinais, processos filhos e execução em paralelo</b>             | <b>343</b>     |
| <b>76. Interação com o usuário: menus, cores e caixas de diálogo</b>   | <b>345</b>     |
| <b>77. Estruturando scripts grandes e bibliotecas de funções</b>       | <b>347</b>     |
| <br><b>XIII Volume 13 — Redes no Linux</b>                             | <br><b>349</b> |
| <b>78. Fundamentos: interfaces, endereçamento IP e a pilha de rede</b> | <b>351</b>     |
| <b>79. Configuração de rede: ip, NetworkManager e arquivos</b>         | <b>353</b>     |
| <b>80. Diagnóstico: ping, traceroute, ss, dig e mtr</b>                | <b>355</b>     |
| <b>81. Acesso e transferência remota: SSH, scp e rsync</b>             | <b>357</b>     |
| <b>82. HTTP na linha de comando: curl, wget e netcat</b>               | <b>359</b>     |
| <b>83. Firewall no Linux: nftables e iptables</b>                      | <b>361</b>     |
| <b>84. Captura e análise de tráfego: tcpdump e tshark</b>              | <b>363</b>     |
| <br><b>XIV Volume 14 — Segurança e o Ambiente Kali</b>                 | <br><b>365</b> |
| <b>85. Hardening do Linux: superfície de ataque e princípios</b>       | <b>367</b>     |
| <b>86. Gerenciamento de segredos: chaves SSH e GPG</b>                 | <b>369</b>     |
| <b>87. Capabilities, isolamento e menor privilégio</b>                 | <b>371</b>     |
| <b>88. Confinamento: AppArmor, SELinux e sandboxing</b>                | <b>373</b>     |
| <b>89. Auditoria, integridade de arquivos e detecção de alterações</b> | <b>375</b>     |
| <b>90. O ecossistema de ferramentas do Kali: categorias e fluxos</b>   | <b>377</b>     |
| <b>91. Anonimato e privacidade: proxychains, Tor e VPN</b>             | <b>379</b>     |
| <br><b>XV. Volume 15 — Automação e Produtividade</b>                   | <br><b>381</b> |
| <b>92. Personalizando o shell: .bashrc, .zshrc, aliases e funções</b>  | <b>383</b>     |
| <b>93. Zsh, plugins e o prompt do Kali</b>                             | <b>385</b>     |
| <b>94. Automatizando tarefas com cron e scripts agendados</b>          | <b>387</b>     |
| <b>95. Controle de versão com Git na linha de comando</b>              | <b>389</b>     |

|                                                                               |            |
|-------------------------------------------------------------------------------|------------|
| <b>96. Editores no terminal: vim e nano</b>                                   | <b>391</b> |
| <b>97. Dotfiles e ambientes reproduzíveis</b>                                 | <b>393</b> |
| <br>                                                                          |            |
| <b>XVI Volume 16 — Kernel, Contêineres e Virtualização</b>                    | <b>395</b> |
| <b>98. O kernel Linux: módulos, /proc e /sys</b>                              | <b>397</b> |
| <b>99. Contêineres por dentro: namespaces, cgroups e Docker</b>               | <b>399</b> |
| <b>100. Virtualização: KVM, QEMU e o laboratório de estudos</b>               | <b>401</b> |
| <b>101. Compilação, toolchains e o ecossistema de desenvolvimento</b>         | <b>403</b> |
| <b>102. Observabilidade e desempenho: strace, ltrace e perf</b>               | <b>405</b> |
| <b>103. Para onde ir depois: certificações, comunidade e prática contínua</b> | <b>407</b> |
| <b>Referências</b>                                                            | <b>409</b> |



# Apresentação

Este é um manual de **Linux e shell scripting** escrito em português do Brasil, pensado para levar o leitor do primeiro comando no terminal até a administração avançada do sistema, a automação com scripts robustos e os fundamentos do kernel, contêineres e virtualização.

A distribuição de referência é o **Kali Linux**. Todos os comandos, caminhos de pacote e convenções foram adaptados ao Kali — que é, por baixo, um Debian *rolling release*. Onde o Kali difere de um Debian ou Ubuntu comum (o repositório rolling, o usuário padrão, os metapacotes de ferramentas, o `kali-tweaks`, a persistência em live USB), o texto aponta a diferença explicitamente. Quem usa outra distribuição derivada de Debian encontrará quase tudo aplicável; onde a família RHEL (`dnf`, `firewalld`, SELinux) diverge de forma relevante, há uma nota.

## Para quem é este manual

Para quem quer aprender Linux a sério: o estudante que acabou de instalar o Kali numa máquina virtual, o profissional de infraestrutura que administra servidores e redes, e quem vem da área de segurança e precisa dominar o sistema operacional que está sob todas as ferramentas. Não se pressupõe conhecimento prévio de Linux — o Volume 1 começa do zero — mas o manual não para no básico: os últimos volumes tratam de LVM, `nftables`, namespaces e `cgroups`.

## Como o manual está organizado

O conteúdo avança em cinco fases, do concreto ao abstrato:

- **Fase 1 — Fundamentos do Linux.** O terminal, o sistema de arquivos, usuários e permissões. A base sobre a qual todo o resto se apoia.
- **Fase 2 — Administração do Sistema.** Processos, pacotes, `systemd`, serviços e armazenamento.
- **Fase 3 — Shell Scripting.** Do primeiro script Bash ao processamento de texto com `sed` e `awk` e ao scripting robusto com tratamento de erros e depuração.
- **Fase 4 — Redes, Segurança e Automação.** Redes no Linux, hardening, o ecossistema de ferramentas do Kali e a automação do dia a dia.
- **Fase 5 — Fronteira e Aprofundamento.** Kernel, contêineres, virtualização e para onde seguir depois.

## Como cada capítulo funciona

Cada capítulo abre por um problema concreto, desenvolve o assunto com exemplos executáveis e comentados, e fecha com duas seções fixas: **No Kali**, que destaca o que muda especificamente nessa distribuição, e **Laboratório**, um roteiro prático para reproduzir o conteúdo no seu próprio ambiente. Todo comando mostrado é para ser digitado — a proposta é aprender fazendo.

## *Apresentação*

**Aviso.** Os capítulos de segurança e das ferramentas do Kali têm caráter educacional e defensivo. Técnicas e ferramentas ofensivas aparecem para que o leitor entenda, detecte e mitigue — e devem ser usadas apenas em ambientes próprios ou com autorização explícita. Usar essas ferramentas contra sistemas de terceiros sem permissão é crime no Brasil (Lei 12.737/2012).

Bom proveito — e mantenha um terminal aberto ao lado enquanto lê.

**Parte I.**

**Volume 1 — Primeiros Passos: Linux e  
Kali**





# 1. O que é Linux: kernel, distribuições e o modelo de software livre

Três e vinte da manhã. O monitoramento avisa que o servidor que concentra os coletores de tráfego de doze torres parou de responder. Você entra por SSH pelo link de gerência e o prompt abre — a máquina está viva, o que morreu foi um serviço. O técnico que montou aquele ponto de presença há quatro anos não trabalha mais na empresa, e a única anotação no inventário diz “servidor Linux”. Antes de tentar qualquer conserto, você digita dois comandos:

```
uname -a
cat /etc/os-release
```

O primeiro responde qual **kernel** está rodando. O segundo responde qual **distribuição** foi instalada. São perguntas diferentes, e tratá-las como se fossem a mesma é a origem de boa parte dos mal-entendidos de quem está começando. Do kernel dependem os drivers, o suporte ao hardware e o comportamento de rede; da distribuição dependem o gerenciador de pacotes, o caminho dos arquivos de configuração, o modelo de atualização e quanto tempo aquela instalação ainda vai receber correções de segurança. Este capítulo desmonta a palavra “Linux” nas partes que ela esconde.

## 1.1. O que a máquina responde no primeiro minuto

Numa instalação Debian de servidor, os dois comandos acima devolvem algo assim:

```
$ uname -a
Linux pop-torre-03 6.12.13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.13-1
↳ (2025-02-08) x86_64 GNU/Linux
```

```
$ cat /etc/os-release
PRETTY_NAME="Debian GNU/Linux 12 (bookworm)"
NAME="Debian GNU/Linux"
VERSION_ID="12"
VERSION="12 (bookworm)"
ID=debian
HOME_URL="https://www.debian.org/"
```

Repare que os números não conversam: o kernel é 6.12.13, a distribuição é a versão 12. Não há relação alguma entre eles. A mesma máquina, reinstalada com Kali, responderia:

## 1. O que é Linux: kernel, distribuições e o modelo de software livre

```
$ uname -a
Linux kali 6.12.25-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.25-1kali1
↳ (2025-04-30) x86_64 GNU/Linux

$ cat /etc/os-release
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
VERSION="2025.2"
ID=kali
ID_LIKE=debian
```

Kernel muito parecido, sistema completamente diferente. E o campo `ID_LIKE=debian` já entrega o parentesco: quase tudo que vale para Debian vale para Kali. Guardar essa distinção desde o começo economiza horas de busca em fórum — a resposta que resolve seu problema pode estar escrita para “Linux”, mas depender de um detalhe que só existe no Ubuntu.

### 1.2. O kernel: o Linux no sentido estrito

**Linux, tecnicamente, é só o kernel.** É um programa — grande, mas um programa — que fica entre o hardware e todo o resto do software. Quando o computador liga, o firmware carrega o kernel na memória e entrega a ele o controle da máquina; a partir daí é o kernel que decide qual processo usa a CPU e por quanto tempo, quais páginas de memória cada programa enxerga, como os bytes chegam ao disco, como os pacotes saem pela placa de rede e como um dispositivo USB recém-conectado passa a existir para o sistema (Love, 2010).

Nenhum programa comum fala com o hardware diretamente. O `ls` não sabe ler um disco; ele pede ao kernel. O `nmap` não sabe montar um pacote na placa de rede; ele pede ao kernel. Essa fronteira tem nome: **chamada de sistema** (*system call*). O programa roda no que se chama espaço de usuário, com privilégios limitados; o kernel roda em espaço de kernel, com acesso total; e a única porta entre os dois é o conjunto de algumas centenas de chamadas de sistema. Figura 1.1 mostra o arranjo.

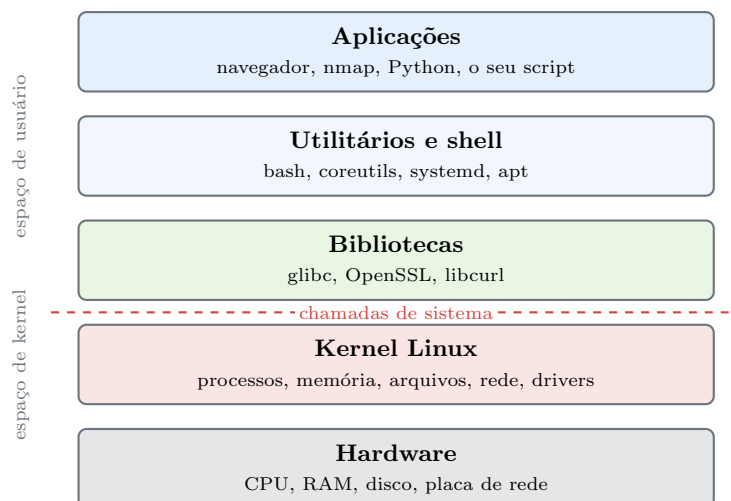


Figura 1.1.: As camadas de um sistema Linux. Tudo acima da linha tracejada precisa pedir ao kernel para tocar no hardware.

Dá para ver essa fronteira funcionando. O comando `strace` intercepta e imprime cada chamada de sistema que um processo faz:

```
strace -c ls /
```

A opção `-c` resume por chamada em vez de listar uma a uma. Um `ls` de um diretório pequeno dispara facilmente umas cem chamadas: `execve` para iniciar o programa, `openat` e `read` para carregar as bibliotecas, `statx` para descobrir o tipo de cada entrada, `getdents64` para ler o diretório, `write` para imprimir na tela. Sem o kernel, o `ls` é um punhado de instruções incapaz de descobrir sequer o próprio nome.

O kernel Linux é **monolítico e modular**. Monolítico porque drivers e subsistemas rodam dentro do mesmo espaço privilegiado, e não como processos separados — decisão de projeto que rendeu, em 1992, um debate público entre Linus Torvalds e Andrew Tanenbaum, autor do Minix, defensor de microkernels. Modular porque partes desse código podem ser carregadas e descarregadas com o sistema em pé:

```
lsmod | head  
modinfo ath9k_htc | head -5
```

O primeiro lista os módulos carregados; o segundo descreve um módulo específico — no exemplo, o driver de um adaptador Wi-Fi USB muito usado em laboratório. É por isso que você pluga um adaptador novo e ele funciona sem reiniciar: o kernel carrega o módulo correspondente sob demanda.

A numeração é simples de ler. Em `6.12.25-amd64`, o `6.12` é a versão de origem (*mainline*), o `25` é o número da correção, e `amd64` é a arquitetura. Algumas versões são marcadas como **LTS** e recebem correções por anos — é o kernel que distribuições de servidor adotam. As versões novas trazem suporte a hardware recente, e é por isso que uma placa de rede lançada este ano pode simplesmente não existir para um servidor com kernel de cinco anos atrás. Esse é o argumento prático mais forte a favor de kernels novos, e o gerenciamento de módulos é tema do Volume 16.

## 1.3. De onde veio: Unix, GNU e 1991

Em 1969, no Bell Labs da AT&T, Ken Thompson e Dennis Ritchie escreveram o Unix. A virada aconteceu poucos anos depois, quando o sistema foi reescrito em C: um sistema operacional que podia ser recompilado para outra máquina em vez de reescrito em assembly. Junto com o código veio um conjunto de ideias que o Linux herdou por inteiro — tudo é arquivo, programas pequenos que fazem uma coisa bem, saída de um encadeada na entrada do outro, configuração em texto simples (Kernighan; Pike, 1984). Universidades adotaram o Unix, a Universidade da Califórnia em Berkeley criou sua própria linhagem (a família BSD), e nos anos 1980 o código virou produto comercial fechado, com licenças caras e disputas judiciais sobre quem podia distribuir o quê.

Foi contra esse fechamento que Richard Stallman lançou, em 1983, o **Projeto GNU** — sigla recursiva para *GNU's Not Unix*. A meta era um sistema completo, compatível com Unix, cujo código qualquer pessoa pudesse ler, modificar e redistribuir. Em 1991 o projeto tinha praticamente tudo: o compilador `gcc`, o editor Emacs, o `bash`, os utilitários que hoje formam o pacote `coreutils`, o `make`, o depurador. Faltava justamente a peça central — o kernel, chamado Hurd, que se arrastava.

## 1. O que é Linux: kernel, distribuições e o modelo de software livre

Nesse mesmo ano, um estudante finlandês de 21 anos chamado Linus Torvalds começou a escrever, por conta própria, um kernel para o seu 386. Em agosto de 1991 ele anunciou o projeto numa lista de discussão do Minix com a frase que ficou célebre por envelhecer mal: era só um passatempo, não seria grande e profissional como o GNU. Em 1992 ele relicenciou o kernel sob a **GPL versão 2**, e a peça que faltava ao GNU encontrou o conjunto de ferramentas que faltava ao kernel. O resultado é o que a saída do `uname -a` chama, literalmente, de **GNU/Linux** — e é a razão de existir a disputa sobre esse nome: quem escreve “GNU/Linux” está lembrando que o kernel sozinho não liga uma máquina útil. Neste manual usamos “Linux” no sentido corrente, de sistema completo, e “kernel” quando a distinção importa.

### 1.4. O que é, então, uma distribuição

Um kernel não abre um terminal. Para ter um sistema operacional utilizável é preciso juntar o kernel a uma biblioteca C, a um programa de inicialização, a um shell, a centenas de utilitários, a um gerenciador de pacotes, a um repositório onde esses pacotes vivem, a um instalador e a um conjunto de decisões padrão. Esse trabalho de integração — escolher versões, aplicar correções, testar combinações, assinar os pacotes e sustentar tudo isso ao longo do tempo — é o que uma **distribuição** faz. Figura 1.2 mostra as peças.

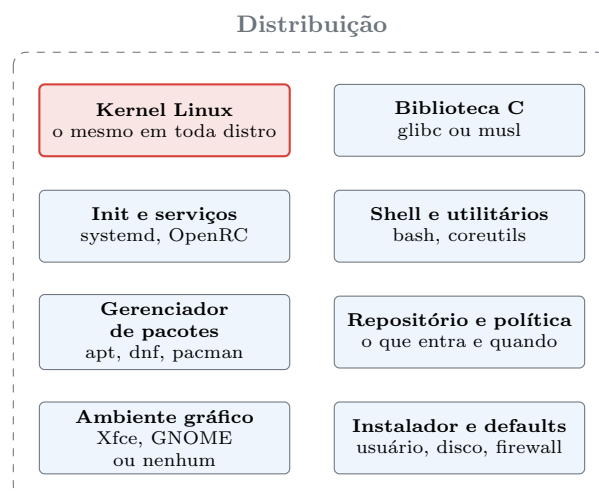


Figura 1.2.: Uma distribuição é o kernel mais o trabalho de integração que o torna utilizável.

Cada uma dessas caixas é uma decisão editorial, e distribuições diferentes decidem diferente. As escolhas que mais mudam o dia a dia são quatro. A primeira é o **modelo de lançamento**: versão fixa, em que os pacotes congelam e só recebem correções de segurança por alguns anos (Debian estável, Ubuntu LTS, RHEL), ou *rolling release*, em que não existe “versão” e o sistema recebe versões novas continuamente (Arch, Kali). A segunda é o **formato de pacote e o gerenciador**: `.deb` com `apt` na família Debian, `.rpm` com `dnf` na família Red Hat, e por aí vai. A terceira é o **conjunto padrão**: uma imagem mínima de servidor, um desktop completo, ou — no caso do Kali — algumas centenas de ferramentas de segurança já instaladas. A quarta é a **política de segurança e suporte**: quem corrige, com que rapidez, e por quanto tempo.

Para ter uma ideia do tamanho do trabalho, conte os pacotes de uma instalação comum:

```
dpkg -l | grep -c '^ii'
```

Uma instalação enxuta de Debian passa dos 500 pacotes; um Kali com o metapacote padrão passa fácil dos 3.000. Nenhum deles foi escrito pela distribuição — todos foram empacotados, testados em conjunto e assinados por ela.

## 1.5. Famílias: o mapa das distribuições

Existem centenas de distribuições, mas quase todas descendem de meia dúzia de troncos, e é o tronco que determina os comandos que você vai digitar. Figura 1.3 resume as linhagens que importam na prática.

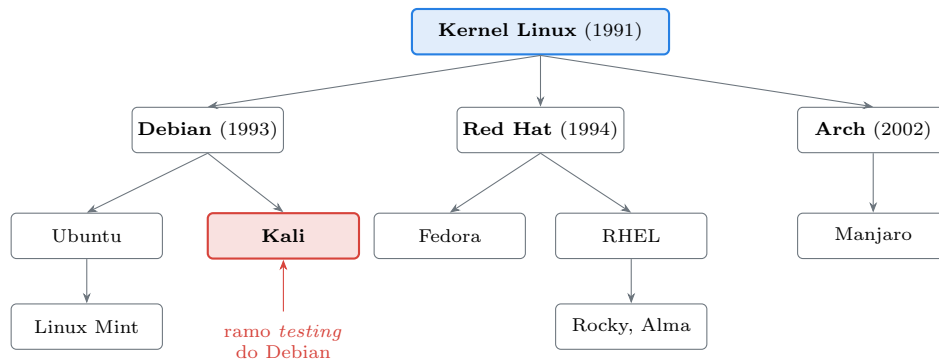


Figura 1.3.: As linhagens principais. O tronco define o gerenciador de pacotes e boa parte das convenções.

A **família Debian** usa pacotes `.deb` e o `apt`. Debian é a base; Ubuntu deriva dela e adiciona um ciclo de lançamento previsível; Kali deriva do ramo de testes do Debian; Linux Mint deriva do Ubuntu; o Raspberry Pi OS também é Debian. É a família com maior presença em servidores de pequeno e médio porte no Brasil, e é a família deste manual.

A **família Red Hat** usa pacotes `.rpm` e o `dnf`. Fedora é o laboratório onde as novidades aparecem primeiro; o RHEL é o produto comercial estabilizado a partir dele; Rocky Linux e AlmaLinux são reconstruções gratuitas e compatíveis do RHEL. É a família dominante em datacenter corporativo, e traz consigo `firewalld` e SELinux, que resolvem os mesmos problemas de outro jeito.

A **família Arch** é *rolling*, minimalista e monta o sistema a partir do que o usuário escolhe instalar. Fora desses três troncos, valem menção o openSUSE, o Alpine — minúsculo, com biblioteca C `musl` no lugar da `glibc`, quase padrão em contêineres — e o Gentoo, que compila tudo a partir do fonte.

A consequência prática é direta: **os conceitos são os mesmos, os comandos nem sempre**. Processo, permissão, descritor de arquivo, socket, sinal — isso é kernel, e é idêntico em toda parte. Já `apt install` contra `dnf install`, `/etc/network/interfaces` contra as conexões do NetworkManager, `nftables` contra `firewalld`, isso é distribuição. Ao escolher a distribuição de um servidor que vai ficar quatro anos num rack de POP, o critério raramente é gosto: é janela de suporte, previsibilidade de atualização e disponibilidade dos pacotes de que você depende (Nemeth et al., 2017).

## 1.6. O modelo de software livre

Nada disso seria possível sem o arranjo jurídico que sustenta o ecossistema. Software livre, na definição da Free Software Foundation, é o que garante quatro liberdades ao usuário: **liberdade 0**, executar o programa para qualquer finalidade; **liberdade 1**, estudar como ele funciona e adaptá-lo, o que exige acesso ao código-fonte; **liberdade 2**, redistribuir cópias; e **liberdade 3**, distribuir versões modificadas. “Livre” aqui é liberdade, não prego — a ambiguidade existe em inglês (*free*) e por isso surgiu o termo alternativo *open source*, cunhado em 1998 com ênfase mais pragmática, no modelo de desenvolvimento colaborativo, e menos na dimensão ética. Na prática as duas listas de licenças aprovadas quase coincidem.

As licenças se dividem em duas famílias. As **copyleft**, como a GPL, obrigam quem distribui uma versão modificada a distribuí-la sob a mesma licença, com o fonte disponível — a liberdade é passada adiante à força. As **permissivas**, como MIT, BSD e Apache, permitem que o código seja incorporado a produtos fechados. O kernel Linux é GPLv2. É por isso que um fabricante que embarca Linux num roteador ou numa OLT precisa disponibilizar o fonte do kernel modificado que usou — e é por isso que existe, para muito equipamento de rede, uma comunidade capaz de compilar firmware alternativo.

Esse mesmo mecanismo é o que permite a existência das distribuições: o Kali pode pegar o Debian inteiro, modificar, rebatizar e redistribuir porque as licenças autorizam. Cada pacote instalado traz sua licença registrada em disco:

```
head -20 /usr/share/doc/coreutils/copyright
apt show nmap | head -15
```

Vale desfazer um mal-entendido comum: software livre não é software sem dinheiro envolvido. Red Hat, Canonical e SUSE são empresas grandes que vendem suporte, certificação e serviços em cima de código aberto; a OffSec, que mantém o Kali, vende treinamento e certificação; a maior parte dos commits do kernel Linux vem de gente paga para isso, empregada por fabricantes de hardware e provedores de nuvem. O que o modelo elimina não é o pagamento — é o aprisionamento.

## 1.7. O que muda no seu dia a dia

Três consequências concretas. Primeira: quando um serviço se comporta de um jeito que a documentação não explica, o código está a um `apt source` de distância, e ler o fonte é uma opção real de diagnóstico, não um recurso teórico. Segunda: o repositório da distribuição é a sua cadeia de suprimentos de software, com pacotes assinados e verificados a cada `apt update` — instalar `.deb` baixado de qualquer lugar contorna essa verificação, e o assunto volta com força no Volume 6. Terceira: você pode auditar. Num ambiente onde a máquina fica trancada num contêiner metálico no meio de um pasto, saber exatamente o que está rodando e de onde veio cada binário não é preciosismo — é a diferença entre um incidente investigável e um mistério.

## 1.8. No Kali

O Kali Linux é uma distribuição **Debian rolling** mantida pela OffSec, montada a partir do ramo `testing` do Debian e voltada a teste de invasão, forense e auditoria de segurança (Offensive Security, 2025). Alguns pontos em que ele difere de um Debian ou Ubuntu comum e que aparecem já no primeiro contato:

- **Não existe “versão” no sentido do Debian.** O 2025.2 de `/etc/os-release` é um marco de imagem de instalação, não um congelamento: a atualização vem do ramo `kali-rolling`, que se atualiza continuamente. Confira com `cat /etc/apt/sources.list` — a linha aponta para `kali-rolling`, não para um codinome como `bookworm`. Existem ainda os ramos `kali-last-snapshot`, congelado no último lançamento, e `kali-experimental`. Isso significa que `apt update && apt full-upgrade` no Kali é uma operação de rotina, não um evento raro.
- **O usuário padrão é kali, não root.** Até a versão 2020.1 o Kali logava como `root` por padrão; hoje adota o modelo de usuário comum com `sudo`, como qualquer Debian. O tema é aprofundado no Volume 4.
- **O kernel tem patches próprios.** O sufixo `-kali1` em `uname -r` indica um kernel com correções aplicadas pela distribuição, notadamente para injeção de pacotes e modo monitor em adaptadores Wi-Fi — recursos que um kernel Debian puro nem sempre oferece.
- **As ferramentas vêm por metapacotes.** `kali-linux-default` traz o conjunto usual, `kali-linux-large` traz quase tudo, `kali-linux-headless` traz só o que roda sem interface gráfica. Instalar um metapacote arrasta centenas de programas de uma vez. Detalhes no Volume 6.
- **kali-tweaks** é um utilitário próprio da distribuição para ajustar rapidamente ramo de repositório, shell padrão, serviços de rede e configurações de hardening.
- **Kali não é distribuição de servidor.** Ela vem com ferramentas ofensivas instaladas, acompanha um ramo de testes e não tem janela de suporte de longo prazo. Para o servidor de monitoramento da história de abertura, a escolha correta é Debian estável ou Ubuntu LTS. Kali é a máquina de trabalho de quem audita, não a máquina auditada.

**Nota sobre a família RHEL.** Em Fedora, RHEL, Rocky ou Alma, os equivalentes são `cat /etc/redhat-release` no lugar do `os-release`, `dnf` no lugar do `apt`, `rpm -qa` no lugar do `dpkg -l`, `firewalld` no lugar do `nftables` direto e SELinux no lugar do AppArmor. O kernel e os conceitos são os mesmos.

## 1.9. Laboratório

Roteiro para executar numa máquina virtual com Kali. Nenhum passo altera o sistema; todos são de leitura.

### 1. Identifique kernel e distribuição separadamente.

```
uname -r
uname -a
cat /etc/os-release
hostnamectl
```

Esperado: `uname -r` devolve algo como `6.12.25-amd64`; o `os-release` devolve `ID=kali` e `ID_LIKE=debian`. O `hostnamectl` reúne as duas informações em campos separados — Operating System e Kernel. Anote os dois valores; eles vão divergir na próxima atualização, e em ritmos diferentes.

### 2. Confirme que o kernel é modular.

```
lsmod | wc -l
lsmod | head -5
```

### 1. O que é Linux: kernel, distribuições e o modelo de software livre

Esperado: dezenas de módulos carregados. A primeira coluna é o nome, a terceira é a contagem de uso — módulo com contagem zero não está sendo usado por ninguém no momento.

### 3. Observe a fronteira das chamadas de sistema.

```
sudo apt install -y strace
strace -c ls / 2>&1 | tail -20
```

Esperado: uma tabela com as chamadas usadas e quantas vezes cada uma. Procure `openat`, `read`, `write`, `getdents64`, `execve`. A conclusão a tirar: um comando trivial como `ls` não faz nada sozinho — ele passa todo o trabalho real para o kernel.

### 4. Veja que o programa depende de bibliotecas, e não só do kernel.

```
ldd /bin/ls
/lib/x86_64-linux-gnu/libc.so.6 --version | head -2
```

Esperado: a lista de bibliotecas compartilhadas de que o `ls` precisa, incluindo `libc.so.6`, e a versão da glibc. Essa camada pertence à distribuição, não ao kernel — trocar de distribuição pode trocar a glibc; trocar de kernel, não.

### 5. Meça o trabalho de integração da distribuição.

```
dpkg -l | grep -c '^ii'
apt list --installed 2>/dev/null | wc -l
```

Esperado: alguns milhares de pacotes num Kali com metapacote padrão. Cada um foi empacotado, testado em conjunto e assinado por alguém.

### 6. Leia a licença de um programa que você usa.

```
head -25 /usr/share/doc/nmap/copyright
apt show bash 2>/dev/null | head -12
```

Esperado: o arquivo `copyright` traz autoria e licença. Compare a licença do `bash` (GPL) com a de algum utilitário sob MIT ou BSD e observe a diferença de exigência sobre versões modificadas.

### 7. Descubra o ramo de repositório em que a sua máquina está.

```
cat /etc/apt/sources.list
apt policy
```

Esperado: uma linha apontando para `kali-rolling`. Se apontar para `kali-last-snapshot`, sua máquina está congelada no último lançamento e não receberá pacotes novos até você mudar isso.

### 8. A prova final de que kernel e distribuição são coisas distintas. Se houver Docker no ambiente:

```
sudo docker run --rm fedora cat /etc/os-release | head -3
sudo docker run --rm fedora uname -r
uname -r
```



Esperado: o `os-release` de dentro do contêiner diz Fedora; o `uname -r` de dentro do contêiner devolve **exatamente o mesmo kernel** da sua máquina Kali. O contêiner traz uma distribuição inteira — utilitários, `dnf`, bibliotecas — mas não traz kernel: ele usa o do hospedeiro. É a demonstração mais limpa da separação, e o mecanismo por trás dela é assunto do Volume 16.

## 1.10. Resumo

- **Linux, no sentido estrito, é o kernel:** o programa que gerencia CPU, memória, arquivos, rede e dispositivos, e que expõe tudo isso ao restante do software por meio das chamadas de sistema.
- **Distribuição é o kernel mais o trabalho de integração:** biblioteca C, init, shell, utilitários, gerenciador de pacotes, repositório, instalador e uma política de atualização e suporte.
- `uname -a` e `/etc/os-release` respondem perguntas diferentes e seus números não têm relação entre si. Saber ler os dois é o primeiro diagnóstico de qualquer máquina desconhecida.
- **A família da distribuição define os comandos.** Conceitos de kernel valem em toda parte; `apt`, `dnf` e `pacman` não. Debian e derivados — Kali incluído — formam a família deste manual.
- **O modelo de software livre é o que torna esse ecossistema possível:** quatro liberdades, licenças copyleft como a GPLv2 do kernel e permissivas como MIT e BSD, e um mercado que cobra por suporte e serviço, não por aprisionamento.
- **Kali é Debian rolling voltado à segurança,** com usuário `kali`, kernel com patches próprios, ferramentas em metapacotes e sem janela de suporte longo — excelente estação de trabalho de auditoria, má escolha para servidor de produção.



## 2. Por que Kali Linux: propósito, filosofia e quando não usá-lo

A empresa contratou um analista júnior e ele chegou animado: instalou Kali Linux como sistema principal do notebook de trabalho, apagou o Windows, e passou a primeira semana feliz. Na segunda semana o Wi-Fi do escritório parou de conectar depois de um `apt full-upgrade`. Na terceira, o cliente pediu para ele acessar uma planilha compartilhada e o pacote de escritório não abria o arquivo direito. Na quarta, o time de TI descobriu que a máquina que circula na rede corporativa tem `responder`, `bettercap` e `metasploit-framework` instalados e nenhum controle sobre quem senta nela. Nada disso é defeito do Kali. É uso fora de propósito.

O Kali é uma distribuição excelente — para o que ela foi feita. Este capítulo explica o que ela foi feita para ser, de onde veio, quais decisões de projeto explicam seus comportamentos estranhos, e — tão importante quanto — em quais situações a resposta certa é outra distribuição. Como o manual inteiro usa o Kali como referência, vale deixar claro desde já o contrato: aprender Linux **no** Kali é ótimo, porque tudo que vale para Debian vale aqui; usar Kali como sistema de produção, de servidor ou de escritório é uma escolha ruim, e as razões são técnicas.

### 2.1. O que o Kali é, em uma frase

Kali Linux é uma distribuição Debian voltada a **teste de invasão, forense digital, engenharia reversa e auditoria de segurança**, mantida pela OffSec, distribuída gratuitamente e construída a partir do ramo `testing` do Debian (Offensive Security, 2025).

Cada parte dessa frase carrega uma consequência prática. “Debian” significa `apt`, `.deb`, `/etc`, `systemd` — nenhuma novidade estrutural, e é por isso que o conhecimento adquirido aqui é transferível. “Voltada a teste de invasão” significa que o conjunto padrão de pacotes tem centenas de ferramentas ofensivas, o que é conveniente para quem audita e é superfície de risco para quem não audita. “Ramo `testing`” significa *rolling release*: não há versão congelada, os pacotes mudam continuamente, e a estabilidade de longo prazo nunca foi um objetivo do projeto. “Mantida pela OffSec” significa que existe uma empresa por trás, com um modelo de negócio — treinamento e certificação, notadamente a OSCP — que sustenta o trabalho de empacotamento.

### 2.2. De onde veio: a linhagem BackTrack

O Kali não nasceu do zero em 2013. Ele é o quarto ou quinto nome de uma linha de distribuições de segurança que começou como live CD no início dos anos 2000, quando montar um ambiente com todas as ferramentas de auditoria significava compilar dezenas de projetos à mão, cada um com dependências conflitantes. Figura 2.1 resume o caminho.

O **Whoppix**, de 2004, era um live CD derivado do Knoppix com ferramentas de teste de invasão. Ele virou WHAX, e em 2006 a fusão com o projeto Auditor Security Collection produziu o **BackTrack**, que dominou a área por sete anos. O BackTrack era competente e desorganizado: as ferramentas iam parar em `/pentest/`, fora de qualquer convenção do sistema de arquivos,

## 2. Por que Kali Linux: propósito, filosofia e quando não usá-lo

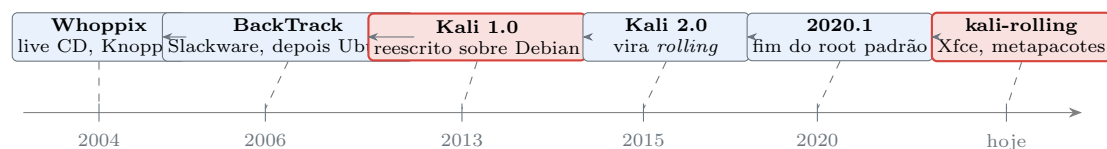


Figura 2.1.: A linhagem do Kali: vinte anos de tentativas de empacotar um ambiente de auditoria reproduzível.

muitas vezes como cópias do repositório de origem sem empacotamento nenhum. Atualizar era um problema, reproduzir um ambiente era um problema, e auditar a procedência de um binário era quase impossível.

Em março de 2013 a equipe jogou fora a base e recomeçou sobre Debian. Essa é a mudança que define o **Kali**: toda ferramenta virou pacote `.deb`, com dependências declaradas, com arquivos nos lugares previstos pelo FHS, com assinatura no repositório, com histórico de versões. O nome mudou justamente para sinalizar que não era uma nova versão do BackTrack, e sim outro sistema.

Duas viradas posteriores importam. Em 2015, o Kali 2.0 adotou o modelo *rolling*, alinhando o repositório ao `testing` do Debian de forma contínua. E em janeiro de 2020, a versão 2020.1 **encerrou o login como root por padrão**, adotando um usuário comum chamado `kali` com `sudo` — mudança que quebrou o hábito de uma geração inteira e alinhou a distribuição às práticas normais de segurança.

### 2.3. As decisões de projeto que explicam o comportamento

Boa parte do que parece esquisito no Kali é consequência direta de uma decisão consciente. Vale conhecer as principais, porque elas antecipam problemas.

**Repositório único e assinado.** Todo pacote do Kali vem de um repositório assinado com a chave GPG do projeto, e o número de espelhos oficiais é deliberadamente pequeno. A lógica é de cadeia de suprimentos: quem instala uma distribuição de segurança está confiando enormemente em quem empacotou aquelas ferramentas, muitas das quais têm capacidade de fazer estrago. Um repositório enxuto e assinado é auditável; uma coleção de PPAs de terceiros não é. Consequência prática: **não saia adicionando repositórios de outras distribuições ao Kali**. Misturar Ubuntu com Kali quebra dependências e destrói a garantia de procedência. A regra vale em dobro no Kali por ele ser *rolling*.

**Ferramentas curadas, não exaustivas.** O repositório tem algo em torno de seiscentas ferramentas de segurança, e a entrada de uma nova passa por avaliação: a ferramenta é mantida? Tem licença compatível? Funciona? Duplica algo que já existe? Isso é oposto à filosofia do BlackArch, que empacota milhares de projetos sem tanta triagem. Cada abordagem tem mérito; o Kali escolheu ser menor e mais confiável.

**Kernel com patches próprios.** O sufixo `-kaliN` em `uname -r` indica correções aplicadas pela distribuição, especialmente relacionadas a **injeção de pacotes** e **modo monitor** em adaptadores Wi-Fi. Um kernel Debian puro frequentemente não permite as duas coisas com os mesmos drivers. Esse é um dos poucos pontos em que o Kali realmente entrega algo que você não obteria instalando as ferramentas manualmente em outra distro.

**Metapacotes em vez de instalação monolítica.** Em vez de uma imagem gigante, o Kali oferece pacotes que são apenas listas de dependências: `kali-linux-core` (o mínimo), `kali-linux-default` (o conjunto da imagem padrão), `kali-linux-large`, `kali-linux-everything`, e metapacotes por área — `kali-tools-web`, `kali-tools-wireless`, `kali-tools-forensics`, `kali-tools-passwords`, entre outros. Instalar um deles arrasta centenas de programas; remover libera vários gigabytes. Isso é o que permite um Kali de 2 GB para uma VM descartável e um de 30 GB para uma estação de trabalho completa.

**Onipresença de plataformas.** O mesmo repositório serve imagens para máquina virtual, instalação em disco, live USB, ARM (Raspberry Pi e semelhantes), WSL2 no Windows, contêiner Docker, nuvem, e o NetHunter, que roda em celulares Android. A infraestrutura de build é a mesma; muda o alvo.

**Personalização assumida.** O projeto publica as receitas de construção das próprias imagens (`live-build-config`), de modo que qualquer pessoa pode gerar um Kali sob medida — com exatamente as ferramentas que quer, o papel de parede que quiser e as configurações que precisa. É um recurso subutilizado e uma das coisas mais úteis da distribuição para quem trabalha com auditoria recorrente.

## 2.4. O contrato de segurança que você assina ao instalar

Aqui está a parte que costuma ser mal compreendida. Uma distribuição de segurança **não é uma distribuição segura**. São ideias opostas.

Uma instalação padrão de Kali traz centenas de binários com capacidade ofensiva, muitos deles com bits de permissão especiais, alguns rodando com privilégio elevado. Traz também um conjunto de decisões voltadas à conveniência do operador — serviços que iniciam fácil, configurações permissivas para laboratório. A superfície de ataque de uma máquina Kali é maior que a de um Debian mínimo, por construção. Se um atacante compromete uma máquina Kali, ele encontra ali uma caixa de ferramentas montada, aquecida e pronta.

Por isso, três hábitos valem desde o primeiro dia:

- **Kali é máquina de trabalho de quem audita, não máquina exposta.** Não a coloque como servidor com serviços abertos à internet. Não deixe SSH aberto para o mundo. Serviços no Kali vêm desabilitados no boot por padrão justamente por isso — é uma decisão de projeto, não um esquecimento.
- **Use VM com snapshot.** Um teste de invasão suja o sistema: configurações alteradas, pacotes instalados às pressas, artefatos espalhados. Restaurar um snapshot é mais rápido e mais confiável do que limpar.
- **Separe cliente de cliente.** Se você audita mais de uma organização, dados de uma não podem encostar nos de outra. VMs distintas, ou pelo menos volumes distintos e criptografados.

E o ponto jurídico, que não é detalhe: usar essas ferramentas contra sistemas de terceiros sem autorização é crime no Brasil. A Lei 12.737/2012 tipificou a invasão de dispositivo informático, e a legislação posterior endureceu as penas. O que torna um teste de invasão legítimo não é a ferramenta nem a técnica — é o **contrato**: escopo definido por escrito, janela de execução acordada, autorização de quem tem poder para autorizá-la, e regras claras sobre o que fazer com o que for encontrado. Sem esse documento, a mesma ação que num contrato é serviço profissional é, fora dele, crime. Este manual ensina os mecanismos e assume que o leitor trabalha em laboratório próprio ou sob autorização.

## 2.5. Quando não usar Kali

Esta seção existe porque a pergunta aparece toda semana em fórum. Figura 2.2 resume a decisão.

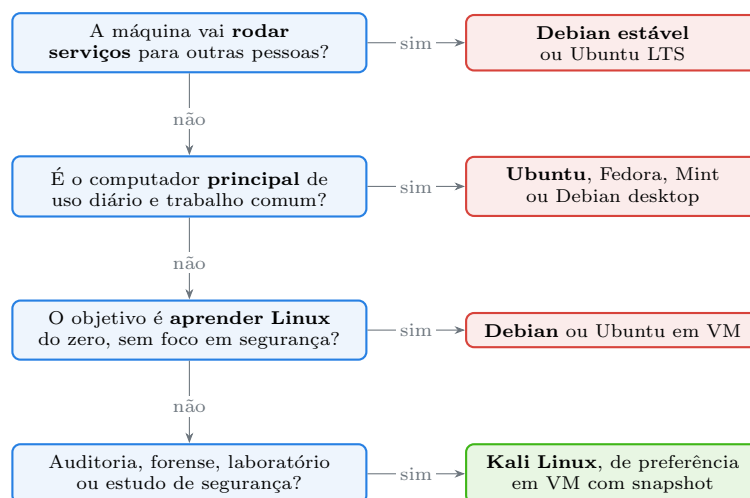


Figura 2.2.: Kali resolve um problema específico. Fora dele, quase sempre existe escolha melhor.

**Não use como servidor.** Rolling release e servidor de produção são incompatíveis: um `apt full-upgrade` pode trocar a versão principal de um banco de dados, de um servidor web ou da própria biblioteca C. Não há janela de suporte de segurança de longo prazo, não há backports estabilizados, não há promessa de compatibilidade. Servidor pede Debian estável, Ubuntu LTS ou RHEL/Rocky.

**Não use como desktop de trabalho comum.** É possível — muita gente faz — mas você paga um preço em manutenção que não tem contrapartida. O ambiente vem otimizado para operação de auditoria, não para o dia a dia de escritório, e cada atualização traz a chance de uma regressão em algo que você depende para trabalhar.

**Não use como primeira distribuição para aprender Linux, se o objetivo é só aprender Linux.** Aqui cabe uma ressalva honesta, já que este manual escolheu o Kali como referência: o problema não é o Kali ser difícil — ele é um Debian, e Debian é dos sistemas mais bem documentados que existem. O problema é que, ao pesquisar um erro, o iniciante recebe respostas contaminadas por particularidades da distribuição e por conselhos de fóruns de segurança. A escolha do manual é deliberada: usamos o Kali porque ele é Debian e porque o público-alvo tem interesse em segurança, mas todo capítulo marca explicitamente o que é geral e o que é específico da distribuição — essa é a função da seção “No Kali” que fecha cada capítulo.

**Não use se você não sabe o que as ferramentas fazem.** Rodar `bettercap` ou responder numa rede corporativa “para ver o que acontece” pode derrubar serviços, capturar credenciais de colegas e render demissão por justa causa, além de responsabilização criminal. Esse não é um cenário hipotético.

**Não use se a organização proíbe.** Muitas empresas têm política explícita contra ferramentas ofensivas em estações de trabalho, e com razão. Verifique antes.

E a contrapartida: **use Kali** quando a tarefa for teste de invasão autorizado, resposta a incidente, forense, engenharia reversa, avaliação de segurança de rede sem fio, laboratório de estudo (CTF, máquinas vulneráveis, ambientes de treino) ou aula prática de segurança. Nesses casos, o tempo que você economiza por ter tudo empacotado e funcionando é enorme.

## 2.6. As alternativas que valem conhecer

O Kali não é a única opção, e conhecer as vizinhas ajuda a entender as decisões dele.

**Parrot Security OS** também é derivado do Debian e cobre território parecido, com ênfase maior em privacidade e um ambiente gráfico mais leve. É a alternativa mais direta ao Kali.

**BlackArch** é um repositório de ferramentas de segurança sobre Arch Linux, com milhares de pacotes. Serve a quem já vive no Arch e quer o volume máximo de ferramentas, aceitando menos curadoria.

**Debian ou Ubuntu com as ferramentas instaladas manualmente** é a escolha de muito profissional experiente. Você perde os patches de kernel e a conveniência dos metapacotes, e ganha um sistema previsível cujo conteúdo você conhece linha por linha.

**Distribuições de tarefa única** como o Tails (amnésica, focada em anonimato, roda em RAM e não deixa rastro) e o REMnux (análise de malware) resolvem problemas específicos melhor que qualquer distribuição generalista.

**Security Onion** e semelhantes vão para o lado defensivo — monitoramento de rede, detecção de intrusão, análise de logs. Vale lembrar que Kali é ferramenta **ofensiva** por vocação; o lado defensivo tem seu próprio ecossistema.

## 2.7. No Kali

Verificações e ajustes que valem fazer logo depois de instalar, e que exercitam o que este capítulo descreveu:

- **Confirme o ramo do repositório.** O arquivo `/etc/apt/sources.list` deve conter uma única linha ativa apontando para `kali-rolling`. Se apontar para `kali-last-snapshot`, a máquina está congelada na última imagem lançada e não receberá pacotes novos — útil para reprodutibilidade, ruim para quem quer ferramentas atuais.
- **Confirme a assinatura do repositório.** A chave pública do Kali fica em `/usr/share/keyrings/kali-archive-keyring.gpg`. Se ela expirar ou for corrompida, `apt update` passa a recusar o repositório — é o mecanismo funcionando, não um defeito. A correção é reinstalar o pacote `kali-archive-keyring`, nunca desabilitar a verificação.
- **Descubra quais metapacotes existem** com `apt search kali-linux` e `apt search kali-tools`. É a forma correta de crescer ou encolher a instalação sem caçar ferramenta por ferramenta.
- **kali-tweaks** é o utilitário próprio da distribuição: ajusta ramo de repositório, shell padrão, comportamento de serviços de rede e o modo *undercover*.
- **Modo *undercover*** (`kali-undercover`) troca o tema do Xfce por uma imitação de Windows. É um recurso sério, apesar da aparência: serve para não chamar atenção ao trabalhar em campo, num saguão de cliente ou numa sala com terceiros presentes.
- **Serviços não iniciam no boot.** No Kali, serviços de rede como SSH e o servidor web vêm desabilitados na inicialização. Isso é deliberado. Habilitá-los é possível, mas é uma decisão consciente sua — e o tema de serviços e `systemd` é o Volume 7.
- **Atualize com `apt update && apt full-upgrade`**, nunca só `apt upgrade`. Em rolling release, transições de pacote frequentemente exigem remover algo, e `apt upgrade` se recusa a remover — o resultado é uma máquina parcialmente atualizada, que é a pior configuração possível. O assunto tem capítulo próprio no Volume 6.

## 2. Por que Kali Linux: propósito, filosofia e quando não usá-lo

**Nota sobre a família RHEL.** Não existe equivalente de Kali com a mesma tração no mundo Red Hat. Quem trabalha nesse ecossistema normalmente usa Fedora com repositórios adicionais, ou simplesmente uma VM Kali ao lado. A diferença de família aparece nas ferramentas de sistema (`dnf`, `firewalld`, `SELinux`), não nas ferramentas de segurança em si, que são as mesmas em toda parte.

## 2.8. Laboratório

Roteiro de leitura e verificação numa VM com Kali. Nenhum passo instala ferramenta ofensiva nem toca em rede de terceiros.

### 1. Identifique a natureza rolling da instalação.

```
cat /etc/os-release
cat /etc/apt/sources.list
apt policy | head -20
```

Esperado: `VERSION` traz um marco de imagem (por exemplo 2025.2) e o `sources.list` aponta para `kali-rolling`. Repare que o número da versão não define de onde vêm os pacotes — quem define é o ramo.

### 2. Confirme que o kernel tem patches da distribuição.

```
uname -r
uname -v
```

Esperado: um sufixo como `-kali1` ou `-kali3`. Esse número é da distribuição, não do kernel de origem. É a evidência dos patches de injeção de pacotes e modo monitor.

### 3. Verifique a cadeia de confiança do repositório.

```
ls -l /usr/share/keyrings/kali-archive-keyring.gpg
sudo apt update
```

Esperado: o `apt update` termina sem aviso de assinatura. Se aparecer `NO_PUBKEY` ou `EXPKEYSIG`, a chave está ausente ou expirada — a correção é `sudo apt install --reinstall kali-archive-keyring`, jamais desativar a verificação.

### 4. Explore os metapacotes.

```
apt search '^kali-linux-' 2>/dev/null | grep '^kali'
apt show kali-linux-default 2>/dev/null | head -20
```

Esperado: a lista dos conjuntos disponíveis e, no `apt show`, um campo `Depends` enorme. Confirme com os próprios olhos que um metapacote não tem conteúdo — ele é só uma lista de dependências.

### 5. Meça o custo do conjunto padrão.

```
dpkg -l | grep -c '^ii'
du -sh /usr 2>/dev/null
```

Esperado: alguns milhares de pacotes e vários gigabytes. Compare mentalmente com uma instalação mínima de Debian, que fica na casa das centenas de pacotes.



## 6. Veja quantas ferramentas de segurança estão de fato instaladas.

```
ls /usr/bin | wc -l
apt list --installed 2>/dev/null | grep -c kali
```

Esperado: milhares de executáveis em `/usr/bin`. Esse número é a materialização do que a seção sobre superfície de ataque discutiu — cada um daqueles binários é código que existe na máquina.

## 7. Confirme que serviços de rede não iniciam sozinhos.

```
systemctl is-enabled ssh 2>/dev/null
systemctl is-active ssh 2>/dev/null
ss -ltnp 2>/dev/null | head
```

Esperado: `disabled` e `inactive` para o SSH numa instalação padrão, e pouquíssimas portas em escuta. Um sistema que não escuta em porta alguma não pode ser atacado pela rede.

8. Experimente o modo *undercover* e volte.

```
kali-undercover
```

Esperado: o Xfce assume aparência de Windows. Rodar o mesmo comando de novo desfaz. Vale entender o que ele faz por baixo: é uma troca de tema, papel de parede e disposição do painel — nada de esconder processos ou tráfego. Serve contra olhar casual, não contra perícia.

9. Registre o estado inicial em um snapshot. Pelo gerenciador da sua VM, crie um snapshot chamado `kali-limpo`. Esperado: um ponto de retorno para todos os laboratórios seguintes deste manual. Isso não é formalidade — a partir do Volume 3 haverá exercícios que alteram o sistema.

## 2.9. Resumo

- **Kali é um Debian rolling voltado a teste de invasão, forense e auditoria**, mantido pela OffSec — herdeiro direto do BackTrack, reconstruído sobre Debian em 2013 justamente para ganhar empacotamento, procedência e reprodutibilidade.
- **As decisões de projeto explicam o comportamento**: repositório único e assinado, ferramentas curadas em vez de exaustivas, kernel com patches para Wi-Fi, metapacotes no lugar de instalação monolítica e serviços desabilitados por padrão.
- **Distribuição de segurança não é distribuição segura**. A superfície de ataque de um Kali padrão é grande por construção, e o hábito correto é tratá-lo como estação de trabalho de auditoria em VM com snapshot, nunca como máquina exposta.
- **O que legitima o uso das ferramentas é a autorização**, não a técnica. Sem escopo por escrito e autorização de quem pode dá-la, a mesma ação é crime no Brasil (Lei 12.737/2012).
- **Não use Kali como servidor, como desktop de trabalho comum ou como sistema de produção**. Rolling release sem janela de suporte é incompatível com esses papéis; Debian estável, Ubuntu LTS ou RHEL resolvem melhor.
- **Existem alternativas com propósitos vizinhos** — Parrot, BlackArch, Tails, REMnux, ou simplesmente Debian com as ferramentas instaladas à mão —, e conhecê-las ajuda a escolher a máquina certa para cada tarefa.



### 3. Instalando o Kali: máquina virtual, bare metal, WSL e live USB

O contrato começa segunda-feira. Você precisa de um ambiente Kali funcionando, com um adaptador Wi-Fi capaz de entrar em modo monitor, com espaço para guardar capturas de tráfego de um cliente sem misturar com as do cliente anterior, e com a possibilidade de voltar tudo ao estado limpo quando a auditoria acabar. Três colegas dão três respostas diferentes: “instala numa VM”, “instala direto no disco”, “usa o WSL, é mais rápido”. As três estão certas — para problemas diferentes. Escolher errado significa descobrir, na quarta-feira, que o adaptador Wi-Fi não aparece dentro da máquina virtual porque ninguém configurou o repasse de USB, ou que não existe como voltar atrás porque a instalação foi feita direto sobre o disco do notebook pessoal.

Este capítulo cobre os cinco caminhos de instalação, o que cada um entrega e o que cada um cobra, e um passo que quase todo tutorial pula: **verificar a imagem antes de instalá-la**. Vamos começar por ele, porque é o único que, se você errar, compromete tudo o que vem depois.

#### 3.1. Antes de tudo: verificar a imagem

Você vai baixar um arquivo de alguns gigabytes que, minutos depois, terá controle total sobre uma máquina sua. Não é exagero conferir a procedência. Uma imagem adulterada — por comprometimento de espelho, por ataque na rede, por download interrompido — instala um sistema que parece normal e não é.

O Kali publica, junto das imagens, um arquivo `SHA256SUMS` com as somas de verificação e um `SHA256SUMS.gpg` com a assinatura desse arquivo. A cadeia de confiança tem dois elos, e pular o segundo é o erro comum. Figura 3.1 mostra o encadeamento.

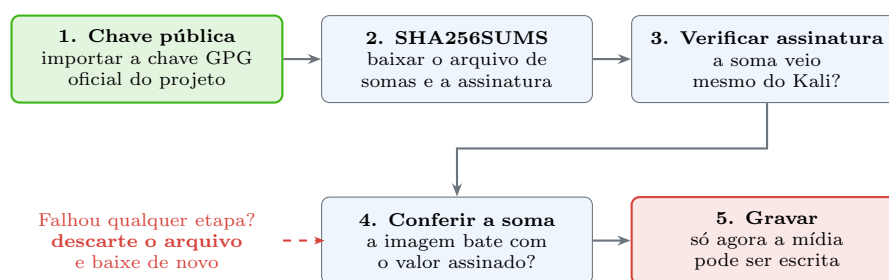


Figura 3.1.: Conferir apenas a soma não prova nada se ela também veio do mesmo lugar adulterado. A assinatura GPG é o elo que fecha a cadeia.

Na prática, a partir de qualquer máquina Linux:

```
# 1. importar a chave pública oficial do projeto
wget -q -O - https://archive.kali.org/archive-keyring.gpg | gpg --import

# 2. baixar o arquivo de somas e a assinatura, ao lado da imagem
```

### 3. Instalando o Kali: máquina virtual, bare metal, WSL e live USB

```
wget https://cdimage.kali.org/current/SHA256SUMS
wget https://cdimage.kali.org/current/SHA256SUMS.gpg

# 3. a soma foi mesmo assinada pelo Kali?
gpg --verify SHA256SUMS.gpg SHA256SUMS

# 4. a imagem bate com a soma?
sha256sum -c SHA256SUMS --ignore-missing
```

O passo 3 deve responder **Good signature**. Um aviso dizendo que a chave não é confiável (**WARNING: This key is not certified with a trusted signature**) é esperado e não é problema: significa apenas que você não assinou a chave do Kali com a sua. O que não pode aparecer é **BAD signature**. O passo 4 deve responder **OK** para o arquivo que você baixou.

No Windows, sem uma máquina Linux à mão, o PowerShell resolve o passo da soma:

```
Get-FileHash .\kali-linux-2025.2-installer-amd64.iso -Algorithm SHA256
```

Compare o resultado com a linha correspondente do **SHA256SUMS**. A verificação da assinatura exige o Gpg4win — e vale o trabalho na primeira vez.

## 3.2. Qual imagem baixar

O projeto publica várias variantes, e o nome do arquivo diz tudo:

**Installer** é a imagem completa de instalação, com os pacotes embutidos: instala sem depender de internet, é a maior. **NetInstaller** é a imagem enxuta que baixa tudo durante a instalação: exige rede, mas sempre instala pacotes atuais e permite escolher exatamente o que entra. **Live** roda direto da mídia sem instalar nada, e é a base para o pendrive com persistência. **Virtual machines** são imagens pré-construídas para VirtualBox e VMware, com as ferramentas de convidado já instaladas — o caminho mais curto para ter um Kali funcionando. **ARM** cobre Raspberry Pi e placas semelhantes. Há ainda imagens para WSL, para nuvem e para o NetHunter em Android.

Existem também versões **weekly**, geradas semanalmente a partir do repositório atual, e as versões de lançamento trimestrais. Para uso normal, pegue a de lançamento e atualize depois; a weekly serve quando você precisa de um pacote muito recente já na imagem.

## 3.3. Método 1: máquina virtual — o padrão recomendado

Para praticamente todo mundo que está começando, e para a maior parte do trabalho profissional, **VM é a resposta certa**. As razões são três: isolamento (o que acontece no Kali fica no Kali), reversibilidade (snapshot antes, restaurar depois) e paralelismo (uma VM por cliente, ou uma VM Kali e uma máquina-alvo vulnerável na mesma rede virtual).

Os hipervisores viáveis são o **VirtualBox** (gratuito, multiplataforma, o mais comum em estudo), o **VMware Workstation** (mais rápido em gráficos e USB, hoje gratuito para uso pessoal), o **KVM/QEMU** com **virt-manager** (nativo do Linux, o mais eficiente quando o hospedeiro já é

Linux) e o **Hyper-V** (embutido no Windows Pro, com integração fraca de USB, o que atrapalha o trabalho com Wi-Fi).

Dimensionamento razoável para uma VM de estudo: **2 a 4 vCPUs**, **4 GB de RAM** no mínimo e 8 GB se houver folga, **60 a 80 GB de disco** em alocação dinâmica. O disco é o item mais subestimado: `kali-linux-default` mais uma atualização acumulada mais capturas de tráfego enchem 40 GB rápido, e ampliar disco de VM depois dá trabalho. Habilite a virtualização assistida por hardware no firmware da máquina (VT-x na Intel, AMD-V na AMD) — sem ela, a VM roda em emulação e fica insuportavelmente lenta.

Depois de instalar, o passo que quase todo mundo esquece são as **ferramentas de convidado**, responsáveis por ajuste automático de resolução, área de transferência compartilhada e pastas compartilhadas:

```
sudo apt update
sudo apt install -y open-vm-tools-desktop # VMware
# ou
sudo apt install -y virtualbox-guest-x11 # VirtualBox
```

### 3.3.1. Os modos de rede da VM

Este é o ponto que mais gera confusão, e errar aqui custa uma tarde inteira de diagnóstico. Figura 3.2 compara os três modos que importam.

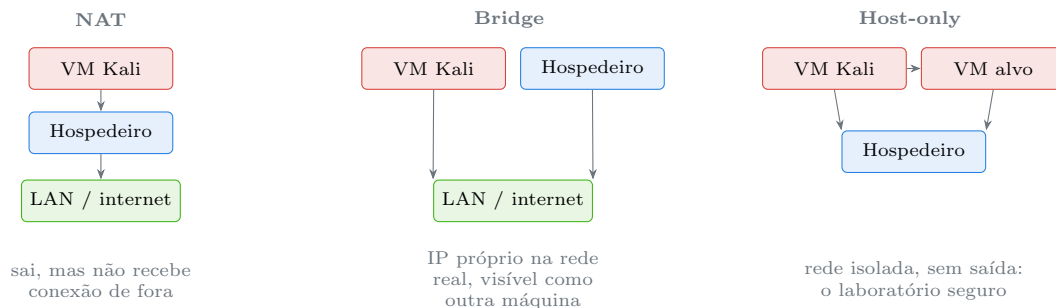


Figura 3.2.: Os três modos de rede que resolvem a grande maioria dos casos. Laboratório com máquina vulnerável pede host-only.

**NAT** é o padrão e serve para a maioria: a VM navega, atualiza pacotes e alcança a internet, escondida atrás do hospedeiro. Nada de fora inicia conexão com ela. **Bridge** coloca a VM na rede física como se fosse uma máquina independente, com IP do mesmo roteador — necessário quando ela precisa ser alcançada por outros equipamentos, e o modo que exige mais cuidado, porque um Kali em bridge numa rede corporativa é exatamente o que a política de TI proíbe. **Host-only** (ou “rede interna”) cria uma rede fechada entre as VMs e o hospedeiro, sem saída para a internet: é o modo correto para montar laboratório com máquinas propositalmente vulneráveis, porque garante que nada daquilo vaze.

A regra prática: **máquina-alvo vulnerável nunca em NAT nem em bridge**. Instalar uma VM cheia de falhas conhecidas e deixá-la alcançável pela rede da casa ou do escritório transforma seu laboratório em porta de entrada.

### 3.3.2. Adaptador Wi-Fi na VM

Adaptador Wi-Fi interno do notebook **não** funciona em modo monitor dentro de uma VM. O hipervisor apresenta ao convidado uma placa Ethernet virtual, e o que o Kali vê não é o rádio real. A solução é um **adaptador USB externo** com chipset compatível, repassado à VM por USB passthrough — em VirtualBox exige o Extension Pack, em VMware é nativo, em KVM se faz pelo `virt-manager`. Depois de repassar, `lsusb` dentro da VM tem de listar o adaptador; se não listar, o problema é de configuração do hipervisor, não do Kali.

## 3.4. Método 2: bare metal — instalação direta no disco

Instalar direto no hardware entrega desempenho total, acesso nativo a todos os dispositivos (inclusive o Wi-Fi interno) e uso pleno da GPU — o que importa para quem faz quebra de hash com `hashcat`. Em troca, você perde a rede de segurança do snapshot e assume os riscos de particionamento.

Vale a pena quando a máquina é dedicada à função. Não vale a pena no notebook que você usa para tudo.

#### Particionar apaga dados

O instalador oferece o modo guiado com “usar o disco inteiro”, e ele faz exatamente o que diz: **destrói todas as partições existentes**, incluindo o outro sistema operacional e todos os arquivos. Antes de instalar em bare metal, faça backup completo e verifique que o backup restaura. O tema de discos e partições tem um volume inteiro adiante (Volume 8); por ora, se houver a menor dúvida, use uma VM.

Três pontos merecem atenção na instalação em disco:

**Dual boot com Windows.** O instalador consegue redimensionar a partição do Windows e conviver com ele, mas o procedimento seguro é reduzir a partição **pelo próprio Windows** antes, deixando espaço não alocado, e só então iniciar o instalador do Kali. Desative também a inicialização rápida do Windows: com ela ligada, o Windows não desliga de verdade e deixa o sistema de arquivos em estado inconsistente, o que pode corromper dados quando o Linux monta a partição.

**Secure Boot.** As imagens atuais do Kali têm suporte a Secure Boot, mas módulos de kernel de terceiros — drivers de adaptadores Wi-Fi fora da árvore, por exemplo — precisam ser assinados para carregar. Se um driver instalado à mão não sobe, Secure Boot é o primeiro suspeito.

**Criptografia de disco.** O instalador oferece LUKS com uma caixa de seleção. Para máquina que sai de casa com dados de cliente, isso não é opcional. O detalhamento de LUKS é o Volume 8.

## 3.5. Método 3: live USB com persistência

O modo live roda o sistema inteiro a partir do pendrive, sem tocar no disco da máquina. Por padrão nada é gravado: reiniciou, perdeu. Com **persistência**, uma partição adicional do pendrive guarda alterações entre sessões — e pode ser criptografada com LUKS.

É o método certo em três situações: perícia forense, quando você não pode escrever nada no disco do equipamento examinado; recuperação de um sistema que não inicia; e trabalho em máquina de terceiros, quando você leva o seu ambiente no bolso e não deixa rastro na máquina do cliente.

Gravar a imagem no pendrive é a operação mais perigosa deste capítulo:

! Escrever no dispositivo errado destrói o disco

O comando abaixo escreve byte a byte no dispositivo indicado, **sem confirmação e sem possibilidade de desfazer**. Apontar para `/dev/sda` em vez de `/dev/sdb` apaga o disco do sistema. Identifique o dispositivo com `lsblk` antes, confira o tamanho (um pendrive de 32 GB aparece com cerca de 29 G), e desconecte outros dispositivos removíveis para reduzir a chance de erro.

```
# 1. identificar o dispositivo - confira tamanho e ponto de montagem
lsblk -o NAME,SIZE,TYPE,MOUNTPOINT,MODEL

# 2. ensaio seguro: escrever numa imagem de arquivo em vez de num disco
truncate -s 4G /tmp/ensaio.img
sudo dd if=kali-linux-live-amd64.iso of=/tmp/ensaio.img bs=4M status=progress
↪ conv=fsync

# 3. só depois, no dispositivo real - SUBSTITUA sdX pelo seu pendrive
sudo dd if=kali-linux-live-amd64.iso of=/dev/sdX bs=4M status=progress
↪ conv=fsync
sync
```

O passo 2 não é enfeite: ele deixa você praticar a sintaxe e ver a saída esperada sem risco nenhum. Alternativamente, ferramentas gráficas como o Etcher listam apenas dispositivos removíveis e reduzem bastante a chance de acidente — é a recomendação para quem está começando.

A persistência é criada depois, particionando o espaço livre do pendrive e rotulando a partição como `persistence`, com um arquivo `persistence.conf` contendo `/ union`. A documentação oficial traz o passo a passo atualizado (Offensive Security, 2025).

## 3.6. Método 4: WSL2 no Windows

O Subsistema Windows para Linux roda uma distribuição Linux sobre um kernel Linux real, gerenciado pelo Windows, com integração forte com o sistema hospedeiro. Instalar é trivial:

```
wsl --install -d kali-linux
```

Depois, dentro do Kali, dá para instalar o **Win-Kex**, que entrega uma sessão gráfica completa:

```
sudo apt update
sudo apt install -y kali-win-kex
kex --win -s
```

### 3. Instalando o Kali: máquina virtual, bare metal, WSL e live USB

O WSL brilha para trabalho de linha de comando: scripts, processamento de texto, ferramentas que só falam TCP, análise de arquivos, desenvolvimento. É rápido, integra com o sistema de arquivos do Windows e não exige gerenciar uma VM.

E tem limites reais. O kernel é o do WSL, não o do Kali — sem os patches de injeção de pacotes discutidos em Capítulo 2. Acesso a hardware é restrito: USB só via `usbipd`, com resultado irregular, e modo monitor de Wi-Fi na prática não funciona. A pilha de rede é intermediada pelo Windows, o que atrapalha ferramentas que manipulam pacotes em baixo nível. Alguns serviços que dependem do `systemd` exigem configuração adicional. Resumo: **WSL é ótimo complemento e mau substituto** — se o trabalho envolve rede em baixo nível ou Wi-Fi, use VM ou bare metal.

## 3.7. Método 5: contêiner

Para uma ferramenta específica, sem instalar nada:

```
docker run --rm -it kalilinux/kali-rolling /bin/bash
apt update && apt install -y kali-linux-headless
```

O contêiner sobe em segundos, é descartável por natureza e compartilha o kernel do hospedeiro — o que, como visto em Capítulo 1, significa que ele traz a distribuição mas não o kernel do Kali. Serve muito bem para rodar uma ferramenta pontual em pipeline de automação, e serve mal como ambiente de trabalho interativo. O funcionamento interno de contêineres é tema do Volume 16.

## 3.8. Depois de instalar: os primeiros cinco minutos

Independentemente do método, quatro passos fecham a instalação:

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y open-vm-tools-desktop      # ou virtualbox-guest-x11
sudo dpkg-reconfigure keyboard-configuration # teclado ABNT2
sudo reboot
```

E, na VM, **crie o snapshot** logo depois do reboot, com nome que diga alguma coisa (`kali-limpo-atualizado`). Esse é o ponto de retorno de todos os laboratórios deste manual.

## 3.9. No Kali

- **Credenciais padrão da imagem live: `kali` / `kali`.** Na instalação em disco você define usuário e senha durante o processo. As imagens pré-construídas de VM também vêm com `kali/kali` — trocar a senha é o primeiro comando depois do primeiro login (`passwd`).
- **Imagens pré-construídas de VM já trazem as ferramentas de convidado.** Se você baixou o pacote compactado para VirtualBox ou VMware, não precisa instalar `open-vm-tools` nem os guest additions: já estão lá. Esse é o caminho mais rápido do zero ao Kali funcionando, e o recomendado para quem só quer estudar.



- **O instalador oferece escolher os metapacotes.** Na tela de seleção de software dá para trocar `kali-linux-default` por `kali-linux-headless` (sem interface gráfica, instalação bem menor) ou marcar conjuntos por área. Escolher ali economiza dezenas de gigabytes e minutos de download.
- **LUKS com senha de destruição.** O instalador do Kali permite configurar uma senha adicional que, ao ser digitada, apaga os cabeçalhos de chave do volume criptografado, tornando os dados irrecuperáveis. É um recurso pensado para quem transporta material sensível — e é uma arma apontada para o próprio pé se configurada sem entender: **não existe recuperação**. O Volume 8 trata do assunto com o cuidado devido.
- **kali-tweaks** ajusta pós-instalação sem editar arquivo: ramo de repositório, shell padrão, comportamento de serviços de rede e hardening básico.
- **Modo undercover** e temas: úteis em campo, tratados em Capítulo 2.
- **Imagens weekly** trazem pacotes mais recentes na própria mídia. Use quando a versão de lançamento estiver velha o bastante para que o primeiro **full-upgrade** baixe vários gigabytes.

**Nota sobre a família RHEL.** O processo é análogo com Fedora ou Rocky: verificar soma e assinatura (com as chaves em `/etc/pki/rpm-gpg/`), gravar a mídia, instalar com o Anaconda em vez do instalador Debian. Os hipervisores e os modos de rede são exatamente os mesmos.

## 3.10. Laboratório

Roteiro para montar o ambiente que será usado no resto do manual. Os passos 1 a 3 podem ser feitos de qualquer sistema; os demais, dentro da VM recém-criada.

1. **Verifique a cadeia de confiança de uma imagem.** Baixe `SHA256SUMS` e `SHA256SUMS.gpg` do espelho oficial, importe a chave e rode:

```
gpg --verify SHA256SUMS.gpg SHA256SUMS
```

Esperado: `Good signature from "Kali Linux Repository ..."`. O aviso sobre a chave não estar certificada é normal. Se aparecer `BAD signature`, não prossiga: baixe tudo de novo, de preferência de outra rede.

2. **Confira a soma da imagem que você baixou.**

```
sha256sum -c SHA256SUMS --ignore-missing
```

Esperado: `<arquivo>: OK`. Anote quanto tempo levou — em imagem de vários gigabytes, alguns minutos são normais, e essa é a razão pela qual tanta gente pula o passo. Pule uma vez e a instalação inteira vira ato de fé.

3. **Crie a VM.** 4 GB de RAM, 2 vCPUs, 60 GB de disco dinâmico, rede em NAT. Instale a partir da imagem verificada, ou importe a imagem pré-construída. Esperado: primeiro login com o usuário criado durante a instalação.
4. **Atualize e instale as ferramentas de convidado.**

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y open-vm-tools-desktop
sudo reboot
```

### 3. Instalando o Kali: máquina virtual, bare metal, WSL e live USB

Esperado: após reiniciar, a janela da VM redimensiona a resolução automaticamente e a área de transferência funciona entre hospedeiro e convidado. Se não funcionar, o pacote instalado é o do hipervisor errado.

5. **Faça o snapshot kali-limpo-atualizado.** Esperado: um ponto de retorno registrado no hipervisor. Confirme que a restauração funciona: crie um arquivo qualquer com `touch ~/teste-snapshot`, restaure o snapshot e verifique que o arquivo sumiu. **Testar o retorno é mais importante que fazer o snapshot** — snapshot que ninguém testou é backup que ninguém testou.
6. **Descubra o modo de rede em uso, por dentro.**

```
ip -brief address
ip route get 1.1.1.1
```

Esperado: em NAT, um endereço de faixa privada do hipervisor (frequentemente 10.0.2.x no VirtualBox ou 192.168.x.x no VMware) e um gateway que é o próprio hipervisor. Anote o endereço; ele muda quando você trocar o modo de rede.

7. **Crie e observe uma rede isolada.** Adicione um segundo adaptador em modo host-only pelo hipervisor, reinicie a VM e rode `ip -brief address` de novo. Esperado: duas interfaces, cada uma com endereço de faixa diferente. Confirme que a interface host-only não alcança a internet — é essa propriedade que torna o laboratório seguro.
8. **Ensaie a gravação de imagem sem risco.**

```
truncate -s 100M /tmp/ensaio.img
sudo dd if=/dev/zero of=/tmp/ensaio.img bs=1M count=20 status=progress
↪ conv=fsync
ls -lh /tmp/ensaio.img
```

Esperado: a barra de progresso, o relatório final de blocos copiados e o arquivo intacto em 100 M — o comando escreveu dentro dele, não o truncou. Repita a leitura da sintaxe até que `if=` (origem) e `of=` (destino) estejam decorados na ordem certa. Trocar os dois num disco real é o acidente clássico.

9. **Documente o ambiente.** Anote em algum lugar fora da VM: hipervisor e versão, recursos alocados, modo de rede, nome do snapshot inicial e a data. Esperado: um registro de meia dúzia de linhas. Quando algo quebrar daqui a dois meses, é esse registro que responde “o que mudou?”.

## 3.11. Resumo

- **Verificar a imagem tem duas etapas, não uma:** conferir a soma SHA-256 prova integridade do download, e só a verificação da assinatura GPG do arquivo de somas prova procedência. Pular a segunda é confiar em quem serviu o arquivo.
- **Máquina virtual é o padrão recomendado** — isolamento, snapshot e a possibilidade de manter ambientes separados por cliente ou por laboratório. Bare metal só quando a máquina for dedicada e o acesso direto ao hardware for necessário.
- **O modo de rede da VM é decisão de segurança:** NAT para uso geral, bridge quando a VM precisa ser alcançável, host-only para qualquer laboratório com máquina propositalmente vulnerável.

- **Wi-Fi em modo monitor exige adaptador USB externo repassado à VM.** O rádio interno do notebook não chega ao convidado, e nenhuma configuração dentro do Kali resolve isso.
- **Live USB com persistência criptografada** é a ferramenta certa para forense e para trabalho em máquina de terceiros; gravar a mídia é a operação mais perigosa do capítulo e merece conferir o dispositivo duas vezes.
- **WSL2 é excelente complemento e mau substituto:** ótimo para linha de comando e scripts, limitado para rede em baixo nível e praticamente inútil para Wi-Fi.
- **Instalação só termina com atualização, ferramentas de convidado e snapshot testado** — e um snapshot cuja restauração nunca foi verificada não conta.



## 4. Primeiro contato: sessão, ambiente Xfce e organização do sistema

A VM subiu, a tela de login apareceu, a senha foi aceita. E agora existe uma área de trabalho azul-escura com um dragão, um painel no topo com ícones que você não reconhece, e um menu com treze categorias numeradas cheias de nomes obscuros. A primeira reação de quem vem do Windows costuma ser tentar achar o “Meu Computador”; a segunda, abrir o navegador de arquivos e descobrir que não existem letras de unidade. Nenhuma dessas coisas é falta — é outro modelo.

Este capítulo faz o inventário do que você está vendo. Vamos percorrer o caminho do boot até o prompt, entender o que é uma sessão e quem a monta, conhecer as peças do Xfce e o que cada uma faz, decifrar o menu de ferramentas do Kali, ler o prompt do terminal caractere a caractere e, por fim, fazer os ajustes que evitam frustração na primeira semana — teclado brasileiro à frente de todos.

### 4.1. Do boot ao prompt: quem entrega a tela de login

Entre ligar a máquina e ver a tela de login existe uma cadeia de responsabilidades bem definida, e conhecê-la resolve metade dos problemas de “travou no boot”. Figura 4.1 mostra a sequência.

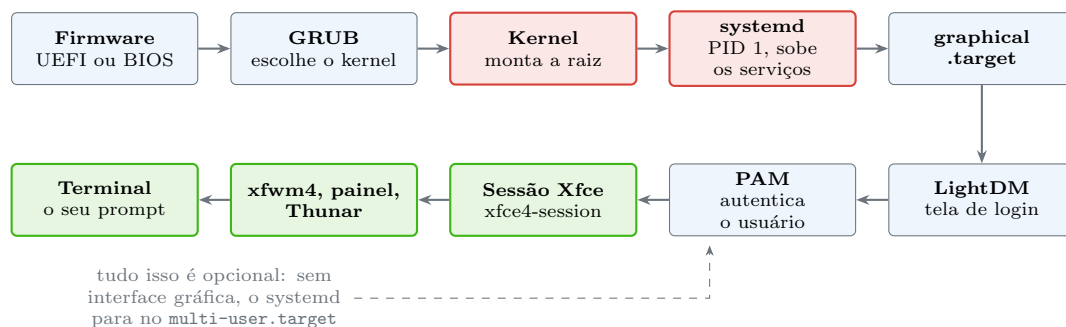


Figura 4.1.: Do firmware ao prompt. O ambiente gráfico é a última camada, e a única que pode ser dispensada por inteiro.

O **firmware** (UEFI na maioria das máquinas atuais) inicializa o hardware e procura um carregador de inicialização. O **GRUB** apresenta o menu de kernels e carrega o escolhido na memória. O **kernel** assume a máquina, monta o sistema de arquivos raiz e inicia o primeiro processo do espaço de usuário. Esse primeiro processo é o **systemd**, que recebe o PID 1 e é responsável por subir todo o resto na ordem correta.

O **systemd** trabalha com **targets**, que são estados desejados do sistema. O **multi-user.target** corresponde a um sistema completo com rede e serviços, mas sem interface gráfica. O **graphical.target** é o **multi-user.target** mais um gerenciador de sessão gráfica. Trocar entre os dois é uma decisão de um comando, e é exatamente o que se faz num servidor:

#### 4. Primeiro contato: sessão, ambiente Xfce e organização do sistema

```
systemctl get-default          # qual é o alvo padrão
sudo systemctl set-default multi-user.target # desligar a interface gráfica
↪ no boot
```

No `graphical.target` sobe o **LightDM**, o gerenciador de sessão que desenha a tela de login. Quando você digita a senha, quem a verifica não é o LightDM: é o **PAM**, o subsistema de autenticação, tema do Volume 4. Com a autenticação aprovada, o LightDM inicia a **sessão Xfce**, que por sua vez sobe as peças do ambiente. Tudo o que você vê na tela é a última camada dessa pilha — e é a mais descartável.

Prova disso está a um atalho de distância. Pressione **Ctrl+Alt+F3**: a interface gráfica some e aparece um login em texto puro. É um **console virtual**, um terminal ligado direto ao kernel, sem X, sem Xfce, sem nada. Há vários deles (F1 a F6), e é aí que você vai trabalhar no dia em que a sessão gráfica não subir. **Ctrl+Alt+F7** (ou F2, conforme a instalação) volta para o ambiente gráfico, que continuou rodando o tempo todo.

### 4.2. O que é uma sessão

Sessão é o conjunto de processos que pertencem a um login seu, mais o ambiente que eles compartilham. O `systemd` rastreia isso explicitamente:

```
loginctl list-sessions
loginctl show-session $XDG_SESSION_ID
whoami
id
```

A saída mostra o identificador da sessão, o usuário dono, o assento (`seat0`, o conjunto teclado-monitor-mouse) e o tipo (`x11`, `wayland` ou `tty`). Esse registro é o que permite ao sistema saber que você está fisicamente na máquina — e é por isso que um usuário logado localmente pode montar um pendrive sem `sudo`, enquanto o mesmo usuário conectado por SSH não pode.

Junto com a sessão vem um punhado de variáveis de ambiente que o resto do sistema consulta o tempo todo:

```
echo $DISPLAY $XDG_SESSION_TYPE $XDG_CURRENT_DESKTOP
echo $HOME $USER $SHELL
env | grep ^XDG | sort
```

`DISPLAY` diz a qual servidor gráfico os programas devem se conectar — se estiver vazia, nenhum programa gráfico abre, e esse é o erro número um de quem tenta rodar uma ferramenta com interface por SSH. `XDG_SESSION_TYPE` diz se a sessão é X11 ou Wayland. `XDG_CURRENT_DESKTOP` diz qual ambiente está no comando. As variáveis `XDG_CONFIG_HOME`, `XDG_DATA_HOME` e `XDG_CACHE_HOME` definem onde as configurações do usuário são gravadas, assunto que volta em Capítulo 5.

Sobre X11 e Wayland: o Kali com Xfce usa **X11** por padrão, através do servidor Xorg. Wayland é o substituto moderno e já é padrão em outros ambientes, mas boa parte das ferramentas de segurança — captura de tela, automação de janelas, compartilhamento remoto — ainda depende de particularidades do X11. Se você mudar para Wayland por conta própria, espere encontrar ferramentas que param de funcionar.

## 4.3. As peças do Xfce

O Kali adotou o **Xfce** como ambiente padrão a partir da versão 2019.4, substituindo o GNOME. A razão é prática: Xfce consome pouca memória, roda bem numa VM com 2 GB, não depende de aceleração gráfica sofisticada e se comporta de forma previsível. Numa distribuição que a maior parte dos usuários roda virtualizada, isso vale mais do que efeitos visuais.

Xfce não é um programa único, e sim um conjunto de processos independentes. Vale saber os nomes, porque quando algo trava você mata e reinicia apenas a peça quebrada:

```
ps -e -o comm= | grep -E 'xfce|xfwm|xfdesktop|xfsettings|lightdm' | sort -u
```

O **xfwm4** é o gerenciador de janelas: desenha as bordas, os botões de fechar e maximizar, e decide qual janela fica na frente. Se as janelas ficarem sem barra de título, é ele que caiu — **xfwm4 --replace &** conserta. O **xfce4-panel** é a barra; ele hospeda o menu, a lista de janelas, o relógio e a bandeja. O **xfdesktop** desenha o papel de parede e os ícones da área de trabalho. O **xfsettingsd** aplica tema, fontes, cursor e configurações de teclado e mouse. O **Thunar** é o gerenciador de arquivos. O **xfce4-session** é quem coordena tudo e restaura o estado ao relogar.

A configuração de todos eles vive em `~/.config/xfce4/`, em arquivos XML. Isso significa que a personalização do seu ambiente é copiável: leve esse diretório para outra máquina e o ambiente vai junto. É a semente da ideia de *dotfiles*, tema do Volume 15.

### 4.3.1. O que fazer no painel na primeira hora

Três coisas rendem retorno imediato.

**Áreas de trabalho.** O Xfce vem com várias, e alternar entre elas é **Ctrl+Alt+←** e **Ctrl+Alt+→**. Numa auditoria, a organização típica é uma área para o terminal, uma para o navegador com a documentação, uma para as ferramentas gráficas e uma para as anotações. É o equivalente barato de ter quatro monitores.

**Atalhos de teclado.** Os que mais economizam tempo já vêm configurados: **Super** abre o menu, **Alt+Tab** alterna janelas, **Alt+F2** executa um comando, e **Super+←/Super+→** encaixam a janela em metade da tela. O terminal suspenso, que aparece deslizando do topo da tela ao toque de uma tecla, costuma estar em **F12** — vale conferir e ajustar em *Configurações* → *Teclado* → *Atalhos de aplicativo*.

**O relógio e a bandeja.** A bandeja é onde aparecem os indicadores de rede (NetworkManager), volume e atualizações. Clicar no ícone de rede é a forma mais rápida de trocar de Wi-Fi ou conferir a conexão VPN.

## 4.4. O menu Kali: treze categorias e uma metodologia

O menu de aplicações do Kali não está organizado alfabeticamente por acaso. As categorias numeradas seguem, aproximadamente, a ordem em que as fases de um teste de invasão acontecem — o menu é uma metodologia disfarçada de menu. Figura 4.2 resume.

O agrupamento não é rígido — **nmap** aparece em mais de uma categoria, e ferramentas grandes servem a várias fases —, mas a lógica ajuda a navegar. Duas observações práticas: o menu mostra

#### 4. Primeiro contato: sessão, ambiente Xfce e organização do sistema

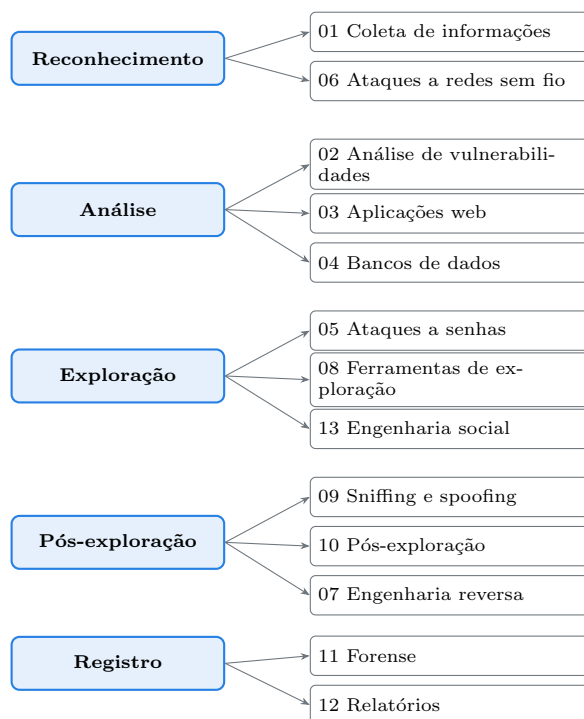


Figura 4.2.: As categorias do menu mapeiam, em linhas gerais, as fases de um trabalho de auditoria.

apenas as ferramentas **instaladas**, então uma instalação **headless** ou enxuta terá categorias quase vazias; e a maioria absoluta dessas ferramentas é de linha de comando, de modo que clicar no item do menu geralmente só abre um terminal com a ajuda do programa. O menu serve para descobrir o que existe, não para operar.

Para explorar o que está instalado sem depender do menu:

```
ls /usr/share/kali-menu/applications/ | head -20
ls /usr/bin | wc -l
```

### 4.5. O terminal e o prompt

O terminal é onde este manual vive a partir do próximo volume, mas vale decifrar o prompt já. Num Kali padrão, com Zsh, ele ocupa duas linhas: a primeira abre com um traçado em L, seguido de (**kali** + um símbolo redondo de separação + **kali**) e, entre colchetes, [**~**]; a segunda fecha o traçado e termina em **\$**. Em texto simples, o equivalente é:

```
(kali@kali)-[~]
$
```

A primeira linha traz o **usuário** (**kali**), um separador, o **nome da máquina** (**kali**) e, entre colchetes, o **diretório atual** — **~** é uma abreviação para o seu diretório pessoal, **/home/kali**. A segunda linha traz o **símbolo do prompt**: **\$** para usuário comum, **#** para root. Esse detalhe importa: **#** no começo da linha significa que você está com privilégio total e cada comando pode destruir o sistema.



Neste manual, os blocos de comando aparecem sem o prompt, para que possam ser copiados e colados diretamente. Quando o prompt aparece, é porque a saída do comando também está sendo mostrada.

O prompt do Kali muda de cor conforme o resultado do último comando — se algo falhou, ele fica vermelho. É um retorno visual que economiza um `echo $?` a cada passo, e vem da configuração do Zsh que o pacote `kali-defaults` instala. Personalizar isso é assunto do Volume 15.

Alguns atalhos do próprio emulador de terminal, que não são atalhos do shell:

- `Ctrl+Shift+T` abre uma aba nova; `Ctrl+Shift+W` fecha.
- `Ctrl+Shift+C` e `Ctrl+Shift+V` copiam e colam. O `Shift` é obrigatório porque `Ctrl+C` já tem significado antigo e importante: interromper o programa em execução.
- `Ctrl+Shift++` e `Ctrl+-` ajustam o tamanho da fonte.
- Selecionar com o mouse já copia, e o clique do botão do meio cola — herança do X11 que funciona em qualquer aplicação.

## 4.6. Organização do sistema: onde as coisas estão

Uma primeira orientação, que Capítulo 5 aprofunda: **não existem letras de unidade**. Há uma única árvore, começando em `/`, e todo dispositivo aparece pendurado em algum ponto dela. O seu diretório pessoal é `/home/kali`, abreviado como `~`, e é o único lugar onde você escreve sem privilégio elevado. Os programas ficam em `/usr/bin`; as configurações do sistema, em `/etc`; as configurações do seu usuário, em `~/.config`; os dados variáveis e logs, em `/var`.

Duas convenções valem desde já. Arquivos e diretórios cujo nome começa com ponto são **ocultos** — `ls` não os mostra, `ls -a` mostra. É onde vivem quase todas as configurações pessoais. E o sistema **diferencia maiúsculas de minúsculas**: `Documentos` e `documentos` são diretórios diferentes, o que surpreende quem vem do Windows.

Uma organização de trabalho que poupa dor de cabeça, criada logo no primeiro dia:

```
mkdir -p ~/lab/{ferramentas,alvos,notas,capturas}
mkdir -p ~/clientes
tree -L 2 ~/lab 2>/dev/null || ls -R ~/lab
```

Manter material de cliente fora do diretório de laboratório é higiene profissional, e separar por diretório é o mínimo. Criptografia por cliente é o passo seguinte, no Volume 8.

## 4.7. Os ajustes do primeiro dia

Quatro configurações evitam sofrimento desnecessário.

**Teclado brasileiro.** É o primeiro item, porque sem ele você não digita `/`, `?`, `ç` nem acentos corretamente — e o caractere `/` é onipresente no Linux.

```
sudo dpkg-reconfigure keyboard-configuration
sudo setupcon # aplica no console de texto
setxkbmap -model abnt2 -layout br # aplica na sessão gráfica, imediatamente
```

#### 4. Primeiro contato: sessão, ambiente Xfce e organização do sistema

Na tela do `dpkg-reconfigure`, escolha *Generic 105-key PC* e depois *Portuguese (Brazil)* — variante ABNT2 para teclados com a tecla `ç` dedicada. Para tornar permanente na sessão gráfica, use também *Configurações* → *Teclado* → *Layout*.

**Fuso horário.** Logs com horário errado atrapalham qualquer correlação de eventos:

```
timedatectl
sudo timedatectl set-timezone America/Sao_Paulo
timedatectl set-ntp true
```

**Idioma do sistema.** O Kali instala em inglês por padrão. Se quiser português, o pacote de localização se ajusta com `sudo dpkg-reconfigure locales`, escolhendo `pt_BR.UTF-8`. Vale um alerta: **mensagens de erro em português são muito mais difíceis de pesquisar**. A recomendação prática é manter o sistema em inglês e o teclado em português — que é o arranjo mais comum entre profissionais brasileiros de infraestrutura.

**Resolução e tela.** Se a janela da VM não redimensiona sozinha, faltam as ferramentas de convidado (Capítulo 3). Com elas instaladas, o ajuste é automático; sem elas, `xrandr` resolve manualmente.

## 4.8. No Kali

- **Xfce desde a versão 2019.4**, com tema próprio, papel de parede do dragão e o menu Whisker. Existem imagens alternativas com GNOME e KDE, mas o Xfce é o padrão testado e o que a documentação assume.
- **Zsh é o shell padrão desde a 2020.4**, com o prompt de duas linhas descrito acima. Bash continua instalado e disponível — `chsh -s /bin/bash` troca, e `kali-tweaks` também oferece a opção. As diferenças entre os dois são tema de Capítulo 6 e do Volume 2.
- **kali-tweaks** concentra os ajustes da distribuição: ramo de repositório, shell padrão, serviços de rede, modo undercover e hardening. É o primeiro lugar a procurar antes de sair editando arquivos.
- **O menu de ferramentas vem do pacote kali-menu**, que instala os arquivos `.desktop` em `/usr/share/kali-menu/applications/`. Categoria vazia significa ferramenta não instalada, não menu quebrado — instale o metapacote da área correspondente.
- **Serviços de rede não sobem no boot**. SSH, PostgreSQL (usado pelo Metasploit) e servidores web ficam desabilitados até você habilitá-los explicitamente. Ferramenta que “não conecta no banco” quase sempre é o PostgreSQL parado.
- **kali-undercover** troca a aparência do Xfce por uma imitação de Windows, útil em campo. Rodar de novo desfaz.
- **Os atalhos de captura de tela** usam `xfce4-screenshooter`, e as imagens caem em `~/Pictures` por padrão. Vale trocar o destino para a pasta do trabalho em curso — evidência espalhada é evidência perdida.

**Nota sobre a família RHEL.** Fedora e RHEL usam GNOME com GDM em vez de Xfce com LightDM, e já rodam Wayland por padrão. A cadeia conceitual é idêntica — firmware, GRUB, kernel, systemd, target gráfico, gerenciador de sessão, PAM, ambiente — e todos os comandos `systemctl`, `loginctl` e `timedatectl` funcionam igual.

## 4.9. Laboratório

Roteiro na VM recém-instalada. Os passos 1 a 6 são de leitura; do 7 em diante há alterações reversíveis.

### 1. Percorra a cadeia da sessão.

```
systemctl get-default
systemctl status lightdm --no-pager | head -8
loginctl list-sessions
loginctl show-session $XDG_SESSION_ID | head -15
```

Esperado: alvo padrão `graphical.target`, LightDM ativo, e uma sessão sua com `Type=x11`, `Seat=seat0` e `Remote=no`. Guarde o número da sessão.

### 2. Visite um console virtual. Pressione Ctrl+Alt+F3, faça login com o mesmo usuário e rode:

```
tty
echo "$XDG_SESSION_TYPE"
who
```

Esperado: `/dev/tty3`, tipo `tty` e duas sessões suas listadas pelo `who` — a gráfica e esta. Volte com Ctrl+Alt+F7 (ou F2) e confirme que a sessão gráfica continuou intacta. Essa é a saída de emergência quando o ambiente gráfico congela.

### 3. Inspeccione as variáveis da sessão gráfica.

```
echo "$DISPLAY | $XDG_SESSION_TYPE | $XDG_CURRENT_DESKTOP"
env | grep -E '^XDG_(CONFIG|DATA|CACHE)' | sort
```

Esperado: `DISPLAY` com algo como `:0.0`, tipo `x11`, ambiente `XFCE`. Compare com a saída do mesmo comando no console virtual do passo anterior, onde `DISPLAY` está vazia — é a razão de programas gráficos não abrirem por ali.

### 4. Identifique os processos do Xfce.

```
ps -e -o pid,comm --sort=comm | grep -E 'xf|lightdm'
```

Esperado: `xfce4-session`, `xfwm4`, `xfce4-panel`, `xfdesktop`, `xfsettingsd`, `lightdm` e `Xorg`. Relacione cada nome com o elemento visual correspondente na tela.

### 5. Provoque e conserte uma falha controlada do gerenciador de janelas.

```
xfwm4 --replace &
```

Esperado: as janelas piscam e voltam com as bordas redesenhadas. Esse é o comando que salva a sessão quando as janelas perdem barra de título ou param de responder ao arrastar — em vez de reiniciar a máquina, reinicia-se apenas a peça.

### 6. Leia o prompt. Observe as duas linhas, identifique usuário, máquina e diretório. Depois:

```
pwd
whoami
comando-que-nao-existe
```

#### 4. Primeiro contato: sessão, ambiente Xfce e organização do sistema

Esperado: `/home/kali`, `kali`, e uma mensagem de erro — repare que o prompt muda de cor depois do comando inválido. Rode `echo $?` em seguida e confirme um valor diferente de zero. Códigos de saída são tema do Volume 9.

#### 7. Configure o teclado brasileiro.

```
setxkbmap -model abnt2 -layout br
```

Esperado: efeito imediato. Teste digitando `~/ç?ãé` — se todos saírem certos, está resolvido para esta sessão. Torne permanente por *Configurações* → *Teclado* → *Layout*, ou com `sudo dpkg-reconfigure keyboard-configuration` seguido de reinício da sessão.

#### 8. Acerte o fuso horário e confirme.

```
sudo timedatectl set-timezone America/Sao_Paulo
timedatectl
date
```

Esperado: Time zone: `America/Sao_Paulo` e a hora local correta. Verifique também que `NTP service: active` — sincronização automática é o que mantém os logs comparáveis com os de outras máquinas.

#### 9. Crie a estrutura de trabalho e conheça os arquivos ocultos.

```
mkdir -p ~/lab/{alvos,capturas,notas} ~/clientes
ls ~
ls -a ~
ls -a ~/.config/xfce4
```

Esperado: o segundo `ls` revela vários itens que o primeiro escondeu, todos começando com ponto. Dentro de `~/.config/xfce4` está a configuração do seu ambiente — o diretório que você levaria para outra máquina para reproduzir a aparência atual.

10. **Feche com um snapshot.** Depois de ajustar teclado, fuso e estrutura de diretórios, crie um snapshot chamado `kali-configurado`. Esperado: um segundo ponto de retorno, agora com o ambiente utilizável, para não ter que repetir esses ajustes toda vez que um laboratório sujar o sistema.

## 4.10. Resumo

- **Do boot ao prompt existe uma cadeia clara:** firmware, GRUB, kernel, systemd, graphical.target, LightDM, PAM e sessão Xfce. O ambiente gráfico é a última camada e a única dispensável — `multi-user.target` entrega um sistema completo sem ela.
- **Consoles virtuais (Ctrl+Alt+F1 a F6) são a saída de emergência,** um terminal ligado direto ao kernel que continua funcionando quando a sessão gráfica trava.
- **Sessão é um objeto real do sistema,** rastreado pelo `loginctl`, com variáveis (`DISPLAY`, `XDG_SESSION_TYPE`, `XDG_*`) das quais dependem os programas gráficos e a localização das suas configurações.
- **Xfce é um conjunto de processos independentes** — `xfwm4`, `xfce4-panel`, `xfdesktop`, `xfsettingsd`, `Thunar` —, e saber os nomes permite reiniciar só a peça quebrada em vez do sistema inteiro.
- **O menu do Kali é uma metodologia disfarçada:** as treze categorias numeradas seguem as fases de um trabalho de auditoria, e categoria vazia significa ferramenta não instalada.

- **O prompt do Kali carrega informação:** usuário, máquina, diretório e um símbolo final que distingue usuário comum (\$) de root (#), com cor que muda quando o último comando falha.
- **Quatro ajustes valem o primeiro dia:** teclado ABNT2, fuso horário com NTP, sistema em inglês com teclado em português, e uma estrutura de diretórios que separa laboratório de material de cliente.



## 5. A estrutura de diretórios do Linux (FHS)

O alerta chegou às onze da noite: partição raiz com 96% de uso num servidor que não deveria crescer. Você entra na máquina e faz a pergunta óbvia — onde está o espaço? Se a resposta for `/var/log`, o problema é um serviço tagarela e a solução é rotação de logs. Se for `/var/lib/docker`, são imagens acumuladas. Se for `/var/cache/apt`, um `apt clean` resolve em dez segundos. Se for `/home`, alguém guardou coisa que não devia. Se for `/tmp` num sistema em que `/tmp` não é memória, algum processo está vazando arquivos temporários. **Cada diretório conta uma história diferente**, e saber ler essa árvore transforma um “disco cheio” genérico num diagnóstico específico.

A boa notícia é que essa árvore não é arbitrária. Ela segue um padrão documentado, o **Filesystem Hierarchy Standard** (Linux Foundation, 2015), que a maioria esmagadora das distribuições respeita. Aprender o FHS uma vez serve para Debian, Kali, Ubuntu, Fedora, RHEL, Alpine e para o contêiner que sobe no Kubernetes. Este capítulo percorre a árvore inteira, explica a lógica por trás dela e mostra os comandos que respondem “onde é que fica isso?” sem depender de memória.

### 5.1. Uma árvore só, sem letras de unidade

A diferença estrutural mais importante em relação ao Windows não é de nomes: é de topologia. No Windows, cada dispositivo ganha uma letra e uma árvore própria — `C:`, `D:`, `E:`. No Linux existe **uma única árvore**, com raiz em `/`, e todo dispositivo adicional é **montado** em algum diretório dessa árvore, passando a fazer parte dela.

Isso significa que o caminho de um arquivo não diz em qual disco ele está. `/home/kali/notas.txt` pode estar no mesmo disco da raiz, num segundo SSD, num volume LVM criptografado ou num compartilhamento de rede — e nenhum programa precisa saber a diferença. Quem responde a pergunta é o comando `findmnt`:

```
findmnt --real
df -hT
lsblk -f
```

O `findmnt` mostra a árvore de montagens: o que está montado, onde e a partir de qual dispositivo. O `df -hT` acrescenta ocupação e tipo de sistema de arquivos. Essa indireção — caminho lógico separado de dispositivo físico — é o que permite mover `/var` para um disco novo sem que nenhum programa perceba, e é a base do modelo de armazenamento que o Volume 8 detalha.

### 5.2. A árvore completa

Figura 5.1 mostra os diretórios de primeiro nível e o que cada um guarda.

Vale percorrer um por um, porque cada um tem uma regra de uso que evita erro depois.

## 5. A estrutura de diretórios do Linux (FHS)

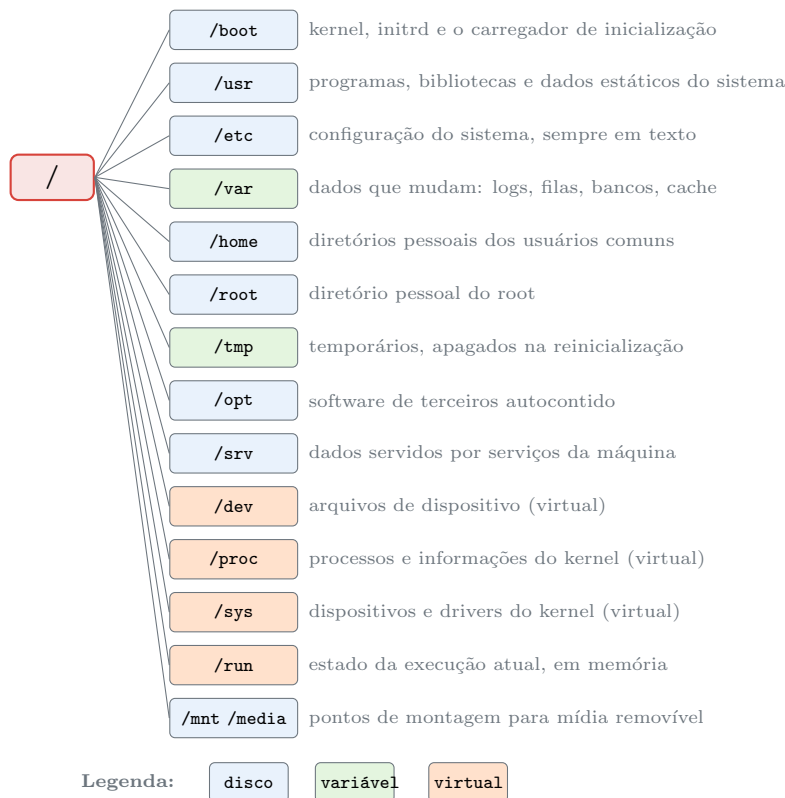


Figura 5.1.: A árvore de primeiro nível. Três dos diretórios não existem em disco: são interfaces do kernel montadas em memória.

### 5.2.1. /usr — os programas e o que é estático

/usr guarda tudo que é instalado pelo gerenciador de pacotes e não muda durante a operação normal: executáveis em /usr/bin, ferramentas administrativas em /usr/sbin, bibliotecas em /usr/lib, e dados que não são programas — ícones, páginas de manual, dicionários, listas de palavras — em /usr/share.

Você vai notar que /bin, /sbin e /lib também existem na raiz. Em sistemas atuais eles são apenas **links simbólicos** para dentro de /usr:

```
ls -ld /bin /sbin /lib /lib64
```

A saída mostra /bin -> usr/bin e assim por diante. Essa unificação, apelidada de *usrmerge*, resolveu um problema histórico: originalmente /bin continha o essencial para o sistema iniciar e /usr podia estar num disco separado montado depois. Com *initramfs*, essa separação perdeu sentido e virou fonte de bugs. Hoje o Debian e derivados exigem a unificação. Consequência prática: /bin/ls e /usr/bin/ls são o mesmo arquivo, e você não precisa mais decorar em qual dos dois procurar.

Dentro de /usr há dois subdiretórios especiais. /usr/local é reservado para software que o **administrador** instalou fora do gerenciador de pacotes — compilado do fonte, baixado de um projeto. A regra é limpa: o **apt** nunca escreve em /usr/local, então nada do que você colocou ali será sobrescrito por atualização. E /usr/src guarda fontes, tipicamente cabeçalhos do kernel usados para compilar módulos.



### 5.2.2. /etc — a configuração, sempre em texto

/etc guarda a configuração de todo o sistema. Duas regras definem o diretório: **nada de binários executáveis** (o FHS é explícito) e, por tradição fortíssima do Unix, **tudo em texto simples**. Isso é o que permite versionar /etc no Git, comparar duas máquinas com diff, e corrigir um sistema quebrado com um editor de texto num console de emergência.

```
ls /etc | head -30
wc -l /etc/passwd /etc/fstab /etc/hostname
ls /etc/apt/ /etc/systemd/
```

Alguns arquivos que você vai encontrar muitas vezes: /etc/passwd e /etc/group (usuários e grupos, Volume 4), /etc/fstab (o que montar no boot, Volume 8), /etc/apt/sources.list (repositórios, Volume 6), /etc/systemd/system/ (serviços, Volume 7), /etc/hosts e /etc/resolv.conf (nomes e DNS, Volume 13).

Um padrão do Debian que aparece por toda parte: em vez de um arquivo monolítico, muitos programas leem um **diretório terminado em .d**, e cada arquivo dentro dele é um fragmento de configuração. /etc/apt/sources.list.d/, /etc/sudoers.d/, /etc/systemd/system/\*.service.d/. A vantagem é que um pacote pode adicionar sua configuração sem editar um arquivo que pertence a outro pacote.

### 5.2.3. /var — tudo que cresce

/var é onde vive o que **muda de tamanho durante a operação**. É o diretório que enche o disco, e por isso o primeiro a investigar num alerta de espaço.

- /var/log — logs do sistema e dos serviços. journalctl lê o log binário do systemd em /var/log/journal; serviços tradicionais escrevem texto em arquivos como /var/log/auth.log e /var/log/syslog.
- /var/lib — estado persistente dos programas: banco de dados do dpkg em /var/lib/dpkg, dados do PostgreSQL, imagens do Docker.
- /var/cache — dados recriáveis. /var/cache/apt/archives guarda os .deb baixados e é a primeira coisa a limpar (apt clean) quando falta espaço.
- /var/spool — filas: impressão, e-mail, cron, at.
- /var/tmp — temporários que **sobrevivem à reinicialização**, ao contrário de /tmp.
- /var/www — raiz de conteúdo web, por convenção do Debian.

### 5.2.4. /home e /root

/home/<usuário> é o diretório pessoal de cada usuário comum, o único lugar onde ele escreve sem privilégio elevado. O atalho para o seu é o til, e a variável de ambiente correspondente é HOME.

O root não fica em /home: tem /root, na raiz. A razão é histórica e prática — se /home estiver num disco separado que falhou em montar, o administrador ainda precisa de um diretório pessoal para trabalhar e consertar.

Dentro do diretório pessoal, a convenção moderna concentra configurações em três lugares definidos pela especificação XDG: ~/.config (configurações), ~/.local/share (dados) e ~/.cache (cache descartável). ~/.local/bin é onde ficam programas instalados só para o seu usuário, e costuma já estar no PATH.

## 5. A estrutura de diretórios do Linux (FHS)

### 5.2.5. /tmp e /var/tmp

/tmp é para arquivos temporários de vida curta. Em sistemas modernos ele frequentemente é um **tmpfs** — ou seja, **vive em memória RAM** e some por completo na reinicialização. Confira na sua máquina:

```
findmnt /tmp
df -hT /tmp /var/tmp
```

Se o tipo aparecer como **tmpfs**, tudo ali consome memória, e gravar um arquivo de 8 GB em /tmp numa VM de 4 GB não vai terminar bem. É um detalhe que morde ao trabalhar com capturas grandes de tráfego ou imagens forenses — o destino correto é /var/tmp ou um diretório de trabalho seu.

Ambos são graváveis por qualquer usuário, o que exigiria uma proteção extra para que um usuário não apague o arquivo de outro. Essa proteção existe, chama-se *sticky bit*, e é tema do Volume 4.

### 5.2.6. /opt e /srv

/opt é para software de terceiros **autocontido**: o programa inteiro num único diretório, /opt/nome-do-produto, com seus próprios binários, bibliotecas e configuração. É onde acabam ferramentas comerciais e programas baixados prontos que não seguem o FHS. Boa parte das ferramentas que um pentester instala manualmente, clonando de um repositório, vai naturalmente para /opt.

/srv é para dados **servidos** pela máquina a terceiros — o conteúdo de um site, os arquivos de um FTP. Na prática o Debian usa /var/www para web, e /srv acaba pouco utilizado.

### 5.2.7. /boot

Guarda o que é necessário para a máquina iniciar: o kernel (**vmlinuz-\***), o **initrd** correspondente, o mapa de símbolos e a configuração do GRUB. Em máquinas UEFI há ainda /boot/efi, uma partição em FAT32 com os carregadores.

```
ls -lh /boot
```

Vale saber de um problema comum: kernels antigos não são removidos automaticamente, e uma partição /boot pequena enche depois de algumas atualizações. Quando isso acontece, o **apt** falha no meio de uma atualização de kernel — situação desconfortável e, felizmente, resolvida com **apt autoremove**.

### 5.2.8. Os três diretórios que não existem em disco

`/proc`, `/sys` e `/dev` são **sistemas de arquivos virtuais**: não ocupam disco nenhum, são gerados pelo kernel em tempo real e desaparecem ao desligar. Eles são a materialização da filosofia “tudo é arquivo” — em vez de uma API especial, o kernel expõe seu estado como arquivos que você lê com `cat`.

`/proc` expõe processos e informações do kernel. Cada processo em execução tem um diretório com seu PID:

```
cat /proc/cpuinfo | head -12
cat /proc/meminfo | head -5
ls /proc/self/
cat /proc/self/status | head -8
```

`/proc/self` é um atalho mágico: dentro dele, o kernel mostra o processo que está lendo. `/proc/<pid>/cmdline` traz a linha de comando completa de um processo, `/proc/<pid>/environ` traz suas variáveis de ambiente e `/proc/<pid>/fd/` lista seus arquivos abertos. Isso é ouro para diagnóstico — e também para análise forense, motivo pelo qual `/proc` aparece bastante no Volume 14.

`/sys` expõe dispositivos, drivers e parâmetros do kernel de forma estruturada. É por ali que se lê a temperatura de um sensor, se descobre se uma interface de rede está com o cabo conectado, ou se ajusta o comportamento de um driver.

```
cat /sys/class/net/*/operstate
ls /sys/class/
```

`/dev` contém os arquivos de dispositivo. `/dev/sda` é um disco inteiro, `/dev/sda1` uma partição, `/dev/null` descarta tudo que recebe, `/dev/zero` fornece bytes nulos infinitos, `/dev/random` e `/dev/urandom` fornecem aleatoriedade criptográfica, `/dev/tty` é o terminal atual.

```
ls -l /dev/null /dev/zero /dev/sda 2>/dev/null
echo "isto some" > /dev/null
head -c 16 /dev/urandom | xxd
```

O primeiro caractere na listagem longa desses arquivos não é `-` nem `d`: é `c` (dispositivo de caractere) ou `b` (dispositivo de bloco). Tipos de arquivo são o assunto de abertura do Volume 3.

`/run` completa o grupo: é um `tmpfs` com o estado da execução atual — arquivos de PID, sockets de comunicação entre processos, informação de montagens temporárias. Substituiu o antigo `/var/run`, que hoje é um link simbólico para ele.

## 5.3. A lógica: duas perguntas organizam tudo

O FHS não é uma lista decorada; ele deriva de duas perguntas sobre cada arquivo. A primeira: **muda durante a operação normal, ou é estático?** A segunda: **pode ser compartilhado com outras máquinas, ou é específico deste computador?** Figura 5.2 organiza os diretórios principais nesses dois eixos.

## 5. A estrutura de diretórios do Linux (FHS)

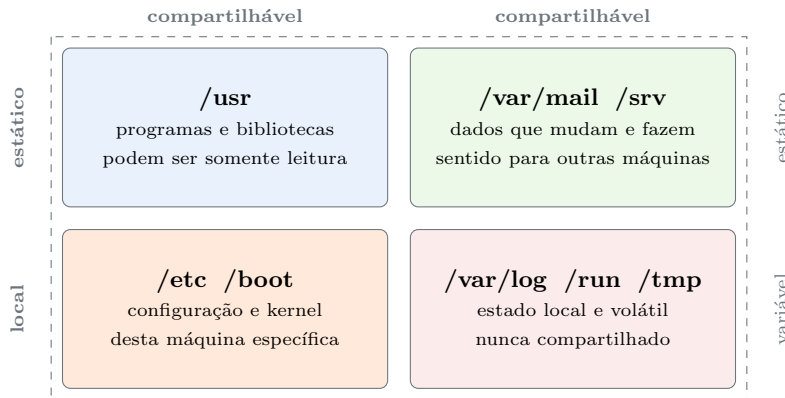


Figura 5.2.: Cada diretório ocupa um lugar definido nos eixos “muda ou não” e “serve a outras máquinas ou não”. Daí decorre o resto.

Essa classificação tem consequência operacional direta. `/usr` ser estático e compartilhável é o que permite montá-lo somente-leitura num sistema endurecido, ou servi-lo por rede a várias estações. `/etc` ser específico da máquina é o que faz dele o alvo prioritário de backup. `/var/log` ser variável e local é o que exige rotação e monitoramento de espaço. E `/tmp` ser volátil é o que garante que nada guardado ali sobreviva ao próximo reinício.

### 5.4. Onde colocar o que você instalou

Uma dúvida recorrente com resposta clara:

| Origem do software                          | Destino correto                                                                    |
|---------------------------------------------|------------------------------------------------------------------------------------|
| Gerenciador de pacotes ( <code>apt</code> ) | <code>/usr/bin</code> , <code>/usr/lib</code> , <code>/usr/share</code> — não mexa |
| Compilado do fonte, para todo o sistema     | <code>/usr/local</code>                                                            |
| Programa de terceiros autocontido           | <code>/opt/&lt;nome&gt;</code>                                                     |
| Script ou ferramenta só para você           | <code>~/.local/bin</code>                                                          |
| Dados servidos a terceiros                  | <code>/srv</code> ou <code>/var/www</code>                                         |

A regra que resolve 90% dos casos: **nunca escreva à mão dentro de `/usr/bin` ou `/usr/lib`**. Esses diretórios pertencem ao gerenciador de pacotes, e um arquivo estranho ali será sobrescrito ou apagado sem aviso na próxima atualização — ou pior, causará conflito silencioso. Use `/usr/local` e o problema desaparece, porque ele vem antes na busca do `PATH` (assunto de Capítulo 6).

### 5.5. No Kali

- **As ferramentas ficam em `/usr/bin`, como qualquer programa.** Essa é a herança mais importante da migração do BackTrack para Debian: acabou o `/pentest/`. `which nmap` responde `/usr/bin/nmap`, e `dpkg -S /usr/bin/nmap` diz qual pacote o instalou — rastreabilidade que o BackTrack não tinha.

- **/usr/share/wordlists é o diretório mais visitado do Kali.** É onde ficam as listas de palavras, incluindo o link para `rockyou.txt.gz`. O arquivo vem compactado e precisa ser descompactado uma vez: `sudo gunzip /usr/share/wordlists/rockyou.txt.gz`. Ali também estão as listas do `dirb`, do `wfuzz` e do `seclists`, se instalado.
- **/usr/share/<ferramenta> guarda os dados de cada programa.** Assinaturas do `nmap` em `/usr/share/nmap/`, scripts NSE em `/usr/share/nmap/scripts/`, módulos do Metasploit em `/usr/share/metasploit-framework/`. Quando a documentação de uma ferramenta cita “o diretório de dados”, é ali.
- **/var/lib/postgresql guarda o banco do Metasploit.** O `msfdb` depende do PostgreSQL, que no Kali não inicia sozinho — daí o erro clássico de conexão recusada ao abrir o `msfconsole` pela primeira vez.
- **/opt é o destino natural das ferramentas clonadas de repositórios.** Grande parte do ferramental de segurança não está empacotado; clonar em `/opt/<ferramenta>` mantém tudo num lugar previsível e fora do alcance do `apt`.
- **/var/log/ num Kali é menos movimentado do que num servidor,** porque poucos serviços rodam. Ainda assim, `/var/log/auth.log` registra cada uso de `sudo`, e vale conhecer o arquivo antes de precisar dele.
- **Cuidado com /tmp em memória.** Numa VM com pouca RAM, salvar uma captura de tráfego longa em `/tmp` derruba o sistema por falta de memória. Use `~/lab/capturas` ou `/var/tmp`.

**Nota sobre a família RHEL.** A árvore é a mesma — mérito do FHS. As diferenças estão no conteúdo: `/etc/yum.repos.d/` no lugar de `/etc/apt/`, `/etc/sysconfig/` para configurações de serviço que o Debian espalha em outros lugares, `/var/log/messages` e `/var/log/secure` no lugar de `/var/log/syslog` e `/var/log/auth.log`.

## 5.6. Laboratório

Roteiro de exploração. Todos os passos são de leitura, exceto o 8 e o 9, que criam arquivos em locais controlados.

### 1. Veja a árvore e a origem de cada ramo.

```
ls -l /
findmnt --real
df -hT
```

Esperado: os diretórios de primeiro nível, e no `findmnt` bem menos linhas do que diretórios — sinal de que a maior parte da árvore vive no mesmo sistema de arquivos. Identifique quais pontos de montagem existem separados na sua instalação.

### 2. Confirme o `usrmerge`.

```
ls -ld /bin /sbin /lib /lib64
readlink -f /bin/ls
```

Esperado: `/bin -> usr/bin` e `readlink` resolvendo para `/usr/bin/ls`. A conclusão: os dois caminhos apontam para o mesmo arquivo, e não há mais escolha a fazer entre eles.

### 3. Descubra quem ocupa o disco.

## 5. A estrutura de diretórios do Linux (FHS)

```
sudo du -xh --max-depth=1 / 2>/dev/null | sort -rh | head -12
sudo du -xh --max-depth=1 /var 2>/dev/null | sort -rh | head -8
```

Esperado: `/usr` como maior consumidor num Kali recém-instalado, e dentro de `/var` provavelmente `/var/lib` e `/var/cache`. A opção `-x` impede que a contagem atravesse para outros sistemas de arquivos — sem ela, `/proc` e `/sys` poluem o resultado. **Esse é o comando do alerta de disco cheio da abertura do capítulo.**

### 4. Explore `/etc` e o padrão dos diretórios `.d`.

```
ls /etc | wc -l
file /etc/passwd /etc/fstab /etc/hostname
ls /etc/apt/sources.list.d/ /etc/sudoers.d/ 2>/dev/null
```

Esperado: algumas centenas de entradas em `/etc`, todas identificadas como texto pelo `file`. Confirme com os próprios olhos a regra: nada de binário em `/etc`.

### 5. Prove que `/proc` não existe em disco.

```
findmnt /proc
cat /proc/uptime
cat /proc/self/status | head -6
ls /proc/self/fd/
```

Esperado: tipo `proc`, e um `/proc/uptime` que muda a cada leitura. Repare que `/proc/self` mostra o próprio processo que está lendo — rode `cat /proc/self/cmdline` duas vezes e observe que o PID no caminho resolvido é diferente a cada execução.

### 6. Leia o hardware pelo sistema de arquivos.

```
grep 'model name' /proc/cpuinfo | head -2
grep MemTotal /proc/meminfo
cat /sys/class/net/*/operstate
ls /sys/class/ | head -20
```

Esperado: modelo da CPU, memória total, e o estado (`up` ou `down`) de cada interface de rede. Nenhuma ferramenta especial envolvida — apenas leitura de arquivos.

### 7. Experimente os dispositivos especiais.

```
ls -l /dev/null /dev/zero /dev/urandom
echo "isto desaparece" > /dev/null
head -c 12 /dev/urandom | xxd
head -c 8 /dev/zero | xxd
```

Esperado: os três aparecem com `c` na primeira coluna (dispositivo de caractere), o `echo` não produz saída nenhuma, `/dev/urandom` devolve bytes diferentes a cada execução e `/dev/zero` devolve sempre zeros. Redirecionar para `/dev/null` é o idioma universal de “descarte esta saída”.

### 8. Instale um script no lugar certo.

```
mkdir -p ~/.local/bin
printf '#!/bin/bash\nnecho "rodando do diretorio pessoal"\n' >
↵ ~/.local/bin/meu-teste
chmod +x ~/.local/bin/meu-teste
```

```
meu-teste
which meu-teste
```

Esperado: o script executa pelo nome e o `which` responde `/home/kali/.local/bin/meu-teste`. Se o comando não for encontrado, `~/.local/bin` não está no PATH desta sessão — feche e reabra o terminal, ou consulte Capítulo 6.

#### 9. Compare com o lugar de todo o sistema.

```
sudo cp ~/.local/bin/meu-teste /usr/local/bin/meu-teste
which -a meu-teste
dpkg -S /usr/local/bin/meu-teste 2>&1 | head -2
```

Esperado: `which -a` lista os dois caminhos, na ordem em que o shell os procura. E o `dpkg -S` responde que o arquivo não pertence a pacote nenhum — exatamente o que se espera de `/usr/local`. Compare com `dpkg -S /usr/bin/nmap`, que devolve o pacote responsável.

#### 10. Verifique se `/tmp` está em memória.

```
findmnt /tmp
df -hT /tmp /var/tmp
```

Esperado: se o tipo for `tmpfs`, todo arquivo ali consome RAM e some ao reiniciar. Anote esse fato — ele explica a diferença de comportamento entre `/tmp` e `/var/tmp` e evita um travamento por falta de memória mais adiante.

#### 11. Encontre os dados das ferramentas do Kali.

```
ls /usr/share/wordlists/
ls /usr/share/nmap/scripts/ | wc -l
dpkg -S /usr/bin/nmap
```

Esperado: os links de listas de palavras, algumas centenas de scripts NSE e a confirmação de que o `nmap` veio do pacote `nmap`. Essa rastreabilidade — cada arquivo pertencendo a um pacote identificável — é o ganho concreto da mudança para Debian discutida em Capítulo 2.

## 5.7. Resumo

- **Existe uma única árvore, com raiz em `/`**, e todo dispositivo é montado em algum ponto dela. O caminho de um arquivo não revela em qual disco ele está; quem responde isso é o `findmnt`.
- **O FHS é um padrão documentado** que quase todas as distribuições respeitam, o que torna esse conhecimento transferível entre Debian, Kali, Ubuntu, RHEL e contêineres.
- **Dois eixos organizam a árvore**: estático versus variável, compartilhável versus local. Daí decorrem `/usr` somente-leitura, `/etc` como alvo prioritário de backup e `/var/log` como candidato eterno a encher o disco.
- **`/proc`, `/sys` e `/dev` não existem em disco**: são interfaces do kernel expostas como arquivos, e são a materialização mais direta do princípio “tudo é arquivo”.
- **Cada origem de software tem um destino correto**: `apt` escreve em `/usr`, você escreve em `/usr/local`, programas autocontidos vão para `/opt`, e o que é só seu vai para `~/.local/bin`. Escrever à mão em `/usr/bin` é receita para conflito silencioso.

## 5. A estrutura de diretórios do Linux (FHS)

- **No Kali, as ferramentas são pacotes comuns em /usr/bin**, com dados em /usr/share/<ferramenta> e listas de palavras em /usr/share/wordlists — o fim do /pentest/ do BackTrack e o começo da rastreabilidade por pacote.



## 6. Anatomia de um comando: shell, argumentos, opções e sintaxe

Você copia de um tutorial a linha que deveria resolver o problema, cola no terminal, aperta Enter — e recebe uma mensagem que não faz sentido nenhum. Talvez `unknown option`, talvez `No such file or directory` apontando para um arquivo que existe, talvez o comando execute e faça algo diferente do prometido. Em quase todos esses casos, o problema não está no programa: está no que o **shell** fez com a sua linha antes de o programa sequer começar a rodar.

Um exemplo que pega todo mundo pelo menos uma vez:

```
rm arquivo com espaco.txt
```

Isso não apaga um arquivo chamado `arquivo com espaco.txt`. Apaga três arquivos — `arquivo`, `com` e `espaco.txt` — ou reclama que nenhum deles existe. O `rm` nunca viu a linha que você digitou; ele recebeu três argumentos separados, porque quem separou foi o shell, e o shell separa por espaço.

Este capítulo desmonta a linha de comando peça por peça: quem lê, quem separa, quem procura o programa, quem executa, quem informa o resultado. É o capítulo que transforma “digitar comandos” em “entender o que está acontecendo”, e todo o Volume 2 depende dele.

### 6.1. O que acontece entre o Enter e a saída

Quando você aperta Enter, o shell executa uma sequência de etapas bem definida. Figura 6.1 mostra o caminho.

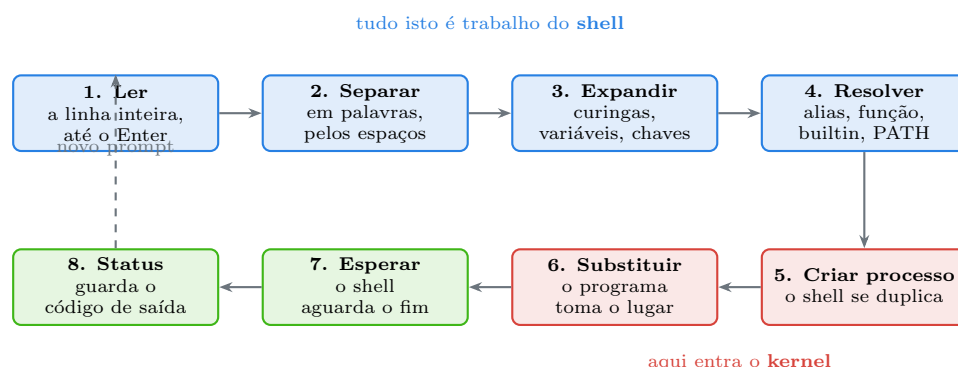


Figura 6.1.: O programa só começa a existir na etapa 6. Tudo que dá errado antes disso é responsabilidade do shell.

As quatro primeiras etapas são inteiramente do shell, e é nelas que mora a maior parte da confusão de quem está começando. A etapa 5 usa a chamada de sistema `fork`, que duplica o

## 6. Anatomia de um comando: shell, argumentos, opções e sintaxe

processo do shell; a 6 usa `execve`, que substitui o programa duplicado pelo programa pedido, mantendo o mesmo processo. É por isso que um comando executado no terminal é **filho** do shell — e é isso que permite ao shell esperar por ele e recolher o resultado (Kerrisk, 2010).

### 6.2. Anatomia da linha

Toda linha de comando tem a mesma estrutura: um nome de comando seguido de zero ou mais palavras. O programa recebe essas palavras numeradas, e é ele — não o shell — quem decide o que cada uma significa. Figura 6.2 diseca um exemplo real.

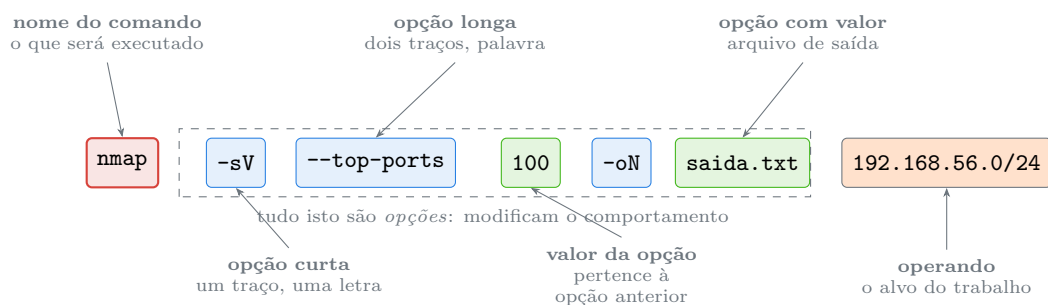


Figura 6.2.: O shell entrega essas sete palavras ao `nmap`; a interpretação de cada uma é decisão do programa.

O ponto essencial: **para o shell, todas as palavras depois do nome do comando são iguais**. Ele não sabe que `-sV` é opção e `192.168.56.0/24` é alvo. Quem sabe é o `nmap`, ao ler seu vetor de argumentos. Isso explica por que a mesma sintaxe funciona diferente em programas diferentes, e por que não existe uma regra universal de “sempre pode inverter a ordem”.

### 6.3. Onde o shell procura o comando

Ao chegar na etapa 4, o shell precisa decidir o que executar. E ele consulta cinco possibilidades, **nesta ordem**:

1. **Palavra-chave** da linguagem do shell: `if`, `for`, `while`, `case`, `function`. Não são programas, são sintaxe.
2. **Alias**: um apelido definido por você ou pela distribuição.
3. **Função** do shell, definida na sessão ou num arquivo de inicialização.
4. **Comando embutido** (*builtin*): implementado dentro do próprio shell — `cd`, `echo`, `export`, `pwd`, `read`, `type`.
5. **Executável no PATH**: um arquivo em disco, procurado nos diretórios listados na variável `PATH`.

O comando que revela essa decisão é o `type`:

```
type -a ls
type -a cd
type -a echo
type -a nmap
type -a if
```

Num Kali padrão, `type -a ls` costuma revelar que `ls` é primeiro um **alias** (com opções de cor já embutidas) e só depois o executável `/usr/bin/ls`. Isso tem consequência prática enorme: quando você escreve um script e chama `ls`, o alias **não** vale — aliases não são herdados por scripts. O comportamento muda, e a origem da diferença é essa.

`echo` aparece duas vezes: como builtin do shell e como `/usr/bin/echo`. Os dois têm diferenças sutis de opções, e o builtin é o que roda. Já `cd` só pode ser builtin — um programa externo não conseguiria mudar o diretório do shell que o chamou, porque mudaria apenas o seu próprio.

### 6.3.1. O PATH e por que o diretório atual não está nele

`PATH` é uma variável com uma lista de diretórios separados por dois-pontos. O shell percorre essa lista **da esquerda para a direita** e usa o primeiro que contiver um arquivo executável com o nome pedido:

```
echo "$PATH"
echo "$PATH" | tr ':' '\n'
which -a python3
```

A ordem importa. Se `/usr/local/bin` vem antes de `/usr/bin` — e vem, em qualquer instalação normal —, um programa que você compilou e instalou em `/usr/local/bin` tem precedência sobre o da distribuição. É exatamente o comportamento desejado, e é o motivo de `/usr/local` existir, como visto em Capítulo 5.

Repare no que **não** está no `PATH`: o diretório atual. No Windows, um executável na pasta corrente é encontrado automaticamente; no Linux, não. Por isso um script no diretório onde você está exige o prefixo explícito:

```
./meu-script.sh
```

Isso não é inconveniência — é segurança. Se `.` estivesse no `PATH`, bastaria alguém deixar um arquivo chamado `ls` num diretório gravável para que o seu próximo `ls` executasse o programa dele. Com um administrador logado como root, isso é escalada de privilégio imediata. A regra vale para sempre: **nunca acrescente `.` ao `PATH`**.

O shell também mantém um cache dos caminhos já resolvidos, o que ocasionalmente confunde:

```
hash -r      # esquece o cache; no Zsh, rehash
```

Se você acabou de instalar um programa e o shell insiste em dizer que ele não existe, essa é a linha que resolve.

## 6.4. Opções: as convenções que existem e as que não existem

Não há uma regra universal, mas há três convenções fortes.

**Opções curtas POSIX:** um traço e uma letra, como `-l`, `-a`, `-v`. Podem ser **agrupadas** quando não têm valor: `ls -l -a -h` equivale a `ls -lah`. Quando uma opção curta exige valor, ele vem como palavra seguinte (`-o saida.txt`) ou colado (`-osaida.txt`), conforme o programa. Se agrupar uma opção que exige valor, ela precisa ser a última do grupo.

**Opções longas GNU:** dois traços e uma palavra, como `--all`, `--recursive`, `--help` (Free Software Foundation, 2024a). São mais legíveis e melhores para scripts, porque um `--recursive` daqui a um ano ainda se explica sozinho, enquanto `-r` exige consulta ao manual. O valor vem com sinal de igual (`--output=saida.txt`) ou como palavra seguinte (`--output saida.txt`), e a maioria dos programas GNU aceita as duas formas.

**O separador `--`:** marca o **fim das opções**. Tudo depois dele é tratado como operando, mesmo que comece com traço. Esse é o antídoto para um problema real:

```
# criar um arquivo cujo nome começa com traco
touch ./-arquivo-estranho

# isto falha: o rm interpreta -arquivo-estranho como opcoes
rm -arquivo-estranho

# isto funciona
rm -- -arquivo-estranho
# ou
rm ./-arquivo-estranho
```

Vale conhecer as exceções que quebram a convenção, porque são comuns. O `tar` histórico aceita opções sem traço nenhum (`tar xzf pacote.tar.gz`). O `ps` aceita três estilos diferentes de sintaxe — Unix (`ps -ef`), BSD (`ps aux`) e GNU (`ps --pid 1`) — e `ps aux` sem traço não é erro de digitação. O `dd` usa `opcao=valor` sem traço algum. O `find` usa opções de uma letra com traço único e nome longo (`-name`, `-type`, `-exec`), e é sensível à ordem dos argumentos. Quando a sintaxe parecer estranha, o manual é a resposta — tema de Capítulo 7.

## 6.5. O código de saída

Todo programa, ao terminar, devolve um número inteiro ao seu pai. Por convenção universal no Unix, **zero significa sucesso** e qualquer outro valor significa algum tipo de falha. O shell guarda esse número na variável especial de status:

```
ls /etc > /dev/null
echo $?          # 0

ls /nao-existe 2> /dev/null
echo $?          # 2

grep raiz-inexistente /etc/passwd
echo $?          # 1
```

Alguns valores têm significado convencional: 1 é erro genérico, 2 costuma ser erro de uso, 126 significa que o arquivo existe mas não é executável, 127 significa comando não encontrado, e valores acima de 128 indicam término por sinal (130 é interrupção por `Ctrl+C`, que é o sinal 2 somado a 128).

O código de saída é o que dá sentido aos operadores de encadeamento:

```
comando_a ; comando_b      # executa b sempre, independente de a
comando_a && comando_b      # executa b só se a teve sucesso
comando_a || comando_b     # executa b só se a falhou
comando_a &                # executa a em segundo plano, devolve o prompt
```

Na prática, `&&` é o que separa uma linha correta de uma linha perigosa:

```
cd /tmp/trabalho && rm -rf ./*      # seguro: só apaga se o cd funcionou
cd /tmp/trabalho ; rm -rf ./*      # perigoso: se o cd falhar, apaga o
↪ diretório atual
```

#### ⚠ A diferença entre `&&` e `;` já destruiu sistemas

A segunda linha acima, com o `cd` falhando por um erro de digitação no caminho, executa `rm -rf ./*` no diretório em que você estiver — que pode ser o seu diretório pessoal ou a raiz. Adote `&&` como padrão em qualquer sequência em que o segundo comando pressuponha o sucesso do primeiro. Em scripts, o mecanismo definitivo é `set -e`, tema do Volume 12.

## 6.6. Quatro armadilhas que valem conhecer agora

**Espaços em nomes de arquivo.** O caso da abertura. A solução é aspas ou escape:

```
ls "arquivo com espaco.txt"
ls arquivo\ com\ espaco.txt
```

Em scripts, a regra que evita 90% dos bugs é sempre colocar variáveis entre aspas duplas: `rm "$arquivo"`, nunca `rm $arquivo`. O assunto tem capítulo próprio no Volume 9.

**sudo não se estende ao redirecionamento.** Esta linha falha mesmo com `sudo`:

```
sudo echo "linha" > /etc/arquivo-protegido
```

O motivo é que o redirecionamento `>` é feito **pelo shell**, que roda como você, antes de o `sudo` sequer começar. O `sudo` eleva o `echo`, não o redirecionamento. As soluções corretas usam `tee` ou um shell elevado:

```
echo "linha" | sudo tee -a /etc/arquivo-protegido > /dev/null
```

**Aliases mudam o comportamento do que você acha que está rodando.** Se `rm` está com alias para `rm -i`, você se acostuma com a confirmação — e perde a rede de proteção em qualquer outra máquina ou dentro de um script. Para forçar o programa real, ignorando o alias, prefixe com barra invertida ou use `command`:

```
\rm arquivo
command rm arquivo
```

**Variável vazia em caminho destrutivo.** Em `rm -rf "$DIR"/`, se `DIR` estiver vazia a linha vira `rm -rf /`. Sistemas modernos protegem esse caso específico com `--preserve-root`, mas a proteção não cobre variações. A defesa é validar antes, e o assunto volta no Volume 12.

## 6.7. O shell é um programa como outro qualquer

Uma última percepção que arruma tudo no lugar: o shell não é o terminal, e não é o sistema. É apenas mais um programa, que por acaso lê linhas e executa outros programas.

```
echo "$SHELL"          # o shell definido para o seu usuario
ps -p $$ -o comm=      # o shell que esta rodando agora
cat /etc/shells        # os shells instalados
```

Repare que `$SHELL` e o shell em execução podem divergir: `$SHELL` vem do cadastro do usuário, e o segundo comando mostra a realidade. Você pode iniciar outro shell a qualquer momento, digitando `bash` ou `sh`, e sair dele com `exit` — voltando ao anterior. Cada um é um processo, com pai e filhos, e a hierarquia é visível:

```
pstree -p $$ 2>/dev/null | head -5
```

Essa perspectiva é o que torna previsível o comportamento de scripts, de `sudo`, de sessões SSH e de contêineres: em todos os casos há um shell rodando, com um pai e um ambiente herdado.

## 6.8. No Kali

- **O shell padrão é o Zsh** desde a versão 2020.4, não o Bash. Para o que este capítulo cobre, os dois são praticamente idênticos — separação de palavras, `PATH`, opções, código de saída e operadores funcionam igual. As diferenças aparecem em recursos avançados e na configuração, temas dos Volumes 2 e 15.
- **rehash no lugar de hash -r**. Se um programa recém-instalado não é encontrado, o Zsh limpa o cache com `rehash`. O Bash usa `hash -r`. Nas versões recentes do Zsh, o cache costuma se atualizar sozinho.
- **A distribuição define aliases próprios**, principalmente para colorir saídas (`ls`, `grep`, `diff`, `ip`). Confira o que está ativo com `alias` sem argumentos e investigue qualquer surpresa com `type -a <comando>`. Lembre-se: aliases não valem dentro de scripts.
- **Os scripts do Kali ficam disponíveis no PATH como qualquer programa**, porque são pacotes normais em `/usr/bin` (Capítulo 5). Ferramenta clonada à mão em `/opt` não está no `PATH` e exige caminho completo ou um link em `/usr/local/bin`.
- **sudo está configurado para o usuário kali** por pertencer ao grupo `sudo`. Isso significa que quase todo comando administrativo neste manual começa com `sudo` — e a linha correspondente é registrada em `/var/log/auth.log`. Privilégios são o Volume 4.
- **Ferramentas de segurança abusam de sintaxes não convencionais**. `hashcat` usa `-m` com números de modo, `sqlmap` mistura opções longas com valores obrigatórios, `msfconsole` tem sua própria linguagem de comandos que não é shell. Ler a ajuda antes de improvisar economiza tempo.
- **Cuidado com comandos copiados de artigos**. Além do risco óbvio de executar algo destrutivo, há o truque de esconder um comando extra num bloco formatado para parecer inofensivo ao ser copiado. Cole sempre num editor antes de executar, ou digite. É a versão prática do que este capítulo ensina: o shell obedece à linha, não à sua intenção.

**Nota sobre a família RHEL.** Bash é o padrão em Fedora e RHEL, e o `PATH` de root inclui `/usr/sbin` por padrão em ambas as famílias. Tudo o mais deste capítulo é idêntico — separação de palavras, resolução de comando, opções, códigos de saída e

operadores fazem parte da especificação POSIX (IEEE and The Open Group, 2018), não da distribuição.

## 6.9. Laboratório

Roteiro na VM. Os passos criam apenas arquivos descartáveis em `/tmp` e no diretório pessoal.

### 1. Veja o shell que está de fato rodando.

```
echo "$SHELL"
ps -p $$ -o pid,ppid,comm
cat /etc/shells
```

Esperado: `/usr/bin/zsh` no Kali padrão, o PID do seu shell e o PID do processo pai (o emulador de terminal). Anote os dois números.

### 2. Descubra o que cada nome realmente é.

```
type -a ls
type -a cd
type -a echo
type -a nmap
type -a if
```

Esperado: `ls` provavelmente como alias antes do executável, `cd` como builtin, `echo` como builtin e arquivo, `nmap` como arquivo em `/usr/bin`, `if` como palavra reservada. Cinco categorias diferentes, cinco respostas diferentes.

### 3. Explore o PATH e a ordem de precedência.

```
echo "$PATH" | tr ':' '\n' | nl
which -a python3
```

Esperado: a lista numerada de diretórios e, no `which -a`, todos os caminhos encontrados na ordem de busca. Confirme que `/usr/local/bin` aparece antes de `/usr/bin`.

### 4. Prove que o diretório atual não está no PATH.

```
cd /tmp
printf '#!/bin/bash\n echo "executei do diretorio atual"\n' > teste-path.sh
chmod +x teste-path.sh
teste-path.sh
./teste-path.sh
```

Esperado: o primeiro falha com “command not found”, o segundo funciona. Essa diferença é uma decisão de segurança, não um descuido — sem ela, um arquivo malicioso deixado num diretório compartilhado seria executado por engano.

### 5. Reproduza a armadilha do espaço.

```
cd /tmp
touch "arquivo com espaco.txt"
ls arquivo com espaco.txt
ls "arquivo com espaco.txt"
```

## 6. Anatomia de um comando: shell, argumentos, opções e sintaxe

Esperado: o primeiro `ls` reclama de três arquivos inexistentes e depois lista o que encontrar; o segundo funciona. Repare que a mensagem de erro **nomeia as três palavras separadamente** — é a evidência de que o shell dividiu a linha antes de chamar o programa.

### 6. Use o separador de fim de opções.

```
cd /tmp
touch ./-arquivo-estranho
ls -l -arquivo-estranho
ls -l -- -arquivo-estranho
rm -- -arquivo-estranho
```

Esperado: o primeiro `ls` trata o nome como um punhado de opções inválidas; com `--` funciona. Guarde esse recurso — ele resolve todo caso de nome de arquivo que começa com traço, situação comum ao lidar com arquivos gerados por ferramentas.

### 7. Observe códigos de saída.

```
ls /etc > /dev/null; echo "status: $?"
ls /nao-existe 2> /dev/null; echo "status: $?"
grep xyzabc /etc/passwd; echo "status: $?"
comando-inexistente; echo "status: $?"
```

Esperado: 0, um valor não-zero, 1 (o `grep` não achou nada) e 127 (comando não encontrado). Memorize o 127: ele é a resposta padrão para erro de digitação e para programa não instalado.

### 8. Compare os operadores de encadeamento.

```
cd /tmp && echo "chegou"
cd /diretorio-que-nao-existe && echo "NAO deve aparecer"
cd /diretorio-que-nao-existe ; echo "APARECE, e este e o perigo"
cd /diretorio-que-nao-existe || echo "tratamento de erro"
```

Esperado: a segunda linha não imprime nada, a terceira imprime mesmo com o `cd` falhando, e a quarta imprime a mensagem de tratamento. Essa é, em quatro linhas, a diferença entre um script que falha com segurança e um que continua fazendo estrago.

### 9. Verifique a armadilha do `sudo` com redirecionamento.

```
sudo echo "teste" > /tmp/protegido-teste.txt
ls -l /tmp/protegido-teste.txt
echo "teste" | sudo tee /tmp/protegido-teste2.txt > /dev/null
ls -l /tmp/protegido-teste2.txt
```

Esperado: os dois arquivos são criados, mas repare no **dono** de cada um. No primeiro caso o arquivo pertence a você, porque quem redirecionou foi o seu shell; no segundo pertence ao root, porque quem escreveu foi o `tee` elevado. Num diretório onde você não tem permissão de escrita, a primeira forma simplesmente falharia.

### 10. Contorne um alias.

```
alias | head -10
type -a ls
ls /etc | head -3
\ls /etc | head -3
command ls /etc | head -3
```



Esperado: a saída com cores no primeiro `ls` e sem cores nos dois seguintes, se houver alias configurado. A conclusão prática: o que você vê interativamente pode não ser o que um script verá.

#### 11. Limpe o cache de comandos.

```
printf '#!/bin/bash\necho novo\n' | sudo tee /usr/local/bin/comando-novo >
↳ /dev/null
sudo chmod +x /usr/local/bin/comando-novo
comando-novo
hash -r 2>/dev/null || rehash
comando-novo
which comando-novo
sudo rm /usr/local/bin/comando-novo
```

Esperado: o comando funciona (Zsh recente atualiza o cache sozinho) ou funciona só depois do `rehash`. De qualquer forma, `which` confirma o caminho `/usr/local/bin/comando-novo` — o lugar correto para algo instalado à mão, conforme Capítulo 5.

## 6.10. Resumo

- **O shell processa a linha antes de o programa existir:** lê, separa em palavras pelos espaços, expande curingas e variáveis, resolve o nome do comando e só então cria o processo. Quase todo erro estranho acontece nessas etapas.
- **Para o shell, todas as palavras após o comando são equivalentes.** Distinguir opção de operando é decisão do programa que as recebe, e por isso não existe sintaxe universal.
- **A resolução do nome segue uma ordem fixa:** palavra-chave, alias, função, builtin, executável no PATH. O comando `type -a` mostra qual delas venceu, e essa resposta explica diferenças entre o terminal e um script.
- **O diretório atual não está no PATH, por segurança.** Executar algo do diretório corrente exige `./`, e acrescentar `.` ao PATH é abrir uma porta de escalada de privilégio.
- **As convenções de opção são três:** curtas agrupáveis com um traço, longas legíveis com dois traços, e `--` marcando o fim das opções. Programas antigos e ferramentas de segurança rompem essas convenções com frequência.
- **Zero é sucesso; qualquer outro valor é falha.** O código de saída é o que dá sentido a `&&` e `||` — e trocar `&&` por `;` numa sequência destrutiva é um dos erros mais caros que se pode cometer no terminal.
- **`sudo` eleva o comando, não o redirecionamento,** porque quem redireciona é o shell. A forma correta de escrever em arquivo protegido passa por `tee` ou por um shell elevado.



## 7. Obtendo ajuda: man, info, tldr e a documentação do Kali

Três da manhã, de novo. Você está numa máquina remota, dentro de um contêiner metálico no meio de um pasto, conectado por um link de gerência que entrega 200 kbps nos bons momentos. Um script de manutenção escrito por alguém que saiu da empresa em 2021 falha com uma mensagem incompreensível, e no meio dele há uma linha com um comando que você nunca viu. Abrir o navegador não é opção: a banda mal sustenta o SSH. Pesquisar no celular também não — a área não tem sinal.

E ainda assim a resposta está ali, na própria máquina. Todo sistema Linux carrega uma biblioteca de documentação instalada localmente: manuais completos de cada comando, de cada arquivo de configuração, de cada chamada de sistema, além dos arquivos que os empacotadores da distribuição escreveram sobre as decisões que tomaram. Saber navegar por essa biblioteca é a habilidade que separa quem depende de conexão de quem resolve o problema onde ele está.

Este capítulo é sobre isso: as fontes de documentação que existem em disco, qual usar em cada situação, e — o detalhe que quase ninguém ensina — como **ler** uma página de manual, que tem uma gramática própria e não é escrita para ser lida do início ao fim.

### 7.1. O mapa das fontes

Existem cinco fontes locais e duas remotas, e a escolha certa depende da pergunta que você tem. Figura 7.1 organiza a decisão.

Essa última observação é mais importante do que parece. A página de manual em disco descreve **a versão que está na sua máquina**. Um artigo publicado na internet descreve a versão que o autor tinha, que pode ser mais nova ou mais velha, e é assim que se perde uma hora tentando usar uma opção que não existe no seu sistema. Quando as duas fontes divergem, a local está certa.

### 7.2. man: a referência canônica

O **man** é o sistema de manuais do Unix, e cada página é a documentação de referência de um comando, arquivo, chamada de sistema ou biblioteca.

```
man ls
man nmap
man sshd_config
```

A navegação usa o **less** como visualizador, e vale decorar seis teclas:

## 7. Obtendo ajuda: *man*, *info*, *tldr* e a documentação do Kali

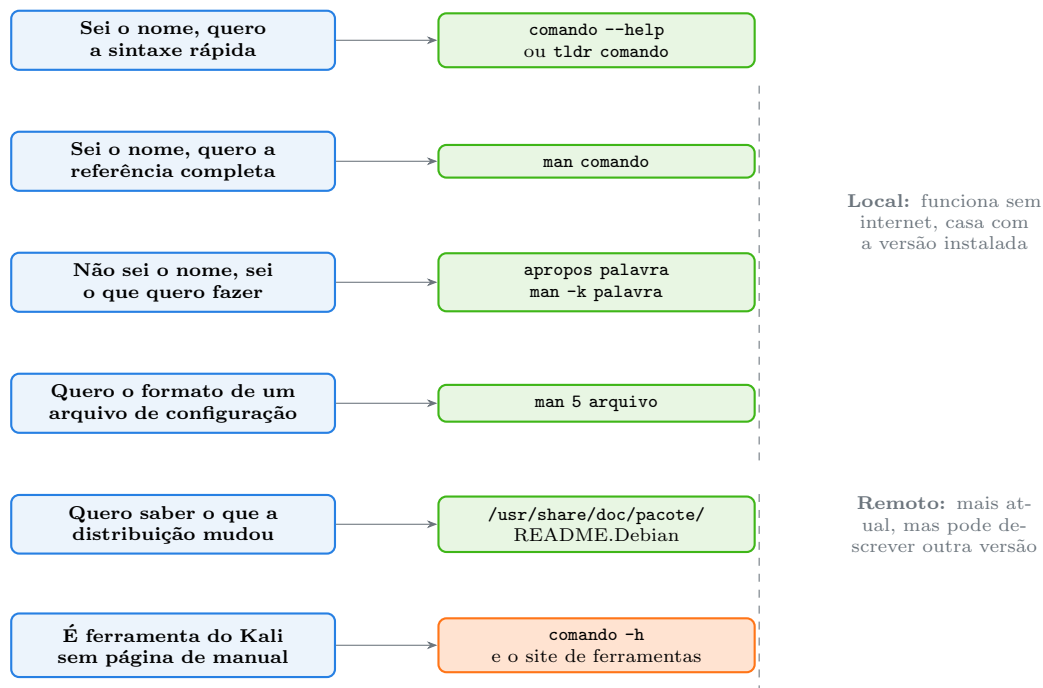


Figura 7.1.: A fonte certa depende da pergunta. Documentação local sempre corresponde à versão instalada; a remota, nem sempre.

| Tecla      | Ação                          |
|------------|-------------------------------|
| espaço / b | avança / volta uma tela       |
| /texto     | busca para frente             |
| n / N      | próxima ocorrência / anterior |
| g / G      | início / fim da página        |
| h          | ajuda do próprio visualizador |
| q          | sai                           |

A busca com `/` é o que torna o `man` prático. Ninguém lê `man bash` inteiro — são milhares de linhas. Você abre e busca a opção específica. Um truque que economiza tempo: para achar a descrição de uma opção curta, busque pelo traço junto de um espaço à frente, como `/ -r`, evitando os acertos no meio de outras palavras.

### 7.2.1. As seções do manual

Este é o conceito que confunde no começo. O manual é dividido em **seções numeradas**, e o mesmo nome pode existir em mais de uma. Figura 7.2 mostra as que importam.

|   |                                                                    |                           |
|---|--------------------------------------------------------------------|---------------------------|
| 1 | Comandos de usuário<br>tudo que se digita no terminal              | <code>man 1 ls</code>     |
| 2 | Chamadas de sistema<br>a fronteira com o kernel                    | <code>man 2 open</code>   |
| 3 | Funções de biblioteca<br>funções em C, sobretudo da glibc          | <code>man 3 printf</code> |
| 4 | Arquivos especiais<br>dispositivos em <code>/dev</code>            | <code>man 4 random</code> |
| 5 | Formatos de arquivo<br>a sintaxe dos arquivos de configuração      | <code>man 5 passwd</code> |
| 6 | Jogos<br>herança histórica                                         | <code>man 6 sl</code>     |
| 7 | Diversos e convenções<br>padrões, protocolos, a própria hierarquia | <code>man 7 hier</code>   |

### 7.2.2. A gramática do SYNOPSIS

A seção SYNOPSIS, logo no começo de cada página, é a parte mais densa e a que mais gente pula. Ela é escrita numa notação convencional que, uma vez conhecida, responde sozinha a maioria das dúvidas de sintaxe:

- **Texto normal (em negrito no terminal)**: digite exatamente assim.
- **Texto em itálico ou sublinhado**: substitua pelo seu valor.
- **[colchetes]**: opcional.
- **reticências...**: pode repetir.
- **{a|b}**: escolha uma das alternativas, e ela é obrigatória.

Aplicando ao `ls`, cujo SYNOPSIS é `ls [OPTION]... [FILE]...`: opções são opcionais e podem ser várias; arquivos também. Por isso `ls` sozinho funciona, `ls -la` funciona, e `ls -la /etc /var` também.

Já `cp`, com `cp [OPTION]... SOURCE... DIRECTORY`, diz outra coisa: opções são opcionais, mas **origem e destino não são**, e a origem pode se repetir enquanto o destino é único. Isso responde, sem ler mais nada, por que `cp arquivo` sozinho dá erro.

### 7.2.3. Encontrar o comando quando você não sabe o nome

Este é o uso do `man` que mais gente desconhece e mais rende. A base de dados de manuais é pesquisável por palavra-chave:

```
apropos "list directory"
man -k compress
man -k firewall
apropos -s 5 network      # busca so na secao 5
```

`apropos` e `man -k` são o mesmo comando. Eles procuram a palavra na descrição curta de todas as páginas instaladas — ou seja, respondem “qual comando faz X?” usando apenas o que está na máquina.

Se a busca falhar dizendo que nada foi encontrado, o índice pode não ter sido gerado:

```
sudo mandb
```

E o complemento útil: `whatis` faz o caminho inverso, devolvendo a descrição de uma linha de um comando que você já sabe o nome.

```
whatis ls tar ss nmap
```

## 7.3. `--help`: rápido e nem sempre suficiente

A maioria absoluta dos programas aceita `--help` (e muitos aceitam `-h`), imprimindo um resumo em algumas dezenas de linhas:

## 7. Obtendo ajuda: *man*, *info*, *tldr* e a documentação do Kali

```
ls --help
ip --help
nmap --help | head -40
```

É mais rápido que abrir o manual e frequentemente basta. Duas ressalvas: a saída às vezes é longa demais para a tela — encadeie com `less` (`ls --help | less`) — e alguns programas antigos não reconhecem `--help`, imprimindo um erro de uso que, ironicamente, costuma trazer a sintaxe correta.

Cuidado com uma armadilha real: em algumas ferramentas, `-h` significa outra coisa. No `ls`, `-h` é “tamanhos legíveis por humanos”; no `du` e no `df`, idem. Quando `-h` não trouxer ajuda, use `--help`.

### 7.4. `help`: os comandos que não têm arquivo

Comandos embutidos no shell não são arquivos e por isso não têm página de manual própria. Como visto em Capítulo 6, `cd`, `export`, `type` e vários outros são builtins. A documentação deles vem do próprio shell:

```
help cd          # no Bash
help type
man bash         # e depois buscar com / cd
```

No Zsh, shell padrão do Kali, o equivalente é:

```
man zshbuiltins
run-help cd
```

Antes de concluir que um comando não tem documentação, confirme com `type -a` se ele não é um builtin.

### 7.5. `info`: a documentação estendida do GNU

Os projetos GNU mantêm manuais mais extensos no formato `info`, organizados em nós ligados entre si, como um hipertexto anterior à web:

```
info coreutils
info bash
info sed
```

A navegação usa `n` (próximo nó), `p` (anterior), `u` (acima), `Enter` sobre um item de menu e `q` para sair. Vale conhecer porque, para as ferramentas do `coreutils` — `ls`, `cp`, `sort`, `cut`, `tr` — o `info` traz explicações e exemplos que a página `man` resume ou omite. Quando `man` disser “the full documentation is maintained as a Texinfo manual”, é exatamente disso que se trata.

## 7.6. /usr/share/doc: o que a distribuição decidiu

Esta é a fonte que quase ninguém consulta e que resolve as dúvidas mais específicas. Todo pacote instalado deposita ali seus documentos:

```
ls /usr/share/doc/nmap/  
zcat /usr/share/doc/nmap/changelog.Debian.gz | head -20  
ls /usr/share/doc/openssh-server/
```

Três arquivos merecem atenção. O **README.Debian** descreve as decisões específicas do empacotador — configurações padrão alteradas, caminhos diferentes do original, incompatibilidades conhecidas. É onde está escrito por que aquele serviço não inicia sozinho, ou por que o arquivo de configuração está num lugar inesperado. O **changelog.Debian.gz** registra o histórico de mudanças do pacote na distribuição. E o diretório **examples/**, quando existe, traz arquivos de configuração de exemplo prontos para copiar e adaptar.

Para descobrir tudo que um pacote instalou, incluindo documentação e exemplos:

```
dpkg -L nmap | grep -E 'doc|example'  
dpkg -L openssh-server | grep /etc
```

## 7.7. tldr: exemplos em vez de referência

Páginas de manual são **referências**, não tutoriais: descrevem exaustivamente cada opção e raramente mostram como se usa o comando na vida real. O projeto **tldr** preenche essa lacuna com páginas curtas, feitas só de exemplos comentados:

```
sudo apt install -y tldr  
tldr --update  
tldr tar  
tldr find  
tldr ss
```

A diferença é radical. **man tar** tem centenas de linhas e não diz de forma direta como extrair um arquivo **.tar.gz**; **tldr tar** mostra a linha exata, com comentário, na primeira tela. O fluxo eficiente combina os dois: **tldr** para lembrar a forma usual, **man** para os detalhes e casos de borda.

Vale saber que existe também o **tealdeer**, uma implementação mais rápida do mesmo cliente, e o **cheat**, que permite manter suas próprias folhas de consulta — útil para registrar os comandos específicos do seu ambiente.

## 7.8. Um último recurso: o whereis e a leitura do próprio programa

Quando nada mais responde, ainda há caminhos:

## 7. Obtendo ajuda: *man*, *info*, *tldr* e a documentação do Kali

```
whereis nmap          # binario, fonte e manual
dpkg -S $(which nmap) # de qual pacote veio
apt show nmap         # descricao e dependencias
file $(which tldr)    # e um script? entao da para ler
```

O último é subestimado: boa parte das ferramentas de segurança é script em Python ou shell. Se o `file` disser que é texto, você pode abrir com `less` e ler o que ele faz — a forma mais confiável de descobrir o comportamento de uma ferramenta antes de apontá-la para algo importante.

### 7.9. Sobre confiar na resposta

Duas advertências que valem o parágrafo.

**Respostas de fórum envelhecem.** Uma solução de 2016 pode se referir a `iptables` quando sua máquina usa `nftables`, ou a `ifconfig` quando o sistema tem `ip`. Confira sempre contra a documentação local antes de executar.

**Comando colado sem leitura é comando não auditado.** Vale para respostas de fórum, para artigos e também para o que um assistente de IA sugere — incluindo este manual. Antes de executar algo que você não entende, leia a página de manual das partes que não reconhece. É exatamente para isso que este capítulo existe, e é o hábito que impede o desastre da linha destrutiva copiada às pressas discutida em Capítulo 6.

### 7.10. No Kali

- **kali.org/tools** documenta cada ferramenta empacotada, com descrição, exemplos de uso e a página de manual reproduzida. É a fonte oficial quando a ferramenta não tem `man` decente — e muitas não têm, porque são projetos de comunidade sem documentação formal.
- **kali.org/docs** cobre o que é da distribuição: instalação, ramos de repositório, personalização de imagens, WSL, NetHunter, ARM (Offensive Security, 2025).
- **Ferramentas de segurança frequentemente usam `-h` em vez de `--help`**, e algumas imprimem a ajuda apenas quando executadas sem argumento nenhum. Se `--help` devolver erro, tente `-h`, depois o nome sozinho.
- **Nem toda ferramenta do Kali tem página de manual.** Confirme com `man -k <ferramenta>` ou `whereis <ferramenta>`; se não houver, o caminho é `-h`, o `/usr/share/doc/<pacote>/` e o site de ferramentas.
- **Módulos do Metasploit têm documentação própria dentro do console:** `info <modulo>` e `show options` descrevem parâmetros que nenhum `man` cobre. É uma linguagem de comandos separada do shell.
- **O livro *Kali Linux Revealed*** é a documentação oficial estendida da distribuição, disponível gratuitamente e escrito pelos mantenedores (Hertzog; O’Gorman; Aharoni, 2017). Boa parte dele é, na verdade, um curso de Debian.
- **`man` pode não estar instalado em imagens enxutas.** Em contêineres e na variante `headless`, as páginas costumam ser removidas para economizar espaço. Reinstale com `sudo apt install --reinstall man-db manpages` e, se necessário, `sudo unminimize` em derivados que aplicam essa redução.



- **A base de dados do apropos precisa existir.** Depois de instalar muitos pacotes de uma vez — o que acontece ao instalar um metapacote —, rode `sudo mandb` para que as ferramentas novas apareçam nas buscas por palavra-chave.

**Nota sobre a família RHEL.** Tudo neste capítulo funciona igual: `man`, `apropos`, `info` e `--help` são universais. As diferenças estão nos lugares: `/usr/share/doc/<pacote>` sem os arquivos `README.Debian`, `rpm -qd <pacote>` no lugar de `dpkg -L` para listar a documentação, e `rpm -qf` no lugar de `dpkg -S` para descobrir a origem de um arquivo.

## 7.11. Laboratório

Roteiro de leitura. Nenhum passo altera o sistema, exceto a instalação do `tldr` no passo 8.

1. **Leia um manual do começo e identifique as seções padrão.**

```
man ls
```

Dentro do visualizador, use `/` para saltar até `SYNOPSIS`, `DESCRIPTION`, `EXAMPLES` e `SEE ALSO`. Esperado: encontrar todas essas seções. Anote o que aparece em `SEE ALSO` — é a lista de comandos relacionados, e frequentemente é ali que está o que você realmente precisa.

2. **Decifre um SYNOPSIS.**

```
man cp | head -20
man ls | head -12
```

Esperado: `cp [OPTION]... SOURCE... DIRECTORY` e `ls [OPTION]... [FILE]...`. Explique para si mesmo, usando só a notação, por que `ls` funciona sem argumento nenhum e `cp` não. Confirme executando os dois sem argumentos e comparando as mensagens.

3. **Prove que o mesmo nome existe em seções diferentes.**

```
whatis passwd
man 1 passwd | head -8
man 5 passwd | head -12
```

Esperado: `whatis` lista as duas ocorrências; a seção 1 descreve o comando que troca senha e a seção 5 descreve o formato do arquivo, com a explicação de cada campo separado por dois-pontos. Essa página da seção 5 volta a ser útil no Volume 4.

4. **Encontre um comando sem saber o nome dele.**

```
apropos "compress"
apropos "disk space"
man -k "network interface" | head
```

Esperado: listas de candidatos com a descrição de uma linha de cada um. Se nada for encontrado, rode `sudo mandb` e repita. **Este é o comando da situação da abertura do capítulo:** sem internet, é assim que se descobre a ferramenta certa.

## 7. Obtendo ajuda: *man*, *info*, *tldr* e a documentação do Kali

### 5. Busque dentro de um manual gigante.

```
man bash
```

Dentro do visualizador, digite `/EXIT STATUS` e pressione Enter, depois `n` algumas vezes. Esperado: saltar direto para o trecho, sem rolar milhares de linhas. Repita buscando por `PATH`. Essa técnica é o que torna manuais longos utilizáveis.

### 6. Compare `--help` com `man`.

```
ls --help | wc -l
man ls | wc -l
ip --help 2>&1 | head -20
```

Esperado: o `--help` com poucas dezenas de linhas contra centenas do manual. Note que `ip --help` imprime a ajuda em erro padrão, motivo do `2>&1` — detalhe que o Volume 2 explica.

### 7. Documente um comando embutido.

```
type -a cd
help cd 2>/dev/null || man zshbuiltins
man cd 2>&1 | head -3
```

Esperado: `cd` identificado como builtin, a ajuda vindo do shell, e o `man cd` falhando ou trazendo uma página genérica. A conclusão prática: quando `man` não achar um comando, verifique antes se ele é builtin.

### 8. Instale e use o `tldr`.

```
sudo apt install -y tldr
tldr --update
tldr tar
tldr find
```

Esperado: meia dúzia de exemplos comentados por comando, cabendo na tela. Compare `tldr tar` com `man tar | wc -l` e observe a diferença de propósito: um responde “como se faz”, o outro responde “o que existe”.

### 9. Descubra o que a distribuição mudou num pacote.

```
ls /usr/share/doc/openssh-server/
zcat /usr/share/doc/openssh-server/changelog.Debian.gz 2>/dev/null | head
↵ -15
dpkg -L openssh-server | grep -E 'doc|example|etc'
```

Esperado: os arquivos de documentação do pacote, o histórico de alterações e a lista dos arquivos de configuração instalados. Se houver um `README.Debian`, leia — é onde ficam as decisões que explicam comportamentos inesperados.

### 10. Rastreie a origem de um binário.

```
which nmap
whereis nmap
dpkg -S $(which nmap)
apt show nmap 2>/dev/null | head -12
```

Esperado: o caminho do executável, a localização do manual, o pacote de origem e a descrição oficial. Quatro comandos que, juntos, respondem “o que é isso e de onde veio” para qualquer programa da máquina.

#### 11. Leia uma ferramenta que é script.

```
file $(which tldr)
head -20 $(which tldr) 2>/dev/null
```

Esperado: se o `file` disser “Python script” ou “shell script”, as primeiras linhas são legíveis. Auditar uma ferramenta antes de usá-la em ambiente sensível começa exatamente assim.

12. **Monte o seu fluxo.** Escolha um comando que você não conhece — por exemplo `ss`, `stat`, `xxd` ou `nl` — e responda três perguntas sobre ele usando, nesta ordem: `whatis`, `tldr` e `man`. Esperado: descobrir para que serve, ver um exemplo funcionando e localizar a opção específica de que precisava, tudo sem sair da máquina. Esse é o hábito que este capítulo quer instalar.

## 7.12. Resumo

- **A documentação está instalada na máquina** e sempre corresponde à versão que você tem — vantagem decisiva sobre qualquer artigo na internet, que descreve a versão do autor.
- **man é a referência canônica**, dividida em seções numeradas. O mesmo nome pode existir em várias delas (`man 1 passwd` contra `man 5 passwd`), e o número entre parênteses numa referência indica onde procurar.
- **O SYNOPSIS tem gramática própria**: colchetes marcam o opcional, reticências marcam o repetível, chaves com barra marcam alternativas obrigatórias. Ler essas quatro convenções responde a maioria das dúvidas de sintaxe.
- **apropos e man -k respondem “qual comando faz X?”** usando só o que está em disco, e dependem do índice mantido pelo `mandb`.
- **--help é o atalho e tldr é o complemento**: um resume as opções, o outro mostra os usos reais. A combinação eficiente é `tldr` para a forma usual e `man` para os detalhes.
- **Comandos embutidos não têm página de manual**; a documentação vem do próprio shell, com `help` no Bash e `man zshbuiltins` ou `run-help` no Zsh.
- **/usr/share/doc/<pacote>/ guarda o que a distribuição decidiu**, incluindo o `README.Debian` que explica comportamentos inesperados e os arquivos de exemplo prontos para adaptar.
- **Comando colado sem leitura é comando não auditado**. Consultar a documentação antes de executar é a defesa mais barata contra a linha destrutiva copiada às pressas.



**Parte II.**

**Volume 2 — O Terminal e o Shell**



## 8. Emulador de terminal e shell: bash, zsh e o padrão do Kali

Você passou uma tarde configurando o terminal do seu Kali: prompt colorido mostrando o diretório e o ramo do Git, autocompletar que adivinha o comando enquanto você digita, **Ctrl+seta** andando de palavra em palavra. Aí você abre uma sessão SSH num servidor e tudo desaparece. O prompt vira um \$ seco, **Ctrl+seta** imprime ;5C na tela, e o autocompletar some. Nada quebrou: você simplesmente descobriu que o que você configurou não era o terminal — era o shell — e que os dois são programas diferentes, rodando em máquinas diferentes, com responsabilidades diferentes.

Essa confusão entre “terminal” e “shell” é a mais comum de todas, e ela não é culpa de quem está aprendendo. As duas coisas aparecem na mesma janela, respondem ao mesmo teclado e imprimem no mesmo lugar. A diferença só fica visível quando algo se comporta de forma inesperada — e aí, sem o modelo mental certo, não há como saber onde procurar.

Este capítulo separa as camadas. Ao final dele você vai saber exatamente qual programa desenha os caracteres na tela, qual interpreta o que você digitou, qual guarda o histórico, qual define as cores, e por que a resposta muda quando você entra num servidor remoto ou dentro de um contêiner.

### 8.1. A pilha: da tecla ao programa

Entre a tecla que você aperta e o programa que executa existem pelo menos quatro camadas independentes. Figura 8.1 mostra o caminho completo.

Ler essa pilha de cima para baixo já resolve metade das dúvidas:

**O emulador de terminal** é um programa gráfico comum, como um navegador ou um editor. Ele abre uma janela, desenha caracteres com uma fonte, mantém o buffer de rolagem, trata a seleção com o mouse, interpreta sequências de escape para colorir texto e traduz teclas em bytes. Ele **não** sabe o que é um comando, não tem histórico de comandos e não conhece **ls**.

**O pseudoterminal (PTY)** é uma peça do kernel. Ele expõe duas pontas: o *master*, que o emulador segura, e o *slave*, que aparece como um arquivo de dispositivo (`/dev/pts/0`, `/dev/pts/1`). É o PTY que faz o eco das teclas digitadas, que junta os caracteres numa linha até você apertar Enter, e que converte **Ctrl+C** no sinal **SIGINT** enviado ao processo em primeiro plano. Quando um programa trava e **Ctrl+C** ainda funciona, é porque quem está agindo é o kernel, não o programa.

**O shell** é o programa que roda ligado à ponta slave. Ele imprime o prompt, lê a linha, faz as expansões, resolve o nome do comando e cria o processo — exatamente o ciclo de Capítulo 6. Prompt, histórico, aliases, autocompletar e atalhos de edição de linha são todos dele.

**O programa** herda três descritores de arquivo já abertos e apontando para o PTY: entrada padrão, saída padrão e saída de erro. É por isso que qualquer comando “sabe” imprimir na sua tela sem nenhuma configuração — o shell entregou os canais prontos.

## 8. Emulador de terminal e shell: bash, zsh e o padrão do Kali

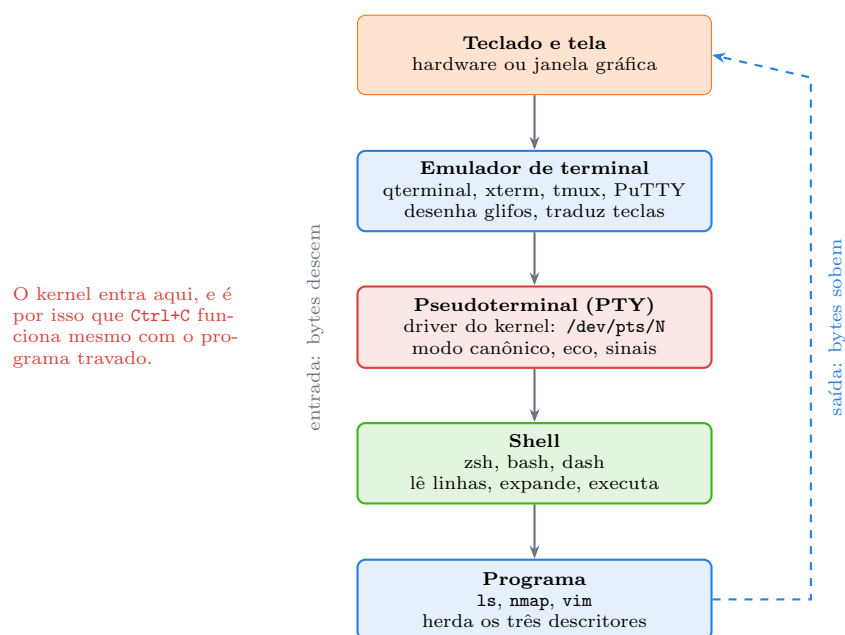


Figura 8.1.: Quatro camadas independentes. Cada problema de terminal pertence a exatamente uma delas.

Voltando ao problema da abertura: no SSH, o emulador de terminal continua sendo o seu, na sua máquina. O que mudou foi o shell, que agora roda no servidor remoto e não tem a sua configuração. Por isso o prompt sumiu.

### 8.2. O emulador de terminal

O nome “emulador” é literal e histórico. Nos anos 1970, um terminal era hardware: um teclado, uma tela e um cabo serial ligado ao computador central. O modelo dominante foi o **DEC VT100**, e a linguagem de controle dele — sequências de escape que começam com o byte **ESC** — virou o padrão de fato que todos os terminais de software imitam até hoje (Kernighan; Pike, 1984). Quando você escreve `\033[31m` para pintar um texto de vermelho, está falando VT100 com uma janela que finge ser um aparelho de 1978.

Emuladores comuns no mundo Linux:

Tabela 8.1.: Emuladores de terminal comuns

| Emulador                                           | Característica                                          |
|----------------------------------------------------|---------------------------------------------------------|
| <code>qterminal</code>                             | Padrão do Kali com Xfce, leve, baseado em Qt            |
| <code>gnome-terminal</code>                        | Padrão do GNOME, integrado ao ambiente                  |
| <code>xterm</code>                                 | O clássico, mínimo, presente em praticamente tudo       |
| <code>alacritty</code> , <code>kitty</code>        | Renderização via GPU, rápidos para muita saída          |
| <code>tmux</code> , <code>screen</code>            | Multiplexadores: terminais <i>dentro</i> de um terminal |
| <code>PuTTY</code> , <code>Windows Terminal</code> | Clientes usados a partir do Windows                     |

A escolha do emulador afeta velocidade de rolagem, suporte a ligaduras de fonte, transparência e atalhos de abas — e nada mais. Nenhum deles muda a sintaxe de um comando.



### 8.2.1. A variável TERM e o banco de dados terminfo

Cada emulador anuncia suas capacidades através da variável de ambiente TERM:

```
echo "$TERM"
infocmp | head -20
```

Num Kali com qterminal, TERM normalmente vale `xterm-256color`. Esse nome é a chave de busca num banco de dados chamado **terminfo**, que descreve o que aquele terminal sabe fazer: quantas cores suporta, qual sequência limpa a tela, qual move o cursor para a linha 5 coluna 12. Programas de tela cheia como **vim**, **htop**, **less** e **nano** consultam esse banco em vez de gravar sequências VT100 no código.

Quando TERM está errado, o sintoma é característico: lixo na tela, cores ausentes ou a reclamação direta.

```
TERM=desconhecido htop      # "Error opening terminal"
```

Isso aparece na prática ao entrar por SSH numa máquina antiga que não conhece `xterm-256color`, ou dentro de contêineres minimalistas onde o pacote **ncurses-base** não foi instalado. A saída de emergência é declarar um terminal conservador que todo mundo conhece:

```
export TERM=xterm
```

### 8.2.2. TTY e PTY

O comando `tty` mostra a qual dispositivo o seu shell está ligado:

```
tty
# /dev/pts/0
```

**pts** significa *pseudo-terminal slave*. Um número diferente para cada janela ou aba aberta. Isso permite mandar texto direto para outra janela, se você tiver permissão:

```
echo "mensagem de outra aba" > /dev/pts/1
```

Existem também os terminais **virtuais** de texto, acessíveis com **Ctrl+Alt+F1** até **F6** numa instalação com interface gráfica. Esses aparecem como `/dev/tty1`, `/dev/tty2` — sem o **pts** — porque não são emulados por um programa gráfico: são fornecidos diretamente pelo kernel. Eles salvam o dia quando o ambiente gráfico congela, e é bom saber que existem antes de precisar.

A distinção também explica um comportamento que confunde em scripts. Um programa pode perguntar ao sistema se sua saída é um terminal ou um arquivo, e mudar de comportamento conforme a resposta:

```
ls --color=auto             # colorido na tela
ls --color=auto | cat       # sem cor: a saída não é um terminal
```

Não é bug. O `ls` detecta que a saída virou um cano e desliga as cores, porque códigos de escape num arquivo de texto seriam lixo. Vale lembrar disso ao processar saídas com **grep** — assunto de Capítulo 12.

### 8.3. O shell

O shell é um programa comum que lê linhas e executa outros programas. Os principais no mundo Linux:

**sh** — o shell POSIX. No Debian e no Kali, `/bin/sh` é um link para o **dash**, um shell minimalista e rápido, escolhido justamente por acelerar os milhares de scripts executados durante o boot (Debian Project, 2025). Ele **não** tem arrays, `[[ ]]`, `local` completo nem substituição de processo. Um script com `#!/bin/sh` que usa sintaxe do Bash falha aqui — e é a causa clássica de “funciona no meu terminal, quebra no cron”.

**bash** — *Bourne Again Shell*, o padrão da maioria das distribuições e o assunto de toda a Fase 3 deste manual. É o alvo natural para scripts, por estar em praticamente todo sistema Linux (Free Software Foundation, 2024b).

**zsh** — o padrão do Kali desde a versão 2020.4. Amplamente compatível com o Bash na sintaxe do dia a dia, mas com autocompletar muito superior, correção de erros de digitação, globbing recursivo nativo e um sistema de temas que produz o prompt colorido característico do Kali.

**fish** — sintaxe amigável e recursos interativos excelentes, mas **incompatível** com POSIX. Ótimo para uso interativo, péssimo para colar comandos de tutoriais.

Descobrir o que está rodando agora:

```
echo "$SHELL"          # o shell cadastrado para o seu usuario
ps -p $$ -o comm=      # o shell realmente em execucao
$SHELL --version
cat /etc/shells        # os shells instalados e permitidos
```

Os dois primeiros divergem com frequência: `$SHELL` vem do `/etc/passwd` e não muda quando você digita `bash` dentro de um Zsh.

#### 8.3.1. Zsh e Bash: o que muda de verdade

Para quem está começando, a diferença prática é pequena, e isso é uma boa notícia. Praticamente tudo neste volume — navegação, redirecionamento, curingas, histórico — funciona igual nos dois. As divergências que aparecem cedo:

Tabela 8.2.: Divergências que aparecem cedo entre Bash e Zsh

| Situação                              | Bash                   | Zsh                                |
|---------------------------------------|------------------------|------------------------------------|
| Índice inicial de array               | 0                      | 1                                  |
| Curinga sem correspondência           | fica literal           | erro <code>no matches found</code> |
| Cache de comandos                     | <code>hash -r</code>   | <code>rehash</code>                |
| Config interativa                     | <code>~/.bashrc</code> | <code>~/.zshrc</code>              |
| <code>\$0</code> no prompt interativo | <code>bash</code>      | <code>zsh</code>                   |

A segunda linha da tabela merece atenção. No Bash, `ls *.xyz` sem nenhum arquivo correspondente passa o texto `*.xyz` literalmente para o `ls`, que reclama de arquivo inexistente. No Zsh, o próprio shell aborta antes com `zsh: no matches found`. O comportamento do Zsh é mais

correto e mais seguro, mas surpreende quem vem de tutoriais escritos para Bash. O tema volta em Capítulo 13.

A regra de convivência é simples: **use o Zsh interativamente e escreva scripts para Bash**, com `#!/usr/bin/env bash` na primeira linha. Um script não herda nada do seu shell interativo — nem aliases, nem funções, nem configuração — então a escolha do shell de uso diário não afeta os scripts.

## 8.4. Arquivos de inicialização

O motivo mais comum de “coloquei o alias no arquivo e ele não funciona” é escolher o arquivo errado. E a escolha depende de duas perguntas que o shell faz a si mesmo ao iniciar. Figura 8.2 mostra a decisão.

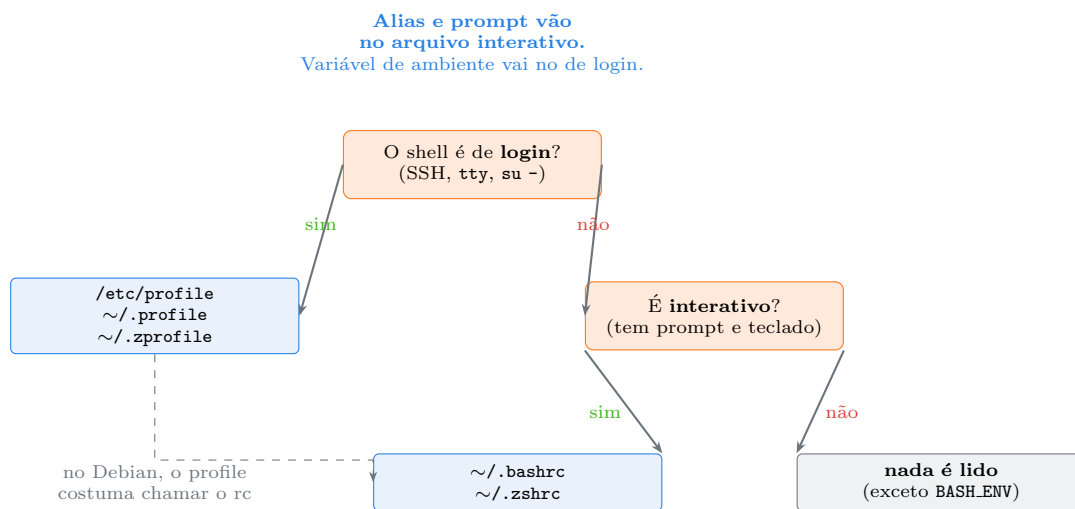


Figura 8.2.: Duas perguntas, três destinos. Errar o arquivo é o motivo número um de configuração que “não pega”.

Um **shell de login** é o que inicia uma sessão: entrar por SSH, logar num terminal virtual de texto, ou usar `su -`. Um **shell interativo** é o que tem prompt e lê do teclado. Uma janela nova do qterminal normalmente abre um shell interativo que **não** é de login — e é exatamente por isso que a configuração vai no `~/.zshrc`, não no `~/.zprofile`.

A divisão de trabalho recomendada:

- `~/.zprofile` / `~/.bash_profile` / `~/.profile` — variáveis de ambiente (PATH, EDITOR, LANG). São herdadas por todos os processos filhos, então basta definir uma vez por sessão.
- `~/.zshrc` / `~/.bashrc` — tudo que só faz sentido com um humano na frente: aliases, funções, prompt, opções de histórico, autocompletar.

Um teste rápido para saber em que situação você está:

```
# 1 se for shell de login
shopt -q login_shell && echo "login" || echo "nao-login"      # bash
[[ -o login ]] && echo "login" || echo "nao-login"            # zsh

# interativo?
[[ $- == *i* ]] && echo "interativo" || echo "nao-interativo"
```

## 8. Emulador de terminal e shell: bash, zsh e o padrão do Kali

Depois de editar um desses arquivos, não é preciso fechar o terminal:

```
source ~/.zshrc      # ou: . ~/.zshrc
```

O `source` executa o arquivo **no shell atual**, e não num filho. Rodar `./~/.zshrc` criaria um novo processo, aplicaria as mudanças nele e o descartaria — inútil. A distinção volta com força no Volume 9.

### 8.4.1. Trocando o shell padrão

```
chsh -s /usr/bin/bash      # muda o shell de login do seu usuario
```

A mudança altera o campo correspondente em `/etc/passwd` e só tem efeito na **próxima** sessão. O shell precisa estar listado em `/etc/shells`. Para experimentar sem compromisso, basta invocar o outro shell dentro do atual e sair com `exit`:

```
bash      # entra no bash como processo filho
exit      # volta ao zsh
```

## 8.5. No Kali

- **Zsh é o padrão desde o Kali 2020.4.** A mudança veio junto com um tema próprio, que mostra usuário, host, diretório e indicadores de estado no prompt, com cor diferente quando você é root — um lembrete visual deliberado de que você está com privilégios totais.
- **kali-tweaks troca o visual do prompt.** Na aba correspondente, é possível alternar entre o prompt de duas linhas, o de uma linha e o modelo enxuto, sem editar o `~/.zshrc` à mão.
- **O emulador padrão do Kali com Xfce é o qterminal.** Vale conhecer os atalhos de aba: `Ctrl+Shift+T` para nova aba, `Ctrl+Shift+W` para fechar, `Ctrl+Shift+C` e `Ctrl+Shift+V` para copiar e colar. O `Shift` é obrigatório porque `Ctrl+C` já tem dono: é o sinal de interrupção.
- **`Ctrl+Shift+V` cola, e isso é um risco real.** Colar comando de artigo direto no terminal é o vetor de ataque mais barato que existe: uma página pode esconder texto adicional no que você acha que selecionou. Cole num editor antes, ou digite.
- **`~/.zshrc` no Kali já vem povoado.** Ele define aliases coloridos, opções de histórico e o tema. Ao personalizar, acrescente ao final em vez de reescrever o arquivo — assim uma atualização de pacote não apaga o seu trabalho e você não perde os defaults da distribuição.
- **O usuário padrão é kali, não root.** Um shell de root aberto com `sudo -i` é de login e lê os arquivos de `/root`, não os seus. Se um alias seu “sumiu” depois de virar root, é isso: outro usuário, outro `~`, outra configuração.
- **No WSL não existe interface gráfica por padrão.** O emulador é o Windows Terminal, o `TERM` costuma vir como `xterm-256color`, e não há `/dev/tty1` para socorro. É um ambiente ótimo para aprender comandos, mas ele não substitui a VM quando o assunto envolve rede, kernel ou drivers.

- **tmux vem instalado.** Ele resolve o problema real de perder o trabalho quando a conexão SSH cai: a sessão continua no servidor e você reconecta com `tmux attach`. O uso completo é tema do Volume 5.

**Nota sobre a família RHEL.** Fedora e RHEL usam Bash como padrão e `~/.bash_profile` como arquivo de login. O `/bin/sh` nessas distribuições é um link para o próprio Bash, e não para o dash — o que faz scripts com `#!/bin/sh` aceitarem sintaxe do Bash lá e falharem no Debian e no Kali. É uma das diferenças que mais geram falsos “funciona aqui”.

## 8.6. Laboratório

Roteiro na VM do Kali. Nenhum passo altera configuração permanente sem que você seja avisado.

### 1. Identifique cada camada da pilha.

```
tty
echo "$TERM"
ps -p $$ -o pid,ppid,comm
ps -p $PPID -o comm=
```

Esperado: um dispositivo `/dev/pts/N`, o tipo de terminal, o PID e o nome do seu shell, e o nome do emulador que o criou. Você acabou de ver três camadas de Figura 8.1 identificadas por nome.

### 2. Prove que o emulador não é o shell.

```
zsh --version
bash --version | head -1
bash                # entra num bash filho
echo "$0"
ps -p $$ -o comm=
exit                # volta ao zsh
echo "$0"
```

Esperado: a mesma janela, o mesmo `/dev/pts`, dois shells diferentes. A janela nunca mudou — só o programa ligado a ela.

### 3. Converse com outra aba pelo dispositivo.

Abra uma segunda aba com `Ctrl+Shift+T`, rode `tty` nela e anote o caminho. Volte à primeira aba:

```
echo "mensagem vinda da outra aba" > /dev/pts/1    # ajuste o numero
```

Esperado: o texto aparece na outra aba. Isso demonstra que o terminal é, literalmente, um arquivo de dispositivo em que se pode escrever.

### 4. Veja o TERM em ação e quebre-o de propósito.

```
tput colors
infocmp | head -5
TERM=vt100 tput colors
TERM=inexistente htop
```

## 8. Emulador de terminal e shell: bash, zsh e o padrão do Kali

Esperado: 256 na primeira linha, 8 com vt100, e um erro de abertura de terminal na última. Repare que a variável foi definida **só para aquele comando** — o seu shell continua intacto.

### 5. Observe um programa detectando que a saída não é um terminal.

```
ls --color=always /etc | head -3
ls --color=auto /etc | cat | head -3
ls /etc | cat | head -3
```

Esperado: cores na primeira, sem cores nas outras. Guarde este comportamento: ele explica por que uma saída colorida na tela chega “limpa” quando você a joga num arquivo.

### 6. Descubra o tipo do seu shell atual.

```
[[ -o login ]] && echo "login" || echo "nao-login"
[[ $- == *i* ]] && echo "interativo" || echo "nao-interativo"
```

Esperado numa aba comum do qterminal: nao-login e interativo. Compare com o resultado de `sudo -i` (que abre um shell de login) e de `zsh -c '[[ -o login ]] && echo login || echo nao-login'` (que não é nem login nem interativo).

### 7. Confirme qual arquivo de inicialização foi lido.

```
echo 'echo "> passei pelo .zshrc"' >> ~/.zshrc
echo 'echo "> passei pelo .zprofile"' >> ~/.zprofile
zsh -i -c exit          # shell interativo, nao login
zsh -l -i -c exit       # shell de login e interativo
```

Esperado: o primeiro imprime só a mensagem do `.zshrc`; o segundo imprime as duas. Desfaça em seguida removendo as duas últimas linhas dos arquivos com o editor de sua preferência (`nano ~/.zshrc`).

### 8. Crie um alias e entenda por que ele some.

```
alias lixo='ls -la /tmp'
lixo
zsh -c lixo
```

Esperado: funciona no seu shell, falha no shell filho não interativo. Aliases pertencem à sessão interativa e **não** são herdados. É por isso que scripts nunca podem depender deles.

### 9. Compare Bash e Zsh no curinga sem correspondência.

```
cd /tmp
ls *.formato-que-nao-existe
bash -c 'ls *.formato-que-nao-existe'
```

Esperado: `zsh:` no matches found no primeiro, e um erro do próprio `ls` no segundo. Duas filosofias — o Zsh protege, o Bash entrega literal.

### 10. Recarregue a configuração sem fechar o terminal.

```
echo "alias oi='echo ola do zshrc'" >> ~/.zshrc
oi
# ainda nao existe
source ~/.zshrc
oi
# agora existe
```

Esperado: falha, depois sucesso. Remova a linha do `~/.zshrc` ao terminar. A lição é que `source` executa no shell atual — executar o arquivo criaria um filho e jogaria o resultado fora.

11. **Veja o terminal virtual de texto.** (*pule este passo no WSL*)

Pressione `Ctrl+Alt+F3`. Faça login em modo texto, rode `tty` — deve mostrar `/dev/tty3`, sem `pts`. Volte à interface gráfica com `Ctrl+Alt+F1` ou `Ctrl+Alt+F7`, conforme a instalação. Esse é o caminho de recuperação quando o ambiente gráfico congela.

## 8.7. Resumo

- **Terminal e shell são programas diferentes.** O emulador desenha caracteres e traduz teclas; o shell interpreta comandos. Prompt, histórico e aliases pertencem ao shell — por isso desaparecem ao entrar num servidor remoto, onde o shell é outro.
- **Entre os dois há um pseudoterminal do kernel**, visível como `/dev/pts/N`. É ele que faz o eco das teclas, monta a linha até o Enter e transforma `Ctrl+C` em SIGINT — motivo pelo qual a interrupção funciona mesmo com o programa travado.
- **A variável TERM descreve as capacidades do terminal** e é a chave do banco terminfo. Quando ela está errada, programas de tela cheia falham ou enchem a tela de lixo; `export TERM=xterm` é a saída de emergência.
- **O Kali usa Zsh por padrão desde 2020.4**, com Bash disponível. As diferenças do dia a dia são poucas — índice de array, curinga sem correspondência e o nome do arquivo de configuração.
- **Use Zsh interativamente e escreva scripts para Bash.** Scripts não herdam aliases, funções nem configuração do shell interativo, então as duas escolhas são independentes.
- **A configuração vai no arquivo certo conforme o tipo de shell:** variável de ambiente no arquivo de login (`~/.zprofile`), alias e prompt no arquivo interativo (`~/.zshrc`). Errar isso é a causa número um de configuração que não faz efeito.
- **source arquivo aplica mudanças no shell atual.** Executar o arquivo cria um processo filho, aplica lá e descarta — o que parece não fazer nada.





## 9. Navegação: `pwd`, `cd`, `ls` e caminhos absolutos e relativos

Um colega te passa um comando que funciona perfeitamente na máquina dele:

```
tar czf backup.tar.gz ../configuracoes
```

Você roda e recebe `../configuracoes: Cannot stat: No such file or directory`. O diretório existe — você acabou de ver. O comando está correto. O que mudou foi apenas de onde ele foi executado, e isso bastou para que `..` apontasse para outro lugar.

Caminho relativo é uma conveniência enorme e uma armadilha silenciosa. Ele encurta a digitação e torna scripts portáteis, mas depende inteiramente de um dado invisível: o diretório de trabalho do processo. Quem não tem clareza sobre esse dado passa a vida corrigindo caminhos por tentativa e erro.

Este capítulo trata da navegação como ela realmente funciona — não como decoreba de comandos, mas como manipulação consciente de um estado que todo processo carrega.

### 9.1. O diretório de trabalho é um atributo do processo

Todo processo em execução no Linux tem um **diretório de trabalho atual** (*current working directory*, ou `cwd`). Não é uma propriedade do usuário, nem do terminal, nem do sistema: é um campo na estrutura que o kernel mantém para aquele processo específico.

```
pwd
```

O `pwd` (*print working directory*) apenas lê esse campo. E como é um atributo do processo, dá para inspecioná-lo de fora, através do `/proc`:

```
ls -l /proc/self/cwd
ls -l /proc/$$/cwd
```

Os dois mostram um link simbólico apontando para o diretório atual — o primeiro do processo `ls`, o segundo do seu shell. Essa consulta é útil de verdade quando um serviço travado está segurando um sistema de arquivos que você quer desmontar: `ls -l /proc/<PID>/cwd` revela onde ele está parado.

Três consequências práticas seguem imediatamente do fato de o `cwd` pertencer ao processo:

**Cada aba do terminal tem o seu.** Duas janelas do mesmo shell podem estar em diretórios diferentes ao mesmo tempo, sem qualquer interferência.

**Um processo filho herda o do pai, mas não o devolve.** Um script que faz `cd /var/log` muda o diretório do próprio processo do script; quando ele termina, o seu shell continua exatamente onde estava. Isso confunde muita gente, e a solução — **source** em vez de execução — é a mesma da configuração do shell, vista em Capítulo 8.

**Por isso *cd* é builtin.** Um programa externo chamado `cd` seria inútil: ele mudaria o próprio diretório e morreria em seguida. Só um comando embutido no shell consegue alterar o shell, como visto em Capítulo 6.

## 9.2. Caminhos absolutos e relativos

Um **caminho absoluto** começa em `/`, a raiz da árvore descrita em Capítulo 5. Ele identifica o mesmo arquivo independentemente de onde você esteja: `/etc/ssh/sshd_config` é sempre o mesmo arquivo.

Um **caminho relativo** não começa com `/` e é interpretado a partir do diretório de trabalho. O mesmo texto aponta para arquivos diferentes conforme o `cwd`. Figura 9.1 ilustra os dois modos sobre uma árvore concreta.

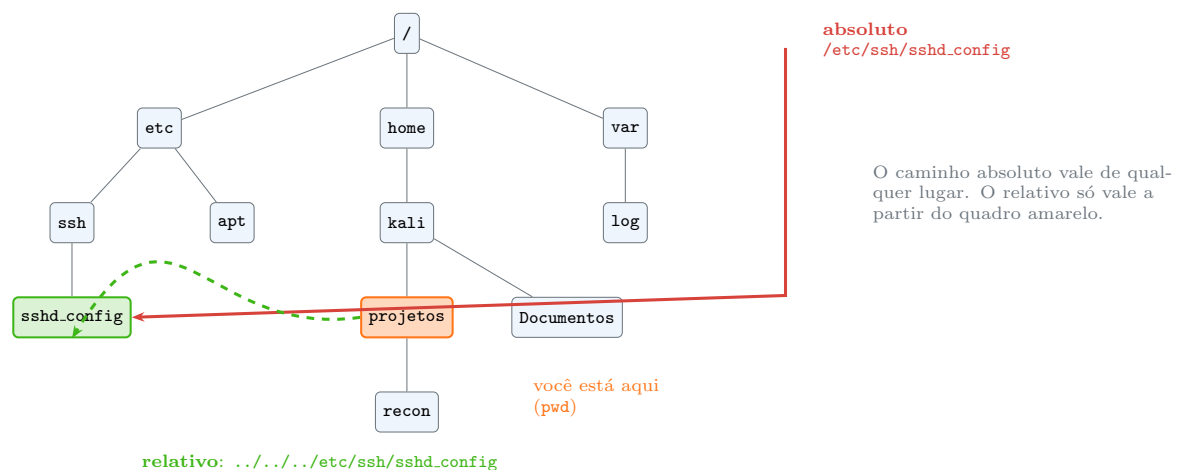


Figura 9.1.: Mesmo destino, dois caminhos. Um deles deixa de funcionar se você mudar de diretório.

Quatro atalhos aparecem em todo caminho relativo:

Tabela 9.1.: Atalhos de caminho

| Símbolo            | Significado                                      |
|--------------------|--------------------------------------------------|
| <code>.</code>     | o diretório atual                                |
| <code>..</code>    | o diretório pai                                  |
| <code>~</code>     | o diretório pessoal do usuário atual             |
| <code>~kali</code> | o diretório pessoal do usuário <code>kali</code> |
| <code>-</code>     | o diretório anterior (só com <code>cd</code> )   |

O `.` e o `..` não são invenção do shell: são **entradas reais de diretório**, criadas pelo sistema de arquivos. Dá para vê-las:

```
ls -a /etc | head -3
ls -ldi . ..
```

Isso explica por que `./script.sh` funciona mesmo sem o diretório atual estar no `PATH`: você não está pedindo uma busca, está dando um caminho — relativo, mas explícito.

O til é diferente: ele é **expandido pelo shell** antes de o comando rodar, e só no início da palavra. Consequência prática que morde:

```
cd ~/Documentos      # funciona
cd "~/Documentos"    # falha: dentro de aspas o til nao expande
cd "$HOME/Documentos" # forma correta com aspas
```

## 9.3. cd: mais do que trocar de pasta

O uso básico dispensa comentário, mas há quatro formas que economizam tempo real:

```
cd /var/log          # absoluto
cd projetos/recon    # relativo
cd ..                # sobe um nível
cd ../..             # sobe dois

cd                   # vai para o diretório pessoal - sem argumento nenhum
cd ~                 # idem
cd -                 # volta ao diretório ANTERIOR e imprime o caminho
```

O `cd -` é o que mais rende no dia a dia. Ele alterna entre dois diretórios usando a variável `OLDPWD`, que o shell atualiza a cada mudança:

```
cd /etc/apt
cd /var/log
cd -                # de volta a /etc/apt
cd -                # de volta a /var/log
echo "$OLDPWD"
```

### 9.3.1. Pilha de diretórios: pushd e popd

Quando são mais de dois diretórios, o `cd -` não basta. O shell mantém uma pilha:

```
pushd /etc/ssh      # vai para la e empilha o diretorio de origem
pushd /var/log      # empilha de novo
dirs -v             # mostra a pilha numerada
popd                # volta ao topo anterior e desempilha
popd
```

É o mecanismo natural para uma investigação que passa por vários pontos do sistema e precisa voltar ao ponto de partida sem digitar caminhos longos.

### 9.3.2. CDPATH: atalho útil e perigoso

A variável CDPATH faz para o *cd* o que o PATH faz para comandos: define diretórios onde procurar o destino.

```
export CDPATH=".:$HOME:$HOME/projetos"
cd recon          # encontra ~/projetos/recon de qualquer lugar
```

A conveniência é real, mas há duas armadilhas. Primeira: com CDPATH definido, *cd* passa a imprimir o caminho para onde foi, e isso quebra scripts que capturam a saída. Segunda: se *.* não for o primeiro elemento, um *cd algum-diretorio* pode levar você para um *algum-diretorio* de outro lugar, e não para o que está bem à sua frente. Use no *~/.zshrc*, nunca em script.

### 9.4. *ls*: ler a saída, não só listar

O *ls* sozinho já é útil, mas o formato longo é onde a informação está:

```
ls -l /etc/passwd
# -rw-r--r-- 1 root root 2891 Jul 14 09:22 /etc/passwd
```

Figura 9.2 dissectiona essa linha.

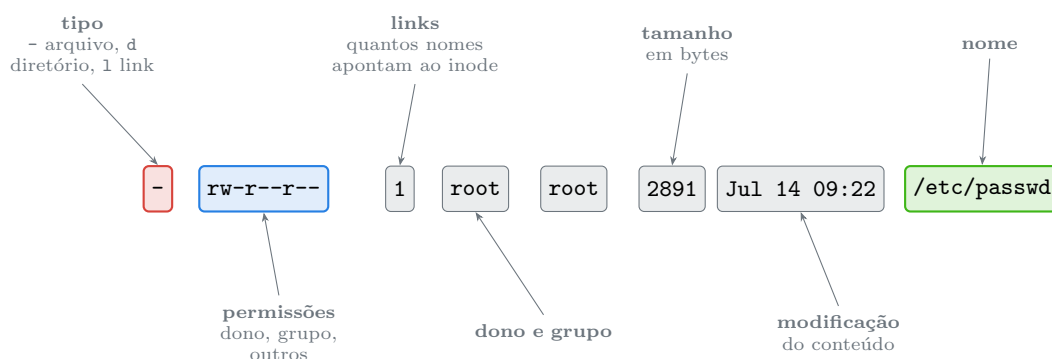


Figura 9.2.: Sete campos numa linha. Os três primeiros são o assunto dos Volumes 3 e 4.

As opções que valem memorizar:

```
ls -a      # inclui ocultos (nomes iniciados por ponto)
ls -A      # ocultos, mas sem . e ..
ls -l      # formato longo
ls -h      # tamanhos legíveis (K, M, G) - só faz sentido com -l
ls -t      # ordena por data de modificação, mais recente primeiro
ls -S      # ordena por tamanho
ls -r      # inverte a ordem
ls -R      # desce recursivamente
ls -d      # o diretório em si, não o conteúdo
ls -i      # mostra o inode
ls -1      # um por linha
```

Combinações que se tornam reflexo com o tempo:

```
ls -lah          # o padrão de trabalho: longo, ocultos, legível
ls -lt | head    # os arquivos mexidos por último
ls -lSh | head   # os maiores arquivos do diretório
ls -ld /var/log  # informação sobre o diretório, não sobre o conteúdo
ls -la ~/.ssh    # arquivos ocultos onde eles realmente importam
```

O `-d` merece destaque porque resolve uma confusão frequente: sem ele, `ls -l /var/log` lista o **conteúdo** do diretório; com ele, mostra a linha do próprio diretório — permissões, dono e data. É o que você quer ao investigar por que não consegue entrar em algum lugar.

#### 9.4.1. Arquivos ocultos não são secretos

Um arquivo cujo nome começa com ponto simplesmente não aparece no `ls` comum. Não há permissão especial, não há atributo escondido: é uma convenção do `ls`, herdada de um acidente histórico. A pasta pessoal está cheia deles:

```
ls -a ~ | head -20
```

É onde vivem `.zshrc`, `.bashrc`, `.ssh`, `.config` e o histórico do shell. Ignorar o `-a` significa achar que o diretório pessoal está vazio quando ele está cheio de configuração.

#### 9.4.2. Cores e classificação

O Kali já vem com `ls` colorido por alias. As cores seguem a variável `LS_COLORS` e distinguem diretório, executável, link, arquivo compactado e link quebrado. Um complemento útil quando as cores não estão disponíveis — por exemplo, dentro de um cano:

```
ls -F /etc | head
```

O `-F` acrescenta um sufixo ao nome: `/` para diretório, `*` para executável, `@` para link simbólico, `|` para FIFO. Funciona em qualquer terminal e sobrevive a redirecionamento.

### 9.5. Links simbólicos e o caminho “de verdade”

Boa parte da árvore do Linux moderno é feita de links. Isso cria uma ambiguidade sobre o que é o diretório atual:

```
cd /bin
pwd          # /bin
pwd -P      # /usr/bin - o caminho físico, resolvendo os links
pwd -L      # /bin      - o caminho lógico, como você chegou (padrão)
```

O shell guarda o caminho **lógico**: aquele pelo qual você chegou. O `-P` (*physical*) resolve todos os links e mostra a localização real. A mesma distinção vale para o `cd`:

```
cd -P /bin && pwd      # entra resolvendo os links
```

Isso importa em scripts que sobem níveis com `...`. Depois de entrar num diretório por um link, `cd ..` pelo caminho lógico volta para o pai do **link**, enquanto o caminho físico volta para o pai do **alvo**. Os dois podem ser lugares completamente diferentes. Links simbólicos têm capítulo próprio no Volume 3.

Para resolver um caminho até a forma canônica, sem ambiguidade:

```
readlink -f ~/../kali/./Documentos
realpath ../..
```

Ambos devolvem o caminho absoluto, sem `.`, sem `..` e sem links pelo meio. Em script, é a maneira correta de transformar o que o usuário digitou em algo confiável.

## 9.6. Onde o script está: o caminho que mais quebra

Um script que precisa ler um arquivo ao lado dele não pode usar caminho relativo simples, porque o diretório de trabalho é de quem **chamou** o script, não de onde ele está. A forma robusta:

```
#!/usr/bin/env bash
DIR_SCRIPT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd -P)"
cat "$DIR_SCRIPT/dados.txt"
```

Duas ferramentas complementares aparecem aí e valem por si:

```
basename /etc/ssh/sshd_config      # sshd_config
dirname  /etc/ssh/sshd_config      # /etc/ssh
```

O tema completo é do Volume 12, mas o padrão acima resolve o problema hoje.

## 9.7. No Kali

- **O diretório pessoal do usuário kali é `/home/kali`; o do root é `/root`.** Depois de um `sudo -i`, o `~` aponta para outro lugar, e arquivos criados ali não estão no seu diretório. É a explicação para “salvei o relatório e não acho mais”.
- **Ferramentas clonadas do GitHub costumam ir para `/opt`.** Diferentemente de `/usr/bin`, `/opt` não está no `PATH`, então navegar até lá e chamar com `./` é a rotina normal — e é o caso de uso mais frequente de `pushd` e `popd`.
- **O Zsh do Kali aceita `cd` implícito.** Digitar só `/var/log` e apertar Enter muda de diretório, sem escrever `cd`. É a opção `AUTO_CD`, ativada no `~/ .zshrc` padrão. Prático, mas não funciona em Bash nem em script.
- **O autocompletar do Zsh é mais esperto que o do Bash.** `cd /u/l/b` seguido de Tab expande para `/usr/local/bin`. Em máquinas remotas com Bash, esse recurso não existe.
- **Wordlists ficam em `/usr/share/wordlists`,** com a `rockyou.txt` frequentemente compactada em `.gz` numa instalação nova. Vale conhecer `ls -lh /usr/share/wordlists` como primeiro reflexo antes de procurar por toda parte.

- **ls -la ~/.ssh deve ser rotina.** Chaves privadas com permissão frouxa fazem o cliente SSH recusar a conexão — e, num sistema compartilhado, expõem a chave. O assunto é do Volume 4, mas o hábito de olhar começa aqui.
- **Cuidado com o pwd num shell de contêiner.** Dentro de um Docker, o caminho é relativo ao sistema de arquivos do contêiner e não corresponde ao do host. O Volume 16 trata do porquê.

**Nota sobre a família RHEL.** Navegação é idêntica: `pwd`, `cd`, `ls`, `.`, `..` e o `til` fazem parte do POSIX (IEEE and The Open Group, 2018). A diferença aparece no conteúdo dos diretórios, não na forma de andar por eles. O Bash é o padrão lá, então `AUTO_CD` e o autocompletar de caminho abreviado não estão disponíveis.

## 9.8. Laboratório

Roteiro na VM. Tudo acontece dentro de `/tmp` e do seu diretório pessoal.

1. **Confirme que o cwd pertence ao processo.**

```
pwd
ls -l /proc/$$/cwd
cd /var/log
ls -l /proc/$$/cwd
cd -
```

Esperado: o link em `/proc` acompanha o `cd`. Você está olhando o dado que o kernel guarda, não uma variável do shell.

2. **Prove que um script não muda o seu diretório.**

```
cd /tmp
printf '#!/bin/bash\ncd /etc\npwd\n' > muda-dir.sh
chmod +x muda-dir.sh
./muda-dir.sh
pwd
source ./muda-dir.sh
pwd
cd /tmp
```

Esperado: o script imprime `/etc`, mas o seu `pwd` continua `/tmp`. Com `source`, o shell atual se move de verdade. Essa diferença de duas letras é uma das mais importantes do manual.

3. **Monte uma árvore de teste e navegue por caminhos relativos.**

```
mkdir -p /tmp/lab009/alfa/beta/gama
cd /tmp/lab009/alfa/beta/gama
pwd
cd ../../
pwd
cd ../beta/gama
pwd
cd ../../../../
pwd
```

## 9. Navegação: *pwd*, *cd*, *ls* e caminhos absolutos e relativos

Esperado: a cada passo o *pwd* confirma a posição. Repare que *../..* do último nível chega em *alfa*, e mais dois níveis chegam em */tmp*.

### 4. Reproduza a armadilha da abertura do capítulo.

```
mkdir -p /tmp/lab009/configuracoes
cd /tmp/lab009/alfa
ls ../configuracoes          # funciona
cd /tmp/lab009/alfa/beta
ls ../configuracoes          # falha
ls ../../configuracoes       # funciona
```

Esperado: o mesmo texto acerta e erra conforme o diretório. Esse é o motivo de scripts de produção usarem caminho absoluto ou resolverem o caminho do próprio script.

### 5. Veja *.* e *..* como entradas reais.

```
cd /tmp/lab009/alfa
ls -a
ls -ldi . .. ../alfa
```

Esperado: *.* e *..* aparecem na listagem, e o inode de *..* coincide com o do diretório pai. Não são invenção do shell — estão gravados no sistema de arquivos.

### 6. Alterne com *cd -* e explore a pilha.

```
cd /etc/apt
cd /var/log
cd -
cd -
pushd /usr/share
pushd /tmp
dirs -v
popd
popd
pwd
```

Esperado: *cd -* alterna entre dois pontos; *dirs -v* mostra a pilha numerada; cada *popd* desempilha um nível.

### 7. Descubra o til que não expande.

```
echo ~
echo "~"
echo "$HOME"
ls "~" 2>/dev/null || echo "falhou por causa das aspas"
```

Esperado: sem aspas, o til vira o caminho; entre aspas, permanece literal e o *ls* procura um arquivo chamado til. Em script, *"\$HOME"* é a forma correta.

### 8. Explore a saída do *ls -l* campo a campo.

```
ls -l /etc/passwd
ls -ld /etc
ls -l /bin
ls -lh /usr/share/wordlists 2>/dev/null || ls -lh /usr/share | head
```



Esperado: o primeiro caractere muda entre traço, d e l. Confira cada campo contra Figura 9.2.

#### 9. Compare as ordenações.

```
cd /tmp/lab009
touch antigo.txt && sleep 2 && touch novo.txt
head -c 200000 /dev/urandom > grande.bin
ls -lt
ls -lSh
ls -ltr
```

Esperado: -t coloca novo.txt primeiro, -S coloca grande.bin primeiro, -r inverte. O ls -ltr é o padrão de quem investiga log: o mais recente fica na última linha, junto ao prompt.

#### 10. Encontre os arquivos ocultos do seu diretório pessoal.

```
ls ~ | wc -l
ls -A ~ | wc -l
ls -ld ~/.ssh ~/.config ~/.zshrc 2>/dev/null
```

Esperado: a segunda contagem é bem maior. A configuração toda estava lá, apenas fora da listagem padrão.

#### 11. Veja a diferença entre caminho lógico e físico.

```
cd /bin
pwd
pwd -P
cd ..
pwd
cd /bin && cd -P . && cd .. && pwd
```

Esperado: pwd diz /bin, pwd -P diz /usr/bin, e o cd .. leva a lugares diferentes conforme o modo. É a mesma armadilha que faz um script de instalação se perder.

#### 12. Canonize caminhos bagunçados.

```
readlink -f /tmp/lab009/alfa/../alfa/./beta
realpath ~/.
basename /usr/share/wordlists/rockyou.txt
dirname /usr/share/wordlists/rockyou.txt
```

Esperado: caminhos limpos e absolutos. Em script, sempre canonize o que veio de fora antes de usar.

#### 13. Limpe.

```
cd /tmp
rm -rf /tmp/lab009 /tmp/muda-dir.sh
```

Repare que o rm -rf só aparece aqui com um caminho absoluto e específico, digitado por inteiro. Esse é o hábito a cultivar, e o tema do próximo capítulo.

## 9.9. Resumo

- **O diretório de trabalho é um atributo do processo**, visível em `/proc/<PID>/cwd`. Cada aba do terminal tem o seu, e um processo filho herda o do pai sem nunca devolver mudanças.
- **Um script que faz *cd* não move o seu shell** — ele move o próprio processo, que morre em seguida. Para mover o shell atual, é `source`.
- **Caminho absoluto começa em / e vale de qualquer lugar; relativo depende do *cwd***. O mesmo texto acerta ou erra conforme o ponto de partida.
- **. e .. são entradas reais no sistema de arquivos**, não sintaxe do shell — por isso `./script.sh` funciona sem o diretório atual estar no `PATH`.
- **O til é expandido pelo shell e só fora de aspas**. Dentro de aspas, use `"$HOME"`.
- ***cd* - alterna entre dois diretórios; *pushd* e *popd* gerenciam uma pilha** quando são mais de dois.
- **A saída de *ls -l* tem sete campos**: tipo, permissões, contagem de links, dono, grupo, tamanho, data e nome. `ls -lah`, `ls -ltr` e `ls -ld` cobrem a maior parte das necessidades reais.
- ***pwd* mostra o caminho lógico; *pwd -P* resolve os links**. A diferença muda o resultado de um `cd ..` e é fonte comum de scripts que se perdem.

## 10. Manipulando arquivos e diretórios: cp, mv, rm, mkdir

Você termina o relatório de um teste e quer guardá-lo em segurança:

```
mv relatorio-final.md backups/
```

Nenhum erro. Nenhuma mensagem. Você segue em frente. Uma semana depois, procura o relatório em `backups/` e não encontra nada — porque o diretório `backups` nunca existiu. O `mv` fez exatamente o que você pediu: **renomeou** `relatorio-final.md` para um arquivo chamado `backups`. O conteúdo está lá, inteiro, com o nome errado e sem extensão. Se naquela mesma semana você tivesse rodado a linha de novo com outro relatório, o primeiro teria sido sobrescrito sem aviso nenhum.

Essa família de comandos — `cp`, `mv`, `rm`, `mkdir` — é a que você mais vai usar e a única que pode destruir trabalho de forma irreversível. Não há lixeira. Não há `Ctrl+Z`. O `rm` não pede confirmação, o `mv` não avisa antes de sobrescrever, e o `cp` faz o mesmo. A ferramenta obedece.

Este capítulo trata desses quatro comandos com o cuidado que eles merecem: primeiro o modelo mental do que cada um faz de verdade no sistema de arquivos, depois as opções, e por fim os hábitos que separam quem já perdeu dados de quem ainda não perdeu.

### 10.1. Nomes e conteúdo são coisas separadas

Antes das opções, um modelo que explica os três comandos de uma vez. No Linux, o **nome** de um arquivo e o **conteúdo** dele vivem em lugares diferentes. Um diretório é uma tabela que associa nomes a números de inode; o inode é a estrutura que guarda metadados e aponta para os blocos de dados. Figura 10.1 mostra o efeito de cada operação nesse arranjo.

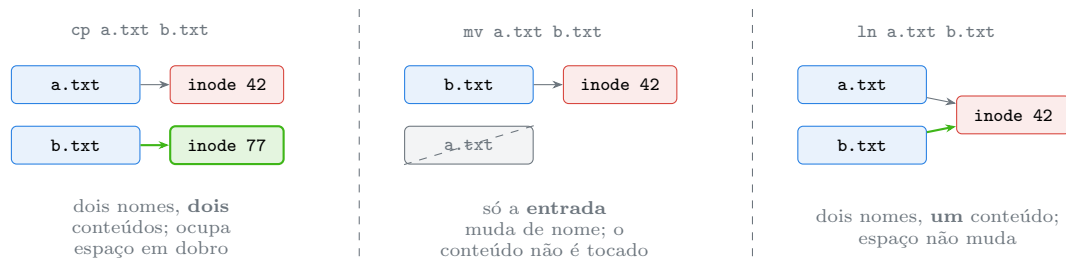


Figura 10.1.: Azul são nomes, vermelho é conteúdo. Copiar duplica o conteúdo; mover mexe só no nome.

Três consequências práticas seguem daí:

**mv dentro do mesmo sistema de arquivos é instantâneo**, mesmo para um arquivo de 40 GB. O comando só reescreve uma entrada de diretório. Já **mv entre sistemas de arquivos**

## 10. Manipulando arquivos e diretórios: *cp*, *mv*, *rm*, *mkdir*

diferentes — do disco para um pendrive, por exemplo — não pode fazer isso: ele copia os dados e depois apaga a origem, e aí demora e pode ser interrompido no meio.

**cp é a operação cara.** Ela lê e escreve o conteúdo inteiro, ocupa espaço adicional e leva tempo proporcional ao tamanho.

**rm não apaga dados; apaga um nome.** Ele remove a entrada de diretório e decrementa o contador de links do inode. O conteúdo só é liberado quando esse contador chega a zero e nenhum processo tem o arquivo aberto. É por isso que apagar um log enquanto um serviço escreve nele não libera espaço em disco — o assunto reaparece no Volume 3.

### 10.2. *mkdir*: criando diretórios

```
mkdir relatorios
mkdir -p projetos/cliente/2026/evidencias    # cria toda a cadeia
mkdir -v teste                               # informa o que criou
mkdir -m 700 privado                         # ja nasce com a permissao
```

O `-p` (*parents*) faz duas coisas úteis: cria os diretórios intermediários que faltarem e **não reclama** se o alvo já existir. Essa segunda propriedade é o que torna a opção obrigatória em scripts:

```
mkdir /tmp/trabalho                         # falha se ja existir, e o script para com set -e
mkdir -p /tmp/trabalho                     # idempotente: pode rodar mil vezes
```

O `-m` define a permissão no momento da criação, o que é diferente de criar e depois rodar `chmod`. Entre uma coisa e outra existe uma janela em que o diretório está aberto a todos — irrelevante na sua máquina, relevante num servidor compartilhado.

Combinado com a expansão de chaves de Capítulo 13, o *mkdir* cria estruturas inteiras numa linha:

```
mkdir -p projeto/{doc,src,teste}/{entrada,saida}
```

Para remover, *rmdir* só funciona com diretório vazio — e é justamente essa recusa que o torna seguro:

```
rmdir vazio
rmdir -p a/b/c                             # remove c, depois b, depois a, enquanto estiverem vazios
```

### 10.3. *touch*: criar e carimbar

```
touch arquivo.txt                          # cria vazio, ou atualiza a data se ja existir
touch a.txt b.txt c.txt                   # varios de uma vez
```

O propósito original do `touch` não é criar arquivos, e sim mexer nos carimbos de tempo. Todo arquivo tem três:

Tabela 10.1.: Os três carimbos de tempo de um arquivo

| Carimbo            | Significado                                                    | Ver com             |
|--------------------|----------------------------------------------------------------|---------------------|
| <code>mtime</code> | última modificação do <b>conteúdo</b>                          | <code>ls -l</code>  |
| <code>atime</code> | último <b>acesso</b> (leitura)                                 | <code>ls -lu</code> |
| <code>ctime</code> | última mudança nos <b>metadados</b><br>(permissão, dono, nome) | <code>ls -lc</code> |

```
touch -d "2020-01-01 10:00" antigo.txt # define uma data especifica
touch -r modelo.txt copia.txt          # copia a data de outro arquivo
touch -a arquivo.txt                   # so o atime
touch -m arquivo.txt                   # so o mtime
stat arquivo.txt                       # mostra os tres de uma vez
```

O `ctime` **não** pode ser definido pelo `touch` — ele é atualizado pelo kernel a cada mudança de metadado, inclusive pelo próprio `touch`. Isso tem peso em perícia digital: alguém pode falsificar `mtime` e `atime` com facilidade, mas um `ctime` mais recente que os outros dois é sinal de que o arquivo foi mexido depois da data que ele exibe.

## 10.4. cp: copiar com intenção

```
cp origem.txt destino.txt              # copia para um novo nome
cp arquivo.txt /tmp/                   # copia para dentro de um diretorio
cp a.txt b.txt c.txt /tmp/backup/      # varios para um diretorio (ele deve
↪ existir)
cp -r pasta/ /tmp/                     # recursivo: obrigatorio para diretorios
```

As opções que mudam o resultado de verdade:

```
cp -i # pergunta antes de sobrescrever (interactive)
cp -n # nunca sobrescreve (no-clobber), sem perguntar
cp -u # so copia se a origem for mais nova (update)
cp -v # mostra cada arquivo copiado (verbose)
cp -p # preserva mtime, dono e permissoes
cp -a # arquivamento: -r + -p + preserva links simbolicos
cp --backup=numbered # renomeia o antigo em vez de destruir
```

O `-a` é o padrão para copiar árvores que precisam continuar iguais — um diretório de configuração, uma coleta de evidências, o conteúdo de um sistema de arquivos. Sem ele, permissões viram as do seu `umask`, datas viram “agora” e links simbólicos são seguidos e copiados como arquivos comuns, o que pode inflar a cópia absurdamente.

### 10.4.1. A barra final muda o resultado

Este é um detalhe que causa surpresa genuína:

```
cp -r origem destino/      # se destino existe: cria destino/origem
cp -r origem/ destino/     # no cp do GNU, o mesmo resultado
rsync -a origem destino/   # cria destino/origem
rsync -a origem/ destino/  # copia o CONTEUDO de origem para dentro de destino
```

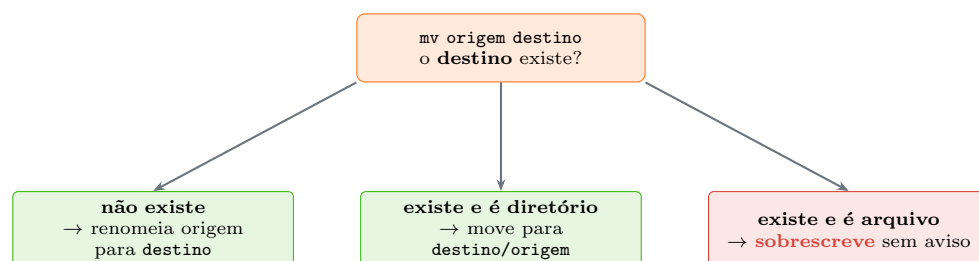
No *cp*, a barra final na origem tem pouco efeito. No *rsync*, ela muda tudo. Como as duas ferramentas convivem no mesmo trabalho, o hábito seguro é sempre testar com *--dry-run* no *rsync* e conferir o resultado do *cp* com *ls* antes de confiar.

O que **sempre** importa é se o destino existe e é um diretório:

```
cp a.txt b.txt             # b.txt nao existe -> cria uma copia com esse nome
cp a.txt b.txt             # b.txt existe como arquivo -> SOBRESCREVE, sem avisar
cp a.txt pasta             # pasta existe como diretorio -> cria pasta/a.txt
```

## 10.5. *mv*: mover e renomear são a mesma coisa

Não existe comando *rename* padrão no Unix porque não é preciso: mover um arquivo para o mesmo diretório com outro nome é renomear. Figura 10.2 mostra a decisão que o *mv* toma — e é ela que explica o desastre da abertura do capítulo.



Antídotos: *mv -i*, *mv -n*, *mv origem destino/*

O primeiro caso é o que morde: um diretório de destino digitado errado vira um arquivo com nome de diretório, silenciosamente.

Figura 10.2.: Três resultados completamente diferentes para a mesma linha de comando.

O antídoto barato: **sempre terminar o destino com barra** quando a intenção é mover para dentro de um diretório.

```
mv relatorio.md backups/    # se backups nao existir, ERRA em vez de renomear
```

Com a barra, um destino inexistente produz “Not a directory” — que é exatamente o que se quer ouvir.

As opções úteis:

```
mv -i    # pergunta antes de sobrescrever
mv -n    # nunca sobrescreve
mv -v    # mostra o que foi movido
mv -u    # so move se a origem for mais nova
mv --backup=numbered # preserva o antigo como destino.~1~
```

### 10.5.1. *mv* é atômico e isso tem uso

Dentro do mesmo sistema de arquivos, *mv* é uma operação **atômica**: em nenhum instante o destino existe pela metade. Isso dá um padrão valioso para escrever arquivos de configuração sem risco:

```
comando-que-gera-a-saida > /etc/servico/config.novo
mv /etc/servico/config.novo /etc/servico/config
```

Se algo falhar na geração, o arquivo antigo continua íntegro. Qualquer processo que leia a configuração vê a versão antiga ou a nova, nunca um arquivo cortado no meio. Escrever direto no destino não oferece essa garantia.

### 10.5.2. Renomear muitos arquivos

O *mv* renomeia um por vez. Para lotes, o Debian e o Kali trazem o *rename* em Perl:

```
rename 's/\.jpeg$/\.jpg/' *.jpeg      # troca a extensao
rename -n 's/ /_/g' *                  # -n = simulacao, nao altera nada
rename 'y/A-Z/a-z/' *                  # tudo para minusculas
```

O *-n* mostra o que aconteceria sem fazer nada. É o *--dry-run* desta ferramenta, e usá-lo antes de cada execução real deveria ser reflexo.

## 10.6. *rm*: o comando sem volta

```
rm arquivo.txt
rm a.txt b.txt c.txt
rm -r pasta/          # recursivo: obrigatorio para diretorios
rm -f arquivo         # forca: nao pergunta, nao reclama de inexistente
rm -rf pasta/         # a combinacao famosa e perigosa
rm -i arquivo         # pergunta a cada arquivo
rm -I *.txt           # pergunta UMA vez se forem mais de tres
rm -v arquivo         # mostra o que apagou
rm -- -arquivo-estranho # nome que comeca com traco
```

O *-I* maiúsculo é subestimado: ele dá uma única confirmação para operações em lote, sem a irritação do *-i* perguntando arquivo por arquivo. É o meio-termo que realmente se usa.

⚠ Não existe lixeira, e o `rm -rf` não pede licença

`rm` não move nada para lugar nenhum. Recuperar exige ferramenta de perícia, backup, ou sorte — e nos sistemas de arquivos modernos, com journaling e SSD com TRIM, geralmente nem isso funciona.

Antes de qualquer `rm -rf`, aplique três verificações:

```
pwd                # 1. onde estou?
ls -la caminho/    # 2. e isto mesmo que eu quero apagar?
rm -rf caminho/     # 3. so entao
```

Trocar o passo 2 por `echo rm -rf caminho/` também funciona: o `echo` mostra a linha final, já expandida, sem executar.

Quatro linhas que já destruíram sistemas de verdade:

```
rm -rf / caminho/  # o espaco depois da barra apaga a raiz inteira
rm -rf "$DIR"/*     # se DIR estiver vazia, isto vira rm -rf /*
rm -rf .*          # em shells antigos, o .. entrava no glob e subia a
↪ arvore
cd /caminho ; rm -rf * # se o cd falhar, apaga o diretorio em que voce esta
```

A quarta é a mesma armadilha de Capítulo 6: `&&` em vez de `;`. As demais têm a mesma raiz — o shell expande antes, e o `rm` obedece ao resultado da expansão, não à sua intenção.

O `rm` moderno recusa `rm -rf /` graças a `--preserve-root`, que é o padrão. Mas a proteção cobre só a raiz literal: `rm -rf /*` passa, e `rm -rf "$VAZIA"/*` também.

### 10.6.1. Lixeira de verdade no terminal

Existe um caminho seguro, e vale adotá-lo:

```
sudo apt install trash-cli
trash-put arquivo.txt      # vai para ~/.local/share/Trash
trash-list
trash-restore
trash-empty 30             # esvazia o que tem mais de 30 dias
```

O `trash-put` usa a mesma lixeira da interface gráfica, o que significa que dá para recuperar pelo gerenciador de arquivos. Muita gente experiente coloca `alias rm='trash-put'` no `~/.zshrc` — mas isso tem um custo: você se acostuma com uma rede de proteção que **não existe** em servidores, contêineres nem scripts. A alternativa mais honesta é criar um alias novo, `alias lixo='trash-put'`, e deixar o `rm` sendo o que ele é.

## 10.7. Copiar árvores grandes: quando `cp` não basta

Para diretórios grandes, através da rede, ou quando é preciso poder retomar de onde parou, o `rsync` é a ferramenta certa mesmo localmente:



```
rsync -av --dry-run origem/ destino/      # SEMPRE simule primeiro
rsync -av origem/ destino/                # so entao execute
rsync -av --progress origem/ destino/     # com barra de progresso
```

Ele copia só o que mudou, preserva metadados com `-a`, mostra o que fará com `--dry-run` e retoma transferências interrompidas. O uso remoto é do Volume 13.

## 10.8. No Kali

- **Não instale alias `rm='rm -i'` e confie nele.** Numa VM descartável, tudo bem; num pentest com shell numa máquina alvo autorizada, o alias não existe e o reflexo aprendido falha. Pior: `rm -f` ignora o `-i` do alias silenciosamente.
- **Coleta de evidências pede `cp -a` ou `rsync -a`.** Preservar `mtime`, dono e permissões faz parte da integridade do material. Uma cópia com `cp` simples carimba todos os arquivos com a data de hoje e destrói justamente o dado que interessa.
- **Para imagem de disco, `cp` não serve.** Use `dd` ou, melhor, `dcfldd`, que já vem no Kali e calcula o hash durante a cópia. `dd` escrito no dispositivo errado destrói o disco inteiro sem confirmação — confira o alvo com `lsblk` antes, e prefira ensaiar com um arquivo-imagem criado por `losetup`. O tema completo é do Volume 8.
- **Wordlists comprimidas precisam ser descompactadas antes do uso,** e `gunzip` remove o `.gz` original por padrão. Para manter o arquivo compactado: `gunzip -k`, ou `zcat rockyou.txt.gz > /tmp/rockyou.txt`. Nunca descompacte dentro de `/usr/share`, que é território do gerenciador de pacotes (Capítulo 5) — trabalhe em `/tmp` ou no seu diretório pessoal.
- **`/opt` é onde as ferramentas clonadas moram,** e mover um clone de Git com `mv` é seguro: o diretório `.git` vai junto. Já copiar com `cp -r` sem `-a` desfaz links simbólicos internos e pode inflar o resultado.
- **Trabalhe em `/tmp` para testes destrutivos.** Ele é limpo a cada boot, e no Kali costuma estar em disco, não em RAM — então cuidado com arquivos grandes se o espaço for apertado. Confira com `df -h /tmp`.
- **`trash-cli` não vem instalado,** mas está no repositório. Numa VM de estudo é uma adição sensata.

**Nota sobre a família RHEL.** Os quatro comandos são os mesmos do GNU coreutils e se comportam identicamente. A diferença notável é que o Fedora e o RHEL vêm com alias `rm='rm -i'` para o root por padrão, o que dá uma falsa sensação de rede de proteção ao migrar entre as famílias — no Debian e no Kali, esse alias não existe.

## 10.9. Laboratório

Roteiro em `/tmp`, sem tocar em nada do sistema.

### 1. Prepare a área de trabalho.

```
mkdir -p /tmp/lab010/{origem,destino,backup}
cd /tmp/lab010
ls -R
```

## 10. Manipulando arquivos e diretórios: *cp*, *mv*, *rm*, *mkdir*

Esperado: três diretórios criados numa linha. A expansão de chaves é do próximo tema do volume, mas o resultado já é visível.

### 2. Confirme que *mkdir -p* é idempotente.

```
mkdir /tmp/lab010/origem ; echo "status: $?"  
mkdir -p /tmp/lab010/origem ; echo "status: $?"
```

Esperado: erro e status não-zero na primeira, silêncio e status 0 na segunda. É por isso que scripts usam *-p* mesmo quando não há níveis a criar.

### 3. Crie arquivos e observe os três carimbos.

```
cd /tmp/lab010/origem  
touch a.txt b.txt c.txt  
stat a.txt  
touch -d "2020-01-01 10:00" antigo.txt  
ls -l  
ls -lc antigo.txt
```

Esperado: *antigo.txt* mostra 2020 no *ls -l* (mtime) mas a data de hoje no *ls -lc* (ctime). Essa discrepância é a assinatura de um carimbo alterado.

### 4. Veja o efeito do *cp* no inode.

```
cd /tmp/lab010/origem  
echo "conteudo original" > a.txt  
cp a.txt copia.txt  
ls -li a.txt copia.txt  
echo "alterado" >> copia.txt  
cat a.txt
```

Esperado: inodes diferentes, e alterar a cópia não mexe no original. Compare com Figura 10.1.

### 5. Veja o efeito do *mv* no inode.

```
ls -li copia.txt  
mv copia.txt renomeada.txt  
ls -li renomeada.txt
```

Esperado: **o mesmo número de inode**. Só o nome mudou; o conteúdo nunca foi lido nem reescrito.

### 6. Reproduza o desastre da abertura.

```
cd /tmp/lab010/origem  
echo "relatorio importante" > relatorio.md  
mv relatorio.md nao-existe/  
mv relatorio.md nao-existe  
ls -l  
cat nao-existe
```

Esperado: com barra, erro claro. Sem barra, silêncio — e um arquivo chamado *nao-existe* contendo o seu relatório. Adote a barra como hábito permanente.

7. Veja o `cp` sobrescrever sem avisar, e depois se proteger.

```
cd /tmp/lab010/origem
echo "versao 1" > alvo.txt
echo "versao 2" > fonte.txt
cp fonte.txt alvo.txt
cat alvo.txt
echo "versao 3" > fonte.txt
cp -n fonte.txt alvo.txt
cat alvo.txt
cp --backup=numbered fonte.txt alvo.txt
ls -l alvo*
```

Esperado: a primeira cópia destrói a versão 1 em silêncio; o `-n` recusa; o `--backup=numbered` copia e preserva o antigo como `alvo.txt.~1~`.

8. Compare `cp -r` com `cp -a`.

```
cd /tmp/lab010
mkdir -p arvore/sub
echo dados > arvore/sub/arquivo.txt
chmod 700 arvore/sub
touch -d "2019-05-05" arvore/sub/arquivo.txt
cp -r arvore copia-r
cp -a arvore copia-a
ls -ld arvore/sub copia-r/sub copia-a/sub
ls -l arvore/sub/arquivo.txt copia-r/sub/arquivo.txt
↵ copia-a/sub/arquivo.txt
```

Esperado: a cópia com `-a` mantém permissão 700 e a data de 2019; a cópia com `-r` não. Para evidências ou backup, é sempre `-a`.

9. Use o `mv` atômico.

```
cd /tmp/lab010
echo "config antiga" > config.txt
printf 'config nova\nlinha 2\n' > config.txt.novo
mv config.txt.novo config.txt
cat config.txt
```

Esperado: a troca acontece de uma vez. Nenhum leitor jamais veria um arquivo pela metade.

## 10. Simule uma renomeação em lote antes de executar.

```
cd /tmp/lab010/origem
touch foto1.jpeg foto2.jpeg foto3.jpeg
rename -n 's/\.jpeg$/\.jpg/' *.jpeg
rename 's/\.jpeg$/\.jpg/' *.jpeg
ls
```

Esperado: o `-n` lista as mudanças planejadas sem aplicar; a segunda linha aplica. Sempre nessa ordem.

11. Experimente as confirmações do `rm`.

## 10. Manipulando arquivos e diretórios: *cp*, *mv*, *rm*, *mkdir*

```
cd /tmp/lab010/origem
touch d1.txt d2.txt d3.txt d4.txt d5.txt
rm -I d*.txt
touch e1.txt e2.txt
rm -i e1.txt e2.txt
```

Esperado: o `-I` pergunta uma única vez para o lote; o `-i` pergunta uma vez por arquivo. O `-I` é o que se usa na prática.

## 12. Veja o `echo` como ensaio de um comando destrutivo.

```
cd /tmp/lab010
DIR=""
echo rm -rf "$DIR"/*
DIR="/tmp/lab010/destino"
echo rm -rf "$DIR"/*
```

Esperado: a primeira linha imprime `rm -rf /*`. Você acabou de ver, sem consequências, o que uma variável vazia faz com um comando destrutivo. **Não remova o `echo`** desse primeiro caso.

## 13. Compare `rmdir` com `rm -r`.

```
cd /tmp/lab010
mkdir -p cheio/sub && touch cheio/sub/arquivo
rmdir cheio          # falha: nao esta vazio
rmdir cheio/sub      # falha: nao esta vazio
rm -r cheio          # funciona
```

Esperado: o `rmdir` recusa duas vezes. Essa recusa é uma função de segurança — use-o quando a intenção for remover algo que **deveria** estar vazio.

## 14. Instale e teste uma lixeira.

```
sudo apt install -y trash-cli
cd /tmp/lab010
touch recuperavel.txt
trash-put recuperavel.txt
trash-list
trash-restore
ls
```

Esperado: o arquivo some, aparece na lista da lixeira e volta com o `trash-restore`. Considere um alias `lixo` no `~/.zshrc` — sem renomear o `rm`.

## 15. Limpe.

```
pwd
ls -la /tmp/lab010
rm -rf /tmp/lab010
```

Os três passos, na ordem, como no aviso do capítulo. Faça disso um ritual.

## 10.10. Resumo

- **Nome e conteúdo são separados.** `cp` duplica o conteúdo, `mv` mexe apenas na entrada de diretório e `rm` remove um nome — o conteúdo só desaparece quando nenhum nome e nenhum processo apontam mais para ele.
- **`mv` no mesmo sistema de arquivos é instantâneo e atômico**, o que dá o padrão seguro de gerar em arquivo temporário e mover por cima. Entre sistemas de arquivos, ele vira cópia mais remoção.
- **`mv` origem destino faz três coisas diferentes** conforme o destino não existir, ser diretório ou ser arquivo. Terminar o destino com barra transforma o caso perigoso em erro.
- **`mkdir -p` é idempotente** e por isso obrigatório em scripts; `rmdir` só remove diretório vazio, e essa recusa é um recurso de segurança.
- **`cp -a` preserva permissões, datas e links**; `cp -r` sozinho carimba tudo com a data de hoje e aplica o seu `umask`. Para backup ou evidência, é sempre `-a`.
- **Não existe lixeira.** Antes de qualquer `rm -rf: pwd, ls -la` no alvo, e só então o comando. Trocar `rm` por `echo rm` é o ensaio mais barato que existe.
- **Variável vazia em caminho destrutivo é a falha clássica.** `rm -rf "$DIR"/*` com `DIR` vazia vira `rm -rf /*`, e `--preserve-root` não protege desse caso.
- **`rename -n` e `rsync --dry-run` simulam antes de agir.** Toda ferramenta de lote tem um modo de ensaio, e usá-lo primeiro custa dois segundos.



## 11. Visualizando conteúdo: cat, less, head, tail e more

Duas cenas que todo mundo vive uma vez:

```
cat /var/log/syslog
```

O terminal cospe centenas de milhares de linhas em alta velocidade. Você aperta **Ctrl+C**, mas já perdeu o buffer de rolagem inteiro, e o que interessava — as três linhas do erro que você procurava — passou voando.

```
cat /usr/bin/nmap
```

Agora a tela vira lixo, o cursor some, letras viram símbolos e o prompt não volta ao normal nem depois de **clear**. Você fecha a janela achando que travou algo.

Nenhum dos dois é acidente: os dois são o **cat** fazendo exatamente o que ele foi projetado para fazer, que **não** é exibir arquivos. Este capítulo separa a ferramenta certa para cada situação — arquivo pequeno, arquivo enorme, arquivo binário, arquivo comprimido, arquivo que ainda está crescendo — e mostra por que o **less** merece o investimento de aprender dez teclas.

### 11.1. Escolher a ferramenta antes de abrir o arquivo

O primeiro reflexo, antes de qualquer visualizador, é descobrir com o que você está lidando:

```
file /var/log/syslog          # ASCII text
file /usr/bin/nmap           # ELF 64-bit LSB pie executable
file /usr/share/wordlists/rockyou.txt.gz # gzip compressed data
ls -lh /var/log/syslog       # tamanho
wc -l /var/log/syslog        # quantas linhas
```

O **file** não confia na extensão: ele lê os primeiros bytes e compara com um banco de assinaturas. Um arquivo chamado **relatorio.txt** que na verdade é um PDF é identificado corretamente, e um binário sem extensão nenhuma também.

Figura 11.1 resume a decisão.

## 11. Visualizando conteúdo: *cat*, *less*, *head*, *tail* e *more*

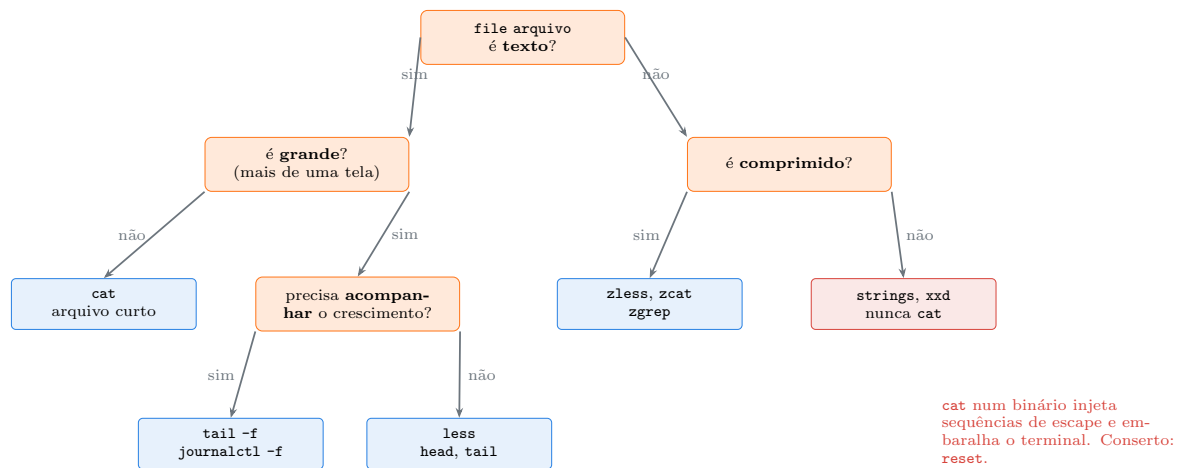


Figura 11.1.: Duas perguntas antes de abrir qualquer arquivo evitam quase todos os acidentes de terminal.

### 11.2. *cat*: concatenar, não visualizar

O nome vem de *concatenate*. A função original é juntar arquivos e mandar o resultado para a saída padrão:

```
cat parte1.txt parte2.txt parte3.txt > completo.txt
```

Exibir na tela é apenas o que acontece quando você dá um arquivo só e não redireciona nada. As opções que valem:

```
cat -n arquivo      # numera todas as linhas
cat -b arquivo      # numera so as linhas nao vazias
cat -s arquivo      # colapsa linhas em branco repetidas
cat -A arquivo      # mostra caracteres invisiveis
cat -T arquivo      # so as tabulacoes, como ^I
cat -E arquivo      # marca o fim de linha com $
```

O `-A` é uma ferramenta de diagnóstico séria. Ele revela três coisas que causam bugs difíceis:

```
printf 'linha com espaco no fim  \r\n\ttabulacao\n' > /tmp/sujo.txt
cat -A /tmp/sujo.txt
# linha com espaco no fim  ^M$
# ^Itabulacao$
```

O `^M` é o retorno de carro do Windows (CRLF), causa clássica de script que responde `bad interpreter: /bin/bash^M`. O `^I` é uma tabulação onde você achava que havia espaços — fatal em Makefile e em YAML. O `$` marca o fim de cada linha, revelando espaços em branco no final.

Para escrever um arquivo curto sem abrir editor:



```
cat > nota.txt
digite o texto
e mais uma linha
# Ctrl+D encerra
```

Ctrl+D não é “cancelar”: é o sinal de fim de arquivo na entrada padrão. Ctrl+C no lugar dele aborta e deixa o arquivo pela metade.

### 11.2.1. *tac* e o uso inútil de *cat*

```
tac arquivo.txt          # mesmo conteudo, ultima linha primeiro
```

Útil para ler log do mais recente para o mais antigo sem inverter a ordenação.

E uma correção de estilo que aparece em toda revisão de script:

```
cat arquivo.txt | grep erro      # inutil: um processo a mais
grep erro arquivo.txt           # o grep abre o arquivo sozinho
grep erro < arquivo.txt         # tambem correto
```

Chamam isso de *useless use of cat*. Não é pecado mortal, mas em pipelines longos sobre arquivos grandes o processo extra e a cópia de dados custam. Todo comando que aceita nome de arquivo dispensa o *cat*.

### 11.2.2. Binários e o terminal embaralhado

Quando um binário passa pelo terminal, os bytes que por acaso formarem sequências de escape VT100 são **interpretados** — trocam a fonte, mudam o mapa de caracteres, reposicionam o cursor. O terminal não travou; ele obedeceu.

O conserto:

```
reset          # reinicializa o terminal por completo
tput reset     # equivalente
stty sane      # restaura os modos do driver do terminal
```

Se nem o prompt aparecer, digite **reset** às cegas e aperte Enter — geralmente funciona. Para inspecionar binários de verdade:

```
strings /usr/bin/nmap | head -20    # so as sequencias de texto legivel
xxd /usr/bin/nmap | head -5         # hexadecimal com ASCII ao lado
hexdump -C /usr/bin/nmap | head -5  # equivalente
```

O *strings* é a primeira ferramenta de qualquer análise de binário: mensagens de erro, caminhos embutidos, nomes de funções e, com frequência espantosa, credenciais deixadas no código.

### 11.3. *less*: o visualizador que se aprende uma vez

O *less* carrega o arquivo sob demanda, o que significa que abrir um log de 10 GB é instantâneo. Ele permite rolar nos dois sentidos, buscar, filtrar e acompanhar crescimento.

```
less /var/log/syslog
less -N arquivo           # com numeros de linha
less -S arquivo           # nao quebra linhas longas (rola na horizontal)
less -R arquivo           # interpreta cores ANSI
less +F arquivo           # ja abre em modo de acompanhamento
less +G arquivo           # ja abre no fim
less +/erro arquivo       # ja abre na primeira ocorrencia de "erro"
less arquivo1 arquivo2    # varios: :n proximo, :p anterior
```

As teclas que cobrem 95% do uso:

Tabela 11.1.: Teclas essenciais do *less*

| Tecla      | Ação                                                       |
|------------|------------------------------------------------------------|
| Espaço / f | avança uma tela                                            |
| b          | volta uma tela                                             |
| j / k      | uma linha para baixo / para cima                           |
| g / G      | início / fim do arquivo                                    |
| /texto     | busca para frente                                          |
| ?texto     | busca para trás                                            |
| n / N      | próxima / anterior ocorrência                              |
| &texto     | mostra <b>só</b> as linhas com o texto                     |
| -i         | liga e desliga a busca sem diferenciar maiúsculas          |
| m + letra  | marca a posição; ' + letra volta a ela                     |
| F          | acompanha o arquivo crescendo ( <b>Ctrl+C</b> sai do modo) |
| v          | abre o arquivo no editor definido em <b>EDITOR</b>         |
| h          | ajuda com a lista completa                                 |
| q          | sai                                                        |

O **&** é o recurso mais subestimado da lista. Dentro de um log gigantesco, **&sshd** deixa na tela apenas as linhas do serviço SSH, e um **&** seguido de Enter volta ao arquivo inteiro. É um **grep** interativo, sem sair do arquivo nem perder a posição.

O **F** transforma o *less* em *tail -f*, com uma vantagem decisiva: **Ctrl+C** sai do modo de acompanhamento e devolve a navegação livre, com todo o histórico ainda disponível. Com *tail -f*, sair significa perder o contexto.

Duas variáveis de ambiente que vale colocar no **~/.zshrc**:

```
export LESS='-R -i -M'      # cores, busca insensivel, barra de status
↪ detalhada
export PAGER=less
```

O *less* é também o paginador que o **man** usa (Capítulo 7), o que significa que essas teclas já valiam para toda a documentação do sistema.

### 11.3.1. *more*: só quando não houver *less*

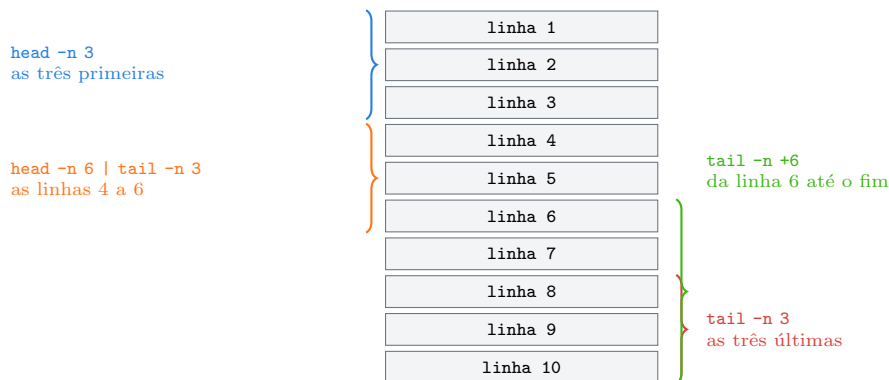
O *more* é o paginador original: avança, mas não volta e não busca para trás. Existe uma frase antiga na comunidade — *less is more* — que resume a relação. Ainda assim, vale conhecer: em contêineres minimalistas, em sistemas embarcados e em algumas máquinas antigas, *more* é o único disponível.

## 11.4. *head* e *tail*: as pontas do arquivo

```
head arquivo           # primeiras 10 linhas
head -n 20 arquivo     # primeiras 20
head -n -5 arquivo     # tudo, MENOS as ultimas 5
head -c 100 arquivo    # primeiros 100 bytes

tail arquivo           # ultimas 10 linhas
tail -n 50 arquivo     # ultimas 50
tail -n +100 arquivo   # da linha 100 ate o fim
tail -c 100 arquivo    # ultimos 100 bytes
```

Os dois formatos de *-n* no *tail* confundem, e a diferença é o sinal de mais: *tail -n 100* são as **últimas** cem linhas; *tail -n +100* é **a partir** da linha 100. Figura 11.2 mostra as faixas selecionadas por cada forma.



O sinal de mais muda tudo: *-n 3* conta do fim, *-n +3* conta do começo.

Figura 11.2.: Quatro formas de recortar faixas de um arquivo sem abrir editor.

A combinação da faixa do meio é um idioma que se usa muito:

```
head -n 20 arquivo | tail -n 5    # linhas 16 a 20
sed -n '16,20p' arquivo          # o mesmo, com sed (Volume 11)
```

E os dois aceitam vários arquivos, com cabeçalho automático:

## 11. Visualizando conteúdo: *cat*, *less*, *head*, *tail* e *more*

```
head -n 3 /etc/passwd /etc/group
tail -n 2 /var/log/*.log
head -q -n 3 *.txt          # -q suprime os cabecalhos
```

### 11.4.1. *tail -f*: acompanhando em tempo real

```
tail -f /var/log/syslog          # segue o arquivo
tail -F /var/log/syslog          # segue o NOME do arquivo
tail -f -n 100 /var/log/syslog   # mostra 100 linhas antes de começar a seguir
tail -f arquivo1 arquivo2        # varios, com cabecalho ao alternar
```

A diferença entre *-f* e *-F* importa mais do que parece. O *-f* minúsculo segue o **inode**: quando o *logrotate* renomeia *syslog* para *syslog.1* e cria um arquivo novo, o *tail -f* continua olhando o arquivo antigo, que nunca mais recebe nada. O *-F* maiúsculo segue o **nome** e reabre o arquivo novo automaticamente. Em servidor, é sempre *-F*.

Um padrão útil combina acompanhamento com filtro:

```
tail -F /var/log/auth.log | grep --line-buffered "Failed password"
```

O *--line-buffered* é obrigatório: sem ele, o *grep* acumula a saída num buffer de alguns kilobytes e você não vê nada durante minutos. O motivo está no buffering de pipes, tema de Capítulo 12.

## 11.5. Arquivos comprimidos e outros visualizadores

Descomprimir só para olhar é desperdício. Existe uma família inteira de equivalentes:

```
zcat rockyou.txt.gz | head      # cat para .gz
zless rockyou.txt.gz           # less para .gz
zgrep senha rockyou.txt.gz      # grep para .gz
zdiff a.gz b.gz                # diff para .gz
```

Para *.bz2* os prefixos são *bz* (*bzcat*, *bzless*); para *.xz*, *xz* (*xzcat*, *xzless*). O *less* moderno costuma detectar a compressão sozinho.

Complementos que valem instalar:

```
sudo apt install bat
batcat arquivo.py              # cat com cores por sintaxe e numeros de linha
```

No Debian e no Kali o binário se chama *batcat*, porque *bat* já pertencia a outro pacote. Um *alias bat='batcat'* resolve. Ele colore por sintaxe, mostra número de linha e integra com o Git — excelente para ler código, desnecessário para processar dados.

E para arquivos que não são texto puro:

```

pdftotext relatorio.pdf - | less      # o hifen manda para a saída padrão
jq . dados.json | less -R            # JSON formatado e colorido
column -t -s, dados.csv | less -S    # CSV alinhado em colunas
xxd -l 64 arquivo.bin                # os primeiros 64 bytes em hexadecimal

```

## 11.6. No Kali

- **journalctl é o less dos logs modernos.** O systemd guarda os logs num formato binário, então `cat` em `/var/log/journal` não serve para nada. Use `journalctl -u ssh`, `journalctl -f` para acompanhar, `journalctl -p err` para filtrar por severidade. O tema completo é do Volume 7.
- **/var/log/auth.log é onde as tentativas de autenticação aparecem** — inclusive todo `sudo` que você digitou. `tail -F /var/log/auth.log` numa segunda aba, enquanto trabalha, ensina muito sobre o que o sistema registra a seu respeito.
- **A rockyou.txt tem mais de 14 milhões de linhas.** `cat` nela é o caminho garantido para perder o terminal. Use `zcat rockyou.txt.gz | head`, `wc -l` para contar, `zgrep` para procurar.
- **Saída de ferramenta com cor precisa de less -R.** `nmap`, `nikto` e vários outros emitem códigos ANSI; sem o `-R`, você lê `ESC[1;32m` no meio de cada linha.
- **strings num binário suspeito é o primeiro passo de análise, não o último.** Ele revela caminhos, URLs, mensagens e às vezes chaves. Faça isso numa VM isolada, sem executar o binário — e apenas com material que você tem autorização para analisar. Análise de malware em máquina de trabalho conectada à rede da empresa é como abrir uma amostra de vírus na cozinha.
- **Relatórios de ferramenta costumam ser enormes.** `nmap -oN saida.txt` numa faixa /16 produz arquivo grande; abra com `less` e use `&` para filtrar por porta ou serviço em vez de rolar tudo.
- **bat não vem instalado**, mas está no repositório como `bat`, com o binário `batcat`.
- **Cuidado ao pagnar arquivo em diretório de evidência.** Abrir com `less` atualiza o atime do arquivo (Capítulo 10). Em contexto pericial, trabalhe sobre cópia montada como somente leitura.

**Nota sobre a família RHEL.** As ferramentas são as mesmas do `coreutils` e do pacote `less`. A diferença prática está nos caminhos: o log de autenticação é `/var/log/secure`, e não `/var/log/auth.log`. O `journalctl` funciona igual, já que o `systemd` é o mesmo.

## 11.7. Laboratório

Roteiro em `/tmp`. Um dos passos embaralha o terminal de propósito — o conserto está no passo seguinte.

### 1. Prepare arquivos de teste.

```

mkdir -p /tmp/lab011 && cd /tmp/lab011
seq 1 200 > numeros.txt
printf 'linha com espaco no fim \r\n\ttabulacao\nnormal\n' > sujo.txt
wc -l numeros.txt
file numeros.txt sujo.txt /usr/bin/ls

```

## 11. Visualizando conteúdo: *cat*, *less*, *head*, *tail* e *more*

Esperado: 200 linhas, dois arquivos de texto e um ELF 64-bit. O file é o primeiro comando de qualquer investigação.

### 2. Revele caracteres invisíveis.

```
cat sujo.txt
cat -A sujo.txt
```

Esperado: a primeira saída parece normal; a segunda mostra `^M` (fim de linha do Windows), `^I` (tabulação) e `$` (fim de linha). Guarde o `cat -A`: ele explica o famoso `bad interpreter: /bin/bash^M`.

### 3. Numere linhas de dois jeitos.

```
printf 'um\n\ndo\n\ntres\n' > vazias.txt
cat -n vazias.txt
cat -b vazias.txt
cat -s vazias.txt
```

Esperado: `-n` numera até as linhas vazias, `-b` pula as vazias, `-s` colapsa as repetidas.

### 4. Escreva um arquivo com *cat* e encerre corretamente.

```
cat > nota.txt
primeira linha
segunda linha
```

Aperte `Ctrl+D` numa linha vazia. Depois:

```
cat nota.txt
```

Esperado: as duas linhas gravadas. Repita usando `Ctrl+C` no lugar de `Ctrl+D` e compare — o arquivo fica truncado.

### 5. Embaralhe o terminal de propósito e conserte.

```
head -c 400 /usr/bin/ls
```

O terminal fica ilegível. Digite `reset` às cegas e aperte `Enter`.

Esperado: o terminal volta ao normal. Você acabou de aprender que “terminal travado” quase nunca é travamento — e que `reset` é a resposta.

### 6. Inspeção o binário do jeito certo.

```
strings /usr/bin/ls | head -20
xxd -l 64 /usr/bin/ls
file /usr/bin/ls
```

Esperado: texto legível no `strings`, os bytes `7f 45 4c 46` (.ELF) no início do `xxd`. Essa assinatura é como o `file` identifica o formato.

### 7. Explore o *less* de verdade.

```
less /var/log/syslog 2>/dev/null || less /var/log/dpkg.log
```

Dentro do `less`, execute na ordem: `G` (fim), `g` (início), `/error` seguido de Enter, `n` algumas vezes, `&sudo` seguido de Enter, `&` sozinho seguido de Enter, `-N` seguido de Enter, `h` (ajuda), `q` para sair da ajuda, `q` de novo para sair.

Esperado: navegação nos dois sentidos, busca destacada e — no `&sudo` — apenas as linhas correspondentes na tela. Esse filtro é o recurso que substitui rolar milhares de linhas.

#### 8. Compare as faixas de `head` e `tail`.

```
cd /tmp/lab011
head -n 3 numeros.txt
tail -n 3 numeros.txt
tail -n +198 numeros.txt
head -n 6 numeros.txt | tail -n 3
head -n -195 numeros.txt
```

Esperado, na ordem: 1–3, 198–200, 198–200, 4–6, 1–5. Confira contra Figura 11.2.

#### 9. Acompanhe um arquivo crescendo.

Numa segunda aba (`Ctrl+Shift+T`):

```
tail -f /tmp/lab011/crescendo.log
```

Na primeira aba:

```
for i in $(seq 1 10); do echo "evento $i em $(date +%T)" >>
↪ /tmp/lab011/crescendo.log; sleep 1; done
```

Esperado: as linhas aparecem na segunda aba conforme são escritas. `Ctrl+C` encerra o `tail`.

#### 10. Veja a diferença entre `-f` e `-F`.

Com o `tail -f` ainda rodando na segunda aba, na primeira:

```
mv /tmp/lab011/crescendo.log /tmp/lab011/crescendo.log.1
echo "linha nova depois da rotacao" >> /tmp/lab011/crescendo.log
```

Esperado: o `tail -f` **não** mostra a linha nova — ele continua no arquivo antigo. Encerre com `Ctrl+C`, rode `tail -F /tmp/lab011/crescendo.log` e repita o teste: agora a linha aparece. Essa é a diferença que faz o monitoramento de servidor funcionar depois de uma rotação de log.

#### 11. Filtre um fluxo em tempo real.

```
tail -F /var/log/auth.log 2>/dev/null | grep --line-buffered sudo &
sleep 1
sudo true
sleep 2
kill %1
```

Esperado: a linha do seu `sudo` aparece filtrada. Se remover `--line-buffered`, nada aparece — o `grep` fica segurando a saída no buffer.

#### 12. Trabalhe com arquivo comprimido sem descomprimir.

## 11. Visualizando conteúdo: *cat*, *less*, *head*, *tail* e *more*

```
cd /tmp/lab011
gzip -k numeros.txt
ls -lh numeros.txt*
zcat numeros.txt.gz | head -3
zgrep -c 5 numeros.txt.gz
zless numeros.txt.gz      # q para sair
```

Esperado: o `.gz` é lido diretamente. Numa `rockyou.txt.gz` de centenas de megabytes, isso economiza tempo e espaço em disco.

## 13. Experimente um visualizador com destaque de sintaxe.

```
sudo apt install -y bat
batcat /etc/os-release
batcat -n /tmp/lab011/numeros.txt | head -5
```

Esperado: cores, número de linha e cabeçalho. Ótimo para código; para dados que seguirão num cano, continue usando `cat`.

## 14. Limpe.

```
cd /tmp
ls -la /tmp/lab011
rm -rf /tmp/lab011
```

## 11.8. Resumo

- **cat concatena, não visualiza.** Usá-lo em arquivo grande arrasa o buffer de rolagem; usá-lo em binário injeta sequências de escape e embaralha o terminal — o conserto é **reset**.
- **file antes de abrir.** Ele lê os bytes iniciais em vez da extensão e evita quase todos os acidentes de terminal.
- **cat -A revela o invisível:** `^M` de fim de linha do Windows, `^I` de tabulação e `$` marcando espaços em branco no fim da linha. É diagnóstico de bugs que não aparecem de outro jeito.
- **less carrega sob demanda** e abre arquivos de gigabytes instantaneamente. Dez teclas — Espaço, b, g, G, /, n, &, F, h, q — cobrem quase todo o uso, e o `&` filtra o arquivo sem sair dele.
- **head -n 3** são as três primeiras; **tail -n 3** são as três últimas; **tail -n +3** é da terceira em diante. Encadear `head` com `tail` recorta qualquer faixa do meio.
- **tail -f segue o inode; tail -F segue o nome.** Depois de uma rotação de log, só o `-F` continua funcionando.
- **Ao filtrar um fluxo ao vivo, use grep --line-buffered.** Sem isso, a saída fica presa no buffer e parece que nada está acontecendo.
- **Arquivos comprimidos têm uma família própria:** `zcat`, `zless`, `zgrep`, `zdiff`. Descomprimir só para olhar desperdiça tempo e espaço.



## 12. Redirecionamento e pipes: stdin, stdout e stderr

Você deixa uma varredura longa rodando e guarda tudo num arquivo para analisar depois:

```
nmap -sV 192.168.56.0/24 > resultado.txt
```

Volta uma hora depois, abre o arquivo e encontra os resultados — mas as mensagens de erro sobre hosts inalcançáveis não estão lá. Elas apareceram na tela e sumiram junto com o buffer de rolagem. Você tenta corrigir e escreve:

```
nmap -sV 192.168.56.0/24 2>&1 > resultado.txt
```

Parece a linha certa. Não é. Ela continua mandando os erros para a tela, e a razão é uma ordem de operações que quase ninguém explica.

Redirecionamento e pipes são o mecanismo que transforma um punhado de comandos pequenos num sistema. É o que permite pegar a saída de um, filtrar com outro, ordenar com um terceiro e gravar o resultado — tudo numa linha, sem arquivo temporário. Entender esse mecanismo é a diferença entre usar o shell e programar com ele.

### 12.1. Os três descritores padrão

Todo processo no Linux começa com três canais de comunicação já abertos, identificados por número:

Tabela 12.1.: Os três descritores de arquivo padrão

| Descritor | Nome          | Padrão  | Para que serve        |
|-----------|---------------|---------|-----------------------|
| 0         | <b>stdin</b>  | teclado | entrada de dados      |
| 1         | <b>stdout</b> | tela    | resultado do trabalho |
| 2         | <b>stderr</b> | tela    | erros e avisos        |

Quem abre esses canais é o shell, antes de executar o programa, como visto em Capítulo 6. Por isso qualquer comando “já sabe” ler do teclado e escrever na tela sem nenhuma configuração.

A separação entre **stdout** e **stderr** é uma decisão de projeto excelente e frequentemente ignorada. Ela permite gravar o resultado útil num arquivo enquanto os erros continuam visíveis na tela — exatamente o que aconteceu no exemplo da abertura, só que involuntariamente. Figura 12.1 mostra o arranjo padrão e o efeito de um redirecionamento.

O detalhe crucial: **o programa não percebe a diferença**. Ele escreve no descritor 1 do mesmo jeito, sem saber se ali está um terminal, um arquivo ou um cano. Só isso já explica por que o mesmo comando funciona interativamente e dentro de um script.

## 12. Redirecionamento e pipes: *stdin*, *stdout* e *stderr*

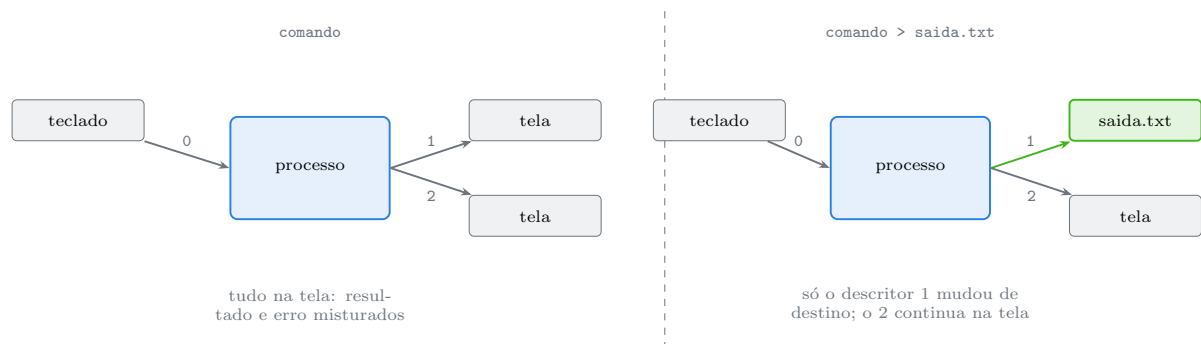


Figura 12.1.: Redirecionar é apenas reapontar um descritor antes de o programa começar. O programa não sabe que isso aconteceu.

### 12.2. Redirecionando a saída

```
comando > arquivo           # stdout para o arquivo, SOBRESCREVENDO
comando >> arquivo           # stdout para o arquivo, ACRESCENTANDO ao final
comando 2> arquivo           # stderr para o arquivo
comando 2>> arquivo          # stderr acrescentando
comando > saida.txt 2> erros.txt # cada um para o seu arquivo
comando > tudo.txt 2>&1       # os dois para o mesmo arquivo
comando &> tudo.txt          # o mesmo, forma abreviada do Bash e do Zsh
comando &>> tudo.txt          # abreviada, acrescentando
```

O **>** **destrói** o conteúdo anterior do arquivo antes mesmo de o comando rodar. E ele faz isso mesmo que o comando falhe:

```
comando-que-nao-existe > importante.txt # importante.txt fica VAZIO
```

O shell trunca o arquivo ao montar o redirecionamento, antes de descobrir que o comando não existe. É um jeito rápido e silencioso de perder dados.

A proteção existe:

```
set -o noclobber
echo teste > existente.txt # recusa: "file exists"
echo teste >| existente.txt # >| força a sobrescrita quando necessario
set +o noclobber           # desliga
```

Vale ativar `noclobber` no `~/.zshrc` — mas, como todo alias de proteção, saiba que ele não estará em servidores nem em scripts.

#### 12.2.1. A ordem de `2>&1` é o que confunde todo mundo

A sintaxe `2>&1` significa literalmente: “faça o descritor 2 apontar para **onde o descritor 1 aponta neste momento**”. É uma cópia do destino atual, não um vínculo permanente. E o

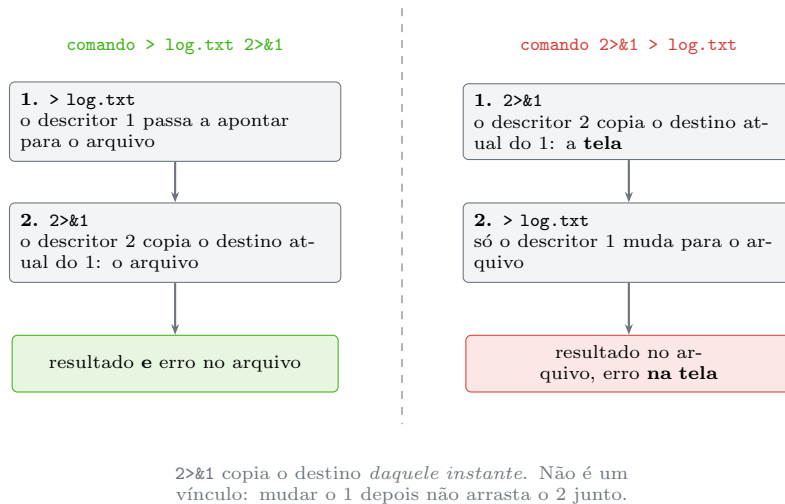


Figura 12.2.: A mesma dupla de redirecionamentos, em duas ordens, com resultados opostos.

shell processa os redirecionamentos **da esquerda para a direita**. Figura 12.2 mostra as duas ordens.

A regra prática que resolve para sempre: **o `2>&1` vem por último**, ou use `&>`.

Existe um uso legítimo da ordem “errada”, e ele é elegante — mandar apenas os erros para um cano:

```
comando 2>&1 > /dev/null | grep "falha"
```

Aqui o `stderr` foi copiado para o cano antes de o `stdout` ser descartado, então só as mensagens de erro chegam ao `grep`.

### 12.2.2. Os arquivos de dispositivo que aparecem em todo lugar

```
comando > /dev/null          # descarta a saída
comando 2> /dev/null         # descarta só os erros
comando &> /dev/null          # descarta tudo
```

`/dev/null` aceita qualquer quantidade de dados e não guarda nada. Ler dele devolve fim de arquivo imediatamente. É o destino padrão para saída que não interessa — especialmente em tarefas do `cron`, que enviariam e-mail a cada linha impressa.

A família completa vale conhecer:

Tabela 12.2.: Dispositivos virtuais usados em redirecionamento

| Dispositivo               | Comportamento                                  |
|---------------------------|------------------------------------------------|
| <code>/dev/null</code>    | descarta o que escrevem; leitura devolve vazio |
| <code>/dev/zero</code>    | leitura devolve bytes zero infinitamente       |
| <code>/dev/urandom</code> | leitura devolve bytes aleatórios               |

## 12. Redirecionamento e pipes: *stdin*, *stdout* e *stderr*

| Dispositivo              | Comportamento                                                          |
|--------------------------|------------------------------------------------------------------------|
| <code>/dev/full</code>   | escrita falha com “no space left”; útil para testar tratamento de erro |
| <code>/dev/stdout</code> | o próprio descritor 1, útil quando um programa exige nome de arquivo   |

```
head -c 1M /dev/zero > vaziao.img      # arquivo de 1 MB de zeros
head -c 32 /dev/urandom | base64       # senha aleatoria
```

### ⚠ Descartar erro esconde problema

`2> /dev/null` é conveniente e perigoso quando vira hábito. Num script, ele silencia justamente a informação que explicaria a falha. Use quando você **sabe** qual erro está descartando — por exemplo, o `find` reclamando de diretórios sem permissão — e não como forma de deixar a saída bonita.

## 12.3. Redirecionando a entrada

```
comando < arquivo          # le do arquivo em vez do teclado
sort < nomes.txt
wc -l < /etc/passwd        # sem o nome do arquivo na saída
```

A diferença entre `wc -l arquivo` e `wc -l < arquivo` é sutil e útil: no primeiro caso o `wc` conhece o nome e o imprime junto do número; no segundo ele só vê um fluxo anônimo e imprime apenas o número. Em script, a segunda forma dispensa recortar a saída.

### 12.3.1. Here-document e here-string

```
cat << FIM
primeira linha
segunda linha com $USER expandido
FIM
```

O here-document (`<<`) alimenta o comando com o texto até o marcador. Variáveis são expandidas por padrão; para desligar a expansão, coloque o marcador entre aspas:

```
cat << 'FIM'
$USER nao sera expandido
FIM
```

Com `<<-` as tabulações iniciais são removidas, o que permite indentar o bloco dentro de uma função sem sujar o conteúdo. O here-string (`<<<`) manda uma única linha:

```
grep raiz <<< "uma linha de teste com raiz dentro"
tr 'a-z' 'A-Z' <<< "minusculas"
base64 -d <<< "S2FsaSBMaW51eAo="
```

É o jeito limpo de alimentar um comando sem `echo` e sem cano. O tema volta com profundidade no Volume 12.

## 12.4. Pipes: a ideia central do Unix

O cano `|` liga o `stdout` de um comando ao `stdin` do próximo:

```
comando_a | comando_b | comando_c
```

Os três processos rodam **ao mesmo tempo**, não em sequência. O kernel cria um buffer entre cada par, e cada processo consome conforme o anterior produz. Isso significa que um pipeline pode processar um arquivo de 100 GB usando poucos megabytes de memória — e que a saída começa a aparecer antes de a entrada terminar.

```
ps aux | grep ssh
ls -l /etc | wc -l
cat /var/log/syslog | grep erro | sort | uniq -c | sort -rn | head
history | awk '{print $2}' | sort | uniq -c | sort -rn | head
```

O último é um clássico: mostra quais comandos você mais usa.

Por padrão, o cano transporta **apenas** o `stdout`. Erros continuam indo para a tela. Para incluir também o `stderr`:

```
comando |& grep algo      # Bash 4+ e Zsh
comando 2>&1 | grep algo  # forma portavel, funciona em qualquer shell
```

### 12.4.1. O código de saída de um pipeline

Este ponto causa bugs silenciosos em produção. Por padrão, o status de um pipeline é o do **último** comando:

```
comando-que-nao-existe | tee saida.txt
echo $?                # 0 - porque o tee funcionou
```

O pipeline “teve sucesso” mesmo com o primeiro comando falhando. Duas soluções:

```
echo "${PIPESTATUS[@]}" # bash: status de cada elemento do pipeline
echo "${pipestatus[@]}" # zsh: mesma coisa, nome em minusculas

set -o pipefail          # o pipeline falha se QUALQUER elemento falhar
```

Em qualquer script sério, `set -o pipefail` deveria estar no topo, junto de `set -euo`. O tema completo é do Volume 12, mas o hábito começa aqui.

## 12.5. tee: gravar e continuar

O `tee` duplica o fluxo: escreve no arquivo e repassa adiante.

```
comando | tee saida.txt          # ve na tela e grava
comando | tee -a saida.txt       # acrescenta em vez de sobrescrever
comando | tee saida.txt | grep erro # grava tudo, filtra o que ve
comando | tee a.txt b.txt c.txt   # grava em varios de uma vez
comando 2>&1 | tee log-completo.txt # inclui os erros no arquivo
```

O caso mais importante é o que resolve a armadilha de `sudo` com redirecionamento, vista em Capítulo 6:

```
sudo echo "linha" >> /etc/arquivo      # FALHA: quem redireciona e o seu
↪ shell
echo "linha" | sudo tee -a /etc/arquivo > /dev/null # funciona
```

O `> /dev/null` no final evita que o conteúdo apareça duplicado na tela.

## 12.6. Substituição de processo

Alguns comandos exigem **nomes de arquivo** e não aceitam entrada por cano — o `diff`, por exemplo, precisa de dois arquivos. A substituição de processo cria arquivos temporários virtuais a partir de comandos:

```
diff <(ls /etc) <(ls /etc/apt)
diff <(sort a.txt) <(sort b.txt)
comm -13 <(sort antigo.txt) <(sort novo.txt)      # so o que e novo
```

O `<(comando)` executa o comando e entrega o caminho de um descritor — geralmente algo como `/dev/fd/63` — no lugar do texto. Dá para ver o mecanismo funcionando:

```
echo <(ls)
# /dev/fd/63
```

A forma inversa, `>(comando)`, cria um destino de escrita:

```
ls -l | tee >(wc -l > contagem.txt) > listagem.txt
```

É um recurso do Bash e do Zsh, não do `sh` POSIX. Em script com `#!/bin/sh` no Debian, ele não funciona.

## 12.7. Buffering: por que a saída parece travada

Um comportamento que parece bug: um comando que imprime na hora quando roda sozinho fica mudo dentro de um cano.

A causa é a biblioteca C. Ela ajusta o buffer conforme o destino: quando a saída é um terminal, o buffer é por linha (aparece na hora); quando é um cano ou arquivo, é por bloco de alguns kilobytes (só aparece quando o bloco enche). É uma otimização, e ela atrapalha monitoramento ao vivo.

As soluções:

```
tail -F /var/log/auth.log | grep --line-buffered "Failed"
comando | stdbuf -oL grep algo
stdbuf -o0 comando | outro           # sem buffer nenhum
unbuffer comando | outro             # do pacote expect, funciona ate com cores
```

O `--line-buffered` do `grep` é a solução mais comum. Para programas que não têm opção equivalente, `stdbuf` resolve na maioria dos casos, e `unbuffer` — que cria um pseudoterminal, enganando a biblioteca — resolve o resto.

## 12.8. Descritores personalizados

Além de 0, 1 e 2, um processo pode abrir outros descritores. É recurso avançado, mas o padrão vale conhecer:

```
exec 3> /tmp/registro.log           # abre o descritor 3 apontando para o arquivo
echo "evento inicial" >&3           # escreve nele
echo "outro evento" >&3
exec 3>&-                           # fecha
cat /tmp/registro.log
```

A vantagem sobre `>>` repetido é que o arquivo é aberto uma única vez. Em script que registra centenas de eventos, isso importa. Volume 12 trata do assunto.

## 12.9. No Kali

- **Ferramentas de varredura escrevem muito em `stderr`.** `nmap`, `gobuster` e `ffuf` mandam progresso e avisos para o descritor 2 e resultado para o 1. Guardar só o `>` perde metade da informação. Use `&> saida.txt` ou `2>&1 | tee saida.txt`.
- **A maioria tem opção própria de saída em arquivo, e ela é melhor.** `nmap -oA` base grava nos três formatos de uma vez, `gobuster -o resultado.txt`, `ffuf -o resultado.json`. Redirecionamento serve para capturar o que a ferramenta não formata; a opção nativa produz arquivo estruturado.
- **| tee durante um teste autorizado é hábito profissional.** Você acompanha na tela e mantém o registro do que foi executado e quando — material de evidência e de relatório. Combine com `script` ou `ts` (do pacote `moreutils`) para carimbar a hora em cada linha.

## 12. Redirecionamento e pipes: *stdin*, *stdout* e *stderr*

- **--line-buffered é obrigatório ao filtrar log ao vivo.** `tail -F /var/log/auth.log | grep --line-buffered "Failed password"` mostra tentativas de autenticação na hora; sem a opção, você espera minutos pelo buffer encher.
- **Escrever em arquivo de /etc exige tee, não sudo com >.** Ajustar `/etc/apt/sources.list` ou `/etc/hosts` sempre passa por `echo ... | sudo tee -a arquivo > /dev/null`.
- **Wordlists funcionam bem por cano.** `zcat rockyou.txt.gz | head -1000 > /tmp/curta.txt` cria uma lista reduzida para teste sem descomprimir centenas de megabytes, e sem escrever em `/usr/share`.
- **Cuidado com > arquivo em evidência.** Um `>` acidental num arquivo de coleta apaga o conteúdo instantaneamente, antes de qualquer comando rodar. `set -o noclobber` no `~/.zshrc` é uma proteção barata, e `>>` deveria ser o padrão mental ao lidar com material que não pode ser perdido.
- **Contêineres de ferramenta seguem a mesma lógica.** `docker run ... 2>&1 | tee log.txt` funciona porque o Docker repassa os descritores do processo interno. Volume 16 detalha.

**Nota sobre a família RHEL.** Redirecionamento e pipes fazem parte do POSIX (IEEE and The Open Group, 2018) e são idênticos. O `&>` é extensão do Bash e do Zsh, disponível nas duas famílias, mas ausente em `dash` e em `sh` estrito — em script com `#!/bin/sh`, use `> arquivo 2>&1`.

### 12.10. Laboratório

Roteiro em `/tmp`.

#### 1. Separe resultado de erro.

```
mkdir -p /tmp/lab012 && cd /tmp/lab012
ls /etc /nao-existe
ls /etc /nao-existe > saida.txt
cat saida.txt
ls /etc /nao-existe 2> erros.txt
cat erros.txt
ls /etc /nao-existe > saida.txt 2> erros.txt
wc -l saida.txt erros.txt
```

Esperado: no segundo comando o erro aparece na tela e não no arquivo; no terceiro acontece o inverso. Os dois canais são independentes.

#### 2. Reproduza a armadilha da ordem.

```
ls /etc /nao-existe > tudo1.txt 2>&1
ls /etc /nao-existe 2>&1 > tudo2.txt
grep -c "No such" tudo1.txt
grep -c "No such" tudo2.txt
```

Esperado: 1 no primeiro e 0 no segundo — e, na segunda linha, o erro apareceu na tela. Compare com Figura 12.2.

#### 3. Use a ordem “errada” de propósito.

```
ls /etc /nao-existe 2>&1 > /dev/null | cat
```



Esperado: **só** a mensagem de erro. O resultado normal foi descartado. É o idioma para tratar exclusivamente os erros de um comando.

4. **Veja o > destruir um arquivo antes da execução.**

```
echo "conteudo importante" > importante.txt
cat importante.txt
comando-que-nao-existe > importante.txt
cat importante.txt
ls -l importante.txt
```

Esperado: o arquivo fica com zero bytes mesmo com o comando falhando. O shell truncou antes de tentar executar.

5. **Ative a proteção contra sobrescrita.**

```
echo "dados" > protegido.txt
set -o noclobber
echo "novo" > protegido.txt
echo "novo" >| protegido.txt
cat protegido.txt
set +o noclobber
```

Esperado: a terceira linha é recusada, a quarta força a escrita. Considere `set -o noclobber` no `~/.zshrc`.

6. **Explore os dispositivos virtuais.**

```
head -c 1048576 /dev/zero > vazio.img
ls -lh vazio.img
head -c 32 /dev/urandom | base64
head -c 20 /dev/zero | xxd
echo teste > /dev/full ; echo "status: $?"
```

Esperado: um arquivo de 1 MB de zeros, uma cadeia aleatória, vinte bytes zerados e um erro de disco cheio no `/dev/full`. O último é a maneira correta de testar tratamento de erro de escrita.

7. **Compare wc com e sem redirecionamento de entrada.**

```
wc -l /etc/passwd
wc -l < /etc/passwd
```

Esperado: com o nome do arquivo na primeira, só o número na segunda. Em script, a segunda forma dispensa recortar a saída.

8. **Use here-document e here-string.**

```
cat << FIM > mensagem.txt
usuario atual: $USER
diretorio: $PWD
FIM
cat mensagem.txt

cat << 'FIM'
$USER fica literal aqui
FIM
```

## 12. Redirecionamento e pipes: *stdin*, *stdout* e *stderr*

```
tr 'a-z' 'A-Z' <<< "converta esta linha"
base64 -d <<< "S2FsaSBMaW5leAo="
```

Esperado: variáveis expandidas no primeiro caso, literais no segundo com o marcador entre aspas, e as duas últimas linhas processando uma cadeia sem arquivo nem **echo**.

## 9. Monte um pipeline de contagem.

```
ls -l /etc | wc -l
cat /etc/passwd | cut -d: -f7 | sort | uniq -c | sort -rn
```

Esperado: a contagem de entradas em */etc* e o ranking dos shells cadastrados no sistema. Cinco processos rodando simultaneamente, cada um consumindo o que o anterior produz.

## 10. Descubra o código de saída de um pipeline.

```
comando-inexistente | tee lixo.txt
echo "status do pipeline: $?"
echo "status de cada etapa: ${pipestatus[@]}:-${PIPESTATUS[@]}"

set -o pipefail
comando-inexistente | tee lixo.txt
echo "com pipefail: $?"
set +o pipefail
```

Esperado: 0 sem **pipefail**, não-zero com ele. Esse detalhe já mascarou falha de backup em produção por meses.

## 11. Grave e veja ao mesmo tempo com **tee**.

```
ls -l /etc | tee listagem.txt | wc -l
wc -l listagem.txt
ls -l /usr | tee -a listagem.txt > /dev/null
wc -l listagem.txt
```

Esperado: a listagem completa vai ao arquivo enquanto só a contagem aparece na tela; o **-a** acrescenta sem apagar.

## 12. Resolva o **sudo** com redirecionamento.

```
sudo echo "# comentario de teste" >> /etc/hosts 2>/dev/null; echo "status:
↳ $?"
echo "# comentario de teste" | sudo tee -a /etc/hosts > /dev/null
tail -2 /etc/hosts
sudo sed -i '$ d' /etc/hosts          # remove a ultima linha, desfazendo o
↳ teste
tail -2 /etc/hosts
```

Esperado: a primeira forma pode até funcionar aqui (o seu usuário talvez consiga escrever), mas num arquivo realmente protegido ela falha; a segunda sempre funciona. A última linha desfaz o teste.

## 13. Compare arquivos sem criar arquivos.

```
diff <(ls /etc) <(ls /etc/apt) | head
echo <(ls)
comm -13 <(printf 'a\nb\nc\n') <(printf 'b\nc\nd\n')
```

Esperado: o `diff` funciona sobre saídas de comando, o `echo` revela um caminho como `/dev/fd/63`, e o `comm` mostra que apenas `d` é exclusivo do segundo conjunto.

#### 14. Observe o buffering na prática.

Numa segunda aba:

```
tail -F /tmp/lab012/fluxo.log | grep ALERTA
```

Na primeira:

```
for i in $(seq 1 5); do echo "ALERTA evento $i" >> /tmp/lab012/fluxo.log;
  ↪ sleep 1; done
```

Esperado: **nada** aparece durante a execução. Encerre com `Ctrl+C`, repita usando `grep --line-buffered ALERTA` e veja as linhas surgirem na hora.

#### 15. Use um descritor personalizado.

```
cd /tmp/lab012
exec 3> registro.log
echo "inicio em $(date +%T)" >&3
echo "esta linha vai para a tela"
echo "fim em $(date +%T)" >&3
exec 3>&-
cat registro.log
```

Esperado: apenas as duas linhas marcadas com `>&3` no arquivo; a outra ficou na tela. O arquivo foi aberto uma única vez.

#### 16. Limpe.

```
cd /tmp
ls -la /tmp/lab012
rm -rf /tmp/lab012
```

## 12.11. Resumo

- **Tudo processo nasce com três descritores:** 0 para entrada, 1 para resultado e 2 para erro. Redirecionar é apenas reapontar um deles antes de o programa começar — e o programa não percebe.
- **> sobrescreve e >> acrescenta**, e a truncagem acontece **antes** da execução: um comando inexistente já esvaziou o arquivo. `set -o noclobber` protege, com `>|` para forçar quando preciso.
- **2>&1 copia o destino atual do descritor 1**, e o shell lê os redirecionamentos da esquerda para a direita. Por isso ele vem por último — ou use `&>`.
- **/dev/null descarta; /dev/zero, /dev/urandom e /dev/full geram dados e falhas sob demanda.** Descartar erro com `2> /dev/null` só é aceitável quando você sabe qual erro está silenciando.

## 12. Redirecionamento e pipes: *stdin*, *stdout* e *stderr*

- **O cano liga *stdout* a *stdin* e roda todos os processos ao mesmo tempo**, o que permite processar arquivos maiores que a memória. Ele não transporta o *stderr* — para isso, `&2` ou `&1 2`.
- **O status de um pipeline é o do último comando**, o que esconde falhas. `set -o pipefail` corrige, e `PIPESTATUS` mostra cada etapa.
- **`tee` grava e repassa**, e é a resposta correta para escrever em arquivo protegido: `echo ... | sudo tee -a arquivo`.
- **`<(comando)` transforma saída em nome de arquivo**, permitindo `diff` e `comm` sobre resultados de comandos, sem arquivo temporário.
- **Buffering por bloco é o motivo de um filtro ao vivo parecer travado**. `grep --line-buffered`, `stdbuf` ou `unbuffer` restauram a saída imediata.

## 13. Curingas, globbing e expansão de chaves

Você tem cinco relatórios num diretório e quer arquivá-los:

```
mv *.txt arquivo/
```

O diretório `arquivo` não existe — você esqueceu de criá-lo. O shell expande `*.txt` para `a.txt b.txt c.txt d.txt e.txt`, e o `mv` recebe seis argumentos: cinco arquivos e um destino chamado `arquivo/`. Como o destino não existe, ele reclama e não faz nada. Sorte.

Agora imagine a mesma linha sem a barra final:

```
mv *.txt arquivo
```

O `mv` recebe `a.txt b.txt c.txt d.txt e.txt arquivo`. Nenhum deles é diretório. Comportamento com múltiplas origens e destino que não é diretório: erro. Continua havendo sorte.

Mas se existir um arquivo chamado `e.txt` e mais nada, e a linha for `mv *.txt e.txt`, o shell expande para `mv e.txt e.txt` e o `mv` reclama que são o mesmo arquivo. E se forem dois — `mv a.txt e.txt` — o conteúdo de `e.txt` é destruído em silêncio.

O ponto não é decorar cada caso. É perceber que **o programa nunca viu o asterisco**. Quem procurou os arquivos, montou a lista e escolheu a ordem foi o shell, antes de o comando existir. Um curinga não é uma instrução: é um texto que vira outros textos. Este capítulo mostra exatamente como essa transformação acontece, em que ordem, e como conferir o resultado antes de executar qualquer coisa.

### 13.1. A ordem das expansões

O shell não faz uma expansão: faz uma sequência delas, sempre na mesma ordem, sobre a mesma linha. Figura 13.1 mostra o encadeamento completo.

Duas consequências dessa ordem explicam praticamente todas as surpresas:

**Chaves são expandidas antes de tudo e não consultam o sistema de arquivos.** `mkdir {a,b,c}` funciona mesmo que nada exista, porque a expansão de chaves gera texto puro. Já `*` só produz nomes que existem de fato.

**Globbing acontece depois da substituição de variáveis.** Isso significa que um curinga guardado numa variável é expandido quando a variável é usada sem aspas — comportamento útil às vezes, desastroso na maioria:

```
PADRAO="*.txt"
echo $PADRAO      # expande: lista os arquivos
echo "$PADRAO"    # literal: imprime *.txt
```

### 13. Curingas, globbing e expansão de chaves

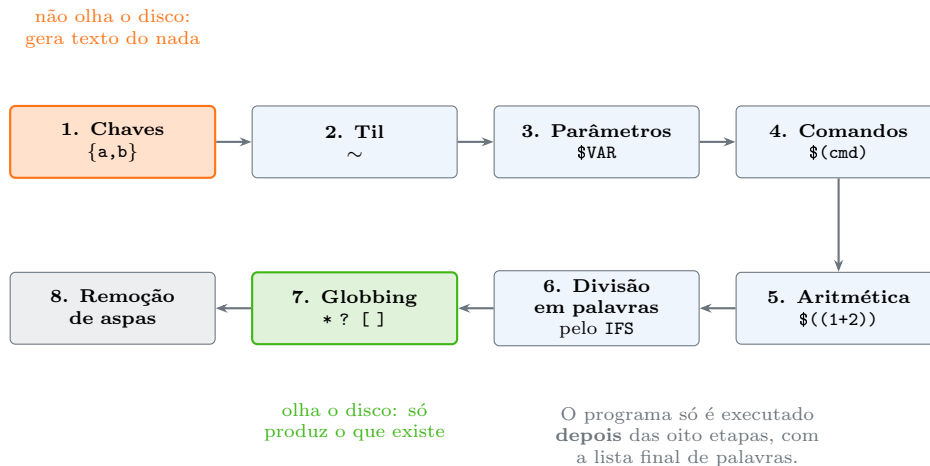


Figura 13.1.: Oito etapas, sempre nesta ordem. Chaves vêm primeiro; globbing, quase por último.

A ferramenta de conferência é sempre a mesma, e custa dois segundos:

```
echo mv *.txt arquivo/          # ve a linha final, ja expandida, sem executar
```

## 13.2. Globbing: os curingas do shell

Os curingas básicos são quatro:

Tabela 13.1.: Curingas básicos do shell

| Padrão     | Casa com                                          |
|------------|---------------------------------------------------|
| *          | qualquer sequência de caracteres, inclusive vazia |
| ?          | exatamente um caractere qualquer                  |
| [abc]      | um caractere entre os listados                    |
| [a-z]      | um caractere na faixa                             |
| [!abc]     | um caractere que <b>não</b> esteja na lista       |
| [:digit:]] | uma classe POSIX: dígito, letra, espaço, etc.     |

Figura 13.2 mostra como cada padrão se comporta sobre a mesma listagem.

|           | *.txt | log?.txt | log[12].txt | *   | .*  |
|-----------|-------|----------|-------------|-----|-----|
| log1.txt  | sim   | sim      | sim         | sim | não |
| log2.txt  | sim   | sim      | sim         | sim | não |
| log10.txt | sim   | não      | não         | sim | não |
| nota.md   | não   | não      | não         | sim | não |
| .oculto   | não   | não      | não         | não | sim |
| dados.csv | não   | não      | não         | sim | não |

Figura 13.2.: O asterisco não casa com nomes ocultos, e ? casa com exatamente um caractere.

Três regras que não são óbvias:

**\* não cruza a barra.** `ls /etc/*` lista o conteúdo de um nível só; para descer, é preciso `*/*` ou globbing recursivo.

**\* não casa com arquivos ocultos.** `rm *` não apaga `.bashrc` — proteção deliberada, para que um `*` descuidado não leve a configuração junto. Para incluir ocultos:

```
setopt dotglob          # zsh
```

### 13.2.1. Classes POSIX

```
ls [[:digit:]]*      # nomes que comecam com digito
ls [[:upper:]]*     # que comecam com maiuscula
ls *[:space:]*      # que contem espaco
```

As classes disponíveis são `alpha`, `digit`, `alnum`, `upper`, `lower`, `space`, `punct`, `xdigit`. Diferentemente de `[a-z]`, elas respeitam a localidade e não deixam letras acentuadas de fora.

### 13.2.2. Quando nada casa

O comportamento diverge entre shells, e a diferença aparece cedo:

```
ls *.formato-inexistente
# bash: ls: cannot access '*.formato-inexistente': No such file or directory
# zsh:  zsh: no matches found: *.formato-inexistente
```

No Bash, o padrão é passado **literalmente** ao comando, que reclama de um arquivo com aquele nome esquisito. No Zsh, o shell aborta antes de executar. O comportamento do Zsh é mais seguro — imagine `rm *.bak` num diretório sem nenhum `.bak`: no Bash, o `rm` receberia o texto `*.bak`; no Zsh, nada acontece.

Para controlar isso:

```
shopt -s nullglob      # bash: padrao sem correspondencia vira lista VAZIA
shopt -s failglob      # bash: padrao sem correspondencia vira erro (como o zsh)
setopt nullglob        # zsh: idem
setopt nomatch         # zsh: o padrao (aborta), ja ativo
shopt -s nocaseglob    # bash: ignora maiusculas e minusculas
setopt nocaseglob      # zsh: idem
```

O `nullglob` é essencial em scripts com laços:

```
for arquivo in *.log; do
    echo "processando $arquivo"
done
```

Sem `nullglob`, num diretório sem nenhum `.log` esse laço executa **uma vez** com a variável valendo o texto `*.log` — e o script processa um arquivo que não existe.

## 13.3. Globbing recursivo

Descer a árvore inteira com um único padrão:

### 13. Curingas, globbing e expansão de chaves

```
setopt extended_glob      # zsh: ja costuma vir ativo
ls **/*.conf              # zsh: todos os .conf, em qualquer profundidade

shopt -s globstar         # bash: precisa ser ativado
ls **/*.conf              # bash: mesmo efeito
```

Sem `globstar`, o `**` no Bash se comporta como um `*` comum — silenciosamente, sem aviso. É uma fonte clássica de script que “funciona no Zsh e não no Bash”.

Para árvores muito grandes, o globbing recursivo é caro: o shell monta a lista inteira em memória antes de executar o comando. O `find` é a ferramenta certa nesse caso, e é tema do Volume 3.

#### 13.3.1. Qualificadores de glob do Zsh

Um recurso poderoso que não existe no Bash. Entre parênteses, depois do padrão, o Zsh aceita filtros:

```
ls *(. )                  # so arquivos comuns
ls */( )                  # so diretorios
ls *(@ )                  # so links simbolicos
ls *(*)                   # so executaveis
ls *(.Lm+10)              # arquivos com mais de 10 MB
ls *(.om[1,5])            # os 5 mais recentes
ls **/*(.Lm+100)          # arquivos grandes em toda a arvore
```

É `find` embutido na sintaxe do shell. Ótimo interativamente, inútil em script portátil.

### 13.4. Expansão de chaves

Chaves não procuram nada: elas **geram** texto. É a única expansão que produz nomes de arquivos que ainda não existem, e por isso é a ferramenta natural para criar estruturas.

```
echo {a,b,c}              # a b c
echo arquivo{1,2,3}.txt   # arquivo1.txt arquivo2.txt arquivo3.txt
echo {1..10}              # 1 2 3 4 5 6 7 8 9 10
echo {01..10}             # 01 02 03 ... 10 (com zero a esquerda)
echo {a..e}               # a b c d e
echo {0..20..5}           # 0 5 10 15 20 (com passo)
echo {10..1}              # contagem regressiva
```

Combinações se multiplicam — é produto cartesiano:

```
echo {a,b}{1,2}           # a1 a2 b1 b2
echo {2024..2026}-{01..12} # 36 combinacoes de ano e mes
mkdir -p projeto/{doc,src,teste}/{entrada,saida}
```

Os idiomas mais úteis do dia a dia:



```

cp config.conf{,.bak}          # vira: cp config.conf config.conf.bak
mv relatorio.txt{-antigo}      # renomeia acrescentando sufixo
diff arquivo.{antigo,novo}     # vira: diff arquivo.antigo
↪ arquivo.novo
touch teste-{01..10}.txt       # dez arquivos numerados
mkdir -p /tmp/lab/{a,b,c}/{in,out}

```

O `cp config.conf{,.bak}` merece atenção: a chave com um elemento vazio produz o nome original e o nome com sufixo, nessa ordem. É o backup mais curto que existe, e vira reflexo rapidamente.

Chaves e curingas se combinam, e aí a ordem de Figura 13.1 importa: as chaves são expandidas primeiro, gerando padrões, e cada padrão passa então pelo globbing:

```

ls *.{txt,md,csv}              # vira: ls *.txt *.md *.csv, e cada um é resolvido no
↪ disco

```

## 13.5. Aspas desligam tudo

```

echo *.txt                     # expande
echo "*.txt"                   # literal
echo '*.txt'                   # literal
echo \*.txt                    # literal

```

Aspas simples desligam **todas** as expansões. Aspas duplas desligam globbing e chaves, mas mantêm variáveis e substituição de comandos:

```

V=teste
echo "$V *.txt {a,b}"          # teste *.txt {a,b}
echo '$V *.txt {a,b}'          # $V *.txt {a,b}

```

Em scripts, a regra que evita a maior parte dos bugs continua sendo a de Capítulo 6: **variáveis sempre entre aspas duplas**, exceto quando você quiser deliberadamente que ocorram divisão em palavras e globbing.

## 13.6. Onde o globbing machuca

**Nome de arquivo com espaço.** O globbing produz nomes corretos, mas se você passar por uma variável sem aspas, a divisão em palavras da etapa 6 quebra o nome:

```

for f in *.txt; do cp $f /tmp/; done # quebra com "meu arquivo.txt"
for f in *.txt; do cp "$f" /tmp/; done # correto

```

### 13. Curingas, globbing e expansão de chaves

Repare que o `for` sobre o `glob` está certo: a lista chega ao laço já dividida corretamente. O problema é o uso da variável sem aspas depois.

**Nome começando com traço.** Um arquivo `-rf` no diretório transforma `rm *` em `rm -rf <resto>`. O antídoto é o `--` de Capítulo 6:

```
rm -- *
rm ./*
```

**Lista de argumentos longa demais.** Existe um limite de tamanho para a linha de comando (na casa de dois megabytes em sistemas típicos). Num diretório com centenas de milhares de arquivos:

```
rm *.log
# bash: /usr/bin/rm: Argument list too long
```

A solução é não montar a lista inteira de uma vez:

```
find . -maxdepth 1 -name '*.log' -delete
find . -maxdepth 1 -name '*.log' -print0 | xargs -0 rm
```

O `-print0` e o `-0` usam o byte zero como separador em vez do espaço — é a única forma completamente segura de passar nomes arbitrários entre programas, já que o zero é o único caractere proibido num nome de arquivo.

 Sempre ensaie um curinga antes de um comando destrutivo

```
ls *.bak          # 1. veja o que casa
echo rm *.bak      # 2. veja a linha final
rm *.bak          # 3. so entao
```

O passo 2 é o mais valioso: ele mostra exatamente a linha que será executada, com a expansão já resolvida, sem executar nada. Um espaço a mais, um arquivo inesperado ou um padrão que casou demais aparecem ali.

## 13.7. No Kali

- **O Zsh aborta em padrão sem correspondência, e isso protege.** `rm *.pcap` num diretório sem captura nenhuma não faz nada no Zsh, enquanto no Bash o `rm` receberia o texto literal. Ao rodar scripts vindos de tutoriais escritos para Bash, essa diferença aparece.
- **Qualificadores de glob resolvem triagem de coleta.** Depois de uma captura longa, `ls -lh *.pcap(.om[1,3])` mostra os três arquivos mais recentes, e `ls **/*(.Lm+50)` encontra tudo acima de 50 MB numa árvore de evidências. Só funciona no Zsh.
- **Wordlists e resultados se acumulam rápido.** `ls /usr/share/wordlists/**/*.*txt` (com `globstar` no Bash) dá o panorama; para árvores muito grandes, prefira `find`, que não monta a lista inteira em memória.
- **nmap -oA base já usa a lógica de chaves na prática:** ele gera `base.nmap`, `base.gnmap` e `base.xml`. Para inspecionar os três, `ls -l base.{nmap,gnmap,xml}`.

- **Backup antes de editar configuração de ferramenta.** `sudo cp /etc/proxychains4.conf{,.bak}` leva dois segundos e já salvou muita gente. Vale para `/etc/apt/sources.list`, `/etc/hosts` e qualquer arquivo que você vá editar com privilégio.
- **Nome de arquivo vindo de ferramenta pode conter qualquer coisa.** Saídas de fuzzing e de download automático produzem nomes com espaço, aspas, quebra de linha e caracteres de controle. Ao processar em lote, é `find -print0` com `xargs -0`, sempre — nunca `ls | xargs`.
- **Cuidado com `rm *` em `/var/log` ou em diretório de evidência.** O `*` não pega ocultos, mas pega tudo o mais, inclusive coisas que você não listou antes. `pwd`, `ls -la`, `echo rm ...` e só então o comando.

**Nota sobre a família RHEL.** O globbing básico é POSIX (IEEE and The Open Group, 2018) e idêntico. Como o Bash é o padrão lá, `globstar`, `nullglob` e `extglob` precisam ser ativados explicitamente com `shopt`, e os qualificadores de glob do Zsh não existem. Um script que dependa de `**` sem `shopt -s globstar` falha silenciosamente, sem mensagem nenhuma.

## 13.8. Laboratório

Roteiro em `/tmp`. Nenhum passo executa comando destrutivo sem ensaio.

### 1. Monte um diretório de teste.

```
mkdir -p /tmp/lab013 && cd /tmp/lab013
touch log1.txt log2.txt log10.txt nota.md dados.csv .oculto
mkdir -p sub/{a,b}
touch sub/a/interno.txt sub/b/outro.txt
ls -a
```

Esperado: seis arquivos, um deles oculto, e uma subárvore criada com chaves.

### 2. Confira o casamento de cada padrão.

```
echo *.txt
echo log?.txt
echo log[12].txt
echo log[0-9]*.txt
echo *
echo .*
echo [[:alpha:]]*
```

Esperado: `log?.txt` não inclui `log10.txt`; `*` não inclui `.oculto`; `.*` traz o oculto e também `.` e `...`. Compare com Figura 13.2.

### 3. Veja a ordenação alfabética morder.

```
echo log*.txt
ls -v log*.txt
```

Esperado: `log1 log10 log2` na expansão do shell, e a ordem numérica correta só com `ls -v`. Quem ordena o glob é o shell, e ele ordena como texto.

### 13. Curingas, globbing e expansão de chaves

#### 4. Prove que \* não cruza a barra.

```
echo *.txt
echo */*.txt
echo **/*.txt
```

Esperado: o primeiro só pega o nível atual, o segundo só o nível abaixo, e o terceiro — no Zsh — pega os dois. Repita com `bash -c 'echo **/*.txt'` e depois com `bash -O globstar -c 'echo **/*.txt'` para ver a diferença.

#### 5. Compare o comportamento quando nada casa.

```
echo *.inexistente
bash -c 'echo *.inexistente'
bash -O nullglob -c 'echo *.inexistente'
```

Esperado: erro no Zsh, texto literal no Bash, linha vazia no Bash com `nullglob`. Três filosofias diferentes para o mesmo padrão.

#### 6. Veja por que nullglob importa em laço.

```
bash -c 'for f in *.inexistente; do echo "processando: $f"; done'
bash -O nullglob -c 'for f in *.inexistente; do echo "processando: $f";
↵ done'
```

Esperado: o primeiro executa uma vez com o texto do padrão; o segundo não executa nenhuma. O primeiro é o bug clássico de script de backup.

#### 7. Experimente a expansão de chaves.

```
echo {a,b,c}
echo item-{1..5}.txt
echo item-{01..05}.txt
echo {0..20..5}
echo {a,b}{1,2}
echo {2025..2026}-{01..03}
```

Esperado: geração de texto puro, sem consultar o disco. Repare no zero à esquerda preservado em `{01..05}`.

#### 8. Use os idiomas de chave do dia a dia.

```
cd /tmp/lab013
echo "config original" > app.conf
cp app.conf{,.bak}
ls app.conf*
echo "config nova" > app.conf
diff app.conf{.bak,}
```

Esperado: o backup criado numa linha curtíssima e o `diff` comparando as duas versões com o nome digitado uma vez só.

#### 9. Veja a diferença entre chaves e curingas.

```
cd /tmp/lab013
echo {x,y,z}.txt      # gera nomes que NAO existem
echo *.txt            # so nomes que existem
mkdir -p novo/{p,q,r} # funciona: chaves nao consultam o disco
ls novo
```

Esperado: as chaves inventam nomes; o asterisco só encontra. Essa é a diferença essencial entre as duas expansões.

10. **Observe a interação entre variável e globbing.**

```
cd /tmp/lab013
PADRAO="*.txt"
echo $PADRAO
echo "$PADRAO"
```

Esperado: expandido sem aspas, literal com aspas. Guardar padrões em variáveis e usá-los sem aspas é uma técnica válida — e uma fonte de bug quando não é intencional.

11. **Reproduza o problema do nome com espaço.**

```
cd /tmp/lab013
touch "meu arquivo.txt"
for f in *.txt; do echo "[$f]"; done
for f in *.txt; do cp $f /tmp/lab013/novo/ 2>&1 | head -2; done
for f in *.txt; do cp "$f" /tmp/lab013/novo/; done
ls /tmp/lab013/novo
```

Esperado: o laço mostra o nome inteiro entre colchetes (o glob está certo), mas o cp sem aspas quebra o nome em dois. As aspas na variável resolvem.

12. **Trate um nome que começa com traço.**

```
cd /tmp/lab013
touch ./-arquivo-problema
ls *
ls -- *
rm -- ./-arquivo-problema
```

Esperado: o primeiro ls interpreta o nome como opções; com -- funciona. Imagine o mesmo com rm -rf no lugar do ls.

13. **Ensaie antes de apagar.**

```
cd /tmp/lab013
touch descartavel{1..5}.tmp
ls *.tmp
echo rm *.tmp
rm *.tmp
ls *.tmp 2>/dev/null || echo "nenhum sobrou"
```

Esperado: os três passos na ordem. Faça do echo na frente do comando destrutivo um reflexo permanente.

14. **Experimente os qualificadores do Zsh.**

### 13. Curingas, globbing e expansão de chaves

```
cd /tmp/lab013
head -c 200000 /dev/urandom > grande.bin
ls *(.)          # so arquivos comuns
ls */            # so diretorios
ls *.om[1,3])    # os tres mais recentes
ls *.Lk+100)     # maiores que 100 KB
```

Esperado: filtros por tipo, data e tamanho sem sair do shell. Rode `bash -c 'ls *()'` para confirmar que isso não existe no Bash.

#### 15. Contorne o limite de argumentos.

```
mkdir -p /tmp/lab013/muitos && cd /tmp/lab013/muitos
touch arquivo{00001..20000}.log
ls | wc -l
find . -maxdepth 1 -name '*.log' -print0 | xargs -0 rm
ls | wc -l
```

Esperado: vinte mil arquivos criados e removidos sem estourar a linha de comando. Com muitos mais, `rm *.log` falharia com “Argument list too long”.

#### 16. Limpe.

```
cd /tmp
ls -la /tmp/lab013
rm -rf /tmp/lab013
```

## 13.9. Resumo

- **Quem expande curingas é o shell, não o programa.** O comando recebe uma lista de nomes já resolvida e nunca vê o asterisco — por isso não existe “desfazer” baseado no padrão original.
- **A ordem das expansões é fixa:** chaves, til, variáveis, comandos, aritmética, divisão em palavras, globbing e remoção de aspas. Chaves geram texto do nada; globbing só produz o que existe no disco.
- **\* não cruza a barra e não casa com ocultos.** Para descer a árvore, é `**` com `globstar` no Bash ou `extended_glob` no Zsh; para incluir ocultos, `dotglob`.
- **Padrão sem correspondência age de forma diferente em cada shell:** o Bash entrega o texto literal, o Zsh aborta. Em script Bash com laço sobre glob, `nullglob` é praticamente obrigatório.
- **Expansão de chaves é produto cartesiano de texto puro.** `cp config{,.bak}`, `mkdir -p projeto/{doc,src}/{in,out}` e `touch teste-{01..10}.txt` são os idiomas que mais economizam digitação.
- **Aspas simples desligam todas as expansões; aspas duplas preservam variáveis e substituição de comandos,** mas desligam globbing e chaves.
- **Nome com espaço quebra em variável sem aspas,** e nome começando com traço vira opção. As defesas são `"$var"` e o separador `--`.
- **echo na frente de um comando destrutivo é o ensaio mais barato que existe,** e mostra exatamente a linha que seria executada.
- **Listas enormes estouram o limite da linha de comando.** `find -print0` com `xargs -0` resolve, e é também a única forma segura de passar nomes arbitrários entre programas.

## 14. Histórico, atalhos de teclado e produtividade no terminal

Você digitou uma linha de noventa caracteres:

```
nmap -sV -sC --top-ports 1000 -oA resultados/varredura-inicial 192.168.56.0/24
```

Aperta Enter e vê o erro: o diretório **resultados** não existe. A correção é criar o diretório e repetir a linha. Muita gente faz isso segurando a seta para a esquerda até o cursor chegar ao ponto certo, ou pior — apagando tudo com Backspace e digitando de novo.

Existem duas teclas que resolvem isso. **Ctrl+A** leva o cursor ao início da linha instantaneamente. E, no fim das contas, nem era preciso: bastava rodar **mkdir -p resultados** e depois **!!** para repetir o comando anterior inteiro.

Este capítulo é sobre a diferença entre operar o terminal e ser fluente nele. As ferramentas são três: o editor de linha, que faz da linha de comando um pequeno editor de texto; o histórico, que transforma tudo o que você já digitou num banco de dados pesquisável; e o autocompletar, que reduz caminhos longos a três teclas. Nenhuma delas é difícil. Todas rendem pelo resto da vida.

### 14.1. O editor de linha

Aquilo que responde às suas teclas antes do Enter não é o terminal nem o comando: é uma biblioteca de edição de linha dentro do shell. No Bash é a **Readline**; no Zsh é a **ZLE** (*Zsh Line Editor*). As duas usam, por padrão, atalhos herdados do editor Emacs — e é por isso que **Ctrl+A** significa “início da linha” em tantos lugares do Unix.

Figura 14.1 mostra o efeito dos principais atalhos sobre uma linha real.

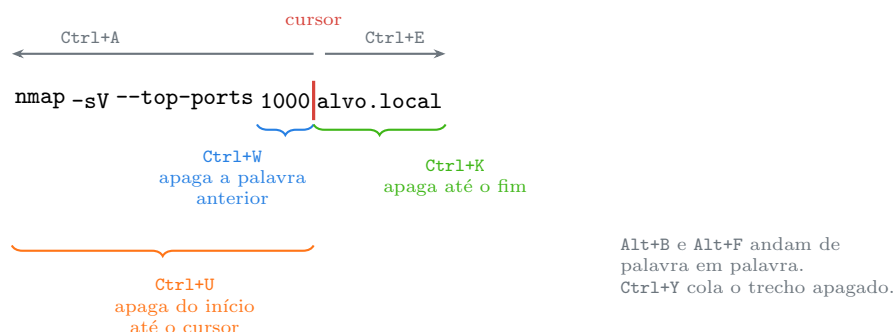


Figura 14.1.: O que os atalhos apagam depende da posição do cursor. **Ctrl+Y** traz de volta o último trecho removido.

A lista que vale memorizar, dividida por função:

Tabela 14.1.: Atalhos de edição de linha

| Atalho          | Ação                                                      |
|-----------------|-----------------------------------------------------------|
| Ctrl+A / Ctrl+E | início / fim da linha                                     |
| Ctrl+B / Ctrl+F | um caractere para trás / para frente                      |
| Alt+B / Alt+F   | uma palavra para trás / para frente                       |
| Ctrl+W          | apaga a palavra antes do cursor                           |
| Alt+D           | apaga a palavra depois do cursor                          |
| Ctrl+K          | apaga do cursor até o fim                                 |
| Ctrl+U          | apaga do início até o cursor (Bash) ou a linha toda (Zsh) |
| Ctrl+Y          | cola o último trecho apagado                              |
| Ctrl+T          | troca os dois caracteres em torno do cursor               |
| Alt+T           | troca as duas palavras em torno do cursor                 |
| Alt+.           | insere o último argumento do comando anterior             |
| Ctrl+_          | desfaz a última edição                                    |
| Ctrl+L          | limpa a tela, mantendo a linha em edição                  |
| Ctrl+X Ctrl+E   | abre a linha atual no editor definido em EDITOR           |

O Alt+. é o mais subestimado. Depois de `mkdir -p /tmp/projeto/evidencias`, digitar `cd` e apertar Alt+. insere o caminho inteiro. Repetir o atalho percorre os últimos argumentos de comandos anteriores.

O Ctrl+X seguido de Ctrl+E resolve o caso do comando que ficou grande demais: ele abre a linha no seu editor, e ao salvar e sair, o shell executa o resultado. Ideal para montar um pipeline longo com calma.

### 14.1.1. Os atalhos de controle de processo

Estes não são do editor de linha: são do driver do terminal, o PTY de Capítulo 8. Por isso funcionam mesmo com o programa ocupado.

Tabela 14.2.: Atalhos tratados pelo driver do terminal

| Atalho | Efeito                                                       |
|--------|--------------------------------------------------------------|
| Ctrl+C | envia SIGINT: pede que o programa termine                    |
| Ctrl+Z | envia SIGTSTP: suspende o programa; <b>fg</b> retoma         |
| Ctrl+D | fim de arquivo na entrada; num prompt vazio, encerra o shell |
| Ctrl+S | <b>congela</b> a saída do terminal                           |
| Ctrl+Q | descongela a saída                                           |



⚠ **Ctrl+S** é o motivo número um de “meu terminal travou”

**Ctrl+S** é um resquício dos terminais físicos: ele manda o controle de fluxo parar a saída. A tela congela, as teclas não aparecem, e tudo parece perdido. O programa continua rodando normalmente — só a exibição parou.

O conserto é **Ctrl+Q**.

Como o atalho é acionado por engano com frequência (a mão vai em **Ctrl+S** de “salvar”), muita gente o desativa de vez:

```
stty -ixon          # coloque no ~/.zshrc
```

Com isso, **Ctrl+S** fica livre para outros usos — no Bash, por exemplo, ele passa a servir para busca no histórico para frente.

### 14.1.2. Modo vi

Quem prefere as teclas do vi:

```
set -o vi          # ativa
set -o emacs       # volta ao padrao
bindkey -v         # zsh
bindkey -e         # zsh, volta ao emacs
```

No modo vi, **Esc** entra em modo de comando e todas as teclas de movimentação do vi valem na linha. É uma escolha pessoal; o importante é saber que existe, porque um dia você vai cair numa máquina configurada assim e vai precisar entender por que **Esc** mudou tudo.

## 14.2. O histórico

Cada comando executado é guardado numa lista em memória e, ao final da sessão, gravado num arquivo.

```
history            # lista tudo, numerado
history 20         # os 20 ultimos
history | grep nmap # busca por texto
echo "$HISTFILE"   # onde fica: ~/.zsh_history ou ~/.bash_history
wc -l "$HISTFILE"
```

Figura 14.2 mostra o caminho dos comandos entre a memória e o disco — e explica por que o histórico às vezes some.

### 14.2.1. Configuração que vale a pena

No ~/.zshrc:

## 14. Histórico, atalhos de teclado e produtividade no terminal

**Duas abas abertas:** a que fechar por último sobrescreve o arquivo e apaga o histórico da outra.



Figura 14.2.: O histórico vive em dois lugares. Sem configuração, a gravação é só na saída — e a última sessão a fechar vence.

```
HISTFILE=~/.zsh_history
HISTSIZE=50000                # comandos na memoria
SAVEHIST=50000                # comandos no arquivo
setopt INC_APPEND_HISTORY      # grava na hora, nao so ao sair
setopt SHARE_HISTORY           # abas compartilham o historico ao vivo
setopt HIST_IGNORE_DUPS        # nao repete comandos identicos seguidos
setopt HIST_IGNORE_SPACE       # comando iniciado com espaco nao entra
setopt HIST_REDUCE_BLANKS      # normaliza espacos
setopt EXTENDED_HISTORY        # grava data e duracao
```

O equivalente no `~/.bashrc`:

```
HISTSIZE=50000
HISTFILESIZE=50000
HISTCONTROL=ignoreboth        # ignoredups + ignorespace
HISTTIMEFORMAT='%F %T '       # carimbo de data em cada linha
shopt -s histappend
PROMPT_COMMAND='history -a'    # grava a cada comando
```

O `HISTTIMEFORMAT` transforma o histórico em registro de auditoria pessoal: você passa a saber **quando** rodou cada coisa. Numa investigação, isso vale ouro.

### 14.2.2. Busca e reuso

O atalho central é `Ctrl+R`: busca reversa incremental.

|                             |                                         |
|-----------------------------|-----------------------------------------|
| <code>Ctrl+R</code>         | entra no modo de busca                  |
| digite parte                | o shell mostra o candidato mais recente |
| <code>Ctrl+R</code> de novo | candidato anterior                      |
| <code>Enter</code>          | executa                                 |
| seta ou <code>Ctrl+E</code> | edita antes de executar                 |
| <code>Ctrl+G</code>         | cancela                                 |

No Bash, **Ctrl+S** faz a busca para frente — desde que **stty -ixon** esteja ativo. E as setas para cima e para baixo, no Zsh do Kali, já buscam por prefixo: digite **nmap** e aperte a seta para cima para percorrer apenas os comandos que começam com **nmap**.

A **expansão de histórico** trabalha por sintaxe, e é o que economiza mais digitação:

Tabela 14.3.: Expansão de histórico

| Expressão   | Significado                                      |
|-------------|--------------------------------------------------|
| !!          | o comando anterior inteiro                       |
| !\$         | o último argumento do comando anterior           |
| !^          | o primeiro argumento do comando anterior         |
| !*          | todos os argumentos do comando anterior          |
| !142        | o comando número 142 do histórico                |
| !-3         | três comandos atrás                              |
| !nmap       | o último comando que começou com <b>nmap</b>     |
| !?erro?     | o último comando que <b>contém erro</b>          |
| ^velho^novo | repete o anterior trocando a primeira ocorrência |
| !!:p        | <b>mostra</b> o comando sem executar             |

Os idiomas que aparecem todo dia:

```
apt update          # esqueceu o sudo
sudo !!            # vira: sudo apt update

mkdir -p /tmp/relatorios/2026/janeiro
cd !$              # vira: cd /tmp/relatorios/2026/janeiro

grep erro /var/log/syslog      # erro de digitacao
^sysog^syslog                 # repete com o nome corrigido

nmap -sV alvo.local
!!:p                          # mostra a linha sem executar - util para conferir
```

O **:p** é a rede de proteção: ele imprime o comando que **seria** executado e o coloca no histórico, para você revisar e chamar de novo com a seta para cima. Antes de um **!!** que envolva **rm** ou **sudo**, use **!!:p** primeiro.

### 14.2.3. O histórico é um risco de segurança

Tudo o que você digita fica gravado em texto puro, inclusive senhas passadas na linha de comando:

```
mysql -u root -pSenhaSecreta123      # esta senha esta agora no ~/.zsh_history
curl -u admin:senha https://alvo/    # esta tambem
```

Três defesas:

**Espaço no início.** Com **HIST\_IGNORE\_SPACE** ativo, um comando que começa com espaço não entra no histórico:

```
mysql -u root -pSenhaSecreta123
```

**Padrões ignorados.** No Bash, HISTIGNORE='\*password\*:~\*secret\*:~\*token\*' filtra por padrão.

**Limpeza pontual.**

```
history -d 145      # remove a entrada 145
history -c          # limpa a lista em memoria
history -w          # grava a lista limpa por cima do arquivo
```

Vale a consciência da outra ponta: num sistema que não é seu, o histórico é **evidência**, e apagá-lo é uma ação registrada e frequentemente ilegal. Num teste autorizado, o escopo do contrato define o que pode ser tocado; fora dele, mexer em registro de terceiro é crime no Brasil (Lei 12.737/2012). A lição defensiva vale igual: o histórico de um usuário comprometido conta a história inteira do que foi feito, e é por isso que ele é um dos primeiros arquivos que uma resposta a incidente examina.

### 14.3. Autocompletar

Tab completa; Tab duas vezes lista as opções.

```
cd /usr/sh<Tab>      # completa para /usr/share/
apt install nma<Tab><Tab> # lista os pacotes que comecam com nma
systemctl status ss<Tab> # completa nomes de servico
git che<Tab>        # completa subcomandos do git
kill -<Tab><Tab>     # lista os sinais
```

O autocompletar não é genérico: cada programa registra as próprias regras. Isso significa que apt install completa **nomes de pacotes**, systemctl completa **nomes de unidades** e git checkout completa **nomes de ramos**. Os scripts que fazem isso ficam em /usr/share/bash-completion/completions/ e em /usr/share/zsh/functions/Completion/.

O Zsh vai além, com menu navegável por setas:

```
autoload -Uz compinit && compinit
zstyle ':completion:*' menu select
```

E aceita caminhos abreviados: cd /u/s/w seguido de Tab expande para /usr/share/wordlists.

### 14.4. Produtividade além dos atalhos

**Aliases** para o que você repete:

```
alias ll='ls -lah'
alias ..='cd ..'
alias ...='cd ../../'
alias ports='ss -tulpn'
alias myip='curl -s ifconfig.me'
```

**Funções** quando é preciso receber argumento — alias não recebe:

```
mkcd() { mkdir -p "$1" && cd "$1"; }
extrair() {
  case "$1" in
    *.tar.gz) tar xzf "$1" ;;
    *.tar.xz) tar xJf "$1" ;;
    *.zip)    unzip "$1"    ;;
    *)       echo "formato desconhecido: $1" ;;
  esac
}
```

Aliases e funções vão no `~/.zshrc`, como visto em Capítulo 8.

**fc** edita o comando anterior no editor e executa ao salvar. É o irmão mais velho do **Ctrl+X Ctrl+E**.

**fzf**, um localizador difuso, substitui o **Ctrl+R** por uma busca interativa muito melhor:

```
sudo apt install fzf
# no ~/.zshrc:
source /usr/share/doc/fzf/examples/key-bindings.zsh
```

Depois disso, **Ctrl+R** abre uma lista filtrável do histórico inteiro, e **Ctrl+T** insere um caminho escolhido interativamente.

**watch** repete um comando em intervalos:

```
watch -n 2 'ss -tunp | head -20'
watch -d -n 1 'ls -lh /tmp/coleta'      # -d destaca as diferenças
```

**Ctrl+Z** e **fg** valem ser tratados como atalho de edição: suspender o **vim**, rodar dois comandos, e voltar com **fg** é mais rápido do que abrir outra aba. O tema completo — **jobs**, **bg**, **nohup** — é do Volume 5.

## 14.5. No Kali

- **O Zsh do Kali já vem com busca por prefixo nas setas.** Digitar **nmap** e apertar a seta para cima percorre apenas os comandos que começam com **nmap**, e não o histórico inteiro. É o recurso que mais economiza tempo depois do **Ctrl+R**.
- **kali-tweaks não configura o histórico** — isso é edição manual do `~/.zshrc`. Vale acrescentar **HISTSIZE**, **SAVEHIST** e **EXTENDED\_HISTORY** num arquivo novo em vez de alterar o padrão da distribuição.

## 14. Histórico, atalhos de teclado e produtividade no terminal

- **Senha de ferramenta na linha de comando vai para o histórico.** `hydra`, `smbclient`, `mysql`, `curl -u` e `sshpass` são os casos mais comuns. Prefira o prompt interativo da própria ferramenta, um arquivo de credenciais com permissão 600, ou o espaço inicial com `HIST_IGNORE_SPACE`.
- **EXTENDED\_HISTORY gera a linha do tempo do seu trabalho.** Num teste autorizado, o histórico com carimbo de data é insumo direto do relatório: o que foi executado, contra quem e quando. Combine com `script -T tempo.log sessao.log` para gravar a sessão inteira, saída incluída.
- **Ctrl+C numa varredura longa perde o progresso.** Ferramentas como `nmap` e `gobuster` aceitam continuar de onde pararam (`--resume`), mas só se a saída estiver sendo gravada em arquivo. Comece sempre com `-oA` ou `-o`, e prefira `tmux` a deixar a varredura presa à aba.
- **Ctrl+S congela a saída de uma varredura em andamento** e assusta bastante quem acha que travou. `Ctrl+Q` devolve, e `stty -ixon` no `~/.zshrc` evita o susto de vez.
- **O autocompletar do apt é lento na primeira vez** depois de um `apt update` no repositório rolling, porque o cache precisa ser reconstruído. Não é travamento.
- **fzf não vem instalado**, mas está no repositório e transforma a navegação em árvores de wordlists e de resultados.

**Nota sobre a família RHEL.** Readline, atalhos e expansão de histórico são iguais, já que o Bash é o mesmo. As diferenças são as opções de configuração — `shopt/HISTCONTROL` em vez de `setopt` — e a ausência da busca por prefixo nas setas, que é um recurso do Zsh.

### 14.6. Laboratório

Roteiro na VM. A maior parte é prática de teclado; faça de verdade, não só leia.

#### 1. Pratique a movimentação sem apagar nada.

Digite a linha abaixo **sem apertar Enter**:

```
nmap -sV -sC --top-ports 1000 -oA resultados/inicial 192.168.56.0/24
```

Agora: `Ctrl+A` (início), `Ctrl+E` (fim), `Alt+B` cinco vezes, `Alt+F` três vezes, `Ctrl+A`, `Ctrl+K` (apaga tudo), `Ctrl+Y` (traz de volta), `Ctrl+C` para descartar.

Esperado: o cursor salta em vez de rastejar, e `Ctrl+Y` restaura o que `Ctrl+K` removeu. Compare com Figura 14.1.

#### 2. Corrija sem apagar tudo.

Digite `ls -la /etc/ssh` e, sem apertar Enter: `Ctrl+W` (apaga a última palavra), depois digite `/etc/ssh` e aperte Enter.

Esperado: uma correção em duas teclas em vez de nove Backspaces.

#### 3. Reutilize o último argumento.

```
mkdir -p /tmp/lab014/relatorios/2026
```

Agora digite `cd` e aperte `Alt+.`, depois Enter.

```
pwd
```

Esperado: você está em `/tmp/lab014/relatorios/2026` sem ter digitado o caminho de novo. Repita `Alt+`. várias vezes para percorrer argumentos anteriores.

#### 4. Edite uma linha longa no editor.

Digite um pipeline qualquer sem apertar `Enter` e pressione `Ctrl+X` seguido de `Ctrl+E`.

Esperado: a linha abre no editor definido em `EDITOR` (`nano` por padrão). Salve e saia — o shell executa. Ideal para pipelines com muitos estágios.

#### 5. Congele e descongele o terminal de propósito.

```
ping -c 20 127.0.0.1
```

Durante a execução, aperte `Ctrl+S`. Espere alguns segundos. Aperte `Ctrl+Q`.

Esperado: a saída para e depois volta **de uma vez**, com tudo o que aconteceu no intervalo. O programa nunca parou. Acrescente `stty -ixon` ao `~/.zshrc` se quiser desativar o atalho.

#### 6. Inspeção o histórico.

```
history | tail -20
history | wc -l
echo "$HISTFILE"
ls -lh "$HISTFILE"
history | grep mkdir
```

Esperado: a lista numerada, o caminho do arquivo e as ocorrências filtradas. Repare no tamanho do arquivo — ele guarda meses de trabalho.

#### 7. Use a busca reversa.

Aperte `Ctrl+R` e digite `mkdir`. Aperte `Ctrl+R` de novo para ver ocorrências anteriores. Aperte a seta para a direita para editar em vez de executar, e `Ctrl+G` para cancelar.

Esperado: navegação instantânea por comandos de semanas atrás. Este é o atalho que mais rende do capítulo.

#### 8. Exercite a expansão de histórico.

```
ls /tmp/lab014
!!
echo /tmp/lab014/relatorios
ls !$
mkdir -p /tmp/lab014/a/b/c
cd !$
pwd
cd /tmp/lab014
```

Esperado: `!!` repete a linha inteira e `!$` reaproveita o último argumento.

#### 9. Confira antes de repetir.

```
ls -la /tmp/lab014
!!:p
!ls:p
```

Esperado: o comando é **mostrado** e não executado, ficando disponível na seta para cima. Adote isso antes de qualquer `!!` que envolva `sudo` ou `rm`.

#### 14. Histórico, atalhos de teclado e produtividade no terminal

##### 10. Corrija um erro de digitação com `^velho^novo`.

```
ls /tmp/lab104  
^lab104^lab014
```

Esperado: a segunda linha reexecuta com o nome corrigido. Funciona só na primeira ocorrência do texto.

##### 11. Esqueça o sudo de propósito.

```
apt update  
sudo !!
```

Esperado: erro de permissão na primeira, execução correta na segunda. É o idioma mais usado do histórico inteiro.

##### 12. Mantenha uma senha fora do histórico.

```
setopt HIST_IGNORE_SPACE  
echo "senha-secreta-de-teste"  
echo "esta entra normalmente"  
history | tail -3
```

Repare no **espaço** antes do segundo comando. Esperado: a linha com a senha não aparece no histórico; a outra sim. Confirme também com `grep senha-secreta "$HISTFILE"`, que não deve retornar nada.

##### 13. Remova uma entrada específica.

```
echo "comando que eu nao queria registrar"  
history | tail -3  
history -d $(history | tail -2 | head -1 | awk '{print $1}')
```

Esperado: a entrada some da lista. Lembre-se de que isso vale para a **sua** máquina — em sistema de terceiros, o histórico é registro e apagá-lo tem consequências legais.

##### 14. Ative o carimbo de data.

```
setopt EXTENDED_HISTORY  
echo "comando com carimbo"  
fc -li -3
```

Esperado: as últimas entradas com data e hora. Com isso ativo permanentemente no `~/.zshrc`, o seu histórico vira uma linha do tempo do trabalho.

##### 15. Explore o autocompletar contextual.

```
cd /usr/sh<Tab>
```

Depois experimente, sempre com `Tab` duas vezes: `apt install nma`, `systemctl status ss`, `kill -`, `git che` (dentro de um repositório).

Esperado: cada programa completa o que faz sentido para ele — pacotes, unidades, sinais, ramos. Não é uma lista genérica de arquivos.



## 16. Crie um alias e uma função.

```
alias ll='ls -lah'
mkcd() { mkdir -p "$1" && cd "$1"; }
ll /tmp/lab014
mkcd /tmp/lab014/novo/nivel
pwd
```

Esperado: o alias funciona e a função cria e entra no diretório. Se quiser mantê-los, acrescente ao final do ~/.zshrc.

## 17. Instale e experimente o fzf.

```
sudo apt install -y fzf
source /usr/share/doc/fzf/examples/key-bindings.zsh
```

Aperte Ctrl+R e digite pedaços soltos de um comando antigo.

Esperado: busca difusa sobre o histórico inteiro, muito superior ao Ctrl+R padrão. Se gostar, coloque a linha do source no ~/.zshrc.

## 18. Limpe.

```
cd /tmp
ls -la /tmp/lab014
rm -rf /tmp/lab014
```

## 14.7. Resumo

- **A linha de comando é um editor**, servido pela Readline no Bash e pela ZLE no Zsh. Ctrl+A, Ctrl+E, Ctrl+W, Ctrl+K, Ctrl+U, Ctrl+Y e Alt+. substituem centenas de toques em setas e Backspace.
- **Ctrl+C, Ctrl+Z, Ctrl+D, Ctrl+S e Ctrl+Q são do driver do terminal**, não do shell — por isso funcionam mesmo com o programa ocupado.
- **Ctrl+S congela a saída e não trava nada**. Ctrl+Q devolve, e stty -ixon desliga o atalho de vez.
- **O histórico vive em memória e só vai ao arquivo ao sair**, a menos que você configure gravação incremental. Sem isso, a última aba a fechar sobrescreve o histórico das outras.
- **Ctrl+R é o atalho de maior retorno do capítulo**. Complementado por !!, !\$, sudo !! e ^velho^novo, ele elimina a maior parte da redigitação.
- **!!:p mostra sem executar**. Use antes de qualquer repetição que envolva sudo ou comando destrutivo.
- **Senha na linha de comando fica gravada em texto puro**. Espaço inicial com HIST\_IGNORE\_SPACE, HISTIGNORE e prompt interativo da ferramenta são as defesas; do lado defensivo, o histórico é uma das primeiras fontes examinadas numa resposta a incidente.
- **O autocompletar é contextual**: cada programa registra as próprias regras, completando pacotes, unidades, ramos ou sinais conforme o comando.
- **Alias para o que se repete, função para o que precisa de argumento**, e as duas coisas no arquivo de configuração interativo do shell.



**Parte III.**

**Volume 3 — Arquivos e o Sistema de  
Arquivos**



## 15. Tipos de arquivo, inodes e a árvore de diretórios

Um servidor para de aceitar upload. O erro é `No space left on device`, mas o `df -h` mostra 40% de uso na partição. Você apaga alguns gigabytes de log, tenta de novo, e o erro continua idêntico. O disco tem espaço sobrando e mesmo assim o sistema jura que não tem.

O comando que resolve o mistério é este:

```
df -i
```

E a saída mostra `IUse% 100%`. Acabaram os **inodes** — não os bytes. Existem espaço de sobra e nenhum lugar para registrar que um arquivo novo existe.

Esse tipo de falha só faz sentido quando se entende que, no Linux, o nome de um arquivo e o arquivo em si são coisas separadas, guardadas em lugares diferentes, e ligadas por um número. Este capítulo desmonta essa estrutura. Ela é a base dos links (Capítulo 16), das permissões (Capítulo 20) e de praticamente tudo que o Volume 8 vai tratar sobre armazenamento.

### 15.1. “Tudo é um arquivo” — e o que isso quer dizer

A frase é herdada do Unix e costuma ser repetida sem explicação. O sentido prático é: o kernel oferece **uma única interface** — abrir, ler, escrever, fechar, posicionar — e faz com que ela sirva para o máximo de coisas possível.

Um arquivo de texto no disco responde a essa interface. Mas também respondem:

- o teclado e a tela, através de `/dev/tty`;
- um disco inteiro, através de `/dev/sda`;
- a memória usada pelo processo 1, através de `/proc/1/maps`;
- a temperatura da CPU, através de um arquivo em `/sys`;
- uma conexão de rede aberta, através de um descritor de socket.

O ganho é enorme: o mesmo `cat`, o mesmo `grep` e o mesmo redirecionamento de Capítulo 12 funcionam sobre qualquer uma dessas coisas. Você lê a temperatura do processador com `cat`, exatamente como leria uma lista de compras.

A frase tem exceções — sockets de rede não aparecem no sistema de arquivos com um caminho, e processos não são arquivos de verdade. Uma formulação mais honesta seria “quase tudo é um arquivo, ou tem um nome no sistema de arquivos”. Mas a ideia central se sustenta (Kerrisk, 2010).

## 15.2. Os sete tipos de arquivo

O primeiro caractere de cada linha do `ls -l` não é enfeite: é o tipo do objeto.

```
ls -l /dev/sda /dev/null /etc/passwd
ls -ld /tmp
```

Tabela 15.1.: Os sete tipos de arquivo do Linux e o caractere que os identifica no `ls -l`

| Símbolo | Tipo                     | O que é                                                                           |
|---------|--------------------------|-----------------------------------------------------------------------------------|
| -       | arquivo comum            | dados: texto, binário, imagem, qualquer coisa                                     |
| d       | diretório                | tabela que mapeia nomes para números de inode                                     |
| l       | link simbólico           | um caminho armazenado como conteúdo (Capítulo 16)                                 |
| c       | dispositivo de caractere | acesso byte a byte: terminais, <code>/dev/null</code> , <code>/dev/urandom</code> |
| b       | dispositivo de bloco     | acesso por blocos: discos, partições, <code>/dev/sda1</code>                      |
| p       | pipe nomeado (FIFO)      | cano com nome no disco, criado por <code>mkfifo</code>                            |
| s       | socket                   | comunicação entre processos locais                                                |

Vale ver cada um ao vivo:

```
ls -l /etc/hostname      # - arquivo comum
ls -ld /etc              # d diretorio
ls -l /usr/bin/vi        # l link simbolico (no Kali, aponta para vim.basic)
ls -l /dev/null          # c dispositivo de caractere
ls -l /dev/sda           # b dispositivo de bloco
ls -l /run/systemd/notify # s socket
```

O `-d` no segundo comando é essencial: sem ele o `ls` lista o **conteúdo** do diretório, não o diretório em si. É um dos tropeços mais comuns de quem está começando.

Os dois tipos de dispositivo merecem uma nota. Em vez de tamanho, o `ls -l` mostra dois números separados por vírgula — o **major** e o **minor**. O major identifica qual driver do kernel atende aquele dispositivo; o minor, qual instância. É por isso que `/dev/sda` e `/dev/sdb` compartilham o major e diferem no minor.

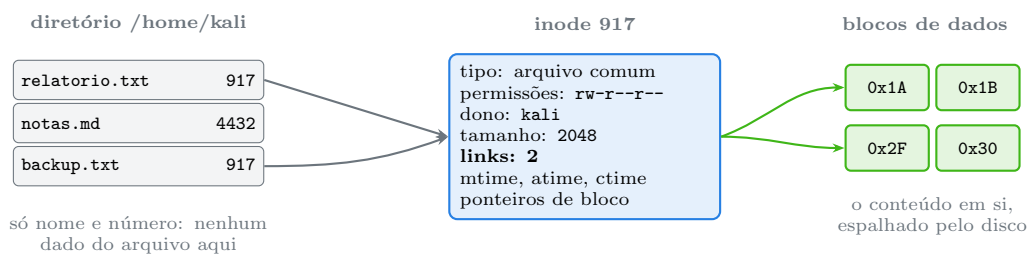
## 15.3. O inode: onde o arquivo realmente mora

Aqui está o ponto que reorganiza o entendimento de todo o resto: **o nome do arquivo não faz parte do arquivo**.

Um arquivo, para o sistema de arquivos, é uma estrutura chamada **inode** (de *index node*). O inode guarda tudo sobre o arquivo — exceto o nome:

- o tipo (dos sete acima);
- as permissões;
- o dono (UID) e o grupo (GID);
- o tamanho em bytes;
- as marcas de tempo;
- o **contador de links** — quantos nomes apontam para ele;
- e os ponteiros para os blocos de dados no disco.

O nome mora em outro lugar: dentro do diretório. Um diretório é, essencialmente, uma tabela de duas colunas — nome e número de inode. Figura 15.1 mostra a cadeia completa.



Dois nomes, um inode: apagar `relatorio.txt` apenas remove a linha do diretório e baixa o contador para 1. O conteúdo continua acessível por `backup.txt`.

Figura 15.1.: A separação entre nome, metadados e conteúdo. O diretório guarda nomes; o inode guarda todo o resto.

Essa arquitetura explica comportamentos que, de outra forma, parecem arbitrários:

- **Renomear um arquivo enorme é instantâneo.** O `mv` dentro do mesmo sistema de arquivos só reescreve uma linha da tabela do diretório. Os dados não se movem.
- **Você pode apagar um arquivo que um programa está usando.** O nome some do diretório, mas o inode só é liberado quando o contador de links chega a zero e nenhum processo o mantém aberto. É por isso que apagar um log gigante nem sempre devolve espaço: se o serviço ainda está com ele aberto, o espaço só volta no restart.
- **Permissão de apagar um arquivo é permissão sobre o diretório**, não sobre o arquivo. Apagar é editar a tabela do diretório. Esse detalhe volta em Capítulo 20.
- **Copiar não é o mesmo que mover.** O `cp` cria um inode novo; o `mv` entre sistemas de arquivos diferentes é, na prática, um `cp` seguido de `rm`.

Para ver o número do inode:

```
ls -li arquivo.txt
ls -li          # listagem longa com o inode na primeira coluna
```

## 15.4. stat: o inode em texto legível

O `ls -l` mostra um resumo. O `stat` mostra o inode quase inteiro:

```
stat /etc/passwd
```

```
File: /etc/passwd
Size: 2913          Blocks: 8          IO Block: 4096   regular file
Device: 8,1        Inode: 1443183       Links: 1
Access: (0644/-rw-r--r--)  Uid: (   0/   root)   Gid: (   0/   root)
Access: 2026-08-01 09:14:22.108374912 -0300
Modify: 2026-07-28 21:03:41.552118304 -0300
Change: 2026-07-28 21:03:41.556118217 -0300
Birth: 2026-07-28 21:03:41.552118304 -0300
```

Três detalhes que costumam passar despercebidos.

**Size e Blocks medem coisas diferentes.** O tamanho é o número de bytes lógicos; **Blocks** conta setores de 512 bytes efetivamente alocados. Para arquivos esparsos — comuns em imagens de máquina virtual — o tamanho lógico é enorme e a alocação real é mínima.

**Links: 1 é o contador que decide quando o inode morre.** Um arquivo comum novo nasce com 1. Um diretório vazio nasce com **2** (o próprio nome no diretório pai e a entrada `.` dentro dele), e ganha mais um a cada subdiretório criado — porque cada subdiretório tem um `..` apontando de volta.

**As marcas de tempo são três, e são frequentemente confundidas.**

Tabela 15.2.: As marcas de tempo de um inode e o que altera cada uma

| Marca              | Nome               | Muda quando                                                       |
|--------------------|--------------------|-------------------------------------------------------------------|
| <code>atime</code> | <i>access time</i> | o conteúdo é <b>lido</b>                                          |
| <code>mtime</code> | <i>modify time</i> | o <b>conteúdo</b> é alterado                                      |
| <code>ctime</code> | <i>change time</i> | o <b>inode</b> muda: permissão, dono, nome, contador de links     |
| <code>btime</code> | <i>birth time</i>  | nunca muda — é a criação, e nem todo sistema de arquivos a guarda |

A confusão clássica é achar que `ctime` é *creation time*. Não é: é *change time*, e ele muda com um simples `chmod`, sem que o conteúdo tenha sido tocado. Do ponto de vista forense isso é ouro — o `ctime` não pode ser definido com `touch`, enquanto `atime` e `mtime` podem:

```
touch -d "2020-01-01 10:00" arquivo.txt # falsifica atime e mtime
stat arquivo.txt                        # o ctime denuncia a operacao de
↪ agora
```

Um aviso sobre o `atime`: por padrão, distribuições modernas montam os sistemas de arquivos com `relatime`, que só atualiza o `atime` se ele estiver mais antigo que o `mtime` ou tiver mais de



24 horas. Sem isso, cada leitura viraria uma escrita — péssimo para desempenho e para SSD. Portanto, não trate o `atime` como registro exato de acesso.

O `stat` aceita formato personalizado, o que o torna útil em script:

```
stat -c '%n %s %U %A' /etc/passwd      # nome, tamanho, dono, permissões
stat -c '%i' arquivo.txt                # so o numero do inode
stat -c '%Y' arquivo.txt                # mtime em segundos desde 1970
stat -f /                               # dados do SISTEMA DE ARQUIVOS, nao do
↪ arquivo
```

## 15.5. Diretórios são arquivos com um formato especial

Um diretório é um arquivo cujo conteúdo o kernel interpreta como uma lista de pares nome/inode. Duas entradas existem sempre:

```
ls -lia /tmp | head -3
```

- `.` — aponta para o inode do próprio diretório;
- `..` — aponta para o inode do diretório pai.

No caso da raiz, `..` aponta para ela mesma. Verifique:

```
ls -id / /..
```

Os dois números são idênticos. É o que impede um `cd ..` de escapar da árvore: não existe nada acima da raiz.

Essas duas entradas explicam a contagem de links de um diretório. Figura 15.2 mostra o mecanismo.

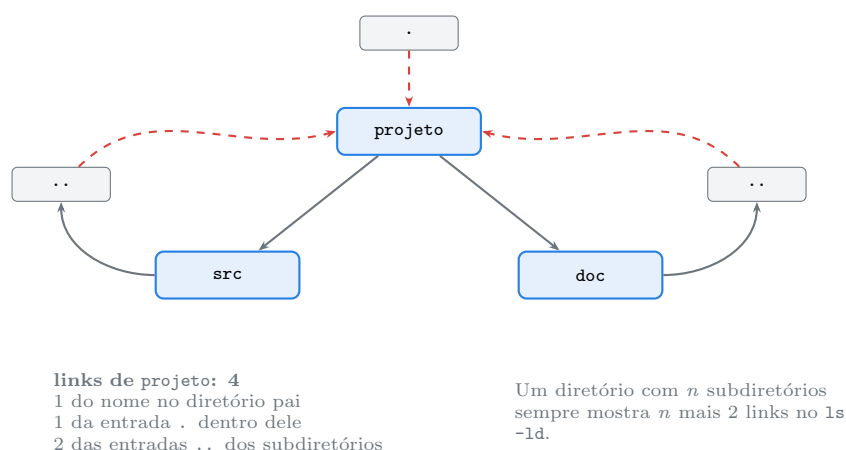


Figura 15.2.: As entradas `.` e `..` não são atalhos do shell: são linhas reais na tabela de cada diretório, e contam para o número de links.

Confirme na prática:

## 15. Tipos de arquivo, inodes e a árvore de diretórios

```
mkdir -p /tmp/demo/a /tmp/demo/b
ls -ld /tmp/demo          # 4 links: nome no pai, o proprio ".", e os dois ".."
mkdir /tmp/demo/c
ls -ld /tmp/demo          # agora 5
```

### 15.6. Quando os inodes acabam

O número de inodes de um sistema de arquivos ext4 é fixado na formatação. O padrão reserva um inode a cada 16 KB de espaço — generoso para arquivos normais, insuficiente para milhões de arquivos minúsculos.

```
df -h          # espaço em bytes
df -i          # espaço em inodes
```

Uma partição com `IUse% 100%` recusa a criação de qualquer arquivo novo, com a mensagem enganosa `No space left on device`. Os culpados habituais: cache de sessão PHP, filas de e-mail, diretórios de spool e — no contexto do Kali — pastas de saída de ferramentas que geram um arquivo por alvo.

Para achar o vilão, conte arquivos por diretório:

```
sudo du --inodes -x -d 2 / 2>/dev/null | sort -rn | head -20
```

A opção `-x` impede que a contagem atravessasse para outros sistemas de arquivos, e o `2>/dev/null` descarta os erros de permissão — um dos usos legítimos daquele descarte discutido em Capítulo 12.

Nem todo sistema de arquivos sofre disso. Btrfs e XFS alocam inodes dinamicamente e praticamente não esgotam. O Volume 8 compara as opções.

#### Aumentar inodes exige reformatar

Não existe comando para acrescentar inodes a um ext4 já formatado. A quantidade é definida por `mkfs.ext4 -N` ou `-i` no momento da criação e é imutável depois. Em sistema que vai hospedar muitos arquivos pequenos, isso precisa ser decidido antes — ou a saída é fazer backup, reformatar e restaurar.

### 15.7. file: descobrir o que é, ignorando o nome

O Linux não usa extensão para determinar o tipo do conteúdo. `relatorio.pdf` pode ser um executável; `dados`, sem extensão nenhuma, pode ser uma imagem PNG. Quem descobre é o `file`, que lê os primeiros bytes e compara com um banco de assinaturas — os *magic numbers*:

```

file /bin/ls
file /etc/passwd
file /dev/sda
file -b arquivo          # so a descricao, sem o nome
file -i arquivo          # tipo MIME
file -L link             # segue o link em vez de descrever o link
sudo file -s /dev/sda1    # inspeciona o dispositivo: mostra o sistema de
↪ arquivos

```

O `file -s` num dispositivo é o jeito rápido de descobrir se uma partição está formatada e com quê, sem montá-la.

Esse hábito tem valor prático imediato: um arquivo baixado com nome `.jpg` que o `file` identifica como `PE32 executable` é um problema, não uma foto.

## 15.8. No Kali

- **A raiz do Kali é ext4 por padrão**, com inodes fixos. Em laboratório com disco pequeno, sessões longas de ferramentas que escrevem um arquivo por alvo (`gobuster` com saída por host, `nuclei` com saída por template, `masscan` fragmentado) esgotam inodes muito antes do espaço. Cheque `df -i` sempre que o `No space left` não bater com o `df -h`.
- **/usr/share/wordlists é um campo minado de links**. No Kali, vários nomes ali são links simbólicos para os diretórios reais dos pacotes (`/usr/share/seclists`, `/usr/share/dirb/wordlists`), e o `rockyou.txt` vem comprimido como `rockyou.txt.gz`. `ls -l /usr/share/wordlists` mostra os alvos; `file -L` resolve o link antes de descrever.
- **Ferramentas ofensivas dependem de arquivos de dispositivo**. `/dev/net/tun` para VPN e `openvpn`, `/dev/rfkill` e as interfaces em `/sys/class/net` para Wi-Fi, `/dev/bus/usb` para adaptadores externos. Quando uma ferramenta reclama de permissão sobre algo em `/dev`, o problema quase sempre é grupo do usuário, não a ferramenta.
- **stat é ferramenta forense de primeira linha**. Num sistema comprometido, `mtime` e `atime` podem ter sido reajustados com `touch`, mas o `ctime` só muda com operação real sobre o inode — e não há opção de `touch` que o defina. Divergência entre `ctime` muito recente e `mtime` antigo é indicador clássico de arquivo adulterado.
- **file antes de executar qualquer coisa baixada**. Em análise de artefato, `file`, `strings` e `sha256sum` são os três primeiros comandos, sempre com o arquivo sem permissão de execução. O Volume 14 aprofunda integridade e verificação.
- **Sockets em /run são o mapa dos serviços locais**. `ls -l /run/*.sock` revela o que está escutando localmente sem passar pela rede — útil tanto para diagnóstico quanto para enumeração em teste autorizado.
- **Em modo live com persistência, o inode não é identificador estável**. A persistência do Kali usa um sistema de arquivos sobreposto (`overlayfs`), e o número de inode que você vê para um mesmo caminho pode mudar entre reinicializações. Não use inode como chave em script que precise sobreviver a reboot nesse cenário.

**Nota sobre a família RHEL.** A estrutura de inode pertence ao sistema de arquivos, não à distribuição, e vale igual em Fedora e RHEL. A diferença relevante é o padrão de formatação: RHEL usa **XFS** desde a versão 7, e XFS aloca inodes dinamicamente — o esgotamento descrito acima praticamente não ocorre lá. Em compensação, um XFS não pode ser reduzido de tamanho depois de criado.

## 15.9. Laboratório

Roteiro em /tmp, sem risco para o sistema.

### 1. Colecione os sete tipos.

```
mkdir -p /tmp/lab015 && cd /tmp/lab015
echo "conteudo" > comum.txt
mkdir umdir
ln -s comum.txt atalho
mkfifo cano
ls -l comum.txt umdir atalho cano /dev/null /dev/sda /run/systemd/notify
```

Esperado: -, d, l, p, c, b e s na primeira coluna. Repare que /dev/null e /dev/sda mostram dois números no lugar do tamanho — major e minor.

### 2. Veja o inode de um arquivo por inteiro.

```
stat comum.txt
ls -li comum.txt
```

Esperado: o número após Inode: no stat é o mesmo da primeira coluna do ls -li. Anote-o.

### 3. Prove que o nome não é o arquivo.

```
cp comum.txt copia.txt
ls -li comum.txt copia.txt
mv comum.txt renomeado.txt
ls -li renomeado.txt
```

Esperado: a cópia recebeu um inode **novo**; o mv manteve o inode **original**. Copiar cria um arquivo; renomear só reescreve a tabela do diretório.

### 4. Separe as três marcas de tempo.

```
stat -c 'A:%x%nM:%y%nC:%z' renomeado.txt
cat renomeado.txt > /dev/null # so leitura
stat -c 'A:%x%nM:%y%nC:%z' renomeado.txt
chmod 600 renomeado.txt # so metadados
stat -c 'A:%x%nM:%y%nC:%z' renomeado.txt
echo "linha nova" >> renomeado.txt # conteudo
stat -c 'A:%x%nM:%y%nC:%z' renomeado.txt
```

Esperado: o chmod mexeu **apenas** no ctime; o >> mexeu no mtime e no ctime. O atime pode não ter mudado na leitura, por causa do relatime.

### 5. Tente falsificar o tempo e veja o ctime denunciar.

```
touch -d "2015-03-10 08:00:00" renomeado.txt
stat -c 'M:%y%nC:%z' renomeado.txt
```

Esperado: o mtime foi para 2015, o ctime marca **agora**. É exatamente essa divergência que se procura em análise forense.

## 6. Conte os links de um diretório.

```
mkdir -p arvore
ls -ld arvore          # 2
mkdir arvore/a arvore/b
ls -ld arvore          # 4
ls -lia arvore | head -4
```

Esperado: 2 no início, 4 depois dos dois subdiretórios. Na listagem, `.` tem o mesmo inode que `arvore`, e `..` tem o inode de `/tmp/lab015`. Compare com Figura 15.2.

## 7. Confirme que a raiz é o topo.

```
ls -id / /.. /../.. /../../..
```

Esperado: os quatro números idênticos. Subir a partir de `/` devolve `/`.

## 8. Segure um arquivo apagado e observe o espaço.

```
head -c 50M /dev/zero > grande.bin
df -h /tmp | tail -1
tail -f grande.bin > /dev/null &
rm grande.bin
ls grande.bin          # nao existe mais
df -h /tmp | tail -1    # espaco ainda ocupado
kill %1
df -h /tmp | tail -1    # agora liberou
```

Esperado: entre o `rm` e o `kill`, o arquivo não aparece na listagem e mesmo assim ocupa espaço. O inode só morre quando o último processo o fecha. É a explicação de “apaguei o log e o disco continua cheio”. Em `/tmp` montado como `tmpfs`, o efeito aparece no consumo de memória em vez de disco — o princípio é o mesmo.

## 9. Localize o arquivo apagado que ainda está aberto.

Repita o passo anterior até o `rm`, e então:

```
sudo lsof +L1 2>/dev/null | head
ls -l /proc/$(pgrep -n tail)/fd/
```

Esperado: o `lsof` lista arquivos com contador de links zero; o `/proc/<pid>/fd/` mostra o caminho seguido de `(deleted)`. Encerre com `kill %1`.

## 10. Cheque a saúde dos inodes do sistema.

```
df -h
df -i
df -i /tmp | tail -1
```

Esperado: `IUse%` bem abaixo de `Use%` num sistema saudável. Se um dia os dois divergirem para o lado errado, você já sabe o que aconteceu.

## 11. Descubra o que é o arquivo, ignorando o nome.

```
cp /bin/ls foto.jpg
file foto.jpg
file -i foto.jpg
file -b /etc/passwd /dev/null umdir
```

## 15. Tipos de arquivo, inodes e a árvore de diretórios

```
sudo file -s /dev/sda1
```

Esperado: `foto.jpg` é identificado como executável ELF, não como imagem. O `file -s` no dispositivo revela o sistema de arquivos sem montá-lo.

### 12. Veja um arquivo esparsos.

```
truncate -s 500M esparsos.img
ls -lh esparsos.img
du -h esparsos.img
stat -c 'tamanho: %s   blocos: %b' esparsos.img
```

Esperado: 500 MB no `ls`, praticamente zero no `du`. Nenhum bloco foi realmente alocado — o mesmo truque que faz imagens de VM ocuparem menos do que declaram.

### 13. Limpe.

```
cd /tmp
ls -la /tmp/lab015
rm -rf /tmp/lab015 /tmp/demo
```

## 15.10. Resumo

- **“Tudo é um arquivo”** significa uma interface única — abrir, ler, escrever — servindo para discos, terminais, processos e sensores. É o que faz `cat` funcionar sobre a temperatura da CPU.
- **São sete tipos**, identificados pelo primeiro caractere do `ls -l`: `-` comum, `d` diretório, `l` link simbólico, `c` e `b` dispositivos, `p` pipe nomeado e `s` socket.
- **O nome não faz parte do arquivo.** O inode guarda tipo, permissões, dono, tamanho, tempos, contador de links e os ponteiros de dados; o nome mora na tabela do diretório.
- **Essa separação explica o cotidiano:** renomear é instantâneo, apagar um arquivo aberto não libera espaço, e permissão de apagar é permissão sobre o diretório.
- **`stat` mostra o inode em texto**, incluindo as três marcas de tempo. `ctime` é *change time*, não *creation time* — e é a única que `touch` não consegue falsificar.
- **Um diretório vazio tem 2 links** (o nome no pai e o `.`), e ganha um a cada subdiretório por causa do `...`. Em `/`, o `...` aponta para a própria raiz.
- **Inodes acabam antes do espaço** em ext4 com muitos arquivos pequenos. O sintoma é `No space left on device` com `df -h` tranquilo; o diagnóstico é `df -i`; e não há como acrescentar inodes sem reformatar.
- **Extensão não determina conteúdo no Linux.** Quem responde o que um arquivo é de fato é o `file`, lendo assinaturas — e `file -s` funciona até em dispositivo de bloco não montado.

## 16. Links simbólicos e hard links

Você organiza o diretório de trabalho, move uma pasta de lugar e, na volta, metade dos atalhos que tinha criado aponta para o vazio. O `ls` mostra os nomes em vermelho piscante, e qualquer tentativa de abri-los devolve:

```
cat: config.yaml: No such file or directory
```

Só que o arquivo existe. Está ali, dois diretórios adiante, intacto. O que quebrou foi o link — e ele quebrou porque guardava um **caminho**, não uma referência ao arquivo.

Existe outro tipo de link, o *hard link*, que não teria quebrado. Ele aponta direto para o inode e sobrevive a qualquer renomeação. Em compensação, tem limitações que o simbólico não tem. Escolher entre os dois é uma decisão prática que aparece toda semana, e depende inteiramente daquela separação entre nome e inode vista em Capítulo 15.

### 16.1. Hard link: outro nome para o mesmo inode

Criar um hard link é acrescentar uma linha na tabela de um diretório apontando para um inode que já existe:

```
echo "conteudo original" > original.txt
ln original.txt segundo-nome.txt
ls -li original.txt segundo-nome.txt
```

Os dois nomes mostram o **mesmo número de inode**, e o contador de links subiu para 2. Aqui está o ponto que costuma surpreender: **não existe “o original”**. Os dois nomes são igualmente legítimos, e o sistema de arquivos não guarda qual veio primeiro.

```
rm original.txt
cat segundo-nome.txt      # o conteudo continua la
```

Apagar um dos nomes só decrementa o contador. O inode e os dados desaparecem quando o contador chega a zero — e nenhum processo mantém o arquivo aberto.

As limitações vêm da própria natureza da coisa:

- **Não atravessa sistemas de arquivos.** Números de inode só fazem sentido dentro de um sistema de arquivos. Tentar ligar `/home/kali/a.txt` a algo em `/mnt/usb` devolve `Invalid cross-device link`.
- **Não funciona para diretórios.** Se fosse permitido, seria possível criar ciclos na árvore, e ferramentas que percorrem diretórios entrariam em laço infinito sem forma segura de detectar. Só o kernel cria hard links de diretório, para as entradas `.` e `...`
- **Não sobrevive a um `cp` ingênuo.** Copiar um arquivo com hard links produz cópias independentes, a menos que se use `cp -a` ou `cp --preserve=links` sobre o conjunto todo.

## 16.2. Link simbólico: um arquivo que contém um caminho

O link simbólico — *symlink*, ou link “mole” — é um arquivo de tipo próprio (l no `ls -l`) cujo conteúdo é uma **cadeia de caracteres com um caminho**. Quando o kernel resolve um caminho e encontra um symlink, ele substitui o link pelo texto guardado e continua a resolução.

```
ln -s /etc/passwd atalho-passwd
ls -l atalho-passwd
readlink atalho-passwd
```

A saída do `ls -l` mostra `atalho-passwd -> /etc/passwd`, e o tamanho do link é exatamente o número de caracteres do caminho armazenado. Um symlink para `/etc/passwd` tem 11 bytes.

Isso libera tudo que o hard link proíbe: pode atravessar sistemas de arquivos, pode apontar para diretórios, pode até apontar para algo que **não existe**. O preço é a fragilidade: se o alvo sair do lugar, o link fica pendurado.

Figura 16.1 coloca os dois lado a lado.

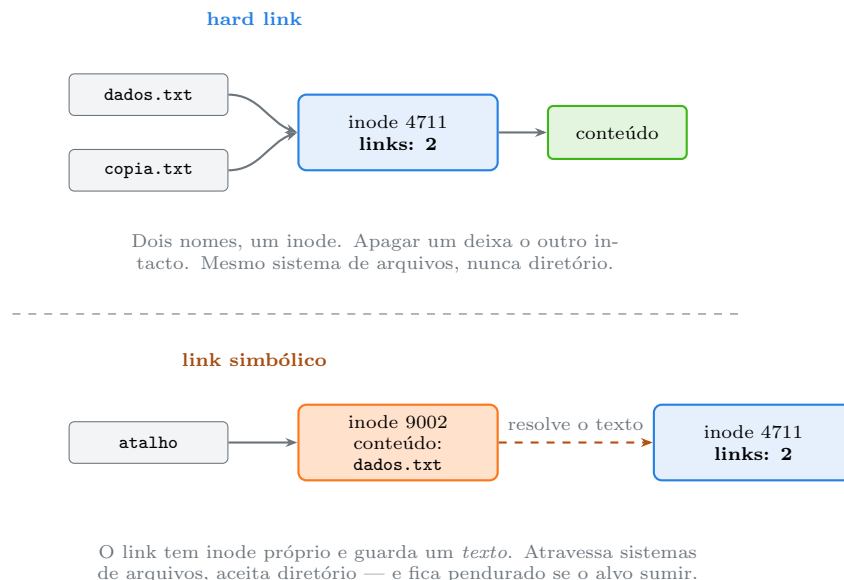


Figura 16.1.: O hard link é um nome a mais para o inode; o link simbólico é um arquivo separado cujo conteúdo é o caminho do alvo.

Um resumo das diferenças:

Tabela 16.1.: Hard link e link simbólico, ponto a ponto

| Característica                  | Hard link         | Link simbólico  |
|---------------------------------|-------------------|-----------------|
| Aponta para                     | inode             | caminho (texto) |
| Tipo no <code>ls -l</code>      | -, indistinguível | l               |
| Atravessa sistema de arquivos   | não               | sim             |
| Funciona para diretório         | não               | sim             |
| Sobrevive ao alvo ser renomeado | sim               | não             |
| Pode apontar para o inexistente | não               | sim             |
| Ocupa inode próprio             | não               | sim             |



| Característica                 | Hard link | Link simbólico |
|--------------------------------|-----------|----------------|
| Incrementa o contador de links | sim       | não            |

## 16.3. `ln`: a ordem dos argumentos e outras armadilhas

A sintaxe segue o padrão do `cp`: **origem primeiro, destino depois**.

```
ln -s ALVO NOME_DO_LINK
```

O erro mais comum é inverter. E ele é traiçoeiro, porque a forma invertida frequentemente **funciona** — criando um link no lugar errado, apontando para o lugar errado. A regra mnemônica que resolve: leia da esquerda para a direita como “faça *isto* ficar disponível *ali*”.

Um segundo detalhe que morde: se o segundo argumento for um **diretório existente**, o `ln` cria o link dentro dele com o nome-base do alvo. É conveniente e, quando não intencional, confuso.

```
ln -s /etc/hosts /tmp/          # cria /tmp/hosts
```

Omitir o segundo argumento cria o link no diretório atual:

```
ln -s /usr/share/wordlists/rockyou.txt.gz    # cria ./rockyou.txt.gz
```

Opções que valem memorizar:

```
ln -s alvo link          # cria link simbolico
ln -sf alvo link         # substitui o link se ja existir
ln -sfn alvo link        # -n e ESSENCIAL quando "link" ja aponta para um
↪ diretorio
ln -sr alvo link         # cria o link com caminho RELATIVO calculado
↪ automaticamente
ln -sv alvo link         # mostra o que fez
ln alvo copia            # hard link
```

O `-n` merece explicação. Sem ele, se `link` já for um link simbólico apontando para um diretório, o `ln -sf` **entra** no diretório e cria o novo link lá dentro, em vez de substituir. Com `-n`, o link existente é tratado como arquivo comum e substituído. Toda receita de “atualizar o link **atual** para a nova versão” precisa de `ln -sfn`.

### 16.3.1. Caminho relativo ou absoluto

O caminho guardado dentro do symlink pode ser absoluto ou relativo, e a escolha tem consequência:

```
ln -s /home/kali/projeto/config.yaml conf-abs
ln -s ../projeto/config.yaml conf-rel
```

## 16. Links simbólicos e hard links

Um link **relativo** é resolvido a partir do diretório onde o link está — não do diretório onde você estava quando o criou. Isso confunde muita gente:

```
cd /tmp
ln -s ../etc/hostname teste      # resolve /tmp/../etc/hostname = /etc/hostname:
↪ funciona
mv teste /var/tmp/              # agora resolve /var/tmp/../etc/hostname:
↪ quebrou
```

Regra prática: **relativo** dentro de uma árvore que se move junto (um repositório, um pacote, uma estrutura de dotfiles); **absoluto** quando o alvo é um caminho fixo do sistema. O `ln -sr` calcula o relativo correto sozinho, o que evita contar `..` na mão.

### 16.4. Como os comandos tratam os links

Este é o ponto onde script quebra em silêncio. A convenção geral usa três letras:

- `-P` (*physical*): **não** segue links — opera sobre o link em si;
- `-L` (*logical*): segue todos os links encontrados;
- `-H`: segue apenas os links dados na linha de comando, não os encontrados durante a descida recursiva.

```
ls -l link          # descreve o LINK
ls -lL link         # descreve o ALVO
cat link            # le o ALVO: leitura sempre segue
rm link            # remove o LINK, nunca o alvo
cp link copia      # copia o CONTEUDO do alvo
cp -P link copia   # copia o LINK como link
cp -a origem/ destino/ # preserva links, permissões e tempos: o padrão para
↪ backup
du -sh link        # tamanho do link, poucos bytes
du -shL link       # tamanho do alvo
```

O caso mais perigoso é o `chown -R`. Por padrão ele **não** segue links simbólicos, o que é o comportamento seguro. Mas `chown -RL` segue, e num diretório que contenha um link para `/etc` isso muda o dono de arquivos de sistema.

 **chown -R com -L pode arruinar o sistema**

```
sudo chown -RL kali:kali /var/www      # PERIGOSO se houver link para fora
↪ da árvore
sudo chown -Rh kali:kali /var/www      # muda o dono do LINK, não do alvo:
↪ seguro
```

Antes de qualquer `chown -R` ou `chmod -R` em diretório que você não criou, liste os links que existem lá dentro: `find /var/www -type l -ls`. Se algum apontar para fora da árvore, trate-o individualmente.

## 16.5. Resolvendo caminhos: *readlink*, *realpath* e *namei*

```
readlink link           # mostra o caminho armazenado, um nível so
readlink -f link        # resolve ate o fim; o alvo final pode nao existir
realpath link           # resolve ate o fim; exige que o alvo exista
realpath --relative-to=/usr /usr/bin/vim
namei -l /usr/bin/vi    # mostra CADA etapa da resolucao, com permissoes
```

O *namei -l* é subestimado. Quando um caminho devolve **Permission denied** e você não sabe qual componente é o culpado, ele responde de imediato — mostrando permissão e dono de cada diretório do caminho, além de resolver os links pelo meio.

O *readlink -f* é o idioma padrão em script para descobrir onde o script realmente está:

```
DIR="$(dirname "$(readlink -f "$0"))"
```

Sem ele, um script chamado através de um link no PATH acha que mora no diretório do link, não no diretório real.

## 16.6. Links quebrados e laços

Um link cujo alvo não existe é um *dangling symlink*. Ele não dá erro na criação nem na listagem — só na hora do uso.

```
find /caminho -xtype l           # lista links quebrados
find /caminho -xtype l -printf '%p -> %l\n'
find /caminho -type l -exec test ! -e {} \; -print # forma portavel
```

O *-xtype l* é o teste certo: *-type l* lista **todos** os links, enquanto *-xtype l* lista aqueles cujo alvo, depois de resolvido, ainda é um link — na prática, os quebrados.

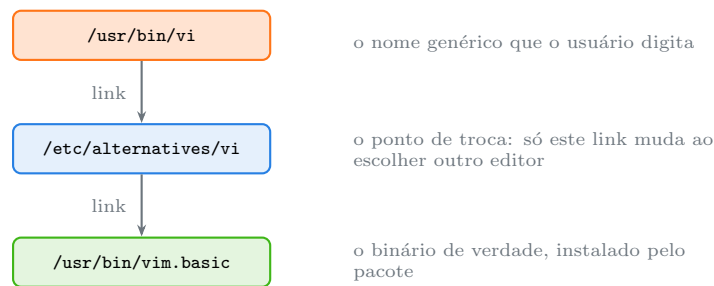
Um link também pode apontar para si mesmo, direta ou indiretamente:

```
ln -s laco laco
cat laco
# cat: laco: Too many levels of symbolic links
```

O kernel limita a cadeia de resolução — tipicamente 40 níveis — e devolve **ELOOP**. É proteção, não defeito.

## 16.7. O caso concreto do Kali: *update-alternatives*

Debian e Kali resolvem “qual programa atende o nome genérico” com uma cadeia dupla de links simbólicos gerenciada pelo *update-alternatives* (Debian Project, 2025). Figura 16.2 mostra a cadeia de *vi*.



O comando `update-alternatives` reescreve apenas o link do meio. Nada em `/usr/bin` é tocado, e a troca vale para o sistema inteiro.

Figura 16.2.: A cadeia dupla do `update-alternatives`: o nível intermediário é o que permite trocar de programa sem mexer nos arquivos dos pacotes.

```
ls -l /usr/bin/vi
namei -l /usr/bin/vi
update-alternatives --list editor
sudo update-alternatives --config editor
```

A elegância do arranjo está no nível do meio. Sem ele, trocar de editor exigiria sobrescrever `/usr/bin/vi` — o que o gerenciador de pacotes desfaria na próxima atualização.

## 16.8. Segurança: por que links merecem desconfiança

Links simbólicos são um vetor clássico de ataque local, e o mecanismo é simples: um programa privilegiado escreve num caminho previsível dentro de `/tmp`; um usuário sem privilégio cria antes um link com aquele nome apontando para um arquivo sensível; o programa privilegiado escreve no alvo do link.

As defesas do Linux moderno:

```
sysctl fs.protected_symlinks # 1 por padrao no Debian/Kali
sysctl fs.protected_hardlinks # 1 por padrao
```

Com `fs.protected_symlinks=1`, um link num diretório com *sticky bit* — como `/tmp`, tema de Capítulo 20 — só é seguido se o dono do link for o mesmo dono do diretório ou o mesmo do processo. `fs.protected_hardlinks=1` impede criar hard link para um arquivo que você não pode ler.

Isso mitiga, mas não substitui a prática correta em script: usar `mktemp` em vez de nome previsível.

```
tmp="$(mktemp -d)" # nome imprevisível, permissões 700
trap 'rm -rf "$tmp"' EXIT
```

O tema completo de escrita segura em `/tmp` volta no Volume 12; o de superfície de ataque, no Volume 14.

## 16.9. No Kali

- **/usr/share/wordlists é quase todo feito de links.** `dirb`, `dirbuster`, `fasttrack.txt`, `metasploit`, `wfuzz` e `seclists` aparecem lá como links simbólicos para os diretórios reais dos pacotes. `ls -l /usr/share/wordlists` mostra os alvos; `namei -l` mostra onde de fato vão parar. Se um pacote for removido, o link fica pendurado e a ferramenta reclama de arquivo inexistente sem dizer que a causa é um link morto.
- **update-alternatives governa editor, x-terminal-emulator, x-www-browser e vários outros.** Trocar o editor padrão do visudo e do git no Kali é `sudo update-alternatives --config editor`, não `edit /usr/bin/vi`.
- **Wordlist gigante em outra partição pede link, não cópia.** Se o `rockyou.txt` descomprimido só existe numa partição de dados montada, `ln -s /mnt/dados/rockyou.txt ~/wordlists/rockyou.txt` resolve sem duplicar 130 MB. Hard link não serviria: sistemas de arquivos diferentes.
- **Estruture o diretório de engajamento com links relativos.** Uma árvore `~/engajamentos/cliente-x/` com links criados por `ln -sr` continua íntegra quando a pasta inteira for arquivada, movida ou copiada para a máquina do relatório. Com caminho absoluto, tudo quebra na primeira mudança de host.
- **Em análise de artefato, nunca copie um diretório suspeito sem -P ou -a.** Um `cp -r` comum segue os links e pode arrastar arquivos de fora da amostra para dentro dela — ou, pior, escrever onde não devia. `cp -a` preserva a estrutura como está.
- **Hard link é a forma de “congelar” evidência sem duplicar espaço.** Enquanto o contador de links for maior que zero, o conteúdo não vai embora, mesmo que o nome original seja apagado. Use dentro do mesmo sistema de arquivos e registre o inode com `stat`.
- **Cheque links quebrados depois de um apt autoremove agressivo.** No Kali rolling, remover metapacotes pode deixar links pendurados em `/usr/share`. `sudo find /usr/share -xtype l` faz a varredura em segundos.

**Nota sobre a família RHEL.** O mecanismo de links é do kernel e idêntico. A diferença é administrativa: RHEL e Fedora usam `alternatives` (sem o `update-`) para o mesmo papel, com sintaxe própria. Além disso, o SELinux atribui contexto ao link e ao alvo separadamente — um link com contexto errado pode ser negado mesmo com permissões corretas, e o diagnóstico aparece em `ls -lZ`, não em `ls -l`.

## 16.10. Laboratório

Roteiro em `/tmp`.

1. Crie um hard link e observe o contador.

```
mkdir -p /tmp/lab016 && cd /tmp/lab016
echo "conteudo original" > dados.txt
ls -li dados.txt
ln dados.txt copia-hard.txt
stat -c '%n inode=%i links=%h' dados.txt copia-hard.txt
```

Esperado: inode idêntico e contador 2 nos dois. São dois nomes do mesmo arquivo.

2. Prove que não existe “o original”.

## 16. Links simbólicos e hard links

```
rm dados.txt
ls -li copia-hard.txt
cat copia-hard.txt
```

Esperado: o conteúdo continua acessível e o contador voltou a 1. Apagar um nome não apaga o arquivo.

### 3. Escreva por um nome e leia pelo outro.

```
ln copia-hard.txt terceiro.txt
echo "linha acrescentada" >> terceiro.txt
cat copia-hard.txt
```

Esperado: a linha aparece nos dois. É o mesmo inode, os mesmos blocos.

### 4. Esbarre nos limites do hard link.

```
mkdir umdir
ln umdir outro-dir
ln copia-hard.txt /dev/shm/tentativa
```

Esperado: hard link not allowed for directory no primeiro e Invalid cross-device link no segundo. As duas restrições, ao vivo.

### 5. Crie um link simbólico e inspecione.

```
ln -s copia-hard.txt atalho
ls -li copia-hard.txt atalho
stat -c '%n tipo=%F tamanho=%s' atalho
readlink atalho
```

Esperado: inode **diferente**, tipo symbolic link, e tamanho igual ao número de caracteres do alvo — 15 bytes para copia-hard.txt.

### 6. Quebre o link e conserte.

```
mv copia-hard.txt renomeado.txt
ls -l atalho
cat atalho
ln -sf renomeado.txt atalho
cat atalho
```

Esperado: depois do mv o link fica pendurado e o cat falha; o ln -sf reaponta. Um hard link não teria notado a renomeação.

### 7. Entenda relativo contra absoluto.

```
mkdir -p arv/sub
echo alvo > arv/alvo.txt
ln -s arv/alvo.txt rel-frag
ln -sr arv/alvo.txt arv/sub/rel-certo
ls -l rel-frag arv/sub/rel-certo
cat arv/sub/rel-certo
mv rel-frag arv/sub/
cat arv/sub/rel-frag
```

Esperado: o `ln -sr` calculou `../alvo.txt` e funciona de dentro de `sub`; o link criado à mão quebrou ao ser movido, porque o caminho relativo é resolvido a partir de onde o link está.

#### 8. Veja o `-n` fazer diferença.

```
mkdir -p versao-1 versao-2
ln -s versao-1 atual
ls -l atual
ln -sf versao-2 atual          # SEM -n
ls -l atual; ls -l versao-1/
rm -f versao-1/versao-2
ln -sfn versao-2 atual        # COM -n
ls -l atual
```

Esperado: sem `-n`, o link foi criado **dentro** de `versao-1`; com `-n`, o link `atual` foi substituído. É a diferença entre uma troca de versão que funciona e uma que se enrola.

#### 9. Compare como os comandos tratam o link.

```
ln -s renomeado.txt ponteiro
ls -l ponteiro
ls -lL ponteiro
du -sh ponteiro
du -shL ponteiro
cp ponteiro copia-conteudo
cp -P ponteiro copia-link
ls -l copia-conteudo copia-link
```

Esperado: `ls -lL` e `du -shL` mostram o alvo; `cp` sem opção copiou o conteúdo, `cp -P` copiou o link como link.

#### 10. Confirme que `rm` nunca toca o alvo.

```
rm ponteiro
ls -l renomeado.txt
```

Esperado: o alvo intacto. Remover um link simbólico é sempre seguro.

#### 11. Provoque um laço.

```
ln -s laco laco
cat laco
```

Esperado: Too many levels of symbolic links. O kernel corta a resolução por volta de 40 níveis.

#### 12. Cace os links quebrados.

```
find . -type l | wc -l
find . -xtype l
find . -xtype l -printf '%p -> %l\n'
```

Esperado: a primeira contagem inclui todos os links; a segunda lista só os pendurados, com o alvo inexistente ao lado.

13. Rastreie a cadeia do vi.

```
ls -l /usr/bin/vi
namei -l /usr/bin/vi
readlink -f /usr/bin/vi
update-alternatives --list editor
```

Esperado: a cadeia /usr/bin/vi até /usr/bin/vim.basic passando por /etc/alternatives, exatamente como em Figura 16.2. O namei -l mostra permissão e dono de cada etapa.

14. Descubra por que um caminho dá “Permission denied”.

```
mkdir -p fechado/dentro
echo segredo > fechado/dentro/arq.txt
chmod 000 fechado
cat fechado/dentro/arq.txt
namei -l "$PWD/fechado/dentro/arq.txt"
chmod 755 fechado
```

Esperado: o cat falha e o namei -l aponta exatamente qual componente do caminho barrou. É o diagnóstico mais rápido que existe para esse erro.

15. Explore os links reais do Kali.

```
ls -l /usr/share/wordlists/
sudo find /usr/share -xtype l 2>/dev/null | head
```

Esperado: vários links apontando para os diretórios dos pacotes. A segunda linha revela se algum ficou pendurado depois de remoções de pacote.

16. Confira as proteções do kernel.

```
sysctl fs.protected_symlinks fs.protected_hardlinks
```

Esperado: = 1 nas duas. É o que impede o ataque clássico de link em /tmp.

17. Limpe.

```
cd /tmp
ls -laR /tmp/lab016 | head -30
rm -rf /tmp/lab016
```

## 16.11. Resumo

- **Hard link é outro nome para o mesmo inode.** Não existe “o original”: os nomes são equivalentes, e o arquivo só some quando o contador de links zera.
- **Hard link não atravessa sistema de arquivos nem funciona para diretório,** porque números de inode são locais e ciclos de diretório quebrariam qualquer travessia recursiva.
- **Link simbólico é um arquivo cujo conteúdo é um caminho.** Tem inode próprio, tamanho igual ao comprimento do caminho, e resolve na hora do uso — por isso pode apontar para o inexistente.
- **ln -s ALVO LINK**, nessa ordem. Se o destino for diretório existente, o link nasce dentro dele; **-sf**n é obrigatório para substituir um link que aponta para diretório.



- **Relativo se move junto com a árvore; absoluto fixa o alvo.** `ln -sr` calcula o relativo correto sem contar `..` na mão.
- **-P não segue, -L segue, -H segue só o da linha de comando.** `cp` copia conteúdo por padrão e `cp -a` preserva a estrutura; `rm` nunca toca o alvo; `chown -RL` pode arrastar arquivos de fora da árvore.
- **`readlink -f`, `realpath` e `namei -l` resolvem a cadeia**, e o `namei -l` é o diagnóstico mais rápido para `Permission denied` num caminho longo.
- **`find -xtype l` acha os links quebrados**, e `fs.protected_symlinks` protege contra o ataque clássico em diretório com sticky bit — mas `mktemp` continua sendo a prática correta em script.



## 17. Localizando arquivos: `find`, `locate`, `which` e `type`

Duas perguntas aparecem toda semana e têm respostas completamente diferentes.

A primeira: “*o disco encheu — quais arquivos com mais de 100 MB foram modificados no último mês?*” Nenhum índice tem isso pronto. É preciso percorrer a árvore, olhar o inode de cada arquivo e filtrar.

A segunda: “*onde está o `nmap`?*” Aqui não se percorre nada: o shell já sabe, porque ele mesmo resolveu o nome quando você digitou.

O erro clássico é usar a ferramenta da segunda pergunta para responder a primeira, ou o contrário — e concluir que “não achou”. Este capítulo separa as três famílias: quem varre a árvore (`find`), quem consulta um índice (`locate`), e quem responde sobre comandos (`type`, `command -v`, `which`).

### 17.1. `find`: a varredura completa

O `find` percorre a árvore a partir de um ou mais pontos de partida, avalia expressões sobre cada arquivo encontrado e executa ações sobre os que passam. A estrutura da linha de comando é rígida e vale internalizar. Figura 17.1 mostra.

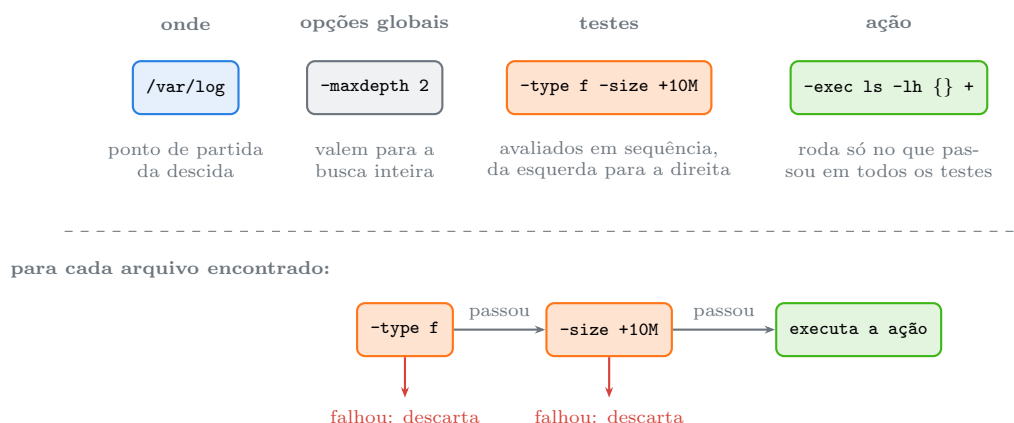


Figura 17.1.: A linha do `find` tem quatro partes em ordem fixa, e os testes funcionam como um E lógico com curto-circuito: o primeiro que falha descarta o arquivo.

Esse curto-circuito não é detalhe acadêmico — é o que faz a diferença entre uma busca de segundos e uma de minutos. Colocar o teste mais barato e mais restritivo primeiro (`-type f` antes de `-size`, e `-name` antes de qualquer coisa que leia o conteúdo) economiza chamadas ao sistema.

## 17. Localizando arquivos: *find*, *locate*, *which* e *type*

### 17.1.1. Testes que se usa de verdade

```
find . -name "*.conf"          # nome, sensível a maiúsculas, glob
find . -iname "*.CONF"         # ignora maiúsculas
find . -path "*/etc/*.conf"    # casa contra o CAMINHO inteiro
find . -regex '.*cap0[12][0-9].*' # expressão regular sobre o caminho

find . -type f                 # arquivo comum
find . -type d                 # diretório
find . -type l                 # link simbólico
find . -xtype l                # link QUEBRADO

find . -size +100M             # maior que 100 MB
find . -size -1k               # menor que 1 KB
find . -size 0                 # exatamente zero blocos
find . -empty                  # arquivo vazio OU diretório vazio

find . -mtime -7               # conteúdo alterado nos últimos 7 dias
find . -mtime +30              # alterado há mais de 30 dias
find . -mmin -15               # alterado nos últimos 15 minutos
find . -newer referencia.txt    # mais recente que outro arquivo
find . -cmin -60               # INODE alterado na última hora

find . -user kali -group sudo
find . -perm 644                # exatamente 644
find . -perm -u+w               # pelo menos escrita para o dono
find . -perm /o+w               # QUALQUER bit de escrita para outros

find . -links +1                # arquivos com hard link
find . -inum 1443183            # busca pelo número de inode
```

Três sutilezas escondidas aí:

**O sinal muda tudo.** Sem sinal, o valor é exato; + é “mais que”; - é “menos que”. Isso vale para `-size`, `-mtime`, `-links` e `-perm` — e `-mtime 7` (exatamente sete dias atrás, arredondado para baixo) quase nunca é o que se quer.

**-name** casa só o nome-base, **-path** casa o caminho inteiro. Procurar `find . -name "*/logs/*"` nunca acha nada, porque nome-base não contém barra.

**As unidades de -size são armadilha.** O padrão é blocos de 512 bytes. Use sempre o sufixo: c para bytes, k, M, G.

### 17.1.2. Operadores lógicos

Por padrão, testes justapostos são um E. Para OU e negação:

```
find . -name "*.log" -o -name "*.txt"          # OU
find . ! -name "*.log"                         # NAO
find . -type f \( -name "*.sh" -o -name "*.py" \) -size +1k
```

Os parênteses precisam ser escapados ou entre aspas — o shell os interpretaria antes. E cuidado com a precedência: E liga mais forte que OU. Sem parênteses, `find . -name a -o -name b -delete` apaga apenas os b.

 `-delete` e `-exec rm` merecem um ensaio antes

Toda busca destrutiva deve ser rodada **primeiro** com `-print` no lugar da ação:

```
find /var/log -name "*.gz" -mtime +90 -print      # confira a lista
find /var/log -name "*.gz" -mtime +90 -delete     # so depois
```

O `-delete` implica `-depth` e altera a ordem de avaliação, o que já produziu remoções inesperadas quando combinado com `-prune`. Se a expressão for complexa, prefira `-print0` | `xargs -0 rm --` depois de conferir a lista.

### 17.1.3. Ações

```
find . -name "*.log" -print                # padrao
find . -name "*.log" -print0              # separado por NUL: seguro para nome
↪ com espaco
find . -name "*.log" -ls                  # formato de "ls -l"
find . -name "*.log" -printf '%s %p\n'    # formato personalizado: tamanho e
↪ caminho
find . -name "*.tmp" -delete

find . -name "*.sh" -exec chmod +x {} \;   # UMA execucao por arquivo
find . -name "*.sh" -exec chmod +x {} +    # AGRUPA: uma execucao para muitos
find . -name "*.conf" -execdir grep -l senha {} + # roda DENTRO do diretorio
↪ de cada achado
find . -name "alvo" -quit                  # para na primeira ocorrencia
```

A diferença entre `\;` e `+` é de desempenho e é grande. Com `\;`, mil arquivos significam mil processos. Com `+`, o `find` agrupa quantos couberem na linha de comando — geralmente uma ou duas execuções no total.

O `-execdir` existe por segurança: ele executa com o diretório de trabalho posicionado no diretório do arquivo encontrado e passa `./nome`, o que elimina a classe de ataques baseada em nomes de diretório manipulados.

### 17.1.4. Controlar a descida

```
find . -maxdepth 1 -type f                # so o nivel atual
find . -mindepth 2                        # ignora o nivel de partida
find / -xdev -name "*.core"               # NAO atravessa para outro sistema de
↪ arquivos
find . -name node_modules -prune -o -type f -print # pula a arvore inteira
```

## 17. Localizando arquivos: *find*, *locate*, *which* e *type*

O **-prune** é o mais confuso dos quatro. Ele significa “não desça neste diretório”, e sozinho não filtra a saída — por isso o idioma vem sempre com **-o ... -print**. Leia como: “se casar com o padrão, pode; senão, se for arquivo comum, imprima”.

O **-xdev** merece destaque: sem ele, um **find /** entra em **/proc**, **/sys**, **/run** e em qualquer pen drive montado, o que gera ruído e demora. Em varredura do sistema inteiro, **-xdev** deveria ser reflexo.

### 17.1.5. *find* com *xargs*

Quando a ação é complexa, **xargs** é mais flexível que **-exec**:

```
find . -name "*.log" -print0 | xargs -0 grep -l "ERRO"
find . -name "*.txt" -print0 | xargs -0 -P 4 -n 20 gzip      # 4 processos em
↳ paralelo
find . -type f -print0 | xargs -0 -I{} cp {} /destino/        # -I define o
↳ marcador
```

A dupla **-print0 / -0** é obrigatória, não opcional. Sem ela, um arquivo chamado **relatório final.txt** vira dois argumentos e o comando falha — ou pior, acerta o alvo errado. Nome de arquivo no Linux pode conter espaço, quebra de linha e quase qualquer byte; o único separador seguro é o NUL.

## 17.2. *locate*: a consulta ao índice

O **find** percorre o disco a cada execução. O **locate** consulta um banco de dados construído antes, e por isso responde instantaneamente:

```
sudo apt install plocate      # no Kali/Debian atuais, plocate substituiu o
↳ mlocate
locate rockyou
locate -i ROCKYOU             # ignora maiusculas
locate -b '\rockyou.txt'      # so o nome-base, casamento exato
locate -c wordlist             # so a contagem
locate -e arquivo             # so o que ainda existe no disco
locate -r '\.conf$'           # expressao regular
```

As duas limitações que importam:

**O índice envelhece.** Ele é atualizado por um *timer* do **systemd**, normalmente uma vez por dia. Um arquivo criado há dez minutos não está lá. Para forçar:

```
sudo updatedb
```

**O índice respeita permissões só parcialmente.** O **plocate** filtra os resultados conforme o que o usuário pode ver, mas o banco em si (**/var/lib/plocate/plocate.db**) contém a árvore inteira. Configurações de exclusão ficam em **/etc/updatedb.conf** — vale olhar **PRUNEPATHS** e **PRUNEFS** lá, especialmente se houver montagens de rede lentas.

A regra de escolha é simples: **locate** para “onde fica um arquivo com esse nome?”, **find** para qualquer pergunta que envolva tamanho, data, permissão, dono ou conteúdo.

## 17.3. Onde está um comando: `type`, `command -v`, `which`, `whereis`

Este é o grupo mais mal compreendido, porque as ferramentas respondem perguntas diferentes. Figura 17.2 mostra o caminho que o shell percorre ao resolver um nome — e o que cada ferramenta enxerga desse caminho.

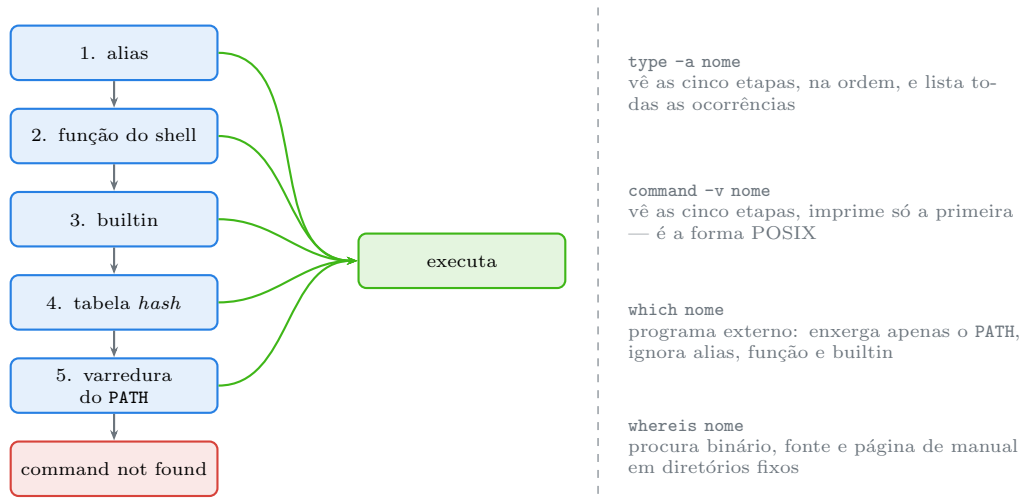


Figura 17.2.: A ordem de resolução do shell e o que cada ferramenta de consulta consegue observar dela.

```

type ls           # "ls is aliased to 'ls --color=auto'"
type -a python3  # TODAS as ocorrências, em ordem
type -t grep     # so a categoria: alias, function, builtin, file,
↳ keyword
command -v nmap  # caminho, ou nada com status diferente de zero
which nmap       # programa externo, so olha o PATH
whereis nmap     # binario, fonte e pagina de manual
  
```

A recomendação prática é direta: **`type` no terminal**, **`command -v` em script**. O `which` é um programa externo, tem comportamento inconsistente entre distribuições e não sabe da existência de aliases, funções e builtins. Um script que faz `if which foo` pode responder errado justamente nos casos que importam.

```

if command -v nmap > /dev/null 2>&1; then
    echo "nmap disponivel"
fi
  
```

### 17.3.1. A tabela hash e o comando que “sumiu”

O Bash memoriza onde encontrou cada comando, para não varrer o PATH toda vez. Isso gera um comportamento desconcertante: você move ou reinstala um binário e o shell continua tentando o caminho antigo.

## 17. Localizando arquivos: *find*, *locate*, *which* e *type*

```
hash          # mostra a tabela
hash -r       # limpa a tabela inteira
hash -d nmap  # esquece só um comando
```

O sintoma é `bash: /usr/bin/ferramenta: No such file or directory` para um comando que você acabou de ver com `ls`. A cura é `hash -r`.

### 17.4. No Kali

- **locate não vem instalado por padrão em várias imagens do Kali.** `sudo apt install plocate && sudo updatedb` resolve. O `plocate` substituiu o `mlocate` no Debian e é ordens de grandeza mais rápido; o banco fica em `/var/lib/plocate/`.
- **Achar a wordlist certa é caso de locate, não de find.** `locate -b '\rockyou.txt'` responde na hora; `find / -name rockyou.txt` percorre o disco inteiro, e ainda entra em `/proc`. Se o `locate` não estiver disponível, ao menos limite: `find /usr/share -name 'rockyou*'`.
- **Varredura de permissão frouxa é um find de uma linha.** `find / -xdev -perm /o+w -type f 2>/dev/null` lista arquivos graváveis por qualquer usuário; `find / -xdev -perm -4000 -type f 2>/dev/null` lista binários SUID. Os dois são inspeção legítima do próprio sistema e material do Volume 14 — em máquina de terceiros, só com autorização escrita.
- **-xdev é obrigatório em varredura a partir de /.** Sem ele, o `find` entra em `/proc` e `/sys`, que são sistemas de arquivos virtuais com milhares de entradas sintéticas, e em qualquer imagem forense que você tenha montado — misturando os dados da análise com os do seu sistema.
- **Arquivos de saída de ferramenta acumulam rápido.** `find ~/engajamentos -type f -size +50M -mtime +30 -printf '%s\t%p\n' | sort -rn | head` mostra o que está ocupando espaço antes de o disco encher. Sempre com `-printf` e revisão antes de qualquer remoção.
- **type -a denuncia atalho perigoso.** Num sistema comprometido, um alias ou uma função de shell com nome de comando legítimo em `~/.bashrc` intercepta chamadas sem alterar binário nenhum. `type -a ls`, `type -a sudo` e `type -a ssh` mostram a cadeia inteira — o `which` não mostraria.
- **hash -r depois de apt install.** No Kali rolling, atualizações movem binários entre `/usr/bin` e `/usr/sbin` com frequência. Se um comando recém-instalado “não existe”, `hash -r` costuma ser a resposta antes de qualquer investigação mais profunda.
- **find com -print0 sempre, nas saídas de ferramenta.** Nomes gerados por scanner frequentemente contêm dois-pontos, colchetes e espaços vindos de URLs e cabeçalhos HTTP. Sem `-print0/-0`, o pipeline se desmonta.

**Nota sobre a família RHEL.** O `find` é idêntico (GNU `findutils` nas duas famílias). A diferença é o `locate`: RHEL e Fedora ainda distribuem `mlocate` (pacote `mlocate`, banco em `/var/lib/mlocate/`), com opções compatíveis mas desempenho inferior ao `plocate`. Em ambientes com SELinux, vale acrescentar `-context` às buscas: `find /var/www -context '*httpd_sys_content_t*'`.



## 17.5. Laboratório

Roteiro em /tmp.

### 1. Monte um terreno de teste.

```
mkdir -p /tmp/lab017/{docs,logs,scripts,node_modules} && cd /tmp/lab017
echo texto > docs/relatorio.txt
echo "nome com espaco" > "docs/relatorio final.txt"
printf '%s\n' "#!/bin/bash" "echo ola" > scripts/exemplo.sh
printf '%s\n' "#!/usr/bin/env python3" "print('ola')" > scripts/exemplo.py
head -c 12M /dev/zero > logs/grande.log
echo pequeno > logs/pequeno.log
touch -d "60 days ago" logs/antigo.log
touch node_modules/lixo.js
ln -s /nao/existe docs/quebrado
find . -ls | head -20
```

### 2. Filtre por tipo e por nome.

```
find . -type f
find . -type d
find . -name "*.log"
find . -iname "*.LOG"
find . -path "*/logs/*"
find . -name "*/logs/*"          # nao acha nada: -name so ve o nome-base
```

Esperado: o penúltimo funciona, o último devolve vazio. É a diferença entre `-path` e `-name`.

### 3. Filtre por tamanho, com e sem sufixo.

```
find . -type f -size +10M
find . -type f -size +10          # SEM sufixo: blocos de 512 bytes
find . -type f -size -1k
find . -empty
```

Esperado: o segundo comando devolve muito mais coisa que o primeiro, porque 10 blocos são 5 KB. Sempre use sufixo.

### 4. Filtre por tempo.

```
find . -type f -mmin -5
find . -type f -mtime +30
touch referencia
find . -type f -newer referencia
```

Esperado: quase tudo no primeiro (você acabou de criar), só o `antigo.log` no segundo, e nada no terceiro além do que for tocado depois.

### 5. Combine com operadores.

```
find . -type f \( -name "*.sh" -o -name "*.py" \)
find . -type f ! -name "*.log"
find . -name "*.log" -o -name "*.sh" -ls          # SEM parenteses: veja o
↵ efeito
```

## 17. Localizando arquivos: *find*, *locate*, *which* e *type*

Esperado: no último, o `-ls` só se aplica ao segundo termo, por causa da precedência. É a armadilha que transforma `-delete` num acidente.

### 6. Pule uma subárvore com `-prune`.

```
find . -type f | wc -l
find . -name node_modules -prune -o -type f -print | wc -l
find . -name node_modules -prune -o -type f -print
```

Esperado: a segunda contagem é menor. O `-prune` impede a descida, e o `-o ... -print` é o que faz a saída sair certa.

### 7. Compare `\;` com `+`.

```
time find . -type f -exec echo {} \; > /dev/null
time find . -type f -exec echo {} + > /dev/null
```

Esperado: o segundo é visivelmente mais rápido. Um processo por arquivo contra um processo para todos.

### 8. Veja por que `-print0` importa.

```
find docs -name "*.txt" | xargs ls -l          # quebra no nome com espaco
find docs -name "*.txt" -print0 | xargs -0 ls -l
```

Esperado: o primeiro reclama de `docs/relatorio` e `final.txt` como se fossem dois arquivos; o segundo acerta. Esse par é obrigatório, não opcional.

### 9. Formate a saída com `-printf`.

```
find . -type f -printf '%s\t%p\n' | sort -rn | head -5
find . -type f -printf '%TY-%Tm-%Td %p\n' | sort | head
find . -xtype l -printf '%p -> %l\n'
```

Esperado: ranking por tamanho, listagem por data e o link quebrado com o alvo inexistente ao lado.

### 10. Ensaie uma remoção antes de executá-la.

```
find . -name "*.log" -mtime +30 -print
find . -name "*.log" -mtime +30 -delete
find . -name "*.log"
```

Esperado: só `antigo.log` na primeira lista, e ele desaparece depois. **Sempre** rode a versão com `-print` antes.

### 11. Limite a profundidade e o sistema de arquivos.

```
find . -maxdepth 1
find . -mindepth 2 -type f
time find / -xdev -name "hostname" 2>/dev/null | head
```

Esperado: o `-xdev` evita a descida em `/proc` e `/sys` e reduz muito o tempo.

### 12. Use o *locate* e sinte a diferença.

```
command -v locate || sudo apt install -y plocate
sudo updatedb
time locate -b '\hostname'
time find / -xdev -name hostname 2>/dev/null | head
```

Esperado: milissegundos contra segundos. Índice contra varredura.

### 13. Descubra o limite do índice.

```
touch /tmp/lab017/arquivo-novissimo.txt
locate arquivo-novissimo
sudo updatedb
locate arquivo-novissimo
```

Esperado: nada na primeira consulta, resultado depois do `updatedb`. O índice não sabe do que acabou de nascer.

### 14. Compare as ferramentas de comando.

```
type ls
type -a ls
type -t ls
command -v ls
which ls
whereis ls
```

Esperado: o `type` revela o alias `ls --color=auto` que o Kali configura; o `which` mostra só `/usr/bin/ls`, ignorando o alias que de fato será executado.

### 15. Crie uma função com nome de comando e veja quem a detecta.

```
grep() { echo "esta funcao interceptou o grep"; }
type -a grep
command -v grep
which grep
grep teste
unset -f grep
type grep
```

Esperado: `type -a` e `command -v` denunciam a função; o `which` não faz ideia. É exatamente o cenário de um `.bashrc` adulterado.

### 16. Provoque e resolva o problema da tabela hash.

```
hash
hash -r
hash
```

Esperado: a tabela lista os comandos já resolvidos nesta sessão e fica vazia depois do `-r`. Guarde `hash -r` para quando um binário recém-instalado “não existir”.

### 17. Limpe.

```
cd /tmp
rm -rf /tmp/lab017
```

## 17.6. Resumo

- **Duas perguntas, duas famílias:** *find* varre a árvore e responde sobre tamanho, data, dono e permissão; *locate* consulta um índice e responde sobre nome, instantaneamente.
- **A linha do *find* tem quatro partes em ordem fixa** — caminho, opções globais, testes, ação — e os testes funcionam como um E com curto-circuito. Teste barato primeiro é otimização real.
- **O sinal muda o significado:** `-size +100M` é maior, `-size -100M` é menor, `-size 100M` é exato. Sem sufixo, a unidade é bloco de 512 bytes.
- **-name casa o nome-base, -path casa o caminho inteiro**, e `-prune` só funciona no idioma `-prune -o ... -print`.
- **-exec ... + agrupa e -exec ... \;** não. Para mil arquivos, isso é a diferença entre um processo e mil. `-execdir` acrescenta segurança.
- **-print0 com xargs -0 é obrigatório**, porque nome de arquivo pode conter espaço e quebra de linha; NUL é o único separador seguro.
- **Toda busca destrutiva roda antes com -print.** `-delete` altera a ordem de avaliação e já causou remoção inesperada em expressão com `-prune`.
- **type no terminal e command -v em script; which só enxerga o PATH** e por isso ignora alias, função e builtin — os três lugares onde um comando pode ter sido sequestrado.
- **hash -r** resolve o comando que “sumiu” depois de uma reinstalação: o shell estava lembrando o caminho antigo.

## 18. Compactação e arquivamento: tar, gzip, xz e zip

Você baixa um arquivo chamado `ferramenta.tar.gz`, roda o comando de sempre no diretório em que estava, e o terminal despeja três mil linhas. Quando para, o seu diretório pessoal tem duzentos arquivos novos misturados com os seus, sem nenhuma pasta que os separe. Alguns sobrescreveram coisas que você tinha.

É o *tarbomb*: um arquivo que, em vez de conter um diretório raiz, contém os arquivos soltos. O prejuízo é chato e a prevenção custa dois segundos — listar o conteúdo antes de extrair.

Esse é o tipo de detalhe que separa quem usa arquivos compactados de quem apanha deles. E a origem de boa parte da confusão é conceitual: no mundo Unix, **juntar arquivos** e **comprimir dados** são duas operações separadas, feitas por programas diferentes. No mundo Windows, o ZIP faz as duas de uma vez. Entender essa divisão explica quase tudo o que vem a seguir.

### 18.1. Arquivar não é comprimir

- **Arquivar** é pegar muitos arquivos, com seus nomes, permissões, donos e datas, e produzir **um único fluxo de bytes**. Quem faz isso é o **tar** (de *tape archive* — a origem é fita magnética, e ela explica várias esquisitices do formato).
- **Comprimir** é reduzir o tamanho de um fluxo de bytes. Quem faz isso é o **gzip**, o **bzip2**, o **xz**, o **zstd**.

Um arquivo `.tar.gz` é o resultado da composição: o **tar** produziu o fluxo, o **gzip** comprimiu o fluxo inteiro. O ZIP faz diferente: comprime **cada arquivo separadamente** e depois junta os pedaços comprimidos num contêiner com índice. Figura 18.1 mostra a diferença e as consequências.

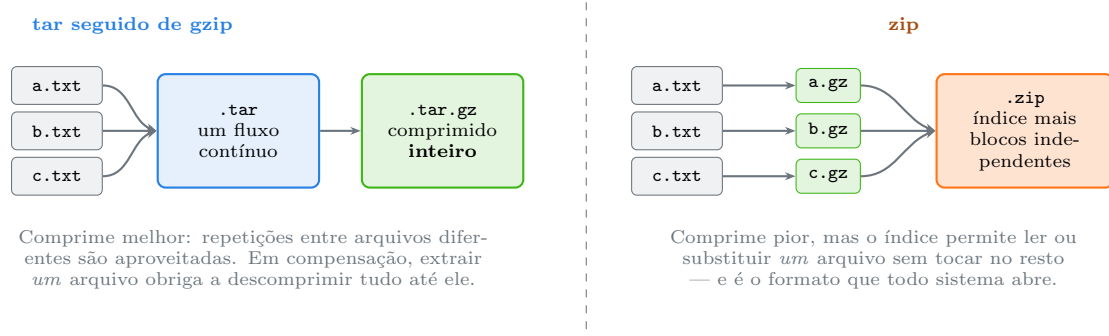


Figura 18.1.: A composição do Unix contra o contêiner do ZIP: melhor taxa de compressão de um lado, acesso aleatório e portabilidade do outro.

Isso responde a três perguntas frequentes de uma vez:

## 18. Compactação e arquivamento: tar, gzip, xz e zip

- **Por que .tar.gz costuma ser menor que .zip?** Porque o compressor vê o conjunto todo e aproveita repetições entre arquivos. Num diretório com cem arquivos de configuração parecidos, a diferença é enorme.
- **Por que não dá para extrair um único arquivo de um .tar.gz rapidamente?** Porque é preciso descomprimir o fluxo desde o começo até chegar nele.
- **Por que .zip é o padrão de intercâmbio?** Porque qualquer sistema operacional abre, e porque o índice permite abrir sem descomprimir tudo.

### 18.2. tar: o comando e suas letras

A sintaxe do tar é herdada dos anos 1970 e aceita opções sem hífen, o que confunde quem vem de outros comandos. As três operações principais:

```
tar -cf pacote.tar diretorio/      # (c)reate: criar
tar -tf pacote.tar                 # (t)able: listar
tar -xf pacote.tar                 # e(x)tract: extrair
```

O **f** indica que o próximo argumento é o **nome do arquivo** — e ele precisa vir por último no agrupamento, senão o tar pega a opção errada como nome. Essa é a origem do erro **tar: Cowardly refusing to create an empty archive.**

A compressão entra com mais uma letra:

```
tar -czf pacote.tar.gz dir/       # z: gzip
tar -cjf pacote.tar.bz2 dir/      # j: bzip2
tar -cJf pacote.tar.xz dir/       # J maiusculo: xz
tar --zstd -cf pacote.tar.zst dir/ # zstd
tar -caf pacote.tar.xz dir/       # a: escolhe pelo sufixo do nome
```

Na extração, o tar moderno **detecta a compressão sozinho**. Não é preciso lembrar a letra:

```
tar -xf qualquer-coisa.tar.gz
tar -xf qualquer-coisa.tar.xz
```

Opções que resolvem problemas reais:

```
tar -xf pacote.tar.gz -C /destino/ # extrai EM outro diretorio
tar -xvf pacote.tar.gz             # v: mostra os nomes
tar -tvf pacote.tar.gz | head      # LISTE antes de extrair: defesa
↪ contra tarbomb
tar -czf b.tar.gz --exclude='*.log' --exclude='.git' dir/
tar -xf p.tar.gz --strip-components=1 # descarta o primeiro nivel de
↪ diretorio
tar -czpf b.tar.gz dir/            # p: preserva permissoes
tar -xf p.tar.gz caminho/especifico.txt # extrai so um membro
tar --diff -f p.tar.gz             # compara o arquivo com o disco
```

O **--strip-components=1** é o companheiro diário de quem compila a partir do fonte: quase todo tarball do mundo livre embrulha tudo em **programa-1.2.3/**, e às vezes você quer o conteúdo direto.

⚠ Sempre liste antes de extrair

```
tar -tvf desconhecido.tar.gz | head -20
```

Três coisas para conferir: (1) existe um diretório raiz único, ou os arquivos estão soltos — tarbomb; (2) há caminhos começando com / ou contendo ../, que tentam escapar do destino; (3) o tamanho declarado faz sentido.

O GNU tar remove a barra inicial e recusa ../ por padrão, mas nem toda implementação faz isso, e o comportamento muda com -P. Na dúvida, extraia num diretório vazio:

```
mkdir -p /tmp/inspecao && tar -xf desconhecido.tar.gz -C /tmp/inspecao
```

## 18.3. Os compressores, e quando usar cada um

Os quatro que aparecem no dia a dia, com trocas bem diferentes:

Tabela 18.1.: Os quatro compressores usuais e suas trocas

| Compressor | Sufixo | Compressão      | Velocidade   | Uso típico                                   |
|------------|--------|-----------------|--------------|----------------------------------------------|
| gzip       | .gz    | modesta         | rápida       | padrão universal, logs, transferência        |
| bzip2      | .bz2   | boa             | lenta        | legado; hoje raramente a melhor escolha      |
| xz         | .xz    | excelente       | muito lenta  | distribuição de software, arquivamento longo |
| zstd       | .zst   | boa a excelente | muito rápida | escolha moderna: quase tudo que o gzip fazia |

A regra prática de 2020 em diante: **zstd quando você controla os dois lados, gzip quando precisa de compatibilidade universal, xz quando o arquivo será baixado muitas vezes e comprimido uma só**. O bzip2 sobrevive por compatibilidade com arquivos antigos.

Os três primeiros compartilham a mesma interface básica:

```
gzip arquivo.txt           # cria arquivo.txt.gz e APAGA o original
gzip -k arquivo.txt        # -k: mantem o original
gzip -9 arquivo.txt        # nivel maximo (1 = rapido, 9 = pequeno)
gunzip arquivo.txt.gz      # o mesmo que gzip -d
zcat arquivo.txt.gz        # le sem descomprimir para disco
zless arquivo.txt.gz       # paginacao
zgrep padrao arquivo.gz    # busca direto no comprimido
```

## 18. Compactação e arquivamento: tar, gzip, xz e zip

Que o **gzip apague o original** por padrão surpreende quem vem do ZIP. O **-k** corrige, e vale como hábito.

A família tem equivalentes para cada compressor: **bzcat/bzgrep**, **xzcat/xzgrep**, **zstdcat/zstdgrep**.

Um aviso sobre o **xz**: níveis altos consomem muita memória na **descompressão** também. Um arquivo criado com **xz -9** pode exigir centenas de megabytes para abrir, o que importa em máquina pequena ou em contêiner com limite.

O **zstd** merece uma nota extra pelo alcance dos seus níveis:

```
zstd -1 arquivo          # rapidissimo
zstd -19 arquivo         # comprime como xz, ainda mais rapido que ele
zstd --long=27 -19 arq   # janela grande: excelente para arquivos muito
↪ repetitivos
zstd -T0 arquivo         # usa todos os nucleos
```

### 18.4. zip e unzip

```
zip -r pacote.zip diretorio/    # -r e OBRIGATORIO para diretorios
zip -9 -r pacote.zip dir/       # nivel maximo
zip -e pacote.zip arquivos     # com senha (ver aviso abaixo)
unzip pacote.zip
unzip -l pacote.zip             # LISTAR antes
unzip -d /destino pacote.zip   # extrair em outro lugar
unzip -o pacote.zip            # sobrescreve sem perguntar
unzip -j pacote.zip            # descarta a estrutura de diretorios
```

O **-r** esquecido é o erro número um: sem ele, **zip pacote.zip diretorio/** cria um ZIP contendo apenas a entrada do diretório, vazia.

#### ⚠ A criptografia do ZIP tradicional é quebrada

O **zip -e** usa, por padrão, o algoritmo *ZipCrypto* dos anos 1990. Ele é vulnerável a ataque de texto claro conhecido: quem tiver **um** dos arquivos do pacote em versão não comprimida recupera a chave em minutos, sem precisar adivinhar a senha. Ferramentas públicas fazem isso automaticamente.

Para proteger conteúdo de verdade, use **7z** com AES-256 ou, melhor ainda, **gpg** sobre o arquivo já compactado:

```
tar -czf dados.tar.gz dir/ && gpg -c dados.tar.gz && rm dados.tar.gz
```

Chaves, GPG e gestão de segredos são tema do Volume 14.

O **7z** cobre os casos que o **zip** não cobre bem:



```

sudo apt install p7zip-full
7z a -t7z -m0=lzma2 -mx=9 pacote.7z dir/
7z a -p -mhe=on pacote.7z dir/      # AES-256 e nomes de arquivo também cifrados
7z l pacote.7z
7z x pacote.7z -o/destino

```

O `-mhe=on` cifra também a lista de nomes — sem ele, mesmo com senha, qualquer um vê o que há dentro.

## 18.5. Compactação em fluxo: sem arquivo temporário

Como tudo isso opera sobre fluxos, o `tar` combina naturalmente com os pipes de Capítulo 12. É o que permite copiar uma árvore inteira por SSH sem tocar no disco intermediário. Figura 18.2 mostra o caminho dos dados.

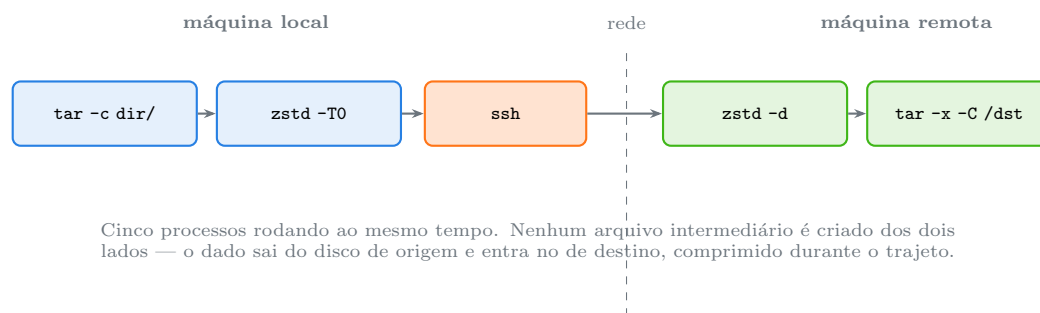


Figura 18.2.: Arquivar, comprimir, transportar e extrair como um pipeline único. É a mesma ideia de Capítulo 12, aplicada a árvores de diretório.

```

# copiar uma arvore pela rede, comprimindo no caminho
tar -cf - dir/ | zstd -T0 | ssh usuario@host 'zstd -d | tar -xf - -C /destino'

# copiar entre dois diretorios locais preservando tudo
tar -cf - -C origem . | tar -xpf - -C destino

# criar um arquivo lendo a lista de outro comando
find . -name "*.conf" -print0 | tar --null -czf confs.tar.gz -T -

```

O `-` no lugar do nome do arquivo significa “entrada ou saída padrão”, e é o que torna o `tar` composável. O `-T -` lê a lista de arquivos do `stdin`, e `--null` faz ele esperar o separador NUL do `-print0` — a mesma preocupação com nomes contendo espaço vista em Capítulo 17.

Para transferências repetidas, no entanto, o `rsync` é melhor: ele copia só as diferenças. Volume 13 trata dele.

## 18.6. Verificar antes de confiar

Compactado ou não, arquivo baixado se verifica:

## 18. Compactação e arquivamento: tar, gzip, xz e zip

```
sha256sum pacote.tar.gz
sha256sum -c CHECKSUMS.txt          # confere contra a lista publicada
gzip -t arquivo.gz                  # testa integridade sem extrair
xz -t arquivo.xz
zstd -t arquivo.zst
unzip -t pacote.zip
tar -tf pacote.tar.gz > /dev/null && echo "estrutura ok"
```

O `-t` de “test” existe em todos os compressores e detecta corrupção sem gastar espaço em disco. Depois de uma transferência longa, é mais rápido que extrair para descobrir que quebrou.

### 18.7. No Kali

- **rockyou.txt vem comprimido.** A imagem do Kali distribui `/usr/share/wordlists/rockyou.txt.gz`. Descomprimir com `sudo gunzip` funciona, mas ocupa 130 MB permanentes e modifica `/usr/share`, que pertence ao pacote. A alternativa limpa é usar em fluxo: `zcat /usr/share/wordlists/rockyou.txt.gz | head -100000 > /tmp/curta.txt`, ou `gunzip -k` para uma cópia em `~`.
- **zgrep e zcat poupam disco em análise de log.** Logs rotacionados chegam como `.gz`, e `zgrep "Failed password" /var/log/auth.log.*.gz` busca em todos sem descomprimir nenhum. Vale para `.xz` e `.zst` com os equivalentes.
- **Empacote a saída de engajamento com tar -czpf.** O `p` preserva permissões, o que importa quando a coleta inclui arquivos com modos específicos que serão parte da evidência. E registre o hash junto: `sha256sum coleta.tar.gz > coleta.tar.gz.sha256`.
- **ZIP com senha não é proteção de dados.** Como dito acima, o ZipCrypto é quebrável com um único arquivo conhecido dentro do pacote — e ferramentas para isso estão nos repositórios do Kali justamente porque o formato é indefensável. Para relatório com dado sensível, `7z` com AES e `-mhe=on`, ou GPG.
- **tar em fluxo é a forma correta de mover coleta grande.** `tar -cf - coleta/ | zstd -T0 | ssh analista@host 'zstd -d | tar -xf - -C /casos/'` não deixa arquivo temporário na máquina de origem — o que é bom para espaço e melhor ainda para higiene em ambiente que não é seu.
- **Cuidado com tarball de origem duvidosa.** Uma amostra baixada para análise se lista antes (`tar -tvf`), se extrai num diretório descartável, e nunca em `~` ou `/tmp` compartilhado. Um `.tar` pode conter caminhos absolutos e links simbólicos apontando para fora do destino — a combinação que já produziu escrita arbitrária de arquivo em vários extratores.
- **--exclude mantém o pacote enxuto.** Ao arquivar um diretório de trabalho, `--exclude='.git' --exclude='*.pcap' --exclude='venv'` costuma cortar mais da metade do tamanho, e evita levar captura de tráfego bruta para onde ela não deveria ir.

**Nota sobre a família RHEL.** As ferramentas são as mesmas (GNU tar, gzip, xz). A diferença prática está nos pacotes: `zip/unzip` e `p7zip` não vêm instalados por padrão em instalação mínima do RHEL, e o `p7zip` esteve por anos apenas no EPEL. O `zstd` é padrão no Fedora e no RHEL 9. Ao restaurar arquivos com `tar -xp` num sistema com SELinux, o contexto de segurança **não** vem no tarball comum — é preciso `--selinux` na criação e na extração, ou um `restorecon -R` depois.

## 18.8. Laboratório

Roteiro em /tmp.

### 1. Monte o material de teste.

```
mkdir -p /tmp/lab018/projeto/{src,docs} && cd /tmp/lab018
for i in $(seq 1 30); do
  cp /etc/services "projeto/src/copia$i.conf"
done
echo "documento" > projeto/docs/leiam.txt
du -sh projeto
```

Esperado: alguns megabytes, com trinta arquivos praticamente idênticos — o cenário perfeito para ver a diferença entre os formatos.

### 2. Separe arquivar de comprimir.

```
tar -cf projeto.tar projeto/
ls -lh projeto.tar
gzip -k projeto.tar
ls -lh projeto.tar projeto.tar.gz
```

Esperado: o .tar tem praticamente o tamanho do diretório; o .gz é uma fração dele. Duas operações, dois programas.

### 3. Compare os quatro compressores.

```
for c in "gzip -9" "bzip2 -9" "xz -9" "zstd -19"; do
  nome=$(echo $c | cut -d' ' -f1)
  /usr/bin/time -f "$nome: %e s" $c -k -c projeto.tar > "teste.$nome" 2>&1
  ↪ || true
done
ls -lhS projeto.tar teste.*
```

Esperado: xz e zstd -19 produzem os menores; o gzip é o mais rápido dos clássicos; o zstd chega perto do xz em muito menos tempo. Se o zstd não estiver instalado, sudo apt install zstd.

### 4. Veja o tar.gz ganhar do zip.

```
command -v zip || sudo apt install -y zip unzip
zip -q -9 -r projeto.zip projeto/
ls -lh projeto.tar.gz projeto.zip
```

Esperado: o .tar.gz é bem menor. Ele comprimiu os trinta arquivos parecidos como um bloco só; o ZIP comprimiu cada um isoladamente.

### 5. Liste antes de extrair.

```
tar -tvf projeto.tar.gz | head
unzip -l projeto.zip | head
```

Esperado: os dois mostram um diretório raiz projeto/. É isso que se procura — sem raiz única, é tarbomb.

## 18. Compactação e arquivamento: tar, gzip, xz e zip

### 6. Fabrique e desarme um tarbomb.

```
cd projeto && tar -czf ../bomba.tar.gz . && cd ..  
tar -tf bomba.tar.gz | head  
mkdir -p seguro && tar -xf bomba.tar.gz -C seguro  
ls seguro | head
```

Esperado: a listagem mostra arquivos soltos, sem raiz. Extrair com -C num diretório vazio neutraliza o problema, sempre.

### 7. Extraia só um membro.

```
mkdir -p parcial  
tar -xf projeto.tar.gz -C parcial projeto/docs/leiam.txt  
find parcial
```

Esperado: só o arquivo pedido, com a estrutura de diretórios recriada.

### 8. Descarte o nível de topo.

```
mkdir -p direto  
tar -xf projeto.tar.gz -C direto --strip-components=1  
ls direto
```

Esperado: src e docs diretamente em direto/, sem a pasta projeto/ no meio.

### 9. Exclua o que não interessa.

```
tar -czf enxuto.tar.gz --exclude='*.conf' projeto/  
ls -lh projeto.tar.gz enxuto.tar.gz  
tar -tf enxuto.tar.gz
```

Esperado: pacote muito menor, só com o leiam.txt e a estrutura.

### 10. Trabalhe direto no comprimido.

```
gzip -k projeto/docs/leiam.txt  
zcat projeto/docs/leiam.txt.gz  
zgrep -c . projeto/docs/leiam.txt.gz  
zless projeto/docs/leiam.txt.gz # saia com q
```

Esperado: leitura e busca sem gerar arquivo descomprimido em disco.

### 11. Teste a integridade e simule corrupção.

```
gzip -t projeto.tar.gz && echo "integro"  
cp projeto.tar.gz corrompido.tar.gz  
printf 'XXXX' | dd of=corrompido.tar.gz bs=1 seek=5000 conv=notrunc  
↵ 2>/dev/null  
gzip -t corrompido.tar.gz; echo "status: $?"
```

Esperado: o segundo teste falha com invalid compressed data. O dd aqui escreve 4 bytes num arquivo de teste em /tmp — nunca aponte um dd para dispositivo sem ter certeza absoluta do destino.

### 12. Copie uma árvore em fluxo, sem intermediário.

```
mkdir -p destino
tar -cf - -C projeto . | tar -xpf - -C destino
diff -r projeto destino && echo "identicos"
```

Esperado: cópia idêntica, com permissões preservadas, sem nenhum arquivo temporário. É o pipeline de Figura 18.2 sem a parte da rede.

#### 13. Alimente o tar com a saída do find.

```
find projeto -name "*.txt" -print0 | tar --null -czf textos.tar.gz -T -
tar -tf textos.tar.gz
```

Esperado: só os .txt no pacote. O --null casa com o -print0 e sobrevive a nomes com espaço.

#### 14. Registre o hash junto do pacote.

```
sha256sum projeto.tar.gz | tee projeto.tar.gz.sha256
sha256sum -c projeto.tar.gz.sha256
```

Esperado: projeto.tar.gz: OK. É o procedimento mínimo para qualquer arquivo que vai ser transportado ou entregue.

#### 15. Limpe.

```
cd /tmp
du -sh /tmp/lab018
rm -rf /tmp/lab018
```

## 18.9. Resumo

- **Arquivar e comprimir são operações distintas.** O tar junta arquivos num fluxo preservando nomes, permissões e datas; o compressor reduz o fluxo. .tar.gz é a composição das duas.
- **O ZIP comprime arquivo por arquivo e junta com um índice,** o que dá acesso aleatório e portabilidade universal, ao custo de taxa de compressão pior em conjuntos parecidos.
- **-c cria, -t lista, -x extrai, e -f precede o nome do arquivo.** Na extração, o tar detecta a compressão sozinho — não é preciso lembrar a letra.
- **Liste sempre antes de extrair** (tar -tvf, unzip -l) e, na dúvida, extraia com -C num diretório vazio. É a defesa contra tarbomb e contra caminhos que tentam escapar do destino.
- **gzip para compatibilidade, zstd para o dia a dia, xz para arquivar por muito tempo.** O gzip apaga o original por padrão; use -k.
- **zcat, zgrep e zless operam direto no comprimido,** e existem equivalentes para bz2, xz e zst.
- **A criptografia do zip -e é quebrada.** Use 7z com AES-256 e -mhe=on, ou GPG sobre o pacote já pronto.
- **O - como nome de arquivo torna o tar composável,** e é o que permite copiar árvores pela rede comprimindo no caminho, sem nenhum arquivo temporário.
- **-t testa integridade em todos os compressores,** e sha256sum acompanha todo pacote que vai ser transportado.



## 19. Montagem de dispositivos e o comando mount

Você copia dois gigabytes para um pen drive. A barra de progresso termina em três segundos — suspeitamente rápido —, você puxa o dispositivo e vai embora. Na outra máquina, o pen drive está vazio, ou tem arquivos truncados pela metade.

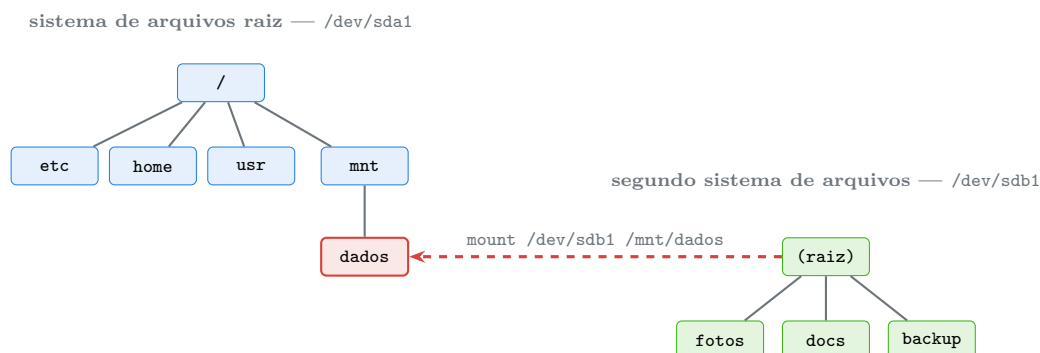
Não foi defeito do hardware. O Linux confirmou a cópia assim que os dados entraram no **cache de escrita** da memória, e a gravação real aconteceria alguns segundos depois. Puxar o dispositivo antes de `umount` interrompeu exatamente isso.

Esse comportamento é uma consequência direta de como o Linux trata dispositivos de armazenamento: eles não são “unidades” com letra, são sistemas de arquivos **enxertados** em algum ponto da árvore única que começa em `/`. Entender o enxerto — quando ele acontece, quem o desfaz, e o que fica pendente enquanto ele existe — é o assunto deste capítulo.

### 19.1. Uma árvore só, sem letras de unidade

No Windows, cada dispositivo ganha uma letra e vira a raiz de uma árvore própria. No Linux existe **uma** árvore, começando em `/` (Capítulo 5), e cada sistema de arquivos adicional é ligado a um diretório existente dela. Esse diretório é o **ponto de montagem**.

Figura 19.1 mostra o enxerto.



Depois do enxerto, `/mnt/dados/fotos` é um caminho válido. O conteúdo que `/mnt/dados` porventura tivesse antes fica **escondido** — não é apagado, apenas inacessível até o `umount`.

Figura 19.1.: Montar é enxertar a raiz de um sistema de arquivos num diretório da árvore existente. Não há letras de unidade porque não há árvores paralelas.

Duas consequências práticas desse desenho:

**O conteúdo anterior do ponto de montagem some de vista.** Se você tinha arquivos em `/mnt/dados` e montou algo por cima, eles continuam no disco — invisíveis — e reaparecem no `umount`. Isso já assustou muita gente que achou ter perdido dados.

Um caminho pode atravessar vários sistemas de arquivos sem que você perceba. `/home/kali/dados/foto.jpg` pode ter `/`, `/home` e `/home/kali/dados` em três dispositivos diferentes. É por isso que o `-xdev` do `find` (Capítulo 17) e o `-x` do `du` existem.

## 19.2. Ver o que está montado

Quatro ferramentas, com propósitos distintos:

```
lsblk          # árvore de DISPOSITIVOS de bloco: discos,
↳ particoes, montagens
lsblk -f        # acrescenta sistema de arquivos, rotulo e UUID
findmnt        # árvore de MONTAGENS, hierarquica e legivel
findmnt /home   # quem esta montado ali
findmnt -t ext4,vfat # so os tipos indicados
df -hT         # espaco livre com o tipo do sistema de arquivos
mount | column -t # lista bruta, herdeira do /etc/mtab
cat /proc/mounts # a fonte da verdade, direto do kernel
blkid          # UUID, rotulo e tipo de cada dispositivo
```

O `lsblk` e o `findmnt` respondem perguntas diferentes e complementares: o primeiro parte do hardware, o segundo parte da árvore. Quando um pen drive é conectado e “não aparece”, o `lsblk` mostra se o kernel ao menos o enxergou.

O `mount` sem argumentos ainda funciona, mas hoje é a opção pior — a saída é longa e cheia de sistemas de arquivos virtuais. `findmnt` é a substituição moderna.

## 19.3. Montar e desmontar

```
sudo mount /dev/sdb1 /mnt/dados
sudo mount -t vfat /dev/sdb1 /mnt/pendrive
sudo mount -o ro /dev/sdb1 /mnt/leitura
sudo mount UUID=1a2b-3c4d /mnt/dados
sudo mount -o remount,rw /          # remonta sem desmontar
sudo umount /mnt/dados
sudo umount /dev/sdb1              # pelo dispositivo tambem funciona
```

O tipo (`-t`) quase nunca é necessário: o `mount` detecta pelo conteúdo. Ele importa quando o mesmo dispositivo pode ser lido de mais de uma forma, ou quando você quer forçar uma implementação específica.

### 19.3.1. As opções que importam



```
-o ro          # somente leitura
-o rw          # leitura e escrita (padrao)
-o noexec      # proibe executar binarios daqui
-o nosuid      # ignora os bits SUID/SGID (@sec-cap020 e Volume 4)
-o nodev       # ignora arquivos de dispositivo
-o noatime     # nao atualiza atime: desempenho
-o sync        # escreve na hora, sem cache - lento, mas resiste a remocao
↳ abrupta
-o uid=1000,gid=1000,umask=022  # dono e permissoes em vfat/ntfs, que nao
↳ guardam isso
-o defaults    # rw,suid,dev,exec,auto,nouser,async
```

A trinca `noexec,nosuid,nodev` é a configuração padrão de higiene para qualquer mídia que não é sua. Sistemas de arquivos como FAT e NTFS não guardam permissões Unix, então tudo neles aparece com o dono e o modo definidos na montagem — daí o `uid=`, `gid=` e `umask=`.

### 19.3.2. umount e o “target is busy”

O erro mais comum ao desmontar:

```
umount: /mnt/dados: target is busy.
```

Significa que algum processo tem um arquivo aberto ali, ou tem o diretório como diretório de trabalho. Para descobrir quem:

```
lsuf +f -- /mnt/dados      # lista processos usando o ponto de montagem
fuser -vm /mnt/dados       # o mesmo, com formato compacto
fuser -km /mnt/dados       # MATA os processos: use com consciencia
```

A causa número um é banal: **o seu próprio shell está com `cd` dentro do diretório**. Um `cd /` resolve.

Existe a saída de emergência:

```
sudo umount -l /mnt/dados  # lazy: desliga da arvore agora, libera quando os
↳ processos soltarem
```

Ela remove o ponto da árvore imediatamente, mas o sistema de arquivos só é realmente liberado quando o último processo fechar. Não é substituto de `umount` normal, e não garante que os dados foram gravados.

#### Remover mídia sem desmontar perde dados

O Linux confirma a escrita quando os dados chegam ao cache, não ao disco. Antes de puxar qualquer mídia removível:

```
sync
sudo umount /mnt/pendrive  # o umount ja inclui o sync do sistema montado
```

Se `umount` falhar por “busy”, **não** puxe o dispositivo — descubra o processo com `fuser -vm` e resolva. O `sync` sozinho reduz o risco, mas não substitui o `umount`, porque metadados do sistema de arquivos podem ficar em estado inconsistente.

## 19.4. /etc/fstab: montagens permanentes

O `/etc/fstab` descreve o que deve ser montado no boot. Cada linha tem seis campos, e Figura 19.2 os detalha.

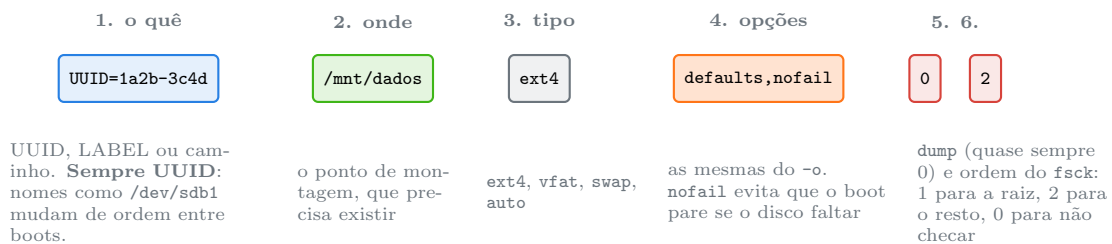


Figura 19.2.: A anatomia de uma linha do `/etc/fstab`. O primeiro e o quarto campos são os que causam problema de boot quando mal preenchidos.

```
cat /etc/fstab
blkid                # pegue o UUID daqui
sudo findmnt --verify # CONFERE a sintaxe do fstab: rode sempre apos
↪ editar
sudo mount -a         # monta tudo que esta no fstab e ainda nao esta
↪ montado
```

### ⚠ Erro no fstab impede o sistema de iniciar

Uma linha inválida — UUID errado, ponto de montagem inexistente, tipo incorreto — pode deixar o boot travado numa **emergency shell**. Depois de **qualquer** edição:

1. `sudo findmnt --verify` para checar a sintaxe;
2. `sudo mount -a` para testar sem reiniciar;
3. acrescente `nofail` em toda montagem de mídia removível ou de rede, para que a ausência do dispositivo não interrompa o boot.

Faça uma cópia antes: `sudo cp /etc/fstab /etc/fstab.bak`.

O uso de **UUID em vez de `/dev/sdX`** não é preciosismo. A ordem de enumeração dos discos depende do tempo de resposta do hardware, e um disco que era `/dev/sdb` pode virar `/dev/sda` na próxima inicialização. Com UUID, o alvo é sempre o mesmo sistema de arquivos.

Em sistemas com `systemd`, o `fstab` é lido por um gerador que produz *units* de montagem. Isso significa que opções como `x-systemd.automount` (montar sob demanda, no primeiro acesso) e `x-systemd.device-timeout=5s` estão disponíveis. O assunto pertence ao Volume 7.

## 19.5. Montagem automática no ambiente gráfico

No Xfce do Kali, conectar um pen drive dispara o `udisks2`, que monta em `/media/<usuário>/<rótulo>` sem pedir senha. É conveniente e tem duas implicações:

- O ponto de montagem depende do **rótulo** do dispositivo, que pode mudar;
- As opções aplicadas são as do `udisks2`, não as suas.

Para controle real — especialmente com mídia que você não confia — desmonte e remonte à mão:

```
udisksctl unmount -b /dev/sdb1
sudo mkdir -p /mnt/inspecao
sudo mount -o ro,noexec,nosuid,nodev /dev/sdb1 /mnt/inspecao
```

## 19.6. Arquivos-imagem: montar sem hardware

Um arquivo pode conter um sistema de arquivos inteiro. Montá-lo usa um **dispositivo de loop**, que faz o kernel tratar o arquivo como se fosse um disco.

```
sudo mount -o loop imagem.iso /mnt/iso
sudo mount -o ro,loop imagem.img /mnt/analise
```

Para imagens com tabela de partição (uma imagem de disco inteiro, não de partição), é preciso mapear as partições:

```
sudo losetup -Pf --show disco.img      # -P le a tabela de particoes; imprime
↪ /dev/loopN
sudo mount -o ro /dev/loop0p1 /mnt/analise
sudo umount /mnt/analise
sudo losetup -d /dev/loop0             # desfaz
losetup -a                             # lista os loops ativos
```

Esse é o mecanismo que permite todo o laboratório deste capítulo acontecer dentro de um arquivo em `/tmp`, sem tocar em nenhum disco real.

## 19.7. Bind mount e tmpfs

Duas montagens que não envolvem dispositivo nenhum.

**Bind mount** faz um diretório aparecer em dois lugares da árvore:

```
sudo mount --bind /origem /destino
sudo mount --rbind /origem /destino    # inclui as montagens que estao dentro
sudo mount -o remount,ro,bind /destino # a copia fica somente-leitura
```

Diferente de um link simbólico, o bind mount é resolvido pelo kernel na camada de montagem: funciona dentro de um **chroot**, funciona quando o programa recusa seguir links, e permite montar o mesmo conteúdo com opções diferentes em cada lugar. É a base de boa parte do que os contêineres fazem (Volume 16).

**tmpfs** é um sistema de arquivos que mora na memória:

```
findmnt -t tmpfs
sudo mount -t tmpfs -o size=256M tmpfs /mnt/rapido
```

Rápido e volátil: o conteúdo desaparece no desligamento. `/run`, `/dev/shm` e frequentemente `/tmp` são tmpfs. Para dado sensível que não deve encostar no disco, é a escolha óbvia — desde que a máquina não use *swap*, que reintroduziria o problema.

### 19.8. No Kali

- **Monte mídia de terceiros somente-leitura, sempre.** `sudo mount -o ro,noexec,nosuid,nodev /dev/sdb1 /mnt/inspecao` é o comando padrão para qualquer pen drive achado, recebido ou apreendido. O `ro` protege a evidência de alteração acidental; o `noexec,nosuid` protege você.
- **Em análise forense, ro no mount não basta.** O ideal é um bloqueador de escrita de hardware ou, na falta dele, trabalhar sobre uma **cópia** da imagem e nunca sobre a mídia original. `losetup -r` cria o dispositivo de loop já somente-leitura, o que fecha mais uma porta.
- **Persistência do Kali live é uma montagem sobreposta.** A partição rotulada `persistence`, com o arquivo `persistence.conf`, é combinada com o sistema somente-leitura da imagem via `overlayfs`. `findmnt -t overlay` mostra o arranjo, e `lsblk -f` mostra os rótulos que o mecanismo procura.
- **NTFS funciona, com o pacote certo.** `sudo apt install ntfs-3g` habilita leitura e escrita em partições do Windows. Para uma imagem de disco de Windows em análise, monte com `-o ro,show_sys_files,streams_interface=windows` para enxergar fluxos alternativos de dados — um lugar clássico de esconder conteúdo.
- **/dev/shm e /tmp são tmpfs e ficam em memória.** Deixar coleta ou credencial ali evita rastro no disco, mas some no desligamento e aparece no *swap* se a memória apertar. Para trabalho sensível, tmpfs mais `swapon` (ou uma máquina sem swap) é o par correto.
- **lsblk primeiro, sempre.** Antes de qualquer mount, `dd` ou `mkfs`, `lsblk -f` mostra tamanho, rótulo e sistema de arquivos de cada dispositivo. Confundir `/dev/sda` com `/dev/sdb` é o erro que destrói o disco do próprio analista — e ele acontece justamente quando se pula esse passo.
- **Arquivo-imagem com losetup é o laboratório mais seguro que existe.** Para praticar particionamento, formatação, LUKS ou LVM (Volume 8), um arquivo de 200 MB em `/tmp` se comporta como um disco de verdade e não tem como destruir nada importante.

**Nota sobre a família RHEL.** `mount`, `fstab`, `lsblk` e `findmnt` são idênticos (utilizam as duas famílias). As diferenças: RHEL usa XFS como padrão, que aceita `-o remount` para crescer mas nunca para encolher; o SELinux exige contexto correto no ponto de montagem, e uma montagem sem `-o context=...` pode resultar em acesso negado mesmo com permissões corretas — `ls -lZ` e `restorecon` são as ferramentas de diagnóstico. Em mídia removível, o Fedora monta em `/run/media/<usuário>/`, não em `/media/<usuário>/`.

## 19.9. Laboratório

Roteiro inteiro dentro de um arquivo-imagem em /tmp. **Nenhum comando abaixo toca em disco real** — mas leia cada linha antes de executar, especialmente as que envolvem `mkfs` e `losetup`.

### 1. Veja o estado atual do sistema.

```
lsblk -f
findmnt
findmnt -t ext4,vfat,tmpfs
df -hT
cat /etc/fstab
```

Esperado: `lsblk` mostra os dispositivos; `findmnt` mostra a árvore de montagens, com /proc, /sys e /run entre elas.

### 2. Crie um “disco” de mentira.

```
mkdir -p /tmp/lab019 && cd /tmp/lab019
truncate -s 200M disco.img
ls -lh disco.img
file disco.img
```

Esperado: 200 MB declarados, quase nada ocupado (arquivo esparsa, como em Capítulo 15), e o `file` dizendo apenas `data` — ainda não há sistema de arquivos.

### 3. Formate a imagem.

O `mkfs` é destrutivo por natureza. Aqui o alvo é o arquivo `/tmp/lab019/disco.img` e nada mais — confira o caminho antes de dar Enter.

```
mkfs.ext4 -q -L LABTESTE /tmp/lab019/disco.img
file disco.img
blkid disco.img
```

Esperado: agora o `file` identifica `ext4 filesystem` e o `blkid` mostra o rótulo `LABTESTE` e um UUID.

### 4. Monte e use.

```
mkdir -p ponto
sudo mount -o loop disco.img ponto
findmnt ponto
df -hT ponto
sudo chown "$USER" ponto
echo "arquivo dentro da imagem" > ponto/teste.txt
ls -l ponto
```

Esperado: `findmnt` mostra a montagem com origem `/dev/loopN`, e o arquivo criado vive dentro da imagem.

### 5. Comprove que o ponto de montagem esconde o conteúdo anterior.

## 19. Montagem de dispositivos e o comando mount

```
sudo umount ponto
echo "eu estava aqui antes" > ponto/escondido.txt
ls ponto
sudo mount -o loop disco.img ponto
ls ponto
sudo umount ponto
ls ponto
```

Esperado: `escondido.txt` desaparece durante a montagem e reaparece depois. Nada foi apagado.

### 6. Monte somente-leitura e tente escrever.

```
sudo mount -o ro,loop disco.img ponto
findmnt -o TARGET,SOURCE,FSTYPE,OPTIONS ponto
touch ponto/tentativa.txt
sudo umount ponto
```

Esperado: Read-only file system. É o modo em que qualquer mídia de terceiro deve ser aberta.

### 7. Veja o `noexec` em ação.

```
sudo mount -o loop disco.img ponto
sudo cp /bin/echo ponto/
ponto/echo "executou"
sudo umount ponto
sudo mount -o loop,noexec disco.img ponto
ponto/echo "executou"
sudo umount ponto
```

Esperado: funciona na primeira montagem e devolve `Permission denied` na segunda. `noexec` é uma barreira real, não decorativa.

### 8. Provoque o “target is busy”.

```
sudo mount -o loop disco.img ponto
cd ponto
sudo umount /tmp/lab019/ponto
cd ..
sudo umount ponto
```

Esperado: a primeira tentativa falha porque o seu shell estava dentro; depois do `cd ..` funciona. É a causa número um do erro.

### 9. Descubra quem está segurando a montagem.

```
sudo mount -o loop disco.img ponto
sleep 300 > ponto/ocupado.log &
sudo umount ponto
fuser -vm ponto
sudo lsof +f -- ponto 2>/dev/null | head
kill %1
sudo umount ponto
```

Esperado: o `fuser -vm` mostra o PID do `sleep` segurando o ponto. Depois do `kill`, o `umount` funciona.

#### 10. Trabalhe com uma imagem particionada.

```
truncate -s 300M completo.img
printf 'label: dos\n,\n' | sfdisk completo.img
sudo losetup -Pf --show completo.img
lsblk /dev/loop*
```

Esperado: o `losetup -P` cria `/dev/loopN` e `/dev/loopNp1` para a partição. Anote o número que apareceu; ele é usado a seguir como `loopN`.

#### 11. Formate, monte e desfaça o loop.

```
LOOP=$(losetup -j completo.img | cut -d: -f1)
echo "usando $LOOP"
sudo mkfs.ext4 -q "${LOOP}p1"
mkdir -p ponto2
sudo mount "${LOOP}p1" ponto2
df -hT ponto2
sudo umount ponto2
sudo losetup -d "$LOOP"
losetup -a
```

Esperado: montagem normal, e a lista de loops vazia ao final. Sempre desfaça o `losetup`: loops esquecidos seguram o arquivo e confundem a próxima tentativa.

#### 12. Experimente um bind mount.

```
mkdir -p origem destino
echo "conteudo original" > origem/arq.txt
sudo mount --bind origem destino
ls destino
echo "escrito pelo destino" >> destino/arq.txt
cat origem/arq.txt
findmnt destino
sudo mount -o remount,ro,bind destino
touch destino/novo.txt
sudo umount destino
```

Esperado: o mesmo diretório visível nos dois caminhos, e a segunda montagem somente-leitura enquanto a origem continua gravável. Um link simbólico não faria isso.

#### 13. Monte um tmpfs.

```
mkdir -p memoria
sudo mount -t tmpfs -o size=64M,mode=700 tmpfs memoria
sudo chown "$USER" memoria
df -hT memoria
head -c 10M /dev/urandom > memoria/dados.bin
df -hT memoria
free -h | head -2
sudo umount memoria
ls memoria
```

## 19. Montagem de dispositivos e o comando mount

Esperado: o espaço sai da memória, não do disco, e o conteúdo some no `umount`. Repare no limite de 64 MB imposto pelo `size=`.

### 14. Confira o `fstab` sem editá-lo.

```
sudo findmnt --verify  
blkid | head
```

Esperado: `Success, no errors or warnings detected`. Guarde esse comando: ele é o que evita um boot travado depois de uma edição do `fstab`.

### 15. Limpe.

```
cd /tmp  
findmnt | grep lab019 || echo "nada montado"  
losetup -a | grep lab019 || echo "nenhum loop ativo"  
rm -rf /tmp/lab019
```

Esperado: nenhuma montagem e nenhum loop pendente antes de remover. Se algo aparecer, desfaça antes.

## 19.10. Resumo

- **O Linux tem uma árvore só.** Montar é enxertar a raiz de outro sistema de arquivos num diretório existente; não há letras de unidade porque não há árvores paralelas.
- **O ponto de montagem esconde o que havia nele**, sem apagar. O conteúdo anterior reaparece no `umount`.
- **`lsblk` parte do hardware, `findmnt` parte da árvore**, e `/proc/mounts` é a fonte da verdade do kernel. `blkid` fornece os UUIDs.
- **`ro,noexec,nosuid,nodev` é a montagem padrão para mídia que não é sua**, e `uid=/gid=/umask=` são necessários em FAT e NTFS, que não guardam permissões Unix.
- **Escrita é confirmada no cache, não no disco.** `sync` e `umount` antes de remover qualquer mídia; `umount -l` é paliativo, não solução.
- **“Target is busy” quase sempre é o seu próprio `cd`.** `fuser -vm` e `lsof +f --` identificam o culpado quando não é.
- **No `fstab`, use `UUID` e `nofail`**, e valide com `findmnt --verify` e `mount -a` antes de reiniciar — uma linha errada trava o boot numa shell de emergência.
- **`-o loop` e `losetup -Pf` transformam um arquivo em disco**, o que dá um laboratório completo e inofensivo para praticar particionamento, formatação e montagem.
- **Bind mount duplica um diretório na árvore** com opções próprias, funciona onde link simbólico não funciona, e é a base do isolamento de contêineres.



## 20. Permissões de arquivo: leitura, escrita e execução

Um colega não consegue ler um arquivo do seu diretório. Você confere e o arquivo está `-rw-rw-r--`: leitura para todo mundo. Mesmo assim, ele recebe:

```
cat: /home/kali/relatorio.txt: Permission denied
```

Você tenta a solução que todo tutorial ruim sugere — `chmod 777` no arquivo — e continua sem funcionar. Porque o problema nunca esteve no arquivo. Está no **diretório**: sem permissão de execução em `/home/kali`, ninguém atravessa aquele caminho, por mais aberto que o arquivo esteja.

Permissões no Linux são simples de descrever e cheias de detalhes que só aparecem no uso. Este capítulo cobre o modelo básico — as três classes, os três bits, o `chmod`, o `chown` e o `umask` — com atenção especial ao que significa cada bit **num diretório**, que é onde mora a maior parte da confusão. Os bits especiais (SUID, SGID, sticky) e as ACLs são do Volume 4.

### 20.1. Ler a saída do `ls -l`

```
ls -l relatorio.txt
```

```
-rw-r--r-- 1 kali kali 2913 ago  1 09:14 relatorio.txt
```

A primeira coluna carrega dez caracteres, e Figura 20.1 decompõe cada um.

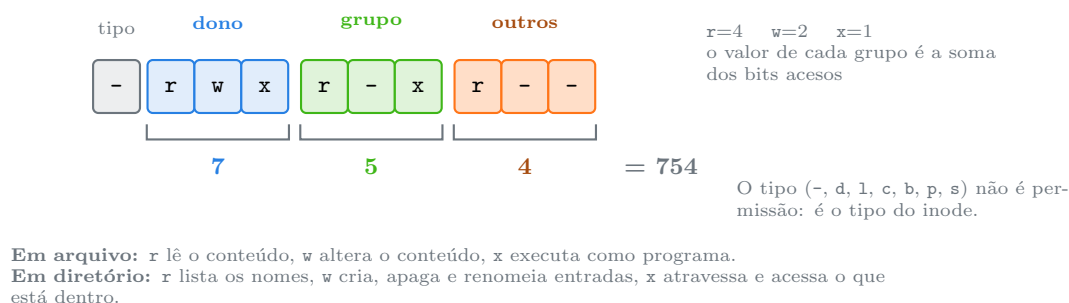


Figura 20.1.: Os dez caracteres do modo: um de tipo e três grupos de três bits, cada grupo somando para um dígito octal.

Os números vêm de potências de dois: `r` vale 4, `w` vale 2, `x` vale 1. Um grupo com `rwX` soma 7; com `r-X`, 5; com `r--`, 4. Daí a notação 754, 644, 755 que aparece em toda documentação.

Para ver o modo já traduzido:

```
stat -c '%a %A %U %G %n' relatorio.txt
# 644 -rw-r--r-- kali kali relatorio.txt
```

## 20.2. A regra que quase ninguém aprende: só uma classe se aplica

Este é o ponto que produz os erros mais desconcertantes. O kernel **não** combina permissões das três classes. Ele escolhe **uma** e para. Figura 20.2 mostra a decisão.

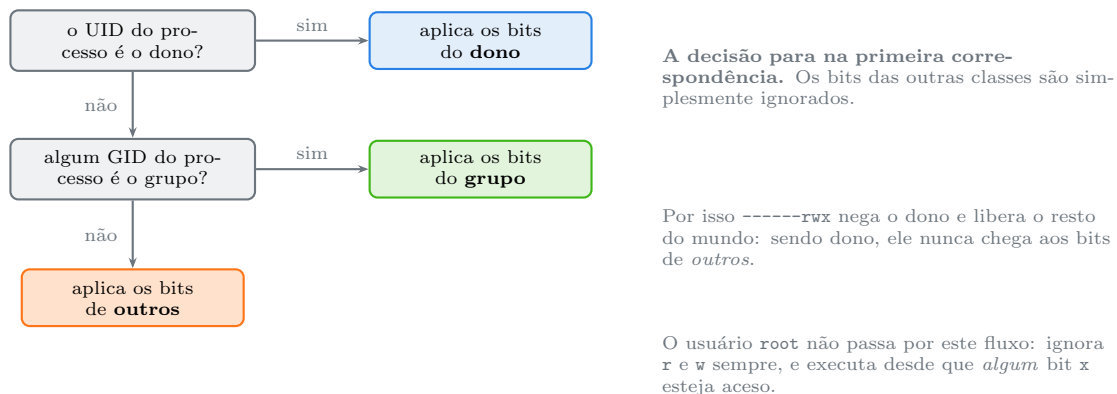


Figura 20.2.: O kernel escolhe uma única classe — a primeira que casa — e aplica só os bits dela.

A consequência mais contraintuitiva:

```
echo teste > curioso.txt
chmod 007 curioso.txt      # -----rwx: nada para o dono, tudo para outros
ls -l curioso.txt
cat curioso.txt            # Permission denied - para VOCE, que e o dono
```

Você é o dono, os bits do dono são todos zero, e a decisão termina ali. Qualquer outro usuário da máquina lê, escreve e executa; você, não. (O dono continua podendo rodar `chmod` de volta, porque **mudar permissão** é prerrogativa do dono, não uma permissão do arquivo.)

## 20.3. chmod: as duas notações

### 20.3.1. Simbólica

```
chmod u+x script.sh      # acrescenta execucao para o dono
chmod g-w arquivo        # remove escrita do grupo
chmod o= arquivo          # zera TODOS os bits de outros
chmod a+r arquivo        # a = all: dono, grupo e outros
chmod u=rw,go=r arquivo  # define exatamente, varias clausulas
chmod +x script.sh       # sem classe: respeita o umask
chmod -R u+rwX,go-w pasta/ # X maiusculo: so em diretorio ou no que ja e
↪ executavel
```

O **X** maiúsculo merece destaque. Num `chmod -R +x`, todos os arquivos de dados viram executáveis, o que é errado e perigoso. Com **X**, o bit só é aplicado a diretórios (onde **x** é necessário para atravessar) e a arquivos que já tinham algum **x**. É o jeito correto de consertar permissões de uma árvore inteira.

### 20.3.2. Octal

```
chmod 644 arquivo           # rw-r--r--
chmod 600 ~/.ssh/id_ed25519 # rw----- : obrigatorio para chave privada
chmod 755 script.sh         # rwxr-xr-x
chmod 700 ~/privado         # rwx-----
chmod 640 config.conf       # rw-r----- : dono escreve, grupo le
```

Tabela 20.1.: Modos octais mais frequentes e o que cada um significa

| Octal | Símbolo   | Uso típico                                                     |
|-------|-----------|----------------------------------------------------------------|
| 644   | rw-r--r-- | arquivo comum: documento, configuração pública                 |
| 600   | rw-----   | segredo: chave privada, <code>.netrc</code> , token            |
| 755   | rwxr-xr-x | script, binário, diretório de uso geral                        |
| 700   | rwx-----  | diretório privado, <code>~/.ssh</code> , <code>~/.gnupg</code> |
| 640   | rw-r----- | configuração legível só pelo grupo do serviço                  |
| 666   | rw-rw-rw- | <b>evitar</b> : qualquer usuário altera                        |
| 777   | rwxrwxrwx | <b>nunca</b> : qualquer usuário altera e executa               |

A diferença entre as notações é de intenção: a simbólica **ajusta** o que existe; a octal **define** tudo de uma vez. Num script que precisa garantir o estado final, a octal é mais segura; num ajuste pontual, a simbólica evita quebrar o que já estava certo.

 `chmod 777` não é solução de nada

Toda vez que 777 “resolveu” um problema, o problema real era outro — dono errado, grupo errado, ou falta de **x** num diretório do caminho. E o custo é alto: qualquer usuário ou processo da máquina passa a poder **alterar** e **executar** aquele arquivo.

Em `/var/www` isso significa que uma falha em qualquer aplicação web permite escrever código executável no diretório servido. É o passo que transforma uma vulnerabilidade pequena em comprometimento completo.

Diagnóstico correto: `ls -ld` em **cada** diretório do caminho, ou `namei -l` no caminho inteiro (Capítulo 16), e `id` para conferir grupos.

## 20.4. Permissões em diretório: o que realmente significam

Aqui está a maior fonte de confusão, e ela é resolvida lembrando que um diretório é uma tabela de nomes (Capítulo 15):

Tabela 20.2.: O mesmo bit significa coisas diferentes em arquivo e em diretório

| Bit      | Em arquivo             | Em diretório                                            |
|----------|------------------------|---------------------------------------------------------|
| <b>r</b> | ler o conteúdo         | <b>listar os nomes</b> das entradas                     |
| <b>w</b> | alterar o conteúdo     | <b>criar, apagar e renomear</b> entradas                |
| <b>x</b> | executar como programa | <b>atravessar</b> : acessar o que está dentro pelo nome |

Quatro consequências que aparecem no dia a dia:

**r sem x é quase inútil.** Você vê os nomes e não consegue abrir nada, nem rodar `ls -l` (que precisa consultar o inode de cada entrada). O `ls` mostra os nomes e “?” em todas as outras colunas.

**x sem r é útil e é usado de propósito.** Você não pode listar o diretório, mas acessa qualquer caminho cujo nome já conheça. É como uma porta sem placa: quem sabe o número, entra. `/home` frequentemente é assim, e servidores web usam esse modo para diretórios que não devem ser navegáveis.

**Apagar um arquivo depende do diretório, não do arquivo.** Apagar é remover uma linha da tabela — operação de escrita **no diretório**. Um arquivo `444` (somente leitura para todos) dentro de um diretório onde você tem `w` pode ser apagado por você sem nenhum aviso. O contrário também vale: você pode ter `rw` num arquivo e ser incapaz de apagá-lo.

**É por isso que existe o sticky bit.** Em `/tmp`, todo mundo precisa de `w` para criar arquivos — o que, pela regra acima, permitiria apagar os arquivos dos outros. O sticky bit (`t` no final: `drwxrwxrwt`) restringe a remoção ao dono do arquivo. Volume 4 detalha ele e os outros bits especiais.

```
ls -ld /tmp
# drwxrwxrwt 20 root root 4096 ago  1 09:14 /tmp
```

## 20.5. `chown` e `chgrp`: mudar dono e grupo

```
sudo chown kali arquivo           # muda o dono
sudo chown kali:sudo arquivo      # dono e grupo
sudo chown :sudo arquivo          # so o grupo
chgrp sudo arquivo                # idem, comando dedicado
sudo chown -R kali:kali pasta/    # recursivo
sudo chown --reference=modelo alvo # copia o dono e o grupo de outro arquivo
```

Duas regras do modelo Unix clássico:

- **Só o root muda o dono de um arquivo.** Se um usuário comum pudesse doar arquivos, ele poderia burlar cotas de disco e criar arquivos SUID em nome de terceiros.

- **Um usuário pode mudar o grupo** de um arquivo que possui, desde que ele mesmo pertença ao grupo de destino.

Sobre a interação com links simbólicos, vale repetir o alerta de Capítulo 16: `chown -R` não segue links por padrão (bom), mas `chown -RL` segue (perigoso), e `chown -Rh` altera o próprio link.

Para descobrir seus grupos:

```
id
id -Gn
groups
```

Uma pegadinha frequente: acabou de ser adicionado a um grupo e o acesso continua negado. Os grupos são carregados no **login**; é preciso encerrar a sessão e entrar de novo (ou usar `newgrp` grupo como paliativo naquele shell).

## 20.6. *umask*: as permissões que os arquivos novos recebem

Um arquivo criado não nasce com o modo que o programa pede — nasce com esse modo **menos** o que o `umask` retira.

```
umask          # 0022 no padrao do Debian/Kali
umask -S       # u=rwx,g=rx,o=rx
umask 077      # so o dono ve o que eu criar daqui em diante
umask 027      # dono tudo, grupo le, outros nada
```

A conta é uma subtração de bits sobre uma base fixa:

- arquivos comuns partem de **666** (nunca **x** — o kernel não dá execução automaticamente);
- diretórios partem de **777**.

Tabela 20.3.: Valores comuns de `umask` e o que resulta deles

| <code>umask</code> | Arquivo novo | Diretório novo | Perfil                         |
|--------------------|--------------|----------------|--------------------------------|
| 022                | 644          | 755            | padrão: todos leem             |
| 027                | 640          | 750            | grupo lê, outros nada          |
| 077                | 600          | 700            | estritamente privado           |
| 002                | 664          | 775            | trabalho colaborativo em grupo |

O `umask` vale para o shell atual e para os processos que ele inicia. Para torná-lo permanente, entra no `~/.bashrc` ou `~/.zshrc` — assunto do Volume 15. Para todo o sistema, em `/etc/login.defs` e nos módulos do PAM, tema do Volume 4.

Um detalhe importante: `umask` **não** altera nada que já existe. Ele só afeta a criação.

## 20.7. Testar permissão sem tentar a operação

Em script, testar antes é melhor que falhar depois:

```
[ -r arquivo ] && echo "posso ler"
[ -w arquivo ] && echo "posso escrever"
[ -x arquivo ] && echo "posso executar"
[ -O arquivo ] && echo "sou o dono"
[ -G arquivo ] && echo "meu grupo bate com o do arquivo"

test -x /usr/bin/nmap; echo $?
```

Os testes usam os **IDs efetivos** do processo, e por isso respondem exatamente o que o kernel responderia. Condicionais e códigos de saída são o tema dos capítulos 055 e 056.

## 20.8. No Kali

- **O SSH recusa chave privada com permissão frouxa.** `~/.ssh` precisa ser 700 e `id_ed25519` precisa ser 600. Com qualquer coisa mais aberta, o cliente aborta com UNPROTECTED PRIVATE KEY FILE e não tenta a conexão. É uma verificação do próprio OpenSSH, não do kernel — e ela existe porque uma chave legível por outros é uma chave comprometida.
- **Script baixado precisa de `chmod +x`, e de leitura antes.** O fluxo correto para qualquer script de terceiros: `file`, depois `less` para ler o que ele faz, depois `chmod +x`, e só então executar. Um `curl ... | bash` pula os três passos de uma vez.
- **Caça a arquivos graváveis por todos é inspeção legítima do próprio sistema.** `find / -xdev -type f -perm /o+w -not -path '/proc/*' 2>/dev/null` lista o que qualquer usuário pode alterar; `find / -xdev -type d -perm -0002 ! -perm -1000 2>/dev/null` lista diretórios graváveis por todos **sem** sticky bit — combinação que permite substituir arquivos alheios. Em máquina de terceiros, isso só com autorização escrita: acesso não autorizado a sistema informático é crime no Brasil (Lei 12.737/2012).
- **Diretório de engajamento pede `umask 077`.** Coleta de teste de intrusão contém credencial, captura de tráfego e dado de cliente. `umask 077` no início da sessão garante que nada nasça legível por outros usuários da máquina; `chmod -R go= ~/engajamentos` conserta o que já existe.
- **`/tmp` tem sticky bit, e você depende dele.** `ls -ld /tmp` mostra `drwxrwxrwt`. Sem o `t`, qualquer usuário poderia apagar os arquivos temporários de qualquer outro — inclusive de serviços privilegiados. Ainda assim, `mktemp` continua sendo a forma correta de criar arquivo temporário em script.
- **`chmod -R 777` num diretório servido pela web é a ponte entre uma falha e o comprometimento.** Se o processo do servidor puder escrever no diretório que ele serve, qualquer upload mal validado vira execução remota. 755 para diretórios e 644 para arquivos, com dono separado do usuário do servidor, é o arranjo mínimo.
- **Use `chmod -R u+rwX,go-rwX`, nunca `chmod -R 777` para “consertar”.** O `X` maiúsculo dá execução só onde faz sentido — diretórios e o que já era executável — e evita transformar cada PDF e cada wordlist num executável.
- **`getcap` complementa o `ls -l`.** Programas modernos do Kali usam *capabilities* em vez de SUID: `dumpcap`, por exemplo, tem `cap_net_raw` sem ser SUID. Um binário aparentemente inofensivo no `ls -l` pode ter privilégio via capability — `getcap -r /usr/bin 2>/dev/null` revela. O assunto é do Volume 14.

**Nota sobre a família RHEL.** Os bits `rx` são POSIX (IEEE and The Open Group, 2018) e idênticos. Duas diferenças de contexto: (1) o RHEL usa por padrão *user private groups* — cada usuário tem um grupo próprio de mesmo nome —, o que torna `umask 002` seguro para trabalho colaborativo, prática comum lá e incomum no Debian; (2) o SELinux acrescenta uma camada **acima** das permissões: mesmo com `777`, um acesso pode ser negado pelo contexto de segurança. Quando `ls -l` diz que deveria funcionar e não funciona, o próximo passo em RHEL é `ls -lZ` e `ausearch -m avc -ts recent`.

## 20.9. Laboratório

Roteiro em `/tmp`. Alguns passos usam um segundo usuário; onde ele não existir, o efeito pode ser observado com `sudo -u nobody`.

### 1. Leia o modo de um arquivo.

```
mkdir -p /tmp/lab020 && cd /tmp/lab020
echo "conteudo" > arquivo.txt
ls -l arquivo.txt
stat -c '%a %A %U %G %n' arquivo.txt
umask
```

Esperado: `644` e `-rw-r--r--`, resultado do `umask 022` sobre a base `666`.

### 2. Compare as duas notações.

```
chmod 754 arquivo.txt; stat -c '%a %A' arquivo.txt
chmod u=rwx,g=rx,o=r arquivo.txt; stat -c '%a %A' arquivo.txt
chmod go-r arquivo.txt; stat -c '%a %A' arquivo.txt
chmod 644 arquivo.txt
```

Esperado: as duas primeiras linhas produzem exatamente o mesmo modo. A octal define, a simbólica ajusta.

### 3. Prove que só uma classe se aplica.

```
echo segredo > curioso.txt
chmod 007 curioso.txt
ls -l curioso.txt
cat curioso.txt
sudo -u nobody cat /tmp/lab020/curioso.txt
chmod 644 curioso.txt
```

Esperado: **você**, o dono, recebe `Permission denied`; o `nobody` lê sem problema. A decisão parou na classe do dono.

### 4. Torne um script executável.

```
printf '%s\n' '#!/bin/bash' 'echo "o script rodou"' > script.sh
ls -l script.sh
./script.sh
chmod +x script.sh
ls -l script.sh
./script.sh
```

```
bash script.sh
```

Esperado: `Permission denied` antes do `chmod`. Repare que `bash script.sh` funciona mesmo sem o bit `x` — porque quem é executado é o `bash`, e ele só precisa de `r` no script.

5. Veja `r` sem `x` num diretório.

```
mkdir -p pasta && echo dado > pasta/dentro.txt
chmod 644 pasta          # r, sem x
ls pasta
ls -l pasta
cat pasta/dentro.txt
cd pasta
chmod 755 pasta
```

Esperado: os **nomes** aparecem, mas `ls -l`, `cat` e `cd` falham. Sem `x` não se atravessa o diretório.

6. Veja `x` sem `r`.

```
chmod 711 pasta          # x, sem r
ls pasta
cat pasta/dentro.txt
chmod 755 pasta
```

Esperado: a listagem é negada, mas o acesso pelo nome exato funciona. É a “porta sem placa” usada de propósito em vários lugares do sistema.

7. Descubra que apagar depende do diretório.

```
echo "protegido" > pasta/somente-leitura.txt
chmod 444 pasta/somente-leitura.txt
ls -l pasta/somente-leitura.txt
rm pasta/somente-leitura.txt
```

Esperado: o `rm` pergunta, e ao confirmar apaga — mesmo o arquivo sendo 444. A permissão que valeu foi o `w` no diretório `pasta`.

8. Confirme o inverso.

```
mkdir -p travado && echo dado > travado/arq.txt
chmod 555 travado        # sem w no diretorio
ls -l travado/arq.txt    # o arquivo continua rw para o dono
echo "mais texto" >> travado/arq.txt
rm travado/arq.txt
chmod 755 travado
```

Esperado: você **escreve** no arquivo e não consegue **apagá-lo**. Conteúdo e nome estão em lugares diferentes, como visto em Capítulo 15.

9. Reproduza o problema da abertura do capítulo.

```
mkdir -p caminho/fundo
echo "todo mundo deveria ler" > caminho/fundo/publico.txt
chmod 644 caminho/fundo/publico.txt
chmod 700 caminho
```



```
sudo -u nobody cat /tmp/lab020/caminho/fundo/publico.txt
namei -l /tmp/lab020/caminho/fundo/publico.txt
chmod 755 caminho
sudo -u nobody cat /tmp/lab020/caminho/fundo/publico.txt
```

Esperado: negado com o arquivo aberto, liberado depois de consertar o **diretório**. O `namei -l` aponta o culpado sem adivinhação.

10. Use **X** maiúsculo em vez de **+x** recursivo.

```
mkdir -p arvore/sub && echo dado > arvore/sub/dados.txt
printf '#!/bin/bash\necho ok\n' > arvore/roda.sh && chmod +x
↪ arvore/roda.sh
chmod -R a+x arvore; find arvore -printf '%m %p\n'
chmod -R go-x arvore; chmod -R u+rwX,go-w arvore
find arvore -printf '%m %p\n'
```

Esperado: com `a+x`, o arquivo de dados virou executável; com `u+rwX`, só os diretórios e o script que já era executável mantêm o bit.

11. Veja o `umask` mudando o que nasce.

```
umask 022; touch a022.txt; mkdir d022
umask 077; touch a077.txt; mkdir d077
umask 002; touch a002.txt; mkdir d002
umask 022
stat -c '%a %n' a022.txt a077.txt a002.txt d022 d077 d002
```

Esperado: 644/755, 600/700, 664/775. E note que nenhum arquivo nasceu com `x`: a base de arquivos é 666.

12. Confirme que o `umask` não muda o passado.

```
umask 077
stat -c '%a %n' a022.txt
umask 022
```

Esperado: `a022.txt` continua 644. O `umask` só age na criação.

13. Troque dono e grupo.

```
id
id -Gn
echo teste > propriedade.txt
sudo chown root propriedade.txt
ls -l propriedade.txt
echo "tentando escrever" >> propriedade.txt
sudo chown "$USER:$USER" propriedade.txt
echo "agora vai" >> propriedade.txt
cat propriedade.txt
```

Esperado: escrita negada enquanto o dono era `root`, liberada depois. Repare que só o `sudo` conseguiu mudar o dono.

14. Copie permissões de um modelo.

## 20. Permissões de arquivo: leitura, escrita e execução

```
chmod 640 modelo.conf 2>/dev/null || { echo x > modelo.conf; chmod 640
↪ modelo.conf; }
touch alvo.conf
stat -c '%a %n' modelo.conf alvo.conf
chmod --reference=modelo.conf alvo.conf
stat -c '%a %n' modelo.conf alvo.conf
```

Esperado: os dois com 640. Útil para replicar o modo de um arquivo de configuração conhecido.

### 15. Observe o sticky bit em /tmp.

```
ls -ld /tmp
stat -c '%a %A %n' /tmp
find /tmp -maxdepth 1 -type d -perm -1000 2>/dev/null | head
```

Esperado: drwxrwxrwt e modo 1777. O 1 na frente é o sticky bit, detalhado no Volume 4.

### 16. Faça uma varredura de permissões frouxas no seu próprio sistema.

```
find /tmp/lab020 -type f -perm /o+w
sudo find /etc -xdev -type f -perm /o+w 2>/dev/null | head
sudo find / -xdev -type d -perm -0002 ! -perm -1000 2>/dev/null | head
```

Esperado: nada ou muito pouco num sistema saudável. Qualquer arquivo de /etc gravável por todos é um achado que merece investigação.

### 17. Teste permissão em script.

```
for f in arquivo.txt script.sh curioso.txt; do
  printf '%s: ' "$f"
  [ -r "$f" ] && printf 'r'
  [ -w "$f" ] && printf 'w'
  [ -x "$f" ] && printf 'x'
  [ -O "$f" ] && printf ' (sou dono)'
  echo
done
```

Esperado: a linha de cada arquivo reflete exatamente o que o `ls -l` mostra para a sua classe.

### 18. Limpe.

```
cd /tmp
chmod -R u+rwX /tmp/lab020
rm -rf /tmp/lab020
```

O `chmod` antes do `rm` evita o erro em diretórios que ficaram sem `w` durante o laboratório.

## 20.10. Resumo

- **Dez caracteres no `ls -l`:** um de tipo e três grupos de `rwX`, para dono, grupo e outros. `r=4`, `w=2`, `x=1`, somados por grupo dão a notação octal.

- **Só uma classe se aplica**, a primeira que casa: dono, senão grupo, senão outros. Por isso `chmod 007` nega o dono e libera o mundo — os bits das outras classes são ignorados, não somados.
- **A notação simbólica ajusta, a octal define.** Em script que precisa garantir o estado final, use octal; em ajuste pontual, simbólica.
- **Em diretório os bits significam outra coisa:** `r` lista nomes, `w` cria e apaga entradas, `x` atravessa. `r` sem `x` é quase inútil; `x` sem `r` é útil e intencional.
- **Apagar um arquivo é permissão sobre o diretório**, não sobre o arquivo — consequência direta de o nome morar na tabela do diretório. O sticky bit existe para corrigir isso em `/tmp`.
- **`chmod 777` nunca é a solução.** O problema real costuma ser dono errado ou falta de `x` num diretório do caminho; `namei -l` e `id` diagnosticam em segundos.
- **`chmod -R u+rwX,go-w` é a forma correta de consertar uma árvore:** o `X` maiúsculo evita tornar arquivos de dados executáveis.
- **Só o root muda o dono;** um usuário muda o grupo apenas para grupos aos quais pertence, e grupos novos só valem depois de novo login.
- **`umask` subtrai da base 666 para arquivos e 777 para diretórios,** afeta apenas o que for criado dali em diante, e 077 é o valor certo para diretório com material sensível.



**Parte IV.**

**Volume 4 — Usuários, Grupos e  
Permissões**



## **21. Usuários, grupos e o arquivo `/etc/passwd`**

*(em elaboracao)*





## **22. O root, sudo e a elevação de privilégios**

*(em elaboracao)*



## **23. Permissões especiais: SUID, SGID e sticky bit**

*(em elaboracao)*



## **24. ACLs e atributos estendidos de arquivo**

*(em elaboracao)*



## **25. Gerenciamento de usuários, senhas e o PAM**

*(em elaboracao)*





## **26. Trabalhando como root no Kali com segurança**

*(em elaboracao)*



**Parte V.**

**Volume 5 — Processos e Controle de  
Tarefas**



## **27. O que é um processo: PID, estados e a árvore de processos**

*(em elaboracao)*



## 28. Monitoramento: ps, top e htop

*(em elaboracao)*





## **29. Sinais e controle: kill, killall, nice e prioridades**

*(em elaboracao)*



## **30. Jobs, primeiro e segundo plano, nohup e disown**

*(em elaboracao)*



## 31. Multiplexação de terminal: tmux e screen

*(em elaboracao)*



## **32. Recursos do sistema: memória, CPU, uptime e limites**

*(em elaboracao)*





**Parte VI.**

**Volume 6 — Gerenciamento de Pacotes**



## **33. APT e o modelo Debian: repositórios e o Kali rolling**

*(em elaboracao)*



## **34. Instalando, removendo e atualizando pacotes com apt**

*(em elaboracao)*



## **35. dpkg, arquivos .deb e resolução de dependências**

*(em elaboracao)*





## **36. Repositórios do Kali, metapacotes e kali-tweaks**

*(em elaboracao)*



## 37. Compilando a partir do código-fonte

*(em elaboracao)*



## **38. Alternativas: pipx, snap, flatpak e Appliance**

*(em elaboracao)*



**Parte VII.**

**Volume 7 — Boot, systemd e Serviços**





## **39. O processo de inicialização: UEFI, GRUB e o kernel**

*(em elaboracao)*



## **40. systemd: units, targets e o modelo de serviços**

*(em elaboracao)*



## 41. Gerenciando serviços com systemctl

*(em elaboracao)*



## 42. Logs com journald e o syslog tradicional

*(em elaboracao)*





## **43. Agendamento: cron, at e timers do systemd**

*(em elaboracao)*



## **44. Targets, modo de recuperação e resolução de problemas de boot**

*(em elaboracao)*



**Parte VIII.**

**Volume 8 — Armazenamento e  
Sistemas de Arquivos**



## **45. Discos, partições e a nomenclatura de dispositivos**

*(em elaboracao)*





## **46. Particionamento com fdisk, parted e gdisk**

*(em elaboracao)*



## **47. Sistemas de arquivos: ext4, Btrfs, XFS e formatação**

*(em elaboracao)*



## **48. LVM: volumes lógicos flexíveis**

*(em elaboracao)*



## **49. RAID por software com mdadm**

*(em elaboracao)*





## **50. Criptografia de disco com LUKS e a persistência no Kali**

*(em elaboracao)*



**Parte IX.**

**Volume 9 — Fundamentos de Shell  
Scripting**



## **51. Do comando ao script: o primeiro script Bash**

*(em elaboracao)*



## **52. Variáveis, o ambiente e o escopo de exportação**

*(em elaboracao)*





## **53. Aspas, expansões e substituição de comandos**

*(em elaboracao)*



## **54. Entrada e saída: read, echo e printf**

*(em elaboracao)*



## **55. Condicionais: test, colchetes duplos e if**

*(em elaboracao)*



## **56. Códigos de saída e o encadeamento de comandos**

*(em elaboracao)*





## **57. Boas práticas: shebang, permissões e portabilidade**

*(em elaboracao)*



**Parte X.**

**Volume 10 — Scripting Intermediário**



## 58. Laços: for, while e until

*(em elaboracao)*



## **59. Funções, retorno e escopo de variáveis**

*(em elaboracao)*





## **60. Arrays indexados e associativos**

*(em elaboracao)*



## 61. Parâmetros posicionais e getopt

*(em elaboracao)*



## 62. Expansão de parâmetros avançada

*(em elaboracao)*



## **63. Aritmética e manipulação de números no shell**

*(em elaboracao)*





**Parte XI.**

**Volume 11 — Processamento de Texto**



## **64. Expressões regulares: fundamentos e os diferentes sabores**

*(em elaboracao)*



## 65. grep e a busca de padrões

*(em elaboracao)*



## **66. sed: o editor de fluxo**

*(em elaboracao)*





## **67. awk: processamento por campos e registros**

*(em elaboracao)*



## **68. cut, sort, uniq, tr e a caixa de ferramentas de texto**

*(em elaboracao)*



## **69. Dados estruturados na linha de comando: jq, JSON e CSV**

*(em elaboracao)*



## **70. Juntando tudo: pipelines de processamento de dados**

*(em elaboracao)*





**Parte XII.**

**Volume 12 — Scripting Avançado e  
Robusto**



## **71. Tratamento de erros: set -euo pipefail e traps**

*(em elaboracao)*



## 72. Depuração de scripts: set -x e ShellCheck

*(em elaboracao)*



## **73. Manipulando arquivos, descritores e processos em scripts**

*(em elaboracao)*





## **74. Subshells, agrupamento e here-documents**

*(em elaboracao)*



## **75. Sinais, processos filhos e execução em paralelo**

*(em elaboracao)*



## **76. Interação com o usuário: menus, cores e caixas de diálogo**

*(em elaboracao)*



## **77. Estruturando scripts grandes e bibliotecas de funções**

*(em elaboracao)*





**Parte XIII.**

**Volume 13 — Redes no Linux**



## **78. Fundamentos: interfaces, endereçamento IP e a pilha de rede**

*(em elaboracao)*



## **79. Configuração de rede: ip, NetworkManager e arquivos**

*(em elaboracao)*



## **80. Diagnóstico: ping, traceroute, ss, dig e mtr**

*(em elaboracao)*





## **81. Acesso e transferência remota: SSH, scp e rsync**

*(em elaboracao)*



## **82. HTTP na linha de comando: curl, wget e netcat**

*(em elaboracao)*



## 83. Firewall no Linux: nftables e iptables

*(em elaboracao)*



## **84. Captura e análise de tráfego: tcpdump e tshark**

*(em elaboracao)*





**Parte XIV.**

**Volume 14 — Segurança e o Ambiente  
Kali**



## **85. Hardening do Linux: superfície de ataque e princípios**

*(em elaboracao)*



## **86. Gerenciamento de segredos: chaves SSH e GPG**

*(em elaboracao)*



## **87. Capabilities, isolamento e menor privilégio**

*(em elaboracao)*





## **88. Confinamento: AppArmor, SELinux e sandboxing**

*(em elaboracao)*



## **89. Auditoria, integridade de arquivos e detecção de alterações**

*(em elaboracao)*



## **90. O ecossistema de ferramentas do Kali: categorias e fluxos**

*(em elaboracao)*



## **91. Anonimato e privacidade: proxychains, Tor e VPN**

*(em elaboracao)*





**Parte XV.**

**Volume 15 — Automação e  
Produtividade**



## 92. Personalizando o shell: .bashrc, .zshrc, aliases e funções

*(em elaboracao)*



## 93. Zsh, plugins e o prompt do Kali

*(em elaboracao)*



## **94. Automatizando tarefas com cron e scripts agendados**

*(em elaboracao)*





## **95. Controle de versão com Git na linha de comando**

*(em elaboracao)*



## 96. Editores no terminal: vim e nano

*(em elaboracao)*



## **97. Dotfiles e ambientes reproduzíveis**

*(em elaboracao)*



**Parte XVI.**

**Volume 16 — Kernel, Contêineres e  
Virtualização**





## **98. O kernel Linux: módulos, /proc e /sys**

*(em elaboracao)*



## **99. Contêineres por dentro: namespaces, cgroups e Docker**

*(em elaboracao)*



## **100. Virtualização: KVM, QEMU e o laboratório de estudos**

*(em elaboracao)*



## **101. Compilação, toolchains e o ecossistema de desenvolvimento**

*(em elaboracao)*





## **102. Observabilidade e desempenho: strace, ltrace e perf**

*(em elaboracao)*



## **103. Para onde ir depois: certificações, comunidade e prática contínua**

*(em elaboracao)*



# Referências

DEBIAN PROJECT. **Debian Policy Manual**. [S.l.]: Debian Project, 2025.

FREE SOFTWARE FOUNDATION. **Bash Reference Manual**. [S.l.]: GNU Project, 2024b.

FREE SOFTWARE FOUNDATION. **GNU Coding Standards**. [S.l.]: GNU Project, 2024a.

HERTZOG, Raphaël; O'GORMAN, Jim; AHARONI, Mati. **Kali Linux Revealed: Mastering the Penetration Testing Distribution**. Cornelius, NC: Offsec Press, 2017.

IEEE AND THE OPEN GROUP. **The Open Group Base Specifications Issue 7, IEEE Std 1003.1-2017**. [S.l.]: The Open Group, 2018.

KERNIGHAN, Brian W.; PIKE, Rob. **The Unix Programming Environment**. Englewood Cliffs, NJ: Prentice Hall, 1984.

KERRISK, Michael. **The Linux Programming Interface**. San Francisco, CA: No Starch Press, 2010.

LINUX FOUNDATION. **Filesystem Hierarchy Standard, Version 3.0**. [S.l.]: Linux Foundation, 2015.

LOVE, Robert. **Linux Kernel Development**. 3. ed. Upper Saddle River, NJ: Addison-Wesley, 2010.

NEMETH, Evi *et al.* **UNIX and Linux System Administration Handbook**. 5. ed. Boston, MA: Addison-Wesley, 2017.

OFFENSIVE SECURITY. **Kali Linux Documentation**. <https://www.kali.org/docs/>, 2025.

